

rt_lvgl_littlefs 工程说明与逐行代码分析

面向刚开始接触本工程的同学：从整体流程、组件职责、依赖关系，到 applications 目录每个文件的逐行说明。

生成路径：D:\RT-ThreadStudio\rt_rhread_project\rt_lvgl_littlefs\docs\rt_lvgl_littlefs_project_explained.pdf

源码根目录：D:\RT-ThreadStudio\rt_rhread_project\rt_lvgl_littlefs

提示：源码中部分中文注释/字符串当前呈现为乱码形态，本文档会保留源码原样并从代码行为角度解释其作用。若后续要修复显示文字，可优先检查源文件编码、LVGL 字体字库和字符串编码是否统一为 UTF-8。

目录

- 1. 工程整体目标
- 2. 启动与运行流程
- 3. 使用的组件、原因、接口和依赖
- 4. 关键配置开关
- 5. applications 每个文件逐行说明

工程整体目标

这个工程把 STM32F407 板子做成一个小型联网显示终端：底层用 RT-Thread 管理线程、设备和文件系统；屏幕使用 ILI9341 SPI LCD；界面用 LVGL 绘制；外部 W25Q32 SPI Flash 通过 SFUD、MTD、littlefs 挂载成文件系统；网络通过 ESP8266 AT 设备接入；云端消息用 Paho MQTT 接收，既能显示 PC 状态，也能通过 MQTT 下发命令切换页面、查看文件系统、执行简单终端命令。

启动与运行流程

```
上电
-> RT-Thread 内核启动
-> 自动初始化阶段
    -> board_devcie.c: lcd_spi_hw_init() 注册 spi2/lcd
    -> board_devcie.c: w25q32_spi_hw_init() 注册 spi1/w25q32, 探测 SFUD, 注册 MTD
    -> board_devcie.c: esp8266_at_client_init() 初始化 uart2 AT 客户端
    -> board_devcie.c: w25q32_lfs_mount() 挂载 littlefs 到 /
-> main()
    -> LCD_Init() 初始化 ILI9341
    -> lv_init() 初始化 LVGL
    -> lv_port_disp_init() 注册 LVGL 显示驱动
    -> app_ui_create() 创建页面和控件
    -> 创建 ui 线程: 周期调用 lv_timer_handler() 和 app_ui_update()
    -> 创建 mqtt 线程: 延时后 my_mqtt_start()
-> MQTT 收到消息
    -> my_mqtt.c 回调解析 topic/payload
    -> PC 状态更新到 mqtt_pc_status, UI 下一轮刷新显示
    -> 命令类消息调用 app_ui_request_page()/文件系统/POSIX 接口
```

使用的组件、原因、接口和依赖

组件	为什么使用 / 提供什么接口 / 依赖关系
RT-Thread 内核	线程、tick、延时、日志、自动初始化、设备对象、MSH 命令。代码里常见 rt_thread_create、rt_thread_mdelay、rt_tick_get_millisecond、INIT_DEVICE_EXPORT、MSH_CMD_EXPORT。
RT-Thread SPI 设备框架	把硬件 SPI 总线和具体片选设备抽象成 rt_spi_device。LCD 使用 spi2/lcd, W25Q32 使用 spi1/w25q32。

组件	为什么使用 / 提供什么接口 / 依赖关系
STM32 HAL	提供 GPIO、SPI MSP 初始化、延时和错误状态类型。LCD 驱动里用 HAL_GPIO_WritePin 控制 CS/DC/BLK。
ILI9341 LCD 控制器	240x320 RGB565 显示屏控制芯片。驱动通过 CASET/PASET/RAMWR 设置窗口并写入像素。
LVGL 8.3.11	嵌入式 GUI。main.c 调 lv_init, lv_port_disp.c 注册显示驱动, ui.c 创建标签、图片和对象。
SFUD + MTD NOR	SFUD 识别和读写 SPI Flash, MTD NOR 把 Flash 包装成 RT-Thread 块设备, 给 littlefs 使用。
littlefs + DFS/POSIX	littlefs 提供掉电友好的小型文件系统, DFS/POSIX 让代码能用 open/read/write/opendir/statfs 等接口。
SAL/AT/ESP8266	SAL 统一 socket API, AT 设备包通过 ESP8266 提供网络能力, MQTT 和 UDP 测试都依赖网络栈。
Paho MQTT	提供 MQTTClient、paho_mqtt_start、paho_mqtt_publish、messageHandlers 等接口, 用于云端消息和远程命令。
MSH/FinSH	命令行外壳。工程导出 mqtt_start、mqtt_stop、udp_test、http_get、switch 等命令便于调试。

关键 RT-Thread / 软件包配置

宏	值	说明
RT_USING_OVERFLOW_CHECK		RT-Thread 功能开关：启用对应内核、驱动或组件能力。
RT_USING_HOOK		RT-Thread 功能开关：启用对应内核、驱动或组件能力。
RT_USING_IDLE_HOOK		RT-Thread 功能开关：启用对应内核、驱动或组件能力。
RT_USING_TIMER_SOFT		RT-Thread 功能开关：启用对应内核、驱动或组件能力。
RT_USING_SEMAPHORE		RT-Thread 功能开关：启用对应内核、驱动或组件能力。
RT_USING_MUTEX		RT-Thread 功能开关：启用对应内核、驱动或组件能力。
RT_USING_EVENT		RT-Thread 功能开关：启用对应内核、驱动或组件能力。
RT_USING_MAILBOX		RT-Thread 功能开关：启用对应内核、驱动或组件能力。
RT_USING_MESSAGEQUEUE		RT-Thread 功能开关：启用对应内核、驱动或组件能力。
RT_USING_MEMPOOL		RT-Thread 功能开关：启用对应内核、驱动或组件能力。
RT_USING_SMALL_MEM		RT-Thread 功能开关：启用对应内核、驱动或组件能力。
RT_USING_HEAP		RT-Thread 功能开关：启用对应内核、驱动或组件能力。
RT_USING_DEVICE		RT-Thread 功能开关：启用对应内核、驱动或组件能力。
RT_USING_CONSOLE		RT-Thread 功能开关：启用对应内核、驱动或组件能力。
RT_USING_CPU_FFS		RT-Thread 功能开关：启用对应内核、驱动或组件能力。
RT_USING_COMPONENTS_INIT		RT-Thread 功能开关：启用对应内核、驱动或组件能力。
RT_USING_USER_MAIN		RT-Thread 功能开关：启用对应内核、驱动或组件能力。
RT_USING_FINSH		RT-Thread 功能开关：启用对应内核、驱动或组件能力。
RT_USING_DFS		RT-Thread 功能开关：启用对应内核、驱动或组件能力。

宏	值	说明
RT_USING_DFS_DEVFS		RT-Thread 功能开关：启用对应内核、驱动或组件能力。
RT_USING_DEVICE_IPC		RT-Thread 功能开关：启用对应内核、驱动或组件能力。
RT_USING_SYSTEM_WORKQUEUE		RT-Thread 功能开关：启用对应内核、驱动或组件能力。
RT_USING_SERIAL		RT-Thread 功能开关：启用对应内核、驱动或组件能力。
RT_USING_PIN		RT-Thread 功能开关：启用对应内核、驱动或组件能力。
RT_USING_MTD_NOR		RT-Thread 功能开关：启用对应内核、驱动或组件能力。
RT_USING_RTC		RT-Thread 功能开关：启用对应内核、驱动或组件能力。
RT_USING_SOFT_RTC		RT-Thread 功能开关：启用对应内核、驱动或组件能力。
RTC_SYNC_USING_NTP		宏定义，用名字替代固定值，方便统一修改和表达含义。
RTC_NTP_FIRST_SYNC_DELAY	30	宏定义，用名字替代固定值，方便统一修改和表达含义。
RTC_NTP_SYNC_PERIOD	3600	宏定义，用名字替代固定值，方便统一修改和表达含义。
RT_USING_SPI		RT-Thread 功能开关：启用对应内核、驱动或组件能力。
RT_USING_SFUD		RT-Thread 功能开关：启用对应内核、驱动或组件能力。
RT_USING_LIBC		RT-Thread 功能开关：启用对应内核、驱动或组件能力。
RT_USING_POSIX		RT-Thread 功能开关：启用对应内核、驱动或组件能力。
RT_USING_SAL		RT-Thread 功能开关：启用对应内核、驱动或组件能力。
SAL_INTERNET_CHECK		宏定义，用名字替代固定值，方便统一修改和表达含义。
SAL_USING_AT		宏定义，用名字替代固定值，方便统一修改和表达含义。
SAL_USING_POSIX		宏定义，用名字替代固定值，方便统一修改和表达含义。
RT_USING_NETDEV		RT-Thread 功能开关：启用对应内核、驱动或组件能力。
NETDEV_USING_IFCONFIG		宏定义，用名字替代固定值，方便统一修改和表达含义。
NETDEV_USING_PING		宏定义，用名字替代固定值，方便统一修改和表达含义。
NETDEV_USING_NETSTAT		宏定义，用名字替代固定值，方便统一修改和表达含义。
NETDEV_USING_AUTO_DEFAULT		宏定义，用名字替代固定值，方便统一修改和表达含义。
NETDEV_IPV4	1	宏定义，用名字替代固定值，方便统一修改和表达含义。
NETDEV_IPV6	0	宏定义，用名字替代固定值，方便统一修改和表达含义。
RT_USING_AT		RT-Thread 功能开关：启用对应内核、驱动或组件能力。
AT_USING_CLIENT		宏定义，用名字替代固定值，方便统一修改和表达含义。
AT_CLIENT_NUM_MAX	1	宏定义，用名字替代固定值，方便统一修改和表达含义。
AT_USING_SOCKET		宏定义，用名字替代固定值，方便统一修改和表达含义。
AT_USING_CLI		宏定义，用名字替代固定值，方便统一修改和表达含义。
AT_CMD_MAX_LEN	128	宏定义，用名字替代固定值，方便统一修改和表达含义。

宏	值	说明
AT_SW_VERSION_NUM	0x10301	宏定义，用名字替代固定值，方便统一修改和表达含义。
PKG_USING_PAHOMQTT		RT-Thread 软件包开关：把对应第三方/软件包加入工程。
PAHOMQTT_PIPE_MODE		宏定义，用名字替代固定值，方便统一修改和表达含义。
MQTT_DEBUG		宏定义，用名字替代固定值，方便统一修改和表达含义。
PKG_USING_PAHOMQTT_V110		RT-Thread 软件包开关：把对应第三方/软件包加入工程。
PKG_USING_NETUTILS		RT-Thread 软件包开关：把对应第三方/软件包加入工程。
NTP_USING_AUTO_SYNC		宏定义，用名字替代固定值，方便统一修改和表达含义。
NTP_AUTO_SYNC_FIRST_DELAY	20	宏定义，用名字替代固定值，方便统一修改和表达含义。
NTP_AUTO_SYNC_PERIOD	3600	宏定义，用名字替代固定值，方便统一修改和表达含义。
PKG_USING_NETUTILS_LATEST_VERSION		RT-Thread 软件包开关：把对应第三方/软件包加入工程。
PKG_USING_AT_DEVICE		RT-Thread 软件包开关：把对应第三方/软件包加入工程。
AT_DEVICE_USING_ESP8266		宏定义，用名字替代固定值，方便统一修改和表达含义。
AT_DEVICE_ESP8266_SOCKET		宏定义，用名字替代固定值，方便统一修改和表达含义。
AT_DEVICE_ESP8266_SAMPLE		宏定义，用名字替代固定值，方便统一修改和表达含义。
PKG_USING_AT_DEVICE_V204		RT-Thread 软件包开关：把对应第三方/软件包加入工程。
PKG_USING_LITTLEFS		RT-Thread 软件包开关：把对应第三方/软件包加入工程。
PKG_USING_LITTLEFS_V250		RT-Thread 软件包开关：把对应第三方/软件包加入工程。
LFS_READ_SIZE	256	littlefs 参数，决定读写粒度、块大小、缓存和线程安全行为。
LFS_PROG_SIZE	256	littlefs 参数，决定读写粒度、块大小、缓存和线程安全行为。
LFS_BLOCK_SIZE	4096	littlefs 参数，决定读写粒度、块大小、缓存和线程安全行为。
LFS_CACHE_SIZE	256	littlefs 参数，决定读写粒度、块大小、缓存和线程安全行为。
LFS_BLOCK_CYCLES	-1	littlefs 参数，决定读写粒度、块大小、缓存和线程安全行为。
LFS_THREADSAFE		littlefs 参数，决定读写粒度、块大小、缓存和线程安全行为。
LFS_LOOKAHEAD_MAX	128	littlefs 参数，决定读写粒度、块大小、缓存和线程安全行为。

关键 LVGL 配置

宏	值	说明
LV_COLOR_DEPTH	16	LVGL 颜色配置，影响像素格式和屏幕输出字节顺序。
LV_COLOR_16_SWAP	1	LVGL 颜色配置，影响像素格式和屏幕输出字节顺序。
LV_MEM_SIZE	(48U * 1024U) /* [bytes]*/	宏定义，用名字替代固定值，方便统一修改和表达含义。
LV_TICK_CUSTOM	1	宏定义，用名字替代固定值，方便统一修改和表达含义。

宏	值	说明
LV_DRAW_COMPLEX	0	宏定义，用名字替代固定值，方便统一修改和表达含义。
LV_FONT_MONTERRAT_14	1	宏定义，用名字替代固定值，方便统一修改和表达含义。
LV_TXT_ENC	LV_TXT_ENC_UTF8	宏定义，用名字替代固定值，方便统一修改和表达含义。
LV_USE_IMG	1 /*Requires: lv_label*/	LVGL 功能裁剪开关：1 表示启用，0 表示关闭以节省 Flash/RAM。
LV_USE_LABEL	1	LVGL 功能裁剪开关：1 表示启用，0 表示关闭以节省 Flash/RAM。
LV_USE_THEME_DEFAULT	1	LVGL 功能裁剪开关：1 表示启用，0 表示关闭以节省 Flash/RAM。
LV_USE_FS_POSIX	1	LVGL 功能裁剪开关：1 表示启用，0 表示关闭以节省 Flash/RAM。

applications 逐文件逐行说明

SConscript

RT-Thread Studio/SCons 构建入口，把 applications 下的 C 文件组成 Applications 分组并加入编译。

本文件共 9 行。下面的表格左侧是原始源码，中间是行号，右侧是逐行说明。资源数组文件的数组数据较长，说明会重点解释其作为图片/字体资源的角色。

行	源码	说明
1	<code>from building import *</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
2		空行，用来分隔逻辑块，让代码更容易阅读。
3	<code>cwd = GetCurrentDir()</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
4	<code>src = Glob('*.*')</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
5	<code>path = [cwd]</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
6		空行，用来分隔逻辑块，让代码更容易阅读。
7	<code>group = DefineGroup('Applications', src, depend = [''], CPPPATH = path)</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
8		空行，用来分隔逻辑块，让代码更容易阅读。
9	<code>Return('group')</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。

main.c

应用主入口。负责 LCD 初始化、LVGL 初始化、创建 UI、启动 UI 刷新线程和 MQTT 线程。

本文件共 96 行。下面的表格左侧是原始源码，中间是行号，右侧是逐行说明。资源数组文件的数组数据较长，说明会重点解释其作为图片/字体资源的角色。

行	源码	说明
1	/*	块注释开始/内容，解释接下来代码为什么这样写。
2	* Copyright (c) 2006-2026, RT-Thread Development Team	块注释内容，属于说明文字，不会被编译进程序逻辑。
3	*	块注释内容，属于说明文字，不会被编译进程序逻辑。
4	* SPDX-License-Identifier: Apache-2.0	块注释内容，属于说明文字，不会被编译进程序逻辑。
5	*	块注释内容，属于说明文字，不会被编译进程序逻辑。
6	* Change Logs:	块注释内容，属于说明文字，不会被编译进程序逻辑。
7	* Date Author Notes	块注释内容，属于说明文字，不会被编译进程序逻辑。
8	* 2026-04-26 RT-Thread first version	块注释内容，属于说明文字，不会被编译进程序逻辑。
9	*/	块注释结束。
10		空行，用来分隔逻辑块，让代码更容易阅读。
11	#include <rtthread.h>	引入头文件 <rtthread.h>，让当前文件可以使用其中声明的类型、宏和函数。
12	#include <rtdbg.h>	引入头文件 <rtdbg.h>，让当前文件可以使用其中声明的类型、宏和函数。
13		空行，用来分隔逻辑块，让代码更容易阅读。
14	#include "Lcd.h"	引入头文件 "Lcd.h"，让当前文件可以使用其中声明的类型、宏和函数。
15	#include "lvgl.h"	引入头文件 "lvgl.h"，让当前文件可以使用其中声明的类型、宏和函数。
16	#include "lv_port_disp.h"	引入头文件 "lv_port_disp.h"，让当前文件可以使用其中声明的类型、宏和函数。
17	#include "my_mqtt.h"	引入头文件 "my_mqtt.h"，让当前文件可以使用其中声明的类型、宏和函数。
18	#include "ui.h"	引入头文件 "ui.h"，让当前文件可以使用其中声明的类型、宏和函数。
19		空行，用来分隔逻辑块，让代码更容易阅读。
20	#define DBG_TAG "main"	宏定义，用名字替代固定值，方便统一修改和表达含义。
21	#define DBG_LVL DBG_LOG	宏定义，用名字替代固定值，方便统一修改和表达含义。

行	源码	说明
22		空行，用来分隔逻辑块，让代码更容易阅读。
23	<code>#define UI_THREAD_STACK_SIZE 4096</code>	线程栈大小配置，栈越大越不容易溢出，但会占更多 RAM。
24	<code>#define UI_THREAD_PRIORITY 20</code>	线程优先级配置；RT-Thread 数字越小优先级越高。
25	<code>#define UI_UPDATE_PERIOD_MS 500U</code>	宏定义，用名字替代固定值，方便统一修改和表达含义。
26	<code>#define MQTT_THREAD_STACK_SIZE 4096</code>	线程栈大小配置，栈越大越不容易溢出，但会占更多 RAM。
27	<code>#define MQTT_THREAD_PRIORITY 22</code>	线程优先级配置；RT-Thread 数字越小优先级越高。
28	<code>#define MQTT_START_DELAY_MS 500U</code>	宏定义，用名字替代固定值，方便统一修改和表达含义。
29		空行，用来分隔逻辑块，让代码更容易阅读。
30	<code>static void ui_thread_entry(void *parameter)</code>	声明函数 <code>ui_thread_entry</code> ，告诉编译器后面会有这个接口。
31	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
32	<code>rt_tick_t last_update_tick = rt_tick_get();</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
33		空行，用来分隔逻辑块，让代码更容易阅读。
34	<code>(void)parameter;</code>	显式标记参数未使用，避免编译器产生未使用参数警告。
35		空行，用来分隔逻辑块，让代码更容易阅读。
36	<code>while (1)</code>	<code>while</code> 循环，只要条件成立就持续执行；线程主循环通常写成 <code>while (1)</code> 。
37	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
38	<code>lv_timer_handler();</code>	驱动 LVGL 定时器、动画和刷新流程；必须周期性调用。
39		空行，用来分隔逻辑块，让代码更容易阅读。
40	<code>if ((rt_tick_get() - last_update_tick) >=</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
41	<code>rt_tick_from_millisecond(UI_UPDATE_PERIOD_MS))</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
42	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
43	<code>last_update_tick = rt_tick_get();</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
44	<code>app_ui_update();</code>	周期性刷新 UI 状态、消息、时间和心跳。
45	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
46		空行，用来分隔逻辑块，让代码更容易阅读。
47	<code>rt_thread_mdelay(5);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
48	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
49	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。

行	源码	说明
50		空行，用来分隔逻辑块，让代码更容易阅读。
51	<code>static void mqtt_thread_entry(void *parameter)</code>	声明函数 <code>mqtt_thread_entry</code> ，告诉编译器后面会有这个接口。
52	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
53	<code>(void)parameter;</code>	显式标记参数未使用，避免编译器产生未使用参数警告。
54		空行，用来分隔逻辑块，让代码更容易阅读。
55	<code>rt_thread_mdelay(MQTT_START_DELAY_MS);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
56	<code>my_mqtt_start();</code>	启动 MQTT 客户端并订阅主题。
57	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
58		空行，用来分隔逻辑块，让代码更容易阅读。
59	<code>static void create_app_thread(const char *name,</code>	定义当前文件私有的静态对象或函数，限制作用域，降低模块耦合。
60	<code>void (*entry)(void *parameter),</code>	初始化表中的一项，用于配置数组、结构体或常量表。
61	<code>rt_uint32_t stack_size,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
62	<code>rt_uint8_t priority)</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
63	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
64	<code>rt_thread_t tid;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
65		空行，用来分隔逻辑块，让代码更容易阅读。
66	<code>tid = rt_thread_create(name, entry, RT_NULL, stack_size, priority, 20);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
67	<code>if (tid != RT_NULL)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
68	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
69	<code>rt_thread_startup(tid);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
70	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
71	<code>else</code>	条件判断的兜底分支，前面的条件都不满足时执行。
72	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
73	<code>LOG_E("create %s thread failed", name);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
74	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
75	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
76		空行，用来分隔逻辑块，让代码更容易阅读。
77	<code>int main(void)</code>	声明函数 <code>main</code> ，告诉编译器后面会有这个接口。

行	源码	说明
78	{	代码块开始，进入 当前结构 的内部逻辑。
79	rt_thread_mdelay(500);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
80		空行，用来分隔逻辑块，让代码更容易阅读。
81	LOG_D("Hello RT-Thread!");	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
82		空行，用来分隔逻辑块，让代码更容易阅读。
83	LCD_Init();	初始化 ILI9341 屏幕控制器，让屏幕进入可显示状态。
84		空行，用来分隔逻辑块，让代码更容易阅读。
85	lv_init();	初始化 LVGL 核心库。
86	lv_port_disp_init();	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
87		空行，用来分隔逻辑块，让代码更容易阅读。
88	app_ui_create();	创建应用 UI 的所有页面和控件。
89		空行，用来分隔逻辑块，让代码更容易阅读。
90	create_app_thread("ui", ui_thread_entry,	初始化表中的一项，用于配置数组、结构体或常量表。
91	UI_THREAD_STACK_SIZE, UI_THREAD_PRIORITY);	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
92	create_app_thread("mqtt", mqtt_thread_entry,	初始化表中的一项，用于配置数组、结构体或常量表。
93	MQTT_THREAD_STACK_SIZE, MQTT_THREAD_PRIORITY);	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
94		空行，用来分隔逻辑块，让代码更容易阅读。
95	return RT_EOK;	从当前函数返回结果；错误码或状态值会交给调用者判断。
96	}	代码块结束，离开 当前结构 的内部逻辑。

board_devcie.c

板级设备注册层。负责 SPI1/SPI2 引脚复用、LCD SPI 设备挂载、W25Q32 SPI/SFUD/MTD/littlefs 挂载、ESP8266 AT 客户端初始化。

本文件共 341 行。下面的表格左侧是原始源码，中间是行号，右侧是逐行说明。资源数组文件的数组数据较长，说明会重点解释其作为图片/字体资源的角色。

行	源码	说明
1	#include <rtthread.h>	引入头文件 <rtthread.h>，让当前文件可以使用其中声明的类型、宏和函数。
2	#include <rtdevice.h>	引入头文件 <rtdevice.h>，让当前文件可以使用其中声明的类型、宏和函数。
3	#include <board.h>	引入头文件 <board.h>，让当前文件可以使用其中声明的类型、宏和函数。
4	#include "drv_spi.h"	引入头文件 "drv_spi.h"，让当前文件可以使用其中声明的类型、宏和函数。
5	#ifdef RT_USING_SFUD	条件编译开始：只有配置宏满足时，下面这段代码才会参与编译。
6	#include "spi_flash_sfud.h"	引入头文件 "spi_flash_sfud.h"，让当前文件可以使用其中声明的类型、宏和函数。
7	#endif	条件编译结束。
8	#if defined(RT_USING_SFUD) && defined(RT_USING_MTD_NOR)	条件编译开始：只有配置宏满足时，下面这段代码才会参与编译。
9	#include <drivers/mtd_nor.h>	引入头文件 <drivers/mtd_nor.h>，让当前文件可以使用其中声明的类型、宏和函数。
10	#endif	条件编译结束。
11	#if defined(PKG_USING_LITTLEFS) && defined(RT_USING_DFS)	条件编译开始：只有配置宏满足时，下面这段代码才会参与编译。
12	#include <dfs_fs.h>	引入头文件 <dfs_fs.h>，让当前文件可以使用其中声明的类型、宏和函数。
13	#endif	条件编译结束。
14	#if defined(RT_USING_SFUD) && defined(FINSH_USING_MSH)	条件编译开始：只有配置宏满足时，下面这段代码才会参与编译。
15	#include <finsh.h>	引入头文件 <finsh.h>，让当前文件可以使用其中声明的类型、宏和函数。
16	#endif	条件编译结束。
17		空行，用来分隔逻辑块，让代码更容易阅读。
18	#define LCD_CS_PIN GET_PIN(B, 12)	宏定义，用名字替代固定值，方便统一修改和表达含义。
19	#define LCD_DC_PIN GET_PIN(C, 5)	宏定义，用名字替代固定值，方便统一修改和表达含义。
20	#define LCD_BLK_PIN GET_PIN(B, 1)	宏定义，用名字替代固定值，方便统一修改和表达含义。
21	#define W25Q32_CS_PIN GET_PIN(A, 15)	宏定义，用名字替代固定值，方便统一修改和表达含义。

行	源码	说明
22		空行，用来分隔逻辑块，让代码更容易阅读。
23	static struct rt_spi_device *lcd_spi_dev = RT_NULL;	定义当前文件私有的静态变量或函数，并给出初始值；外部文件不能直接访问。
24	static struct rt_spi_device *w25q32_spi_dev = RT_NULL;	定义当前文件私有的静态变量或函数，并给出初始值；外部文件不能直接访问。
25		空行，用来分隔逻辑块，让代码更容易阅读。
26	#if defined(RT_USING_SFUD) && defined(RT_USING_MTD_NOR)	条件编译开始：只有配置宏满足时，下面这段代码才会参与编译。
27	static sfud_flash_t w25q32_sfud = RT_NULL;	定义当前文件私有的静态变量或函数，并给出初始值；外部文件不能直接访问。
28	static struct rt_mtd_nor_device w25q32_mtd;	定义当前文件私有的静态对象或函数，限制作用域，降低模块耦合。
29		空行，用来分隔逻辑块，让代码更容易阅读。
30	static rt_err_t w25q32_mtd_read_id(struct rt_mtd_nor_device *device)	声明函数 w25q32_mtd_read_id，告诉编译器后面会有这个接口。
31	{	代码块开始，进入 当前结构 的内部逻辑。
32	(void)device;	显式标记参数未使用，避免编译器产生未使用参数警告。
33	return RT_EOK;	从当前函数返回结果；错误码或状态值会交给调用者判断。
34	}	代码块结束，离开 当前结构 的内部逻辑。
35		空行，用来分隔逻辑块，让代码更容易阅读。
36	static rt_size_t w25q32_mtd_read(struct rt_mtd_nor_device *device,	定义当前文件私有的静态对象或函数，限制作用域，降低模块耦合。
37	rt_off_t offset,	初始化表中的一项，用于配置数组、结构体或常量表。
38	rt_uint8_t *data,	初始化表中的一项，用于配置数组、结构体或常量表。
39	rt_uint32_t length)	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
40	{	代码块开始，进入 当前结构 的内部逻辑。
41	(void)device;	显式标记参数未使用，避免编译器产生未使用参数警告。
42		空行，用来分隔逻辑块，让代码更容易阅读。
43	if ((w25q32_sfud == RT_NULL) (offset < 0))	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
44	{	代码块开始，进入 当前结构 的内部逻辑。
45	return 0;	从当前函数返回结果；错误码或状态值会交给调用者判断。
46	}	代码块结束，离开 当前结构 的内部逻辑。
47		空行，用来分隔逻辑块，让代码更容易阅读。
48	if (sfud_read(w25q32_sfud, offset, length, data) != SFUD_SUCCESS)	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。

行	源码	说明
49	{	代码块开始，进入 当前结构 的内部逻辑。
50	return 0;	从当前函数返回结果；错误码或状态值会交给调用者判断。
51	}	代码块结束，离开 当前结构 的内部逻辑。
52		空行，用来分隔逻辑块，让代码更容易阅读。
53	return length;	从当前函数返回结果；错误码或状态值会交给调用者判断。
54	}	代码块结束，离开 当前结构 的内部逻辑。
55		空行，用来分隔逻辑块，让代码更容易阅读。
56	static rt_size_t w25q32_mtd_write(struct rt_mtd_nor_device *device,	定义当前文件私有的静态对象或函数，限制作用域，降低模块耦合。
57	rt_off_t offset,	初始化表中的一项，用于配置数组、结构体或常量表。
58	const rt_uint8_t *data,	初始化表中的一项，用于配置数组、结构体或常量表。
59	rt_uint32_t length)	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
60	{	代码块开始，进入 当前结构 的内部逻辑。
61	(void)device;	显式标记参数未使用，避免编译器产生未使用参数警告。
62		空行，用来分隔逻辑块，让代码更容易阅读。
63	if ((w25q32_sfud == RT_NULL) (offset < 0))	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
64	{	代码块开始，进入 当前结构 的内部逻辑。
65	return 0;	从当前函数返回结果；错误码或状态值会交给调用者判断。
66	}	代码块结束，离开 当前结构 的内部逻辑。
67		空行，用来分隔逻辑块，让代码更容易阅读。
68	if (sfud_write(w25q32_sfud, offset, length, data) != SFUD_SUCCESS)	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
69	{	代码块开始，进入 当前结构 的内部逻辑。
70	return 0;	从当前函数返回结果；错误码或状态值会交给调用者判断。
71	}	代码块结束，离开 当前结构 的内部逻辑。
72		空行，用来分隔逻辑块，让代码更容易阅读。
73	return length;	从当前函数返回结果；错误码或状态值会交给调用者判断。
74	}	代码块结束，离开 当前结构 的内部逻辑。
75		空行，用来分隔逻辑块，让代码更容易阅读。
76	static rt_err_t w25q32_mtd_erase_block(struct rt_mtd_nor_device *device,	定义当前文件私有的静态对象或函数，限制作用域，降低模块耦合。

行	源码	说明
77	rt_off_t offset,	初始化表中的一项，用于配置数组、结构体或常量表。
78	rt_uint32_t length)	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
79	{	代码块开始，进入 当前结构 的内部逻辑。
80	(void)device;	显式标记参数未使用，避免编译器产生未使用参数警告。
81		空行，用来分隔逻辑块，让代码更容易阅读。
82	if ((w25q32_sfud == RT_NULL) (offset < 0))	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
83	{	代码块开始，进入 当前结构 的内部逻辑。
84	return -RT_ERROR;	从当前函数返回结果；错误码或状态值会交给调用者判断。
85	}	代码块结束，离开 当前结构 的内部逻辑。
86		空行，用来分隔逻辑块，让代码更容易阅读。
87	if (sfud_erase(w25q32_sfud, offset, length) != SFUD_SUCCESS)	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
88	{	代码块开始，进入 当前结构 的内部逻辑。
89	return -RT_ERROR;	从当前函数返回结果；错误码或状态值会交给调用者判断。
90	}	代码块结束，离开 当前结构 的内部逻辑。
91		空行，用来分隔逻辑块，让代码更容易阅读。
92	return RT_EOK;	从当前函数返回结果；错误码或状态值会交给调用者判断。
93	}	代码块结束，离开 当前结构 的内部逻辑。
94		空行，用来分隔逻辑块，让代码更容易阅读。
95	static const struct rt_mtd_nor_driver_ops w25q32_mtd_ops =	定义当前文件私有的静态变量或函数，并给出初始值；外部文件不能直接访问。
96	{	代码块开始，进入 当前结构 的内部逻辑。
97	w25q32_mtd_read_id,	初始化表中的一项，用于配置数组、结构体或常量表。
98	w25q32_mtd_read,	初始化表中的一项，用于配置数组、结构体或常量表。
99	w25q32_mtd_write,	初始化表中的一项，用于配置数组、结构体或常量表。
100	w25q32_mtd_erase_block,	初始化表中的一项，用于配置数组、结构体或常量表。
101	};	结构体、枚举或初始化表结束。
102	#endif	条件编译结束。
103		空行，用来分隔逻辑块，让代码更容易阅读。
104	void HAL_SPI_MspInit(SPI_HandleTypeDef *hspi)	声明函数 HAL_SPI_MspInit，告诉编译器后面会有这个接口。

行	源码	说明
105	{	代码块开始，进入 当前结构 的内部逻辑。
106	GPIO_InitTypeDef GPIO_InitStructure = {0};	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
107		空行，用来分隔逻辑块，让代码更容易阅读。
108	if (hspi->Instance == SPI2)	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
109	{	代码块开始，进入 当前结构 的内部逻辑。
110	__HAL_RCC_SPI2_CLK_ENABLE();	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
111	__HAL_RCC_GPIOB_CLK_ENABLE();	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
112		空行，用来分隔逻辑块，让代码更容易阅读。
113	/* SPI2: PB13=SCK, PB14=MISO, PB15=MOSI */	块注释开始/内容，解释接下来代码为什么这样写。
114	GPIO_InitStructure.Pin = GPIO_PIN_13 GPIO_PIN_14 GPIO_PIN_15;	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
115	GPIO_InitStructure.Mode = GPIO_MODE_AF_PP;	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
116	GPIO_InitStructure.Pull = GPIO_NOPULL;	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
117	GPIO_InitStructure.Speed = GPIO_SPEED_FREQ_VERY_HIGH;	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
118	GPIO_InitStructure.Alternate = GPIO_AF5_SPI2;	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
119	HAL_GPIO_Init(GPIOB, &GPIO_InitStructure);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
120	}	代码块结束，离开 当前结构 的内部逻辑。
121	else if (hspi->Instance == SPI1)	声明函数 if，告诉编译器后面会有这个接口。
122	{	代码块开始，进入 当前结构 的内部逻辑。
123	__HAL_RCC_SPI1_CLK_ENABLE();	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
124	__HAL_RCC_GPIOB_CLK_ENABLE();	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
125		空行，用来分隔逻辑块，让代码更容易阅读。
126	/* SPI1: PB3=SCK, PB4=MISO, PB5=MOSI */	块注释开始/内容，解释接下来代码为什么这样写。
127	GPIO_InitStructure.Pin = GPIO_PIN_3 GPIO_PIN_4 GPIO_PIN_5;	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
128	GPIO_InitStructure.Mode = GPIO_MODE_AF_PP;	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
129	GPIO_InitStructure.Pull = GPIO_NOPULL;	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
130	GPIO_InitStructure.Speed = GPIO_SPEED_FREQ_VERY_HIGH;	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。

行	源码	说明
131	<code>GPIO_InitStruct.Alternate = GPIO_AF5_SPI1;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
132	<code>HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
133	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
134	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
135		空行，用来分隔逻辑块，让代码更容易阅读。
136	<code>static int lcd_spi_hw_init(void)</code>	声明函数 <code>lcd_spi_hw_init</code> ，告诉编译器后面会有这个接口。
137	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
138	<code>rt_err_t ret;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
139	<code>struct rt_spi_configuration cfg;</code>	定义结构体类型，把一组相关数据字段放在一起。
140		空行，用来分隔逻辑块，让代码更容易阅读。
141	<code>/* LCD control pins. CS is initialized by rt_hw_spi_device_attach(). */</code>	块注释开始/内容，解释接下来代码为什么这样写。
142	<code>rt_pin_mode(LCD_DC_PIN, PIN_MODE_OUTPUT);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
143	<code>rt_pin_mode(LCD_BLK_PIN, PIN_MODE_OUTPUT);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
144		空行，用来分隔逻辑块，让代码更容易阅读。
145	<code>rt_pin_write(LCD_DC_PIN, PIN_HIGH);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
146	<code>rt_pin_write(LCD_BLK_PIN, PIN_HIGH);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
147		空行，用来分隔逻辑块，让代码更容易阅读。
148	<code>ret = rt_hw_spi_device_attach("spi2", "lcd", GPIOB, GPIO_PIN_12);</code>	把 SPI 总线和片选 GPIO 组合成一个 RT-Thread SPI 设备。
149	<code>if (ret != RT_EOK)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
150	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
151	<code>rt_kprintf("attach lcd failed: %d\n", ret);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
152	<code>return ret;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
153	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
154		空行，用来分隔逻辑块，让代码更容易阅读。
155	<code>lcd_spi_dev = (struct rt_spi_device *)rt_device_find("lcd");</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
156	<code>if (lcd_spi_dev == RT_NULL)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
157	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。

行	源码	说明
158	<code>rt_kprintf("find lcd failed\n");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
159	<code>return -RT_ERROR;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
160	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
161		空行，用来分隔逻辑块，让代码更容易阅读。
162	<code>cfg.data_width = 8;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
163	<code>cfg.mode = RT_SPI_MASTER RT_SPI_MODE_3 RT_SPI_MSB;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
164	<code>cfg.max_hz = 42000000;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
165		空行，用来分隔逻辑块，让代码更容易阅读。
166	<code>ret = rt_spi_configure(lcd_spi_dev, &cfg);</code>	配置 SPI 数据宽度、模式和速度。
167	<code>if (ret != RT_EOK)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
168	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
169	<code>rt_kprintf("config lcd spi failed: %d\n", ret);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
170	<code>return ret;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
171	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
172		空行，用来分隔逻辑块，让代码更容易阅读。
173	<code>rt_kprintf("lcd spi init ok\n");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
174	<code>return RT_EOK;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
175	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
176	<code>INIT_DEVICE_EXPORT(lcd_spi_hw_init);</code>	把初始化函数放入设备初始化阶段自动执行。
177		空行，用来分隔逻辑块，让代码更容易阅读。
178	<code>static int w25q32_spi_hw_init(void)</code>	声明函数 w25q32_spi_hw_init，告诉编译器后面会有这个接口。
179	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
180	<code>rt_err_t ret;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
181	<code>struct rt_spi_configuration cfg;</code>	定义结构体类型，把一组相关数据字段放在一起。
182		空行，用来分隔逻辑块，让代码更容易阅读。
183	<code>ret = rt_hw_spi_device_attach("spi1", "w25q32", GPIOA, GPIO_PIN_15);</code>	把 SPI 总线和片选 GPIO 组合成一个 RT-Thread SPI 设备。
184	<code>if (ret != RT_EOK)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
185	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。

行	源码	说明
186	<code>rt_kprintf("attach w25q32 failed: %d\n", ret);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
187	<code>return ret;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
188	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
189		空行，用来分隔逻辑块，让代码更容易阅读。
190	<code>w25q32_spi_dev = (struct rt_spi_device *)rt_device_find("w25q32");</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
191	<code>if (w25q32_spi_dev == RT_NULL)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
192	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
193	<code>rt_kprintf("find w25q32 failed\n");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
194	<code>return -RT_ERROR;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
195	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
196		空行，用来分隔逻辑块，让代码更容易阅读。
197	<code>cfg.data_width = 8;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
198	<code>cfg.mode = RT_SPI_MASTER RT_SPI_MODE_0 RT_SPI_MSB;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
199	<code>cfg.max_hz = 20000000;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
200		空行，用来分隔逻辑块，让代码更容易阅读。
201	<code>ret = rt_spi_configure(w25q32_spi_dev, &cfg);</code>	配置 SPI 数据宽度、模式和速度。
202	<code>if (ret != RT_EOK)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
203	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
204	<code>rt_kprintf("config w25q32 spi failed: %d\n", ret);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
205	<code>return ret;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
206	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
207		空行，用来分隔逻辑块，让代码更容易阅读。
208	<code>#ifdef RT_USING_SFUD</code>	条件编译开始：只有配置宏满足时，下面这段代码才会参与编译。
209	<code>if (rt_sfud_flash_probe("flash0", "w25q32") == RT_NULL)</code>	探测 SPI Flash 并注册为 SFUD flash 设备。
210	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
211	<code>rt_kprintf("sfud probe w25q32 failed\n");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
212	<code>return -RT_ERROR;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
213	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。

行	源码	说明
214		空行，用来分隔逻辑块，让代码更容易阅读。
215	<code>rt_kprintf("sfud flash0 init ok\n");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
216		空行，用来分隔逻辑块，让代码更容易阅读。
217	<code>#ifdef RT_USING_MTD_NOR</code>	条件编译开始：只有配置宏满足时，下面这段代码才会参与编译。
218	<code>w25q32_sfud = rt_sfud_flash_find_by_dev_name("flash0");</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
219	<code>if (w25q32_sfud == RT_NULL)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
220	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
221	<code>rt_kprintf("find sfud flash0 failed\n");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
222	<code>return -RT_ERROR;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
223	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
224		空行，用来分隔逻辑块，让代码更容易阅读。
225	<code>w25q32_mtd.block_size = w25q32_sfud->chip.erase_gran;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
226	<code>w25q32_mtd.block_start = 0;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
227	<code>w25q32_mtd.block_end = w25q32_sfud->chip.capacity / w25q32_mtd.block_size;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
228	<code>w25q32_mtd.ops = &w25q32_mtd_ops;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
229		空行，用来分隔逻辑块，让代码更容易阅读。
230	<code>ret = rt_mtd_nor_register_device("w25q32_mtd", &w25q32_mtd);</code>	把 NOR Flash 包装成 MTD 设备。
231	<code>if (ret != RT_EOK)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
232	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
233	<code>rt_kprintf("register w25q32_mtd failed: %d\n", ret);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
234	<code>return ret;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
235	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
236		空行，用来分隔逻辑块，让代码更容易阅读。
237	<code>rt_kprintf("w25q32_mtd init ok\n");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
238	<code>#endif</code>	条件编译结束。
239	<code>#endif</code>	条件编译结束。
240		空行，用来分隔逻辑块，让代码更容易阅读。
241	<code>rt_kprintf("w25q32 spi init ok\n");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。

行	源码	说明
242	<code>return RT_EOK;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
243	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
244	<code>INIT_DEVICE_EXPORT(w25q32_spi_hw_init);</code>	把初始化函数放入设备初始化阶段自动执行。
245		空行，用来分隔逻辑块，让代码更容易阅读。
246	<code>#if defined(PKG_USING_LITTLEFS) && defined(RT_USING_DFS)</code>	条件编译开始：只有配置宏满足时，下面这段代码才会参与编译。
247	<code>static int w25q32_lfs_mount(void)</code>	声明函数 <code>w25q32_lfs_mount</code> ，告诉编译器后面会有这个接口。
248	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
249	<code>if (dfs_mount("w25q32_mtd", "/", "lfs", 0, RT_NULL) == 0)</code>	把文件系统挂载到指定路径。
250	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
251	<code>rt_kprintf("littlefs mount ok\n");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
252	<code>return RT_EOK;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
253	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
254		空行，用来分隔逻辑块，让代码更容易阅读。
255	<code>rt_kprintf("littlefs mount failed, try mkfs\n");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
256		空行，用来分隔逻辑块，让代码更容易阅读。
257	<code>if (dfs_mkfs("lfs", "w25q32_mtd") != 0)</code>	格式化文件系统。
258	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
259	<code>rt_kprintf("littlefs mkfs failed\n");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
260	<code>return -RT_ERROR;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
261	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
262		空行，用来分隔逻辑块，让代码更容易阅读。
263	<code>if (dfs_mount("w25q32_mtd", "/", "lfs", 0, RT_NULL) != 0)</code>	把文件系统挂载到指定路径。
264	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
265	<code>rt_kprintf("littlefs mount after mkfs failed\n");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
266	<code>return -RT_ERROR;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
267	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
268		空行，用来分隔逻辑块，让代码更容易阅读。
269	<code>rt_kprintf("littlefs mkfs and mount ok\n");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
270	<code>return RT_EOK;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。

行	源码	说明
271	}	代码块结束，离开 当前结构 的内部逻辑。
272	INIT_APP_EXPORT(w25q32_lfs_mount);	把初始化函数放入应用初始化阶段自动执行。
273	#endif	条件编译结束。
274		空行，用来分隔逻辑块，让代码更容易阅读。
275	#if defined(RT_USING_SFUD) && defined(FINSH_USING_MSH)	条件编译开始：只有配置宏满足时，下面这段代码才会参与编译。
276	static void w25q32_test(void)	声明函数 w25q32_test，告诉编译器后面会有这个接口。
277	{	代码块开始，进入 当前结构 的内部逻辑。
278	sfud_flash_t flash;	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
279	rt_uint32_t test_addr;	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
280	rt_uint32_t i;	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
281	static rt_uint8_t write_buf[256];	定义当前文件私有的静态对象或函数，限制作用域，降低模块耦合。
282	static rt_uint8_t read_buf[256];	定义当前文件私有的静态对象或函数，限制作用域，降低模块耦合。
283		空行，用来分隔逻辑块，让代码更容易阅读。
284	flash = rt_sfud_flash_find_by_dev_name("flash0");	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
285	if (flash == RT_NULL)	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
286	{	代码块开始，进入 当前结构 的内部逻辑。
287	rt_kprintf("flash0 not found\n");	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
288	return;	从当前函数返回结果；错误码或状态值会交给调用者判断。
289	}	代码块结束，离开 当前结构 的内部逻辑。
290		空行，用来分隔逻辑块，让代码更容易阅读。
291	test_addr = flash->chip.capacity - flash->chip.erase_gran;	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
292	rt_kprintf("w25q32 test addr: 0x%08x, size: %d bytes\n", test_addr, sizeof(write_buf));	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
293		空行，用来分隔逻辑块，让代码更容易阅读。
294	for (i = 0; i < sizeof(write_buf); i++)	for 循环，按固定次数或范围重复执行，用于遍历数组、列表或填充缓冲区。
295	{	代码块开始，进入 当前结构 的内部逻辑。
296	write_buf[i] = (rt_uint8_t)(i ^ 0x5a);	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
297	read_buf[i] = 0;	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。

行	源码	说明
298	}	代码块结束，离开 当前结构 的内部逻辑。
299		空行，用来分隔逻辑块，让代码更容易阅读。
300	if (sfud_erase(flash, test_addr, flash->chip.erase_gran) != SFUD_SUCCESS)	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
301	{	代码块开始，进入 当前结构 的内部逻辑。
302	rt_kprintf("w25q32 erase failed\n");	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
303	return;	从当前函数返回结果；错误码或状态值会交给调用者判断。
304	}	代码块结束，离开 当前结构 的内部逻辑。
305		空行，用来分隔逻辑块，让代码更容易阅读。
306	if (sfud_write(flash, test_addr, sizeof(write_buf), write_buf) != SFUD_SUCCESS)	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
307	{	代码块开始，进入 当前结构 的内部逻辑。
308	rt_kprintf("w25q32 write failed\n");	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
309	return;	从当前函数返回结果；错误码或状态值会交给调用者判断。
310	}	代码块结束，离开 当前结构 的内部逻辑。
311		空行，用来分隔逻辑块，让代码更容易阅读。
312	if (sfud_read(flash, test_addr, sizeof(read_buf), read_buf) != SFUD_SUCCESS)	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
313	{	代码块开始，进入 当前结构 的内部逻辑。
314	rt_kprintf("w25q32 read failed\n");	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
315	return;	从当前函数返回结果；错误码或状态值会交给调用者判断。
316	}	代码块结束，离开 当前结构 的内部逻辑。
317		空行，用来分隔逻辑块，让代码更容易阅读。
318	if (rt_memcmp(write_buf, read_buf, sizeof(write_buf)) != 0)	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
319	{	代码块开始，进入 当前结构 的内部逻辑。
320	rt_kprintf("w25q32 verify failed\n");	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
321	return;	从当前函数返回结果；错误码或状态值会交给调用者判断。
322	}	代码块结束，离开 当前结构 的内部逻辑。
323		空行，用来分隔逻辑块，让代码更容易阅读。
324	rt_kprintf("w25q32 erase/write/read test ok\n");	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
325	}	代码块结束，离开 当前结构 的内部逻辑。
326	MSH_CMD_EXPORT(w25q32_test, erase/write/read last sector of W25Q32);	把 C 函数导出为 RT-Thread MSH 命令。

行	源码	说明
327	<code>#endif</code>	条件编译结束。
328		空行，用来分隔逻辑块，让代码更容易阅读。
329	<code>#ifdef AT_USING_CLIENT</code>	条件编译开始：只有配置宏满足时，下面这段代码才会参与编译。
330	<code>#include <at.h></code>	引入头文件 <at.h>，让当前文件可以使用其中声明的类型、宏和函数。
331		空行，用来分隔逻辑块，让代码更容易阅读。
332	<code>static int esp8266_at_client_init(void)</code>	声明函数 esp8266_at_client_init，告诉编译器后面会有这个接口。
333	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
334	<code> return at_client_init("uart2", 512);</code>	声明函数 at_client_init，告诉编译器后面会有这个接口。
335	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
336	<code>INIT_APP_EXPORT(esp8266_at_client_init);</code>	初始化 AT 客户端，绑定到串口。
337	<code>#endif</code>	条件编译结束。
338		空行，用来分隔逻辑块，让代码更容易阅读。
339		空行，用来分隔逻辑块，让代码更容易阅读。
340		空行，用来分隔逻辑块，让代码更容易阅读。
341		空行，用来分隔逻辑块，让代码更容易阅读。

Lcd.h

LCD 驱动对外接口头文件，定义屏幕尺寸、控制脚和像素写入接口。

本文件共 52 行。下面的表格左侧是原始源码，中间是行号，右侧是逐行说明。资源数组文件的数组数据较长，说明会重点解释其作为图片/字体资源的角色。

行	源码	说明
1	<code>#ifndef __LCD_H</code>	条件编译开始：只有配置宏满足时，下面这段代码才会参与编译。
2	<code>#define __LCD_H</code>	宏定义，用名字替代固定值，方便统一修改和表达含义。
3		空行，用来分隔逻辑块，让代码更容易阅读。
4	<code>#ifdef __cplusplus</code>	条件编译开始：只有配置宏满足时，下面这段代码才会参与编译。
5	<code>extern "C" {</code>	声明外部符号：它的定义在别的文件里，当前文件只是使用它。
6	<code>#endif</code>	条件编译结束。
7		空行，用来分隔逻辑块，让代码更容易阅读。
8	<code>#include <board.h></code>	引入头文件 <board.h>，让当前文件可以使用其中声明的类型、宏和函数。
9		空行，用来分隔逻辑块，让代码更容易阅读。
10	<code>/*</code>	块注释开始/内容，解释接下来代码为什么这样写。
11	<code> * ILI9341 resolution used by this project.</code>	块注释内容，属于说明文字，不会被编译进程序逻辑。
12	<code> * Current orientation is portrait: 240 x 320.</code>	块注释内容，属于说明文字，不会被编译进程序逻辑。
13	<code>*/</code>	块注释结束。
14	<code>#define LCD_WIDTH 240U</code>	宏定义，用名字替代固定值，方便统一修改和表达含义。
15	<code>#define LCD_HEIGHT 320U</code>	宏定义，用名字替代固定值，方便统一修改和表达含义。
16		空行，用来分隔逻辑块，让代码更容易阅读。
17	<code>/*</code>	块注释开始/内容，解释接下来代码为什么这样写。
18	<code> * LCD control pins.</code>	块注释内容，属于说明文字，不会被编译进程序逻辑。
19	<code> * PB12 -> LCD_CS</code>	块注释内容，属于说明文字，不会被编译进程序逻辑。
20	<code> * PC5 -> LCD_DC</code>	块注释内容，属于说明文字，不会被编译进程序逻辑。
21	<code> * PB1 -> LCD_BLK</code>	块注释内容，属于说明文字，不会被编译进程序逻辑。
22	<code>*/</code>	块注释结束。

行	源码	说明
23	<code>#define LCD_CS_Pin GPIO_PIN_12</code>	宏定义，用名字替代固定值，方便统一修改和表达含义。
24	<code>#define LCD_CS_GPIO_Port GPIOB</code>	宏定义，用名字替代固定值，方便统一修改和表达含义。
25	<code>#define LCD_DC_Pin GPIO_PIN_5</code>	宏定义，用名字替代固定值，方便统一修改和表达含义。
26	<code>#define LCD_DC_GPIO_Port GPIOC</code>	宏定义，用名字替代固定值，方便统一修改和表达含义。
27	<code>#define LCD_BLK_Pin GPIO_PIN_1</code>	宏定义，用名字替代固定值，方便统一修改和表达含义。
28	<code>#define LCD_BLK_GPIO_Port GPIOB</code>	宏定义，用名字替代固定值，方便统一修改和表达含义。
29		空行，用来分隔逻辑块，让代码更容易阅读。
30	<code>void LCD_Init(void);</code>	声明函数 LCD_Init，告诉编译器后面会有这个接口。
31	<code>void LCD_Set_Window(uint16_t x1, uint16_t y1, uint16_t x2, uint16_t y2);</code>	声明函数 LCD_Set_Window，告诉编译器后面会有这个接口。
32		空行，用来分隔逻辑块，让代码更容易阅读。
33	<code>void LCD_WriteCommand(uint8_t cmd);</code>	声明函数 LCD_WriteCommand，告诉编译器后面会有这个接口。
34	<code>void LCD_WriteData8(uint8_t data);</code>	声明函数 LCD_WriteData8，告诉编译器后面会有这个接口。
35	<code>void LCD_WriteData16(uint16_t data);</code>	声明函数 LCD_WriteData16，告诉编译器后面会有这个接口。
36		空行，用来分隔逻辑块，让代码更容易阅读。
37	<code>/*</code>	块注释开始/内容，解释接下来代码为什么这样写。
38	<code> * Blocking pixel write interface.</code>	块注释内容，属于说明文字，不会被编译进程序逻辑。
39	<code> * Typical LVGL flush order:</code>	块注释内容，属于说明文字，不会被编译进程序逻辑。
40	<code> * 1. LCD_Set_Window(...)</code>	块注释内容，属于说明文字，不会被编译进程序逻辑。
41	<code> * 2. LCD_WritePixels(...)</code>	块注释内容，属于说明文字，不会被编译进程序逻辑。
42	<code> * 3. lv_disp_flush_ready(...)</code>	块注释内容，属于说明文字，不会被编译进程序逻辑。
43	<code>*/</code>	块注释结束。
44	<code>HAL_StatusTypeDef LCD_WritePixels(const uint8_t *data, uint32_t size);</code>	声明函数 LCD_WritePixels，告诉编译器后面会有这个接口。
45		空行，用来分隔逻辑块，让代码更容易阅读。
46	<code>void LCD_Fill_Color(uint16_t color);</code>	声明函数 LCD_Fill_Color，告诉编译器后面会有这个接口。
47		空行，用来分隔逻辑块，让代码更容易阅读。

行	源码	说明
48	<code>#ifndef __cplusplus</code>	条件编译开始：只有配置宏满足时，下面这段代码才会参与编译。
49	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
50	<code>#endif</code>	条件编译结束。
51		空行，用来分隔逻辑块，让代码更容易阅读。
52	<code>#endif /* __LCD_H */</code>	条件编译结束。

Lcd_Reg.h

ILI9341 控制器寄存器/命令常量，避免在驱动里直接散落魔法数字。

本文件共 68 行。下面的表格左侧是原始源码，中间是行号，右侧是逐行说明。资源数组文件的数组数据较长，说明会重点解释其作为图片/字体资源的角色。

行	源码	说明
1	<code>#ifndef __LCD_REG_H</code>	条件编译开始：只有配置宏满足时，下面这段代码才会参与编译。
2	<code>#define __LCD_REG_H</code>	宏定义，用名字替代固定值，方便统一修改和表达含义。
3		空行，用来分隔逻辑块，让代码更容易阅读。
4	<code>/*</code>	块注释开始/内容，解释接下来代码为什么这样写。
5	<code> * ILI9341 常用命令寄存器定义。</code>	块注释内容，属于说明文字，不会被编译进程序逻辑。
6	<code> * 这里只保留了当前这版最小驱动真正会用到的命令，</code>	块注释内容，属于说明文字，不会被编译进程序逻辑。
7	<code> * 目的是先把底层点屏和后续 LVGL 刷屏链路跑通。</code>	块注释内容，属于说明文字，不会被编译进程序逻辑。
8	<code>*/</code>	块注释结束。
9	<code>#define ILI9341_NOP 0x00</code>	宏定义，用名字替代固定值，方便统一修改和表达含义。
10	<code>#define ILI9341_SWRESET 0x01</code>	宏定义，用名字替代固定值，方便统一修改和表达含义。
11	<code>#define ILI9341_RDDID 0x04</code>	宏定义，用名字替代固定值，方便统一修改和表达含义。
12	<code>#define ILI9341_RDDST 0x09</code>	宏定义，用名字替代固定值，方便统一修改和表达含义。
13		空行，用来分隔逻辑块，让代码更容易阅读。
14	<code>#define ILI9341_SLPIN 0x10</code>	宏定义，用名字替代固定值，方便统一修改和表达含义。
15	<code>#define ILI9341_SLPOUT 0x11</code>	宏定义，用名字替代固定值，方便统一修改和表达含义。
16	<code>#define ILI9341_PTLON 0x12</code>	宏定义，用名字替代固定值，方便统一修改和表达含义。
17	<code>#define ILI9341_NORON 0x13</code>	宏定义，用名字替代固定值，方便统一修改和表达含义。
18		空行，用来分隔逻辑块，让代码更容易阅读。
19	<code>#define ILI9341_INVOFF 0x20</code>	宏定义，用名字替代固定值，方便统一修改和表达含义。
20	<code>#define ILI9341_INVON 0x21</code>	宏定义，用名字替代固定值，方便统一修改和表达含义。
21	<code>#define ILI9341_GAMSET 0x26</code>	宏定义，用名字替代固定值，方便统一修改和表达含义。

行	源码	说明
22	#define ILI9341_DISPOFF 0x28	宏定义，用名字替代固定值，方便统一修改和表达含义。
23	#define ILI9341_DISPON 0x29	宏定义，用名字替代固定值，方便统一修改和表达含义。
24	#define ILI9341_CASET 0x2A	宏定义，用名字替代固定值，方便统一修改和表达含义。
25	#define ILI9341_PASET 0x2B	宏定义，用名字替代固定值，方便统一修改和表达含义。
26	#define ILI9341_RAMWR 0x2C	宏定义，用名字替代固定值，方便统一修改和表达含义。
27	#define ILI9341_RAMRD 0x2E	宏定义，用名字替代固定值，方便统一修改和表达含义。
28		空行，用来分隔逻辑块，让代码更容易阅读。
29	#define ILI9341_MADCTL 0x36	宏定义，用名字替代固定值，方便统一修改和表达含义。
30	#define ILI9341_PIXFMT 0x3A	宏定义，用名字替代固定值，方便统一修改和表达含义。
31	#define ILI9341_POWERA 0xCB	宏定义，用名字替代固定值，方便统一修改和表达含义。
32	#define ILI9341_POWERB 0xCF	宏定义，用名字替代固定值，方便统一修改和表达含义。
33	#define ILI9341_DTCA 0xE8	宏定义，用名字替代固定值，方便统一修改和表达含义。
34	#define ILI9341_DTCB 0xEA	宏定义，用名字替代固定值，方便统一修改和表达含义。
35	#define ILI9341_POWER_SEQ 0xED	宏定义，用名字替代固定值，方便统一修改和表达含义。
36	#define ILI9341_FRMCTR1 0xB1	宏定义，用名字替代固定值，方便统一修改和表达含义。
37	#define ILI9341_DFUNCTR 0xB6	宏定义，用名字替代固定值，方便统一修改和表达含义。
38	#define ILI9341_ENABLE3G 0xF2	宏定义，用名字替代固定值，方便统一修改和表达含义。
39	#define ILI9341_PRC 0xF7	宏定义，用名字替代固定值，方便统一修改和表达含义。
40	#define ILI9341_PWCTR1 0xC0	宏定义，用名字替代固定值，方便统一修改和表达含义。
41	#define ILI9341_PWCTR2 0xC1	宏定义，用名字替代固定值，方便统一修改和表达含义。
42	#define ILI9341_VMCTR1 0xC5	宏定义，用名字替代固定值，方便统一修改和表达含义。
43	#define ILI9341_VMCTR2 0xC7	宏定义，用名字替代固定值，方便统一修改和表达含义。
44	#define ILI9341_GMCTRP1 0xE0	宏定义，用名字替代固定值，方便统一修改和表达含义。

行	源码	说明
45	<code>#define ILI9341_GMCTRN1 0xE1</code>	宏定义，用名字替代固定值，方便统一修改和表达含义。
46		空行，用来分隔逻辑块，让代码更容易阅读。
47	<code>/*</code>	块注释开始/内容，解释接下来代码为什么这样写。
48	<code>* MADCTL(0x36) 控制位定义。</code>	块注释内容，属于说明文字，不会被编译进程序逻辑。
49	<code>*</code>	块注释内容，属于说明文字，不会被编译进程序逻辑。
50	<code>* 这个寄存器主要影响：</code>	块注释内容，属于说明文字，不会被编译进程序逻辑。
51	<code>* 1. 显示方向</code>	块注释内容，属于说明文字，不会被编译进程序逻辑。
52	<code>* 2. X/Y 镜像</code>	块注释内容，属于说明文字，不会被编译进程序逻辑。
53	<code>* 3. 横竖屏切换</code>	块注释内容，属于说明文字，不会被编译进程序逻辑。
54	<code>* 4. RGB/BGR 颜色顺序</code>	块注释内容，属于说明文字，不会被编译进程序逻辑。
55	<code>*</code>	块注释内容，属于说明文字，不会被编译进程序逻辑。
56	<code>* 常见问题和它的关系：</code>	块注释内容，属于说明文字，不会被编译进程序逻辑。
57	<code>* - 画面翻转：通常与 MX/MY 有关</code>	块注释内容，属于说明文字，不会被编译进程序逻辑。
58	<code>* - 横竖屏不对：通常与 MV 有关</code>	块注释内容，属于说明文字，不会被编译进程序逻辑。
59	<code>* - 红蓝反了：通常与 BGR 位有关</code>	块注释内容，属于说明文字，不会被编译进程序逻辑。
60	<code>*/</code>	块注释结束。
61	<code>#define ILI9341_MADCTL_MY 0x80</code>	宏定义，用名字替代固定值，方便统一修改和表达含义。
62	<code>#define ILI9341_MADCTL_MX 0x40</code>	宏定义，用名字替代固定值，方便统一修改和表达含义。
63	<code>#define ILI9341_MADCTL_MV 0x20</code>	宏定义，用名字替代固定值，方便统一修改和表达含义。
64	<code>#define ILI9341_MADCTL_ML 0x10</code>	宏定义，用名字替代固定值，方便统一修改和表达含义。
65	<code>#define ILI9341_MADCTL_BGR 0x08</code>	宏定义，用名字替代固定值，方便统一修改和表达含义。
66	<code>#define ILI9341_MADCTL_MH 0x04</code>	宏定义，用名字替代固定值，方便统一修改和表达含义。
67		空行，用来分隔逻辑块，让代码更容易阅读。
68	<code>#endif /* __LCD_REG_H */</code>	条件编译结束。

Lcd.c

ILI9341 SPI LCD 底层驱动。负责片选、命令/数据切换、设置绘图窗口、写像素和初始化寄存器。

本文件共 342 行。下面的表格左侧是原始源码，中间是行号，右侧是逐行说明。资源数组文件的数组数据较长，说明会重点解释其作为图片/字体资源的角色。

行	源码	说明
1	#include "Lcd.h"	引入头文件 "Lcd.h"，让当前文件可以使用其中声明的类型、宏和函数。
2	#include "Lcd_Reg.h"	引入头文件 "Lcd_Reg.h"，让当前文件可以使用其中声明的类型、宏和函数。
3	#include <rtthread.h>	引入头文件 <rtthread.h>，让当前文件可以使用其中声明的类型、宏和函数。
4	#include <rtdevice.h>	引入头文件 <rtdevice.h>，让当前文件可以使用其中声明的类型、宏和函数。
5		空行，用来分隔逻辑块，让代码更容易阅读。
6	/*	块注释开始/内容，解释接下来代码为什么这样写。
7	* 保存“当前窗口”内总共有多少个像素点。	块注释内容，属于说明文字，不会被编译进程序逻辑。
8	* LCD_Set_Window() 会更新这个值，	块注释内容，属于说明文字，不会被编译进程序逻辑。
9	* LCD_Fill_Color() 则会根据这个值连续写入颜色数据。	块注释内容，属于说明文字，不会被编译进程序逻辑。
10	*/	块注释结束。
11	static uint32_t lcd_window_pixels = (uint32_t)LCD_WIDTH * (uint32_t)LCD_HEIGHT;	定义当前文件私有的静态变量或函数，并给出初始值；外部文件不能直接访问。
12	static struct rt_spi_device *lcd_spi_dev = RT_NULL;	定义当前文件私有的静态变量或函数，并给出初始值；外部文件不能直接访问。
13	static uint8_t lcd_tx_burst[1024];	定义当前文件私有的静态对象或函数，限制作用域，降低模块耦合。
14		空行，用来分隔逻辑块，让代码更容易阅读。
15	static struct rt_spi_device *LCD_GetSpiDevice(void);	定义当前文件私有的静态对象或函数，限制作用域，降低模块耦合。
16	static HAL_StatusTypeDef LCD_WriteBytesChecked(const uint8_t *data, uint32_t size);	声明函数 LCD_WriteBytesChecked，告诉编译器后面会有这个接口。
17		空行，用来分隔逻辑块，让代码更容易阅读。
18	/* 拉低 CS，表示选中 LCD 设备 */	块注释开始/内容，解释接下来代码为什么这样写。
19	static void LCD_Select(void)	声明函数 LCD_Select，告诉编译器后面会有这个接口。
20	{	代码块开始，进入 当前结构 的内部逻辑。
21	HAL_GPIO_WritePin(LCD_CS_GPIO_Port, LCD_CS_Pin, GPIO_PIN_RESET);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。

行	源码	说明
22	}	代码块结束，离开 当前结构 的内部逻辑。
23		空行，用来分隔逻辑块，让代码更容易阅读。
24	/* 拉高 CS，表示结束本次 LCD 访问 */	块注释开始/内容，解释接下来代码为什么这样写。
25	static void LCD_Unselect(void)	声明函数 LCD_Unselect，告诉编译器后面会有这个接口。
26	{	代码块开始，进入 当前结构 的内部逻辑。
27	HAL_GPIO_WritePin(LCD_CS_GPIO_Port, LCD_CS_Pin, GPIO_PIN_SET);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
28	}	代码块结束，离开 当前结构 的内部逻辑。
29		空行，用来分隔逻辑块，让代码更容易阅读。
30	/* DC=0，表示接下来发送的是“命令” */	块注释开始/内容，解释接下来代码为什么这样写。
31	static void LCD_DC_CommandMode(void)	声明函数 LCD_DC_CommandMode，告诉编译器后面会有这个接口。
32	{	代码块开始，进入 当前结构 的内部逻辑。
33	HAL_GPIO_WritePin(LCD_DC_GPIO_Port, LCD_DC_Pin, GPIO_PIN_RESET);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
34	}	代码块结束，离开 当前结构 的内部逻辑。
35		空行，用来分隔逻辑块，让代码更容易阅读。
36	/* DC=1，表示接下来发送的是“数据” */	块注释开始/内容，解释接下来代码为什么这样写。
37	static void LCD_DC_DataMode(void)	声明函数 LCD_DC_DataMode，告诉编译器后面会有这个接口。
38	{	代码块开始，进入 当前结构 的内部逻辑。
39	HAL_GPIO_WritePin(LCD_DC_GPIO_Port, LCD_DC_Pin, GPIO_PIN_SET);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
40	}	代码块结束，离开 当前结构 的内部逻辑。
41		空行，用来分隔逻辑块，让代码更容易阅读。
42	/*	块注释开始/内容，解释接下来代码为什么这样写。
43	* 通过 HAL SPI 向 LCD 连续发送一段字节流。	块注释内容，属于说明文字，不会被编译进程序逻辑。
44	* 这里底层固定使用 hspi2，	块注释内容，属于说明文字，不会被编译进程序逻辑。
45	* 优先保证流程正确，超时先使用 HAL_MAX_DELAY。	块注释内容，属于说明文字，不会被编译进程序逻辑。
46	*/	块注释结束。
47	static struct rt_spi_device *LCD_GetSpiDevice(void)	定义当前文件私有的静态对象或函数，限制作用域，降低模块耦合。
48	{	代码块开始，进入 当前结构 的内部逻辑。
49	if (lcd_spi_dev == RT_NULL) {	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。

行	源码	说明
50	<code>lcd_spi_dev = (struct rt_spi_device *)rt_device_find("lcd");</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
51	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
52		空行，用来分隔逻辑块，让代码更容易阅读。
53	<code>return lcd_spi_dev;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
54	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
55		空行，用来分隔逻辑块，让代码更容易阅读。
56	<code>static void LCD_WriteBytes(const uint8_t *data, uint16_t size)</code>	声明函数 LCD_WriteBytes，告诉编译器后面会有这个接口。
57	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
58	<code>(void)LCD_WriteBytesChecked(data, size);</code>	显式标记参数未使用，避免编译器产生未使用参数警告。
59	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
60		空行，用来分隔逻辑块，让代码更容易阅读。
61	<code>static HAL_StatusTypeDef LCD_WriteBytesChecked(const uint8_t *data, uint32_t size)</code>	声明函数 LCD_WriteBytesChecked，告诉编译器后面会有这个接口。
62	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
63	<code>struct rt_spi_device *spi_dev;</code>	定义结构体类型，把一组相关数据字段放在一起。
64	<code>uint32_t offset = 0U;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
65		空行，用来分隔逻辑块，让代码更容易阅读。
66	<code>if ((data == NULL) (size == 0U)) {</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
67	<code>return HAL_OK;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
68	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
69		空行，用来分隔逻辑块，让代码更容易阅读。
70	<code>spi_dev = LCD_GetSpiDevice();</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
71	<code>if (spi_dev == RT_NULL) {</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
72	<code>rt_kprintf("lcd spi device not found\n");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
73	<code>return HAL_ERROR;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
74	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
75		空行，用来分隔逻辑块，让代码更容易阅读。
76	<code>while (offset < size) {</code>	while 循环，只要条件成立就持续执行；线程主循环通常写成 while (1)。
77	<code>rt_size_t chunk = (size - offset > 4096U) ? 4096U : (rt_size_t)(size - offset);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。

行	源码	说明
78		空行，用来分隔逻辑块，让代码更容易阅读。
79	if (rt_spi_send(spi_dev, data + offset, chunk) != chunk) {	通过 RT-Thread SPI 设备发送字节流。
80	return HAL_ERROR;	从当前函数返回结果；错误码或状态值会交给调用者判断。
81	}	代码块结束，离开 当前结构 的内部逻辑。
82		空行，用来分隔逻辑块，让代码更容易阅读。
83	offset += chunk;	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
84	}	代码块结束，离开 当前结构 的内部逻辑。
85		空行，用来分隔逻辑块，让代码更容易阅读。
86	return HAL_OK;	从当前函数返回结果；错误码或状态值会交给调用者判断。
87	}	代码块结束，离开 当前结构 的内部逻辑。
88		空行，用来分隔逻辑块，让代码更容易阅读。
89	/*	块注释开始/内容，解释接下来代码为什么这样写。
90	* 写“1 条命令 + 1 字节参数”的辅助函数。	块注释内容，属于说明文字，不会被编译进程序逻辑。
91	* 适合像像素格式、Gamma 选择这类简单寄存器写入。	块注释内容，属于说明文字，不会被编译进程序逻辑。
92	*/	块注释结束。
93	static void LCD_WriteRegister8(uint8_t reg, uint8_t value)	声明函数 LCD_WriteRegister8，告诉编译器后面会有这个接口。
94	{	代码块开始，进入 当前结构 的内部逻辑。
95	LCD_WriteCommand(reg);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
96	LCD_WriteData8(value);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
97	}	代码块结束，离开 当前结构 的内部逻辑。
98		空行，用来分隔逻辑块，让代码更容易阅读。
99	/*	块注释开始/内容，解释接下来代码为什么这样写。
100	* 在一次片选周期内，先发送命令，再发送对应的数据区。	块注释内容，属于说明文字，不会被编译进程序逻辑。
101	* 这种写法更贴近 LCD 初始化表，也更紧凑。	块注释内容，属于说明文字，不会被编译进程序逻辑。
102	*/	块注释结束。
103	static void LCD_WriteCommandBuffer(uint8_t cmd, const uint8_t *data, uint16_t size)	声明函数 LCD_WriteCommandBuffer，告诉编译器后面会有这个接口。
104	{	代码块开始，进入 当前结构 的内部逻辑。
105	LCD_Select();	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。

行	源码	说明
106	LCD_DC_CommandMode();	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
107	LCD_WriteBytes(&cmd, 1U);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
108	if ((data != NULL) && (size > 0U)) {	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
109	LCD_DC_DataMode();	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
110	LCD_WriteBytes(data, size);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
111	}	代码块结束，离开 当前结构 的内部逻辑。
112	LCD_Unselect();	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
113	}	代码块结束，离开 当前结构 的内部逻辑。
114		空行，用来分隔逻辑块，让代码更容易阅读。
115	void LCD_WriteCommand(uint8_t cmd)	声明函数 LCD_WriteCommand，告诉编译器后面会有这个接口。
116	{	代码块开始，进入 当前结构 的内部逻辑。
117	/* 单独发送 1 个命令字节 */	块注释开始/内容，解释接下来代码为什么这样写。
118	LCD_Select();	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
119	LCD_DC_CommandMode();	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
120	LCD_WriteBytes(&cmd, 1U);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
121	LCD_Unselect();	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
122	}	代码块结束，离开 当前结构 的内部逻辑。
123		空行，用来分隔逻辑块，让代码更容易阅读。
124	void LCD_WriteData8(uint8_t data)	声明函数 LCD_WriteData8，告诉编译器后面会有这个接口。
125	{	代码块开始，进入 当前结构 的内部逻辑。
126	/* 单独发送 1 个 8 位数据 */	块注释开始/内容，解释接下来代码为什么这样写。
127	LCD_Select();	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
128	LCD_DC_DataMode();	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
129	LCD_WriteBytes(&data, 1U);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
130	LCD_Unselect();	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
131	}	代码块结束，离开 当前结构 的内部逻辑。

行	源码	说明
132		空行，用来分隔逻辑块，让代码更容易阅读。
133	<code>void LCD_WriteData16(uint16_t data)</code>	声明函数 LCD_WriteData16，告诉编译器后面会有这个接口。
134	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
135	<code>uint8_t buf[2];</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
136		空行，用来分隔逻辑块，让代码更容易阅读。
137	<code>/*</code>	块注释开始/内容，解释接下来代码为什么这样写。
138	<code>* ILI9341 写 16 位数据时通常按高字节在前的顺序发送。</code>	块注释内容，属于说明文字，不会被编译进程序逻辑。
139	<code>* 例如 RGB565 红色 0xF800，会先发 0xF8，再发 0x00。</code>	块注释内容，属于说明文字，不会被编译进程序逻辑。
140	<code>*/</code>	块注释结束。
141	<code>buf[0] = (uint8_t)(data >> 8);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
142	<code>buf[1] = (uint8_t)(data & 0xFFU);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
143		空行，用来分隔逻辑块，让代码更容易阅读。
144	<code>LCD_Select();</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
145	<code>LCD_DC_DataMode();</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
146	<code>LCD_WriteBytes(buf, 2U);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
147	<code>LCD_Unselect();</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
148	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
149		空行，用来分隔逻辑块，让代码更容易阅读。
150	<code>void LCD_Set_Window(uint16_t x1, uint16_t y1, uint16_t x2, uint16_t y2)</code>	声明函数 LCD_Set_Window，告诉编译器后面会有这个接口。
151	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
152	<code>uint8_t data[4];</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
153		空行，用来分隔逻辑块，让代码更容易阅读。
154	<code>/*</code>	块注释开始/内容，解释接下来代码为什么这样写。
155	<code>* 坐标合法性检查：</code>	块注释内容，属于说明文字，不会被编译进程序逻辑。
156	<code>* 1. 左上角不能跑到右下角后面</code>	块注释内容，属于说明文字，不会被编译进程序逻辑。
157	<code>* 2. 右下角不能超出屏幕边界</code>	块注释内容，属于说明文字，不会被编译进程序逻辑。

行	源码	说明
158	<code>*/</code>	块注释结束。
159	<code>if ((x1 > x2) (y1 > y2) (x2 >= LCD_WIDTH) (y2 >= LCD_HEIGHT)) {</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
160	<code>return;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
161	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
162		空行，用来分隔逻辑块，让代码更容易阅读。
163	<code>/*</code>	块注释开始/内容，解释接下来代码为什么这样写。
164	<code>* 记录当前窗口内的像素总数，</code>	块注释内容，属于说明文字，不会被编译进程序逻辑。
165	<code>* 后续 LCD_Fill_Color() 会按这个数量持续灌颜色。</code>	块注释内容，属于说明文字，不会被编译进程序逻辑。
166	<code>*/</code>	块注释结束。
167	<code>lcd_window_pixels = ((uint32_t)x2 - (uint32_t)x1 + 1U) * ((uint32_t)y2 - (uint32_t)y1 + 1U);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
168		空行，用来分隔逻辑块，让代码更容易阅读。
169	<code>/*</code>	块注释开始/内容，解释接下来代码为什么这样写。
170	<code>* CASET(0x2A)：设置列地址范围，也就是 X 方向窗口。</code>	块注释内容，属于说明文字，不会被编译进程序逻辑。
171	<code>* 数据格式：</code>	块注释内容，属于说明文字，不会被编译进程序逻辑。
172	<code>* x_start_high, x_start_low, x_end_high, x_end_low</code>	块注释内容，属于说明文字，不会被编译进程序逻辑。
173	<code>*/</code>	块注释结束。
174	<code>data[0] = (uint8_t)(x1 >> 8);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
175	<code>data[1] = (uint8_t)(x1 & 0xFFU);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
176	<code>data[2] = (uint8_t)(x2 >> 8);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
177	<code>data[3] = (uint8_t)(x2 & 0xFFU);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
178	<code>LCD_WriteCommand(ILI9341_CASET);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
179	<code>LCD_Select();</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
180	<code>LCD_DC_DataMode();</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
181	<code>LCD_WriteBytes(data, 4U);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
182	<code>LCD_Unselect();</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。

行	源码	说明
183		空行，用来分隔逻辑块，让代码更容易阅读。
184	/*	块注释开始/内容，解释接下来代码为什么这样写。
185	* PASET(0x2B): 设置页地址范围，也就是 Y 方向窗口。	块注释内容，属于说明文字，不会被编译进程序逻辑。
186	* 数据格式:	块注释内容，属于说明文字，不会被编译进程序逻辑。
187	* y_start_high, y_start_low, y_end_high, y_end_low	块注释内容，属于说明文字，不会被编译进程序逻辑。
188	*/	块注释结束。
189	data[0] = (uint8_t)(y1 >> 8);	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
190	data[1] = (uint8_t)(y1 & 0xFFU);	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
191	data[2] = (uint8_t)(y2 >> 8);	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
192	data[3] = (uint8_t)(y2 & 0xFFU);	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
193	LCD_WriteCommand(ILI9341_PASET);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
194	LCD_Select();	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
195	LCD_DC_DataMode();	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
196	LCD_WriteBytes(data, 4U);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
197	LCD_Unselect();	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
198		空行，用来分隔逻辑块，让代码更容易阅读。
199	/*	块注释开始/内容，解释接下来代码为什么这样写。
200	* RAMWR(0x2C): 设置完窗口后，告诉 LCD	块注释内容，属于说明文字，不会被编译进程序逻辑。
201	* “接下来通过 SPI 发来的就是像素数据”。	块注释内容，属于说明文字，不会被编译进程序逻辑。
202	*/	块注释结束。
203	LCD_WriteCommand(ILI9341_RAMWR);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
204	}	代码块结束，离开 当前结构 的内部逻辑。
205		空行，用来分隔逻辑块，让代码更容易阅读。
206	void LCD_Fill_Color(uint16_t color)	声明函数 LCD_Fill_Color，告诉编译器后面会有这个接口。
207	{	代码块开始，进入 当前结构 的内部逻辑。

行	源码	说明
208	<code>uint32_t remaining;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
209	<code>uint32_t i;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
210		空行，用来分隔逻辑块，让代码更容易阅读。
211	<code>/*</code>	块注释开始/内容，解释接下来代码为什么这样写。
212	<code> * 先构造一小段重复颜色缓存。</code>	块注释内容，属于说明文字，不会被编译进程序逻辑。
213	<code> * 每 2 个字节表示 1 个 RGB565 像素，</code>	块注释内容，属于说明文字，不会被编译进程序逻辑。
214	<code> * 这样比每个像素都单独组织数据更高效。</code>	块注释内容，属于说明文字，不会被编译进程序逻辑。
215	<code>*/</code>	块注释结束。
216	<code>for (i = 0; i < (sizeof(lcd_tx_burst) / 2U); ++i) {</code>	for 循环，按固定次数或范围重复执行，用于遍历数组、列表或填充缓冲区。
217	<code> lcd_tx_burst[2U * i] = (uint8_t)(color >> 8);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
218	<code> lcd_tx_burst[2U * i + 1U] = (uint8_t)(color & 0xFFU);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
219	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
220		空行，用来分隔逻辑块，让代码更容易阅读。
221	<code>/*</code>	块注释开始/内容，解释接下来代码为什么这样写。
222	<code> * 向当前窗口持续写入同一种颜色。</code>	块注释内容，属于说明文字，不会被编译进程序逻辑。
223	<code> * 这里不重新设置窗口，默认窗口已经由 LCD_Set_Window() 配好。</code>	块注释内容，属于说明文字，不会被编译进程序逻辑。
224	<code>*/</code>	块注释结束。
225	<code>remaining = lcd_window_pixels;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
226	<code>while (remaining > 0U) {</code>	while 循环，只要条件成立就持续执行；线程主循环通常写成 while (1)。
227	<code> uint16_t chunk_pixels;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
228		空行，用来分隔逻辑块，让代码更容易阅读。
229	<code> chunk_pixels = (remaining > (sizeof(lcd_tx_burst) / 2U)) ? (uint16_t)(sizeof(lcd_tx_burst) / 2U) : (uint16_t)remaining;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
230	<code> if (LCD_WritePixels(lcd_tx_burst, (uint32_t)chunk_pixels * 2U) != HAL_OK) {</code>	把 RGB565 像素字节流通过 SPI 写进当前 LCD 窗口。
231	<code> Error_Handler();</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
232	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。

行	源码	说明
233	<code>remaining -= chunk_pixels;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
234	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
235	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
236		空行，用来分隔逻辑块，让代码更容易阅读。
237	<code>HAL_StatusTypeDef LCD_WritePixels(const uint8_t *data, uint32_t size)</code>	声明函数 <code>LCD_WritePixels</code> ，告诉编译器后面会有这个接口。
238	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
239	<code>HAL_StatusTypeDef status;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
240		空行，用来分隔逻辑块，让代码更容易阅读。
241	<code>if ((data == NULL) (size == 0U)) {</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
242	<code>return HAL_OK;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
243	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
244		空行，用来分隔逻辑块，让代码更容易阅读。
245	<code>LCD_Select();</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
246	<code>LCD_DC_DataMode();</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
247	<code>status = LCD_WriteBytesChecked(data, size);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
248	<code>LCD_Unselect();</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
249		空行，用来分隔逻辑块，让代码更容易阅读。
250	<code>return status;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
251	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
252		空行，用来分隔逻辑块，让代码更容易阅读。
253	<code>void LCD_Init(void)</code>	声明函数 <code>LCD_Init</code> ，告诉编译器后面会有这个接口。
254	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
255	<code>static const uint8_t power_b[] = {0x00U, 0xD9U, 0x30U};</code>	定义当前文件私有的静态变量或函数，并给出初始值；外部文件不能直接访问。
256	<code>static const uint8_t power_seq[] = {0x64U, 0x03U, 0x12U, 0x81U};</code>	定义当前文件私有的静态变量或函数，并给出初始值；外部文件不能直接访问。
257	<code>static const uint8_t driver_timing_a[] = {0x85U, 0x10U, 0x78U};</code>	定义当前文件私有的静态变量或函数，并给出初始值；外部文件不能直接访问。
258	<code>static const uint8_t power_a[] = {0x39U, 0x2CU, 0x00U, 0x34U, 0x02U};</code>	定义当前文件私有的静态变量或函数，并给出初始值；外部文件不能直接访问。
259	<code>static const uint8_t prc[] = {0x20U};</code>	定义当前文件私有的静态变量或函数，并给出初始值；外部文件不能直接访问。

行	源码	说明
260	<code>static const uint8_t dtcb[] = {0x00U, 0x00U};</code>	定义当前文件私有的静态变量或函数，并给出初始值；外部文件不能直接访问。
261	<code>static const uint8_t vmctr1[] = {0x32U, 0x3CU};</code>	定义当前文件私有的静态变量或函数，并给出初始值；外部文件不能直接访问。
262	<code>static const uint8_t frmctr1[] = {0x00U, 0x18U};</code>	定义当前文件私有的静态变量或函数，并给出初始值；外部文件不能直接访问。
263	<code>static const uint8_t dfunctr[] = {0x0AU, 0xA2U};</code>	定义当前文件私有的静态变量或函数，并给出初始值；外部文件不能直接访问。
264	<code>static const uint8_t gamma_pos[] = {</code>	定义当前文件私有的静态变量或函数，并给出初始值；外部文件不能直接访问。
265	<code>0x0FU, 0x20U, 0x1EU, 0x09U, 0x12U, 0x0BU, 0x50U, 0xBAU,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
266	<code>0x44U, 0x09U, 0x14U, 0x05U, 0x23U, 0x21U, 0x00U</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
267	<code>};</code>	结构体、枚举或初始化表结束。
268	<code>static const uint8_t gamma_neg[] = {</code>	定义当前文件私有的静态变量或函数，并给出初始值；外部文件不能直接访问。
269	<code>0x00U, 0x19U, 0x19U, 0x00U, 0x12U, 0x07U, 0x2DU, 0x28U,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
270	<code>0x3FU, 0x02U, 0x0AU, 0x08U, 0x25U, 0x2DU, 0x0FU</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
271	<code>};</code>	结构体、枚举或初始化表结束。
272		空行，用来分隔逻辑块，让代码更容易阅读。
273	<code>/*</code>	块注释开始/内容，解释接下来代码为什么这样写。
274	<code>* 先把控制脚拉到稳定状态：</code>	块注释内容，属于说明文字，不会被编译进程序逻辑。
275	<code>* 1. CS 拉高，避免误选中 LCD</code>	块注释内容，属于说明文字，不会被编译进程序逻辑。
276	<code>* 2. DC 拉高，默认数据态</code>	块注释内容，属于说明文字，不会被编译进程序逻辑。
277	<code>* 3. BLK 拉高，打开背光，便于观察是否点亮</code>	块注释内容，属于说明文字，不会被编译进程序逻辑。
278	<code>*/</code>	块注释结束。
279	<code>HAL_GPIO_WritePin(LCD_CS_GPIO_Port, LCD_CS_Pin, GPIO_PIN_SET);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
280	<code>HAL_GPIO_WritePin(LCD_DC_GPIO_Port, LCD_DC_Pin, GPIO_PIN_SET);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
281	<code>HAL_GPIO_WritePin(LCD_BLK_GPIO_Port, LCD_BLK_Pin, GPIO_PIN_SET);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
282		空行，用来分隔逻辑块，让代码更容易阅读。
283	<code>/* 给 LCD 模组一点上电稳定时间 */</code>	块注释开始/内容，解释接下来代码为什么这样写。
284	<code>HAL_Delay(20U);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。

行	源码	说明
285		空行，用来分隔逻辑块，让代码更容易阅读。
286	/*	块注释开始/内容，解释接下来代码为什么这样写。
287	* 软件复位 LCD 控制器，让内部状态回到已知起点。	块注释内容，属于说明文字，不会被编译进程序逻辑。
288	* 这一步能减少偶发的上电初始化不一致问题。	块注释内容，属于说明文字，不会被编译进程序逻辑。
289	*/	块注释结束。
290	LCD_WriteCommand(ILI9341_SWRESET);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
291	HAL_Delay(500);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
292		空行，用来分隔逻辑块，让代码更容易阅读。
293	/*	块注释开始/内容，解释接下来代码为什么这样写。
294	* 下面这组初始化参数参考了仓库里的大越例程。	块注释内容，属于说明文字，不会被编译进程序逻辑。
295	* 这些寄存器主要用于配置：	块注释内容，属于说明文字，不会被编译进程序逻辑。
296	* 1. 电源时序	块注释内容，属于说明文字，不会被编译进程序逻辑。
297	* 2. 驱动时序	块注释内容，属于说明文字，不会被编译进程序逻辑。
298	* 3. VCOM	块注释内容，属于说明文字，不会被编译进程序逻辑。
299	* 4. 帧率	块注释内容，属于说明文字，不会被编译进程序逻辑。
300	* 5. Gamma 曲线	块注释内容，属于说明文字，不会被编译进程序逻辑。
301	*	块注释内容，属于说明文字，不会被编译进程序逻辑。
302	* 这部分通常和具体屏模适配关系很强，	块注释内容，属于说明文字，不会被编译进程序逻辑。
303	* 所以这里尽量向已知可工作的例程靠拢。	块注释内容，属于说明文字，不会被编译进程序逻辑。
304	*/	块注释结束。
305	LCD_WriteCommandBuffer(ILI9341_POWERB, power_b, sizeof(power_b));	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
306	LCD_WriteCommandBuffer(ILI9341_POWER_SEQ, power_seq, sizeof(power_seq));	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
307	LCD_WriteCommandBuffer(ILI9341_DTCA, driver_timing_a, sizeof(driver_timing_a));	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
308	LCD_WriteCommandBuffer(ILI9341_POWERA, power_a, sizeof(power_a));	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。

行	源码	说明
309	LCD_WriteCommandBuffer(ILI9341_PRC, prc, sizeof(prc));	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
310	LCD_WriteCommandBuffer(ILI9341_DTCB, dtcb, sizeof(dtcb));	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
311		空行，用来分隔逻辑块，让代码更容易阅读。
312	LCD_WriteRegister8(ILI9341_PWCTR1, 0x21U);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
313	LCD_WriteRegister8(ILI9341_PWCTR2, 0x12U);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
314	LCD_WriteCommandBuffer(ILI9341_VMCTR1, vmctr1, sizeof(vmctr1));	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
315	LCD_WriteRegister8(ILI9341_VMCTR2, 0xC1U);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
316	LCD_WriteRegister8(ILI9341_MADCTL, ILI9341_MADCTL_BGR);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
317	LCD_WriteRegister8(ILI9341_PIXFMT, 0x55U);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
318	LCD_WriteCommandBuffer(ILI9341_FRMCTR1, frmctr1, sizeof(frmctr1));	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
319	LCD_WriteCommandBuffer(ILI9341_DFUNCTR, dfunctr, sizeof(dfunctr));	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
320	LCD_WriteRegister8(ILI9341_ENABLE3G, 0x00U);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
321	LCD_WriteRegister8(ILI9341_GAMSET, 0x01U);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
322	LCD_WriteCommandBuffer(ILI9341_GMCTRP1, gamma_pos, sizeof(gamma_pos));	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
323	LCD_WriteCommandBuffer(ILI9341_GMCTRN1, gamma_neg, sizeof(gamma_neg));	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
324		空行，用来分隔逻辑块，让代码更容易阅读。
325	/*	块注释开始/内容，解释接下来代码为什么这样写。
326	* 退出休眠，然后开启正常显示。	块注释内容，属于说明文字，不会被编译进程序逻辑。
327	* Exit Sleep 后必须给足延时，否则有些屏会不稳定。	块注释内容，属于说明文字，不会被编译进程序逻辑。
328	*/	块注释结束。
329	LCD_WriteCommand(ILI9341_SLP0UT);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
330	HAL_Delay(90U);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
331	LCD_WriteCommand(ILI9341_INVOFF);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
332	LCD_WriteCommand(ILI9341_NORON);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。

行	源码	说明
333	HAL_Delay(10U);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
334	LCD_WriteCommand(ILI9341_DISPON);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
335	HAL_Delay(10U);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
336		空行，用来分隔逻辑块，让代码更容易阅读。
337	/*	块注释开始/内容，解释接下来代码为什么这样写。
338	* 默认把绘图窗口设置成整屏，	块注释内容，属于说明文字，不会被编译进程序逻辑。
339	* 这样初始化完成后就可以直接做整屏刷色测试。	块注释内容，属于说明文字，不会被编译进程序逻辑。
340	*/	块注释结束。
341	LCD_Set_Window(0U, 0U, LCD_WIDTH - 1U, LCD_HEIGHT - 1U);	告诉 LCD 接下来要写哪一块矩形区域。
342	}	代码块结束，离开 当前结构 的内部逻辑。

lv_port_disp.h

LVGL 显示端口对外接口，给 main.c 调用显示驱动注册函数。

本文件共 18 行。下面的表格左侧是原始源码，中间是行号，右侧是逐行说明。资源数组文件的数组数据较长，说明会重点解释其作为图片/字体资源的角色。

行	源码	说明
1	<code>#ifndef LV_PORT_DISP_H</code>	条件编译开始：只有配置宏满足时，下面这段代码才会参与编译。
2	<code>#define LV_PORT_DISP_H</code>	宏定义，用名字替代固定值，方便统一修改和表达含义。
3		空行，用来分隔逻辑块，让代码更容易阅读。
4	<code>#ifdef __cplusplus</code>	条件编译开始：只有配置宏满足时，下面这段代码才会参与编译。
5	<code>extern "C" {</code>	声明外部符号：它的定义在别的文件里，当前文件只是使用它。
6	<code>#endif</code>	条件编译结束。
7		空行，用来分隔逻辑块，让代码更容易阅读。
8	<code>#include "lvgl.h"</code>	引入头文件 "lvgl.h"，让当前文件可以使用其中声明的类型、宏和函数。
9		空行，用来分隔逻辑块，让代码更容易阅读。
10	<code>void lv_port_disp_init(void);</code>	声明函数 lv_port_disp_init，告诉编译器后面会有这个接口。
11	<code>lv_disp_t * lv_port_disp_get(void);</code>	声明函数 lv_port_disp_get，告诉编译器后面会有这个接口。
12	<code>void lv_port_disp_handler(void);</code>	声明函数 lv_port_disp_handler，告诉编译器后面会有这个接口。
13		空行，用来分隔逻辑块，让代码更容易阅读。
14	<code>#ifdef __cplusplus</code>	条件编译开始：只有配置宏满足时，下面这段代码才会参与编译。
15	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
16	<code>#endif</code>	条件编译结束。
17		空行，用来分隔逻辑块，让代码更容易阅读。
18	<code>#endif /* LV_PORT_DISP_H */</code>	条件编译结束。

lv_port_disp.c

LVGL 到 LCD 的桥。实现 flush_cb，把 LVGL 算好的像素区域写到 ILI9341。

本文件共 66 行。下面的表格左侧是原始源码，中间是行号，右侧是逐行说明。资源数组文件的数组数据较长，说明会重点解释其作为图片/字体资源的角色。

行	源码	说明
1	#include "lv_port_disp.h"	引入头文件 "lv_port_disp.h"，让当前文件可以使用其中声明的类型、宏和函数。
2		空行，用来分隔逻辑块，让代码更容易阅读。
3	#include "Lcd.h"	引入头文件 "Lcd.h"，让当前文件可以使用其中声明的类型、宏和函数。
4		空行，用来分隔逻辑块，让代码更容易阅读。
5	#define LV_PORT_HOR_RES LCD_WIDTH	宏定义，用名字替代固定值，方便统一修改和表达含义。
6	#define LV_PORT_VER_RES LCD_HEIGHT	宏定义，用名字替代固定值，方便统一修改和表达含义。
7	#define LV_PORT_BUF_LINES 30U	宏定义，用名字替代固定值，方便统一修改和表达含义。
8	#define LV_PORT_BUF_PIXELS (LV_PORT_HOR_RES * LV_PORT_BUF_LINES)	宏定义，用名字替代固定值，方便统一修改和表达含义。
9		空行，用来分隔逻辑块，让代码更容易阅读。
10	static lv_disp_draw_buf_t g_draw_buf;	定义当前文件私有的静态对象或函数，限制作用域，降低模块耦合。
11	static lv_color_t g_buf1[LV_PORT_BUF_PIXELS];	定义当前文件私有的静态对象或函数，限制作用域，降低模块耦合。
12	static lv_disp_drv_t g_disp_drv;	定义当前文件私有的静态对象或函数，限制作用域，降低模块耦合。
13	static lv_disp_t * g_disp = NULL;	定义当前文件私有的静态变量或函数，并给出初始值；外部文件不能直接访问。
14		空行，用来分隔逻辑块，让代码更容易阅读。
15	static void lv_port_flush_cb(lv_disp_drv_t * disp_drv, const lv_area_t * area, lv_color_t * color_p);	声明函数 lv_port_flush_cb，告诉编译器后面会有这个接口。
16		空行，用来分隔逻辑块，让代码更容易阅读。
17	void lv_port_disp_init(void)	声明函数 lv_port_disp_init，告诉编译器后面会有这个接口。
18	{	代码块开始，进入 当前结构 的内部逻辑。
19	lv_disp_draw_buf_init(&g_draw_buf, g_buf1, NULL, LV_PORT_BUF_PIXELS);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
20		空行，用来分隔逻辑块，让代码更容易阅读。
21	lv_disp_drv_init(&g_disp_drv);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
22	g_disp_drv.hor_res = LV_PORT_HOR_RES;	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。

行	源码	说明
23	<code>g_disp_drv.ver_res = LV_PORT_VER_RES;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
24	<code>g_disp_drv.flush_cb = lv_port_flush_cb;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
25	<code>g_disp_drv.draw_buf = &g_draw_buf;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
26		空行，用来分隔逻辑块，让代码更容易阅读。
27	<code>g_disp = lv_disp_drv_register(&g_disp_drv);</code>	向 LVGL 注册实际显示设备。
28	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
29		空行，用来分隔逻辑块，让代码更容易阅读。
30	<code>lv_disp_t * lv_port_disp_get(void)</code>	声明函数 <code>lv_port_disp_get</code> ，告诉编译器后面会有这个接口。
31	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
32	<code>return g_disp;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
33	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
34		空行，用来分隔逻辑块，让代码更容易阅读。
35	<code>void lv_port_disp_handler(void)</code>	声明函数 <code>lv_port_disp_handler</code> ，告诉编译器后面会有这个接口。
36	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
37	<code>/* Blocking flush path: no background transfer to poll. */</code>	块注释开始/内容，解释接下来代码为什么这样写。
38	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
39		空行，用来分隔逻辑块，让代码更容易阅读。
40	<code>static void lv_port_flush_cb(lv_disp_drv_t * disp_drv, const lv_area_t * area, lv_color_t * color_p)</code>	声明函数 <code>lv_port_flush_cb</code> ，告诉编译器后面会有这个接口。
41	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
42	<code>uint32_t width;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
43	<code>uint32_t height;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
44	<code>uint32_t pixel_count;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
45		空行，用来分隔逻辑块，让代码更容易阅读。
46	<code>if((disp_drv == NULL) (area == NULL) (color_p == NULL)) {</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
47	<code>if(disp_drv != NULL) {</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
48	<code>lv_disp_flush_ready(disp_drv);</code>	通知 LVGL 本次 flush 已经完成，可以继续下一块刷新。
49	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
50	<code>return;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。

行	源码	说明
51	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
52		空行，用来分隔逻辑块，让代码更容易阅读。
53	<code>if((area->x2 < 0) (area->y2 < 0) </code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
54	<code>(area->x1 >= LV_PORT_HOR_RES) (area->y1 >= LV_PORT_VER_RES)) {</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
55	<code>lv_disp_flush_ready(disp_drv);</code>	通知 LVGL 本次 flush 已经完成，可以继续下一块刷新。
56	<code>return;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
57	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
58		空行，用来分隔逻辑块，让代码更容易阅读。
59	<code>width = (uint32_t)(area->x2 - area->x1 + 1);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
60	<code>height = (uint32_t)(area->y2 - area->y1 + 1);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
61	<code>pixel_count = width * height;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
62		空行，用来分隔逻辑块，让代码更容易阅读。
63	<code>LCD_Set_Window((uint16_t)area->x1, (uint16_t)area->y1, (uint16_t)area->x2, (uint16_t)area->y2);</code>	告诉 LCD 接下来要写哪一块矩形区域。
64	<code>(void)LCD_WritePixels((const uint8_t *)color_p, pixel_count * 2U);</code>	把 RGB565 像素字节流通过 SPI 写进当前 LCD 窗口。
65	<code>lv_disp_flush_ready(disp_drv);</code>	通知 LVGL 本次 flush 已经完成，可以继续下一块刷新。
66	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。

ui.h

应用 UI 模块接口和页面枚举，给 MQTT 命令层切换页面使用。

本文件共 26 行。下面的表格左侧是原始源码，中间是行号，右侧是逐行说明。资源数组文件的数组数据较长，说明会重点解释其作为图片/字体资源的角色。

行	源码	说明
1	<code>#ifndef __APP_UI_H__</code>	条件编译开始：只有配置宏满足时，下面这段代码才会参与编译。
2	<code>#define __APP_UI_H__</code>	宏定义，用名字替代固定值，方便统一修改和表达含义。
3		空行，用来分隔逻辑块，让代码更容易阅读。
4	<code>typedef enum</code>	定义枚举类型，用一组有名字的常量表达状态或选项。
5	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
6	<code>APP_UI_PAGE_MESSAGE = 0,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
7	<code>APP_UI_PAGE_HOME,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
8	<code>APP_UI_PAGE_OPTIONS,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
9	<code>} app_ui_page_t;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
10		空行，用来分隔逻辑块，让代码更容易阅读。
11	<code>typedef enum</code>	定义枚举类型，用一组有名字的常量表达状态或选项。
12	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
13	<code>APP_UI_ACTIVE_OPTIONS = 0,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
14	<code>APP_UI_ACTIVE_SETTING,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
15	<code>APP_UI_ACTIVE_ABOUT,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
16	<code>} app_ui_active_t;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
17		空行，用来分隔逻辑块，让代码更容易阅读。
18	<code>void app_ui_create(void);</code>	声明函数 <code>app_ui_create</code> ，告诉编译器后面会有这个接口。
19	<code>void app_ui_update(void);</code>	声明函数 <code>app_ui_update</code> ，告诉编译器后面会有这个接口。
20	<code>void app_ui_set_page(app_ui_page_t page);</code>	声明函数 <code>app_ui_set_page</code> ，告诉编译器后面会有这个接口。
21	<code>void app_ui_request_page(app_ui_page_t page);</code>	声明函数 <code>app_ui_request_page</code> ，告诉编译器后面会有这个接口。

行	源码	说明
22	<code>void app_ui_set_active(app_ui_active_t active);</code>	声明函数 <code>app_ui_set_active</code> ，告诉编译器后面会有这个接口。
23	<code>void app_ui_request_active(app_ui_active_t active);</code>	声明函数 <code>app_ui_request_active</code> ，告诉编译器后面会有这个接口。
24		空行，用来分隔逻辑块，让代码更容易阅读。
25	<code>#endif /* __APP_UI_H__ */</code>	条件编译结束。
26		空行，用来分隔逻辑块，让代码更容易阅读。

ui.c

LVGL 页面实现。创建消息页、主页、选项页，并根据 MQTT 数据刷新状态、网络、时间、消息历史。

本文件共 849 行。下面的表格左侧是原始源码，中间是行号，右侧是逐行说明。资源数组文件的数组数据较长，说明会重点解释其作为图片/字体资源的角色。

行	源码	说明
1	#include <rtthread.h>	引入头文件 <rtthread.h>，让当前文件可以使用其中声明的类型、宏和函数。
2	#include <time.h>	引入头文件 <time.h>，让当前文件可以使用其中声明的类型、宏和函数。
3		空行，用来分隔逻辑块，让代码更容易阅读。
4	#include "Lcd.h"	引入头文件 "Lcd.h"，让当前文件可以使用其中声明的类型、宏和函数。
5	#include "lvgl.h"	引入头文件 "lvgl.h"，让当前文件可以使用其中声明的类型、宏和函数。
6	#include "lvgl/src/widgets/lv_img.h"	引入头文件 "lvgl/src/widgets/lv_img.h"，让当前文件可以使用其中声明的类型、宏和函数。
7	#include "my_mqtt.h"	引入头文件 "my_mqtt.h"，让当前文件可以使用其中声明的类型、宏和函数。
8	#include "ui.h"	引入头文件 "ui.h"，让当前文件可以使用其中声明的类型、宏和函数。
9		空行，用来分隔逻辑块，让代码更容易阅读。
10	#define UI_HEARTBEAT_PERIOD_MS 1000U	宏定义，用名字替代固定值，方便统一修改和表达含义。
11	#define UI_HEARTBEAT_ON_MS 500U	宏定义，用名字替代固定值，方便统一修改和表达含义。
12	#define UI_NAV_H 44	宏定义，用名字替代固定值，方便统一修改和表达含义。
13	#define UI_MARGIN 8	宏定义，用名字替代固定值，方便统一修改和表达含义。
14	#define UI_MSG_COUNT 10	宏定义，用名字替代固定值，方便统一修改和表达含义。
15	#define UI_TOPIC_BUF_SIZE 24	宏定义，用名字替代固定值，方便统一修改和表达含义。
16	#define UI_PAYLOAD_BUF_SIZE 96	宏定义，用名字替代固定值，方便统一修改和表达含义。
17	#define UI_MSG_ITEM_GAP 4	宏定义，用名字替代固定值，方便统一修改和表达含义。
18	#define UI_MSG_X 8	宏定义，用名字替代固定值，方便统一修改和表达含义。
19	#define UI_MSG_Y 44	宏定义，用名字替代固定值，方便统一修改和表达含义。
20	#define UI_MSG_W (LCD_WIDTH - 32)	宏定义，用名字替代固定值，方便统一修改和表达含义。

行	源码	说明
21		空行，用来分隔逻辑块，让代码更容易阅读。
22	<code>extern rt_uint32_t my_mqtt_get_message_seq(void);</code>	声明函数 <code>my_mqtt_get_message_seq</code> ，告诉编译器后面会有这个接口。
23	<code>extern rt_size_t my_mqtt_get_history_count(void);</code>	声明函数 <code>my_mqtt_get_history_count</code> ，告诉编译器后面会有这个接口。
24	<code>extern rt_bool_t my_mqtt_get_history_item(rt_size_t display_index,</code>	声明外部符号：它的定义在别的文件里，当前文件只是使用它。
25	<code>char *topic, rt_size_t topic_size,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
26	<code>char *payload, rt_size_t payload_size,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
27	<code>rt_bool_t *is_new);</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
28		空行，用来分隔逻辑块，让代码更容易阅读。
29	<code>extern const lv_img_dsc_t home_avatar;</code>	声明外部符号：它的定义在别的文件里，当前文件只是使用它。
30	<code>extern const lv_font_t ui_font_zh_14;</code>	声明外部符号：它的定义在别的文件里，当前文件只是使用它。
31		空行，用来分隔逻辑块，让代码更容易阅读。
32	<code>#define UI_TEXT_FONT (&ui_font_zh_14)</code>	宏定义，用名字替代固定值，方便统一修改和表达含义。
33		空行，用来分隔逻辑块，让代码更容易阅读。
34	<code>static lv_obj_t *g_heartbeat_boxes[3];</code>	定义当前文件私有的静态对象或函数，限制作用域，降低模块耦合。
35	<code>static lv_obj_t *g_topic_labels[UI_MSG_COUNT];</code>	定义当前文件私有的静态对象或函数，限制作用域，降低模块耦合。
36	<code>static lv_obj_t *g_payload_line1_labels[UI_MSG_COUNT];</code>	定义当前文件私有的静态对象或函数，限制作用域，降低模块耦合。
37	<code>static lv_obj_t *g_payload_line2_labels[UI_MSG_COUNT];</code>	定义当前文件私有的静态对象或函数，限制作用域，降低模块耦合。
38	<code>static lv_obj_t *g_pages[3];</code>	定义当前文件私有的静态对象或函数，限制作用域，降低模块耦合。
39	<code>static lv_obj_t *g_active_title;</code>	定义当前文件私有的静态对象或函数，限制作用域，降低模块耦合。
40	<code>static lv_obj_t *g_active_lines[5];</code>	定义当前文件私有的静态对象或函数，限制作用域，降低模块耦合。
41	<code>static lv_obj_t *g_active_marks[3];</code>	定义当前文件私有的静态对象或函数，限制作用域，降低模块耦合。
42	<code>static lv_obj_t *g_nav_boxes[3];</code>	定义当前文件私有的静态对象或函数，限制作用域，降低模块耦合。
43	<code>static lv_obj_t *g_nav_labels[3];</code>	定义当前文件私有的静态对象或函数，限制作用域，降低模块耦合。
44	<code>static lv_obj_t *g_home_title;</code>	定义当前文件私有的静态对象或函数，限制作用域，降低模块耦合。

行	源码	说明
45	<code>static lv_obj_t *g_panel_net_name;</code>	定义当前文件私有的静态对象或函数，限制作用域，降低模块耦合。
46	<code>static lv_obj_t *g_panel_battery_name;</code>	定义当前文件私有的静态对象或函数，限制作用域，降低模块耦合。
47	<code>static lv_obj_t *g_panel_cpu_value;</code>	定义当前文件私有的静态对象或函数，限制作用域，降低模块耦合。
48	<code>static lv_obj_t *g_panel_cpu_bar;</code>	定义当前文件私有的静态对象或函数，限制作用域，降低模块耦合。
49	<code>static lv_obj_t *g_panel_mem_value;</code>	定义当前文件私有的静态对象或函数，限制作用域，降低模块耦合。
50	<code>static lv_obj_t *g_panel_mem_bar;</code>	定义当前文件私有的静态对象或函数，限制作用域，降低模块耦合。
51	<code>static lv_obj_t *g_panel_net_value;</code>	定义当前文件私有的静态对象或函数，限制作用域，降低模块耦合。
52	<code>static lv_obj_t *g_panel_ip_value;</code>	定义当前文件私有的静态对象或函数，限制作用域，降低模块耦合。
53	<code>static lv_obj_t *g_panel_battery_value;</code>	定义当前文件私有的静态对象或函数，限制作用域，降低模块耦合。
54	<code>static lv_obj_t *g_panel_date_value;</code>	定义当前文件私有的静态对象或函数，限制作用域，降低模块耦合。
55	<code>static lv_obj_t *g_panel_time_value;</code>	定义当前文件私有的静态对象或函数，限制作用域，降低模块耦合。
56	<code>static lv_obj_t *g_panel_pc_value;</code>	定义当前文件私有的静态对象或函数，限制作用域，降低模块耦合。
57	<code>static app_ui_page_t g_current_page = APP_UI_PAGE_HOME;</code>	定义当前文件私有的静态变量或函数，并给出初始值；外部文件不能直接访问。
58	<code>static app_ui_active_t g_current_active = APP_UI_ACTIVE_OPTIONS;</code>	定义当前文件私有的静态变量或函数，并给出初始值；外部文件不能直接访问。
59	<code>static volatile app_ui_page_t g_requested_page = APP_UI_PAGE_HOME;</code>	定义当前文件私有的静态变量或函数，并给出初始值；外部文件不能直接访问。
60	<code>static volatile rt_bool_t g_page_request_pending = RT_FALSE;</code>	定义当前文件私有的静态变量或函数，并给出初始值；外部文件不能直接访问。
61	<code>static volatile app_ui_active_t g_requested_active = APP_UI_ACTIVE_OPTIONS;</code>	定义当前文件私有的静态变量或函数，并给出初始值；外部文件不能直接访问。
62	<code>static volatile rt_bool_t g_active_request_pending = RT_FALSE;</code>	定义当前文件私有的静态变量或函数，并给出初始值；外部文件不能直接访问。
63	<code>static rt_uint32_t g_seen_message_seq = (rt_uint32_t)-1;</code>	定义当前文件私有的静态变量或函数，并给出初始值；外部文件不能直接访问。
64	<code>static rt_uint32_t g_seen_pc_status_seq = (rt_uint32_t)-1;</code>	定义当前文件私有的静态变量或函数，并给出初始值；外部文件不能直接访问。
65	<code>static rt_bool_t g_seen_pc_online = RT_FALSE;</code>	定义当前文件私有的静态变量或函数，并给出初始值；外部文件不能直接访问。
66		空行，用来分隔逻辑块，让代码更容易阅读。
67	<code>static lv_obj_t *ui_line(lv_obj_t *parent, lv_coord_t x, lv_coord_t y,</code>	定义当前文件私有的静态对象或函数，限制作用域，降低模块耦合。

行	源码	说明
68	<code>lv_coord_t w, lv_coord_t h);</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
69	<code>static void ui_create_heartbeat(app_ui_page_t page_id, lv_obj_t *parent);</code>	声明函数 <code>ui_create_heartbeat</code> ，告诉编译器后面会有这个接口。
70	<code>static void ui_create_message_page(lv_obj_t *parent);</code>	声明函数 <code>ui_create_message_page</code> ，告诉编译器后面会有这个接口。
71	<code>static void ui_create_active_page(lv_obj_t *parent);</code>	声明函数 <code>ui_create_active_page</code> ，告诉编译器后面会有这个接口。
72	<code>static void ui_create_home_page(lv_obj_t *parent);</code>	声明函数 <code>ui_create_home_page</code> ，告诉编译器后面会有这个接口。
73	<code>static void ui_create_nav(lv_obj_t *parent);</code>	声明函数 <code>ui_create_nav</code> ，告诉编译器后面会有这个接口。
74	<code>static lv_obj_t *ui_create_label(lv_obj_t *parent, const char *text,</code>	定义当前文件私有的静态对象或函数，限制作用域，降低模块耦合。
75	<code>lv_coord_t x, lv_coord_t y);</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
76	<code>static lv_obj_t *ui_create_meter_row(lv_obj_t *parent, const char *name,</code>	定义当前文件私有的静态对象或函数，限制作用域，降低模块耦合。
77	<code>const char *value, rt_uint8_t percent,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
78	<code>lv_coord_t y, lv_obj_t **value_label);</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
79	<code>static void ui_set_bar_percent(lv_obj_t *bar, rt_uint8_t percent);</code>	声明函数 <code>ui_set_bar_percent</code> ，告诉编译器后面会有这个接口。
80	<code>static void ui_format_rate(char *buf, rt_size_t size, rt_uint32_t bytes_per_sec);</code>	声明函数 <code>ui_format_rate</code> ，告诉编译器后面会有这个接口。
81	<code>static void ui_update_datetime(void);</code>	声明函数 <code>ui_update_datetime</code> ，告诉编译器后面会有这个接口。
82	<code>static lv_coord_t ui_msg_item_height(void);</code>	声明函数 <code>ui_msg_item_height</code> ，告诉编译器后面会有这个接口。
83	<code>static void ui_copy_label_text(char *dst, rt_size_t dst_size, const char *src);</code>	声明函数 <code>ui_copy_label_text</code> ，告诉编译器后面会有这个接口。
84	<code>static void ui_update_nav(void);</code>	声明函数 <code>ui_update_nav</code> ，告诉编译器后面会有这个接口。
85	<code>static void ui_update_heartbeat(void);</code>	声明函数 <code>ui_update_heartbeat</code> ，告诉编译器后面会有这个接口。
86	<code>static void ui_update_message(void);</code>	声明函数 <code>ui_update_message</code> ，告诉编译器后面会有这个接口。
87	<code>static void ui_update_home(void);</code>	声明函数 <code>ui_update_home</code> ，告诉编译器后面会有这个接口。
88	<code>static void ui_apply_active_font(void);</code>	声明函数 <code>ui_apply_active_font</code> ，告诉编译器后面会有这个接口。
89	<code>static void ui_apply_home_lang(void);</code>	声明函数 <code>ui_apply_home_lang</code> ，告诉编译器后面会有这个接口。
90		空行，用来分隔逻辑块，让代码更容易阅读。

行	源码	说明
91	<code>void app_ui_create(void)</code>	声明函数 <code>app_ui_create</code> ，告诉编译器后面会有这个接口。
92	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
93	<code>lv_obj_t *scr = lv_scr_act();</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
94		空行，用来分隔逻辑块，让代码更容易阅读。
95	<code>lv_obj_set_style_bg_color(scr, lv_color_hex(0xF7F7F2), LV_PART_MAIN);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
96	<code>lv_obj_set_style_bg_opa(scr, LV_OPA_COVER, LV_PART_MAIN);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
97	<code>lv_obj_set_style_text_color(scr, lv_color_hex(0x101010), LV_PART_MAIN);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
98	<code>lv_obj_set_style_text_font(scr, UI_TEXT_FONT, LV_PART_MAIN);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
99	<code>lv_obj_clear_flag(scr, LV_OBJ_FLAG_SCROLLABLE);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
100		空行，用来分隔逻辑块，让代码更容易阅读。
101	<code>ui_create_message_page(scr);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
102	<code>ui_create_home_page(scr);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
103	<code>ui_create_active_page(scr);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
104	<code>ui_create_nav(scr);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
105	<code>app_ui_set_page(APP_UI_PAGE_HOME);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
106	<code>app_ui_update();</code>	周期性刷新 UI 状态、消息、时间和心跳。
107	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
108		空行，用来分隔逻辑块，让代码更容易阅读。
109	<code>void app_ui_update(void)</code>	声明函数 <code>app_ui_update</code> ，告诉编译器后面会有这个接口。
110	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
111	<code>if (g_active_request_pending)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
112	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
113	<code>app_ui_set_active((app_ui_active_t)g_requested_active);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
114	<code>g_active_request_pending = RT_FALSE;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
115	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
116		空行，用来分隔逻辑块，让代码更容易阅读。

行	源码	说明
117	<code>if (g_page_request_pending)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
118	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
119	<code>app_ui_set_page((app_ui_page_t)g_requested_page);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
120	<code>g_page_request_pending = RT_FALSE;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
121	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
122		空行，用来分隔逻辑块，让代码更容易阅读。
123	<code>ui_update_heartbeat();</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
124	<code>ui_update_message();</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
125	<code>ui_update_home();</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
126	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
127		空行，用来分隔逻辑块，让代码更容易阅读。
128	<code>void app_ui_set_page(app_ui_page_t page)</code>	声明函数 <code>app_ui_set_page</code> ，告诉编译器后面会有这个接口。
129	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
130	<code>int i;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
131		空行，用来分隔逻辑块，让代码更容易阅读。
132	<code>if (page > APP_UI_PAGE_OPTIONS)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
133	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
134	<code>return;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
135	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
136		空行，用来分隔逻辑块，让代码更容易阅读。
137	<code>g_current_page = page;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
138		空行，用来分隔逻辑块，让代码更容易阅读。
139	<code>for (i = 0; i < 3; ++i)</code>	for 循环，按固定次数或范围重复执行，用于遍历数组、列表或填充缓冲区。
140	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
141	<code>if (g_pages[i] == RT_NULL)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
142	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
143	<code>continue;</code>	跳过本轮循环剩余语句，直接进入下一轮。
144	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
145		空行，用来分隔逻辑块，让代码更容易阅读。

行	源码	说明
146	<code>if (i == (int)page)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
147	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
148	<code>lv_obj_clear_flag(g_pages[i], LV_OBJ_FLAG_HIDDEN);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
149	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
150	<code>else</code>	条件判断的兜底分支，前面的条件都不满足时执行。
151	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
152	<code>lv_obj_add_flag(g_pages[i], LV_OBJ_FLAG_HIDDEN);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
153	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
154	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
155		空行，用来分隔逻辑块，让代码更容易阅读。
156	<code>ui_update_nav();</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
157	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
158		空行，用来分隔逻辑块，让代码更容易阅读。
159	<code>void app_ui_request_page(app_ui_page_t page)</code>	声明函数 <code>app_ui_request_page</code> ，告诉编译器后面会有这个接口。
160	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
161	<code>if (page > APP_UI_PAGE_OPTIONS)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
162	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
163	<code>return;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
164	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
165		空行，用来分隔逻辑块，让代码更容易阅读。
166	<code>g_requested_page = page;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
167	<code>g_page_request_pending = RT_TRUE;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
168	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
169		空行，用来分隔逻辑块，让代码更容易阅读。
170		空行，用来分隔逻辑块，让代码更容易阅读。
171	<code>static void ui_apply_active_font(void)</code>	声明函数 <code>ui_apply_active_font</code> ，告诉编译器后面会有这个接口。
172	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
173	<code>int i;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
174		空行，用来分隔逻辑块，让代码更容易阅读。

行	源码	说明
175	<code>if (g_active_title != RT_NULL)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
176	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
177	<code>lv_obj_set_style_text_font(g_active_title, UI_TEXT_FONT, LV_PART_MAIN);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
178	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
179	<code>for (i = 0; i < 5; ++i)</code>	for 循环，按固定次数或范围重复执行，用于遍历数组、列表或填充缓冲区。
180	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
181	<code>if (g_active_lines[i] != RT_NULL)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
182	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
183	<code>lv_obj_set_style_text_font(g_active_lines[i], UI_TEXT_FONT, LV_PART_MAIN);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
184	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
185	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
186	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
187	<code>void app_ui_set_active(app_ui_active_t active)</code>	声明函数 <code>app_ui_set_active</code> ，告诉编译器后面会有这个接口。
188	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
189	<code>int i;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
190		空行，用来分隔逻辑块，让代码更容易阅读。
191	<code>if (active > APP_UI_ACTIVE_ABOUT)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
192	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
193	<code>return;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
194	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
195		空行，用来分隔逻辑块，让代码更容易阅读。
196	<code>g_current_active = active;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
197	<code>if (g_active_title == RT_NULL)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
198	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
199	<code>return;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
200	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
201		空行，用来分隔逻辑块，让代码更容易阅读。
202	<code>for (i = 0; i < 3; ++i)</code>	for 循环，按固定次数或范围重复执行，用于遍历数组、列表或填充缓冲区。
203	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。

行	源码	说明
204	<code>lv_obj_add_flag(g_active_marks[i], LV_OBJ_FLAG_HIDDEN);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
205	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
206	<code>for (i = 0; i < 5; ++i)</code>	for 循环，按固定次数或范围重复执行，用于遍历数组、列表或填充缓冲区。
207	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
208	<code>lv_obj_set_width(g_active_lines[i], LCD_WIDTH - 36);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
209	<code>lv_obj_align(g_active_lines[i], LV_ALIGN_TOP_LEFT, 6,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
210	<code>(lv_coord_t)(38 + i * 28));</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
211	<code>lv_label_set_text(g_active_lines[i], "");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
212	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
213	<code>ui_apply_active_font();</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
214		空行，用来分隔逻辑块，让代码更容易阅读。
215	<code>if (active == APP_UI_ACTIVE_SETTING)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
216	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
217	<code>lv_label_set_text(g_active_title, "设置");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
218	<code>lv_label_set_text(g_active_lines[0], "界面语言：中文");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
219	<code>lv_label_set_text(g_active_lines[1], "显示：320x240");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
220	<code>lv_label_set_text(g_active_lines[2], "存储：littlefs");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
221	<code>lv_label_set_text(g_active_lines[3], "输入：MQTT命令");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
222	<code>lv_label_set_text(g_active_lines[4], "");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
223	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
224	<code>else if (active == APP_UI_ACTIVE_ABOUT)</code>	声明函数 if，告诉编译器后面会有这个接口。
225	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
226	<code>lv_label_set_text(g_active_title, "关于");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
227	<code>lv_label_set_text(g_active_lines[0], "设备：rt_lvgl_littlefs");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
228	<code>lv_label_set_text(g_active_lines[1], "系统：RT-Thread");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
229	<code>lv_label_set_text(g_active_lines[2], "界面：LVGL + MQTT");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。

行	源码	说明
230	<code>lv_label_set_text(g_active_lines[3], "输入: 切换命令");</code>	函数调用语句: 执行一个已有接口, 可能完成硬件操作、对象创建、字符串处理或状态更新。
231	<code>lv_label_set_text(g_active_lines[4], "构建: " __DATE__);</code>	函数调用语句: 执行一个已有接口, 可能完成硬件操作、对象创建、字符串处理或状态更新。
232	<code>}</code>	代码块结束, 离开 当前结构 的内部逻辑。
233	<code>else</code>	条件判断的兜底分支, 前面的条件都不满足时执行。
234	<code>{</code>	代码块开始, 进入 当前结构 的内部逻辑。
235	<code>static const lv_color_t mark_colors[3] = {</code>	定义当前文件私有的静态变量或函数, 并给出初始值; 外部文件不能直接访问。
236	<code>LV_COLOR_MAKE(0xFF, 0x99, 0x00),</code>	初始化表中的一项, 用于配置数组、结构体或常量表。
237	<code>LV_COLOR_MAKE(0x22, 0xC5, 0x5E),</code>	初始化表中的一项, 用于配置数组、结构体或常量表。
238	<code>LV_COLOR_MAKE(0x3B, 0x82, 0xF6),</code>	初始化表中的一项, 用于配置数组、结构体或常量表。
239	<code>};</code>	结构体、枚举或初始化表结束。
240		空行, 用来分隔逻辑块, 让代码更容易阅读。
241	<code>lv_label_set_text(g_active_title, "功能选择");</code>	函数调用语句: 执行一个已有接口, 可能完成硬件操作、对象创建、字符串处理或状态更新。
242	<code>lv_label_set_text(g_active_lines[0], "设置");</code>	函数调用语句: 执行一个已有接口, 可能完成硬件操作、对象创建、字符串处理或状态更新。
243	<code>lv_label_set_text(g_active_lines[1], "设备信息");</code>	函数调用语句: 执行一个已有接口, 可能完成硬件操作、对象创建、字符串处理或状态更新。
244	<code>lv_label_set_text(g_active_lines[2], "番茄钟");</code>	函数调用语句: 执行一个已有接口, 可能完成硬件操作、对象创建、字符串处理或状态更新。
245	<code>for (i = 0; i < 3; ++i)</code>	for 循环, 按固定次数或范围重复执行, 用于遍历数组、列表或填充缓冲区。
246	<code>{</code>	代码块开始, 进入 当前结构 的内部逻辑。
247	<code>lv_obj_clear_flag(g_active_marks[i], LV_OBJ_FLAG_HIDDEN);</code>	函数调用语句: 执行一个已有接口, 可能完成硬件操作、对象创建、字符串处理或状态更新。
248	<code>lv_obj_set_style_bg_color(g_active_marks[i], mark_colors[i],</code>	初始化表中的一项, 用于配置数组、结构体或常量表。
249	<code>LV_PART_MAIN);</code>	普通 C 语句或声明, 服务于当前函数/模块的局部逻辑。
250	<code>lv_obj_align(g_active_lines[i], LV_ALIGN_TOP_LEFT, 24,</code>	初始化表中的一项, 用于配置数组、结构体或常量表。
251	<code>(lv_coord_t)(38 + i * 28));</code>	函数调用语句: 执行一个已有接口, 可能完成硬件操作、对象创建、字符串处理或状态更新。
252	<code>lv_obj_set_width(g_active_lines[i], LCD_WIDTH - 54);</code>	函数调用语句: 执行一个已有接口, 可能完成硬件操作、对象创建、字符串处理或状态更新。
253	<code>}</code>	代码块结束, 离开 当前结构 的内部逻辑。
254	<code>}</code>	代码块结束, 离开 当前结构 的内部逻辑。
255	<code>}</code>	代码块结束, 离开 当前结构 的内部逻辑。

行	源码	说明
256		空行，用来分隔逻辑块，让代码更容易阅读。
257	<code>void app_ui_request_active(app_ui_active_t active)</code>	声明函数 <code>app_ui_request_active</code> ，告诉编译器后面会有这个接口。
258	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
259	<code>if (active > APP_UI_ACTIVE_ABOUT)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
260	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
261	<code>return;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
262	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
263		空行，用来分隔逻辑块，让代码更容易阅读。
264	<code>g_requested_active = active;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
265	<code>g_active_request_pending = RT_TRUE;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
266	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
267		空行，用来分隔逻辑块，让代码更容易阅读。
268	<code>static lv_obj_t *ui_line(lv_obj_t *parent, lv_coord_t x, lv_coord_t y,</code>	定义当前文件私有的静态对象或函数，限制作用域，降低模块耦合。
269	<code>lv_coord_t w, lv_coord_t h)</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
270	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
271	<code>lv_obj_t *line = lv_obj_create(parent);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
272		空行，用来分隔逻辑块，让代码更容易阅读。
273	<code>lv_obj_remove_style_all(line);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
274	<code>lv_obj_set_pos(line, x, y);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
275	<code>lv_obj_set_size(line, w, h);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
276	<code>lv_obj_set_style_bg_color(line, lv_color_hex(0x101010), LV_PART_MAIN);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
277	<code>lv_obj_set_style_bg_opa(line, LV_OPA_COVER, LV_PART_MAIN);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
278		空行，用来分隔逻辑块，让代码更容易阅读。
279	<code>return line;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
280	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
281		空行，用来分隔逻辑块，让代码更容易阅读。
282	<code>static void ui_create_heartbeat(app_ui_page_t page_id, lv_obj_t *parent)</code>	声明函数 <code>ui_create_heartbeat</code> ，告诉编译器后面会有这个接口。
283	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。

行	源码	说明
284	<code>lv_obj_t *box = lv_obj_create(parent);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
285		空行，用来分隔逻辑块，让代码更容易阅读。
286	<code>g_heartbeat_boxes[page_id] = box;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
287	<code>lv_obj_remove_style_all(box);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
288	<code>lv_obj_set_size(box, 12, 12);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
289	<code>lv_obj_set_style_bg_opa(box, LV_OPA_COVER, LV_PART_MAIN);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
290	<code>lv_obj_set_style_bg_color(box, lv_color_hex(0x111111), LV_PART_MAIN);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
291	<code>lv_obj_align(box, LV_ALIGN_TOP_RIGHT, -14, 16);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
292	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
293		空行，用来分隔逻辑块，让代码更容易阅读。
294	<code>static void ui_create_message_page(lv_obj_t *parent)</code>	声明函数 <code>ui_create_message_page</code> ，告诉编译器后面会有这个接口。
295	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
296	<code>lv_obj_t *page;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
297	<code>lv_obj_t *title;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
298		空行，用来分隔逻辑块，让代码更容易阅读。
299	<code>page = lv_obj_create(parent);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
300	<code>lv_obj_remove_style_all(page);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
301	<code>lv_obj_set_pos(page, UI_MARGIN + 2, 6);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
302	<code>lv_obj_set_size(page, LCD_WIDTH - 2 * UI_MARGIN - 4, LCD_HEIGHT - UI_NAV_H - 8);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
303	<code>lv_obj_clear_flag(page, LV_OBJ_FLAG_SCROLLABLE);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
304	<code>g_pages[APP_UI_PAGE_MESSAGE] = page;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
305		空行，用来分隔逻辑块，让代码更容易阅读。
306	<code>title = lv_label_create(page);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
307	<code>lv_label_set_text(title, "消息");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
308	<code>lv_obj_align(title, LV_ALIGN_TOP_LEFT, 6, 10);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。

行	源码	说明
309		空行，用来分隔逻辑块，让代码更容易阅读。
310	<code>ui_create_heartbeat(APP_UI_PAGE_MESSAGE, page);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
311		空行，用来分隔逻辑块，让代码更容易阅读。
312	<code>ui_line(parent, UI_MARGIN, 4, LCD_WIDTH - 2 * UI_MARGIN, 2);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
313	<code>ui_line(parent, UI_MARGIN, 4, 2, LCD_HEIGHT - UI_NAV_H - 4);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
314	<code>ui_line(parent, LCD_WIDTH - UI_MARGIN - 2, 4, 2, LCD_HEIGHT - UI_NAV_H - 4);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
315	<code>ui_line(parent, UI_MARGIN, LCD_HEIGHT - UI_NAV_H, LCD_WIDTH - 2 * UI_MARGIN, 2);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
316		空行，用来分隔逻辑块，让代码更容易阅读。
317	<code>for (int i = 0; i < UI_MSG_COUNT; ++i)</code>	for 循环，按固定次数或范围重复执行，用于遍历数组、列表或填充缓冲区。
318	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
319	<code>lv_coord_t item_h = ui_msg_item_height();</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
320	<code>lv_coord_t line_h = UI_TEXT_FONT->line_height;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
321		空行，用来分隔逻辑块，让代码更容易阅读。
322	<code>lv_coord_t y = (lv_coord_t)(UI_MSG_Y + i * (item_h + UI_MSG_ITEM_GAP));</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
323		空行，用来分隔逻辑块，让代码更容易阅读。
324	<code>g_topic_labels[i] = lv_label_create(page);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
325	<code>lv_label_set_long_mode(g_topic_labels[i], LV_LABEL_LONG_CLIP);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
326	<code>lv_obj_set_size(g_topic_labels[i], 14, line_h);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
327	<code>lv_obj_set_style_text_color(g_topic_labels[i], lv_color_hex(0xFF9900),</code> <code>LV_PART_MAIN);</code>	初始化表中的一项，用于配置数组、结构体或常量表。
328		普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
329	<code>lv_obj_clear_flag(g_topic_labels[i], LV_OBJ_FLAG_SCROLLABLE);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
330	<code>lv_obj_align(g_topic_labels[i], LV_ALIGN_TOP_LEFT, UI_MSG_X, y);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
331		空行，用来分隔逻辑块，让代码更容易阅读。
332	<code>g_payload_line1_labels[i] = lv_label_create(page);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
333	<code>lv_label_set_long_mode(g_payload_line1_labels[i], LV_LABEL_LONG_CLIP);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。

行	源码	说明
334	<code>lv_obj_set_size(g_payload_line1_labels[i], UI_MSG_W - 18, line_h);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
335	<code>lv_obj_set_style_text_color(g_payload_line1_labels[i], lv_color_hex(0x22C55E),</code>	初始化表中的一行，用于配置数组、结构体或常量表。
336	<code>LV_PART_MAIN);</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
337	<code>lv_obj_clear_flag(g_payload_line1_labels[i], LV_OBJ_FLAG_SCROLLABLE);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
338	<code>lv_obj_align(g_payload_line1_labels[i], LV_ALIGN_TOP_LEFT,</code>	初始化表中的一行，用于配置数组、结构体或常量表。
339	<code>UI_MSG_X + 18, y);</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
340		空行，用来分隔逻辑块，让代码更容易阅读。
341	<code>g_payload_line2_labels[i] = lv_label_create(page);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
342	<code>lv_label_set_long_mode(g_payload_line2_labels[i], LV_LABEL_LONG_CLIP);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
343	<code>lv_obj_set_size(g_payload_line2_labels[i], UI_MSG_W, line_h);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
344	<code>lv_obj_clear_flag(g_payload_line2_labels[i], LV_OBJ_FLAG_SCROLLABLE);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
345	<code>lv_obj_align(g_payload_line2_labels[i], LV_ALIGN_TOP_LEFT, UI_MSG_X, y);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
346	<code>lv_label_set_text(g_payload_line2_labels[i], "");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
347	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
348	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
349		空行，用来分隔逻辑块，让代码更容易阅读。
350	<code>static void ui_create_active_page(lv_obj_t *parent)</code>	声明函数 <code>ui_create_active_page</code> ，告诉编译器后面会有这个接口。
351	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
352	<code>lv_obj_t *page;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
353		空行，用来分隔逻辑块，让代码更容易阅读。
354	<code>page = lv_obj_create(parent);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
355	<code>lv_obj_remove_style_all(page);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
356	<code>lv_obj_set_pos(page, UI_MARGIN + 2, 6);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
357	<code>lv_obj_set_size(page, LCD_WIDTH - 2 * UI_MARGIN - 4, LCD_HEIGHT - UI_NAV_H - 8);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
358	<code>lv_obj_clear_flag(page, LV_OBJ_FLAG_SCROLLABLE);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。

行	源码	说明
359	<code>g_pages[APP_UI_PAGE_OPTIONS] = page;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
360		空行，用来分隔逻辑块，让代码更容易阅读。
361	<code>g_active_title = lv_label_create(page);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
362	<code>lv_obj_set_width(g_active_title, LCD_WIDTH - 48);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
363	<code>lv_label_set_long_mode(g_active_title, LV_LABEL_LONG_CLIP);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
364	<code>lv_obj_align(g_active_title, LV_ALIGN_TOP_LEFT, 6, 10);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
365		空行，用来分隔逻辑块，让代码更容易阅读。
366	<code>for (int i = 0; i < 3; ++i)</code>	for 循环，按固定次数或范围重复执行，用于遍历数组、列表或填充缓冲区。
367	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
368	<code>g_active_marks[i] = lv_obj_create(page);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
369	<code>lv_obj_remove_style_all(g_active_marks[i]);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
370	<code>lv_obj_set_size(g_active_marks[i], 10, 10);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
371	<code>lv_obj_set_style_bg_opa(g_active_marks[i], LV_OPA_COVER, LV_PART_MAIN);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
372	<code>lv_obj_align(g_active_marks[i], LV_ALIGN_TOP_LEFT, 8,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
373	<code>(lv_coord_t)(42 + i * 28));</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
374	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
375		空行，用来分隔逻辑块，让代码更容易阅读。
376	<code>for (int i = 0; i < 5; ++i)</code>	for 循环，按固定次数或范围重复执行，用于遍历数组、列表或填充缓冲区。
377	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
378	<code>g_active_lines[i] = lv_label_create(page);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
379	<code>lv_obj_set_width(g_active_lines[i], LCD_WIDTH - 36);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
380	<code>lv_label_set_long_mode(g_active_lines[i], LV_LABEL_LONG_CLIP);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
381	<code>lv_obj_align(g_active_lines[i], LV_ALIGN_TOP_LEFT, 6,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
382	<code>(lv_coord_t)(38 + i * 28));</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
383	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
384		空行，用来分隔逻辑块，让代码更容易阅读。

行	源码	说明
385	<code>ui_create_heartbeat(APP_UI_PAGE_OPTIONS, page);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
386	<code>app_ui_set_active(APP_UI_ACTIVE_OPTIONS);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
387	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
388		空行，用来分隔逻辑块，让代码更容易阅读。
389	<code>static lv_obj_t *ui_create_label(lv_obj_t *parent, const char *text,</code>	定义当前文件私有的静态对象或函数，限制作用域，降低模块耦合。
390	<code>lv_coord_t x, lv_coord_t y)</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
391	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
392	<code>lv_obj_t *label = lv_label_create(parent);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
393		空行，用来分隔逻辑块，让代码更容易阅读。
394	<code>lv_label_set_text(label, text);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
395	<code>lv_obj_align(label, LV_ALIGN_TOP_LEFT, x, y);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
396		空行，用来分隔逻辑块，让代码更容易阅读。
397	<code>return label;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
398	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
399		空行，用来分隔逻辑块，让代码更容易阅读。
400	<code>static lv_obj_t *ui_create_meter_row(lv_obj_t *parent, const char *name,</code>	定义当前文件私有的静态对象或函数，限制作用域，降低模块耦合。
401	<code>const char *value, rt_uint8_t percent,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
402	<code>lv_coord_t y, lv_obj_t **value_label)</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
403	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
404	<code>lv_obj_t *label;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
405	<code>lv_obj_t *bar_bg;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
406	<code>lv_obj_t *bar_fill;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
407	<code>lv_coord_t bar_w = LCD_WIDTH - 52;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
408	<code>lv_coord_t fill_w;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
409		空行，用来分隔逻辑块，让代码更容易阅读。
410	<code>if (percent > 100)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。

行	源码	说明
411	{	代码块开始, 进入 当前结构 的内部逻辑。
412	percent = 100;	赋值语句: 更新变量、结构体字段或缓冲区内容, 为后续逻辑准备数据。
413	}	代码块结束, 离开 当前结构 的内部逻辑。
414		空行, 用来分隔逻辑块, 让代码更容易阅读。
415	label = ui_create_label(parent, name, 6, y);	赋值语句: 更新变量、结构体字段或缓冲区内容, 为后续逻辑准备数据。
416	lv_obj_set_style_text_color(label, lv_color_hex(0x101010), LV_PART_MAIN);	函数调用语句: 执行一个已有接口, 可能完成硬件操作、对象创建、字符串处理或状态更新。
417		空行, 用来分隔逻辑块, 让代码更容易阅读。
418	label = ui_create_label(parent, value, 72, y);	赋值语句: 更新变量、结构体字段或缓冲区内容, 为后续逻辑准备数据。
419	lv_obj_set_style_text_color(label, lv_color_hex(0x101010), LV_PART_MAIN);	函数调用语句: 执行一个已有接口, 可能完成硬件操作、对象创建、字符串处理或状态更新。
420	if (value_label != RT_NULL)	条件判断: 只有条件成立时才执行后面的分支, 常用于参数检查、错误处理或状态切换。
421	{	代码块开始, 进入 当前结构 的内部逻辑。
422	*value_label = label;	赋值语句: 更新变量、结构体字段或缓冲区内容, 为后续逻辑准备数据。
423	}	代码块结束, 离开 当前结构 的内部逻辑。
424		空行, 用来分隔逻辑块, 让代码更容易阅读。
425	bar_bg = lv_obj_create(parent);	赋值语句: 更新变量、结构体字段或缓冲区内容, 为后续逻辑准备数据。
426	lv_obj_remove_style_all(bar_bg);	函数调用语句: 执行一个已有接口, 可能完成硬件操作、对象创建、字符串处理或状态更新。
427	lv_obj_set_pos(bar_bg, 6, (lv_coord_t)(y + 20));	函数调用语句: 执行一个已有接口, 可能完成硬件操作、对象创建、字符串处理或状态更新。
428	lv_obj_set_size(bar_bg, bar_w, 8);	函数调用语句: 执行一个已有接口, 可能完成硬件操作、对象创建、字符串处理或状态更新。
429	lv_obj_set_style_bg_opa(bar_bg, LV_OPA_COVER, LV_PART_MAIN);	函数调用语句: 执行一个已有接口, 可能完成硬件操作、对象创建、字符串处理或状态更新。
430	lv_obj_set_style_bg_color(bar_bg, lv_color_hex(0xCFCFCF), LV_PART_MAIN);	函数调用语句: 执行一个已有接口, 可能完成硬件操作、对象创建、字符串处理或状态更新。
431		空行, 用来分隔逻辑块, 让代码更容易阅读。
432	fill_w = (lv_coord_t)((bar_w * percent) / 100);	赋值语句: 更新变量、结构体字段或缓冲区内容, 为后续逻辑准备数据。
433	bar_fill = lv_obj_create(parent);	赋值语句: 更新变量、结构体字段或缓冲区内容, 为后续逻辑准备数据。
434	lv_obj_remove_style_all(bar_fill);	函数调用语句: 执行一个已有接口, 可能完成硬件操作、对象创建、字符串处理或状态更新。
435	lv_obj_set_pos(bar_fill, 6, (lv_coord_t)(y + 20));	函数调用语句: 执行一个已有接口, 可能完成硬件操作、对象创建、字符串处理或状态更新。
436	lv_obj_set_size(bar_fill, fill_w, 8);	函数调用语句: 执行一个已有接口, 可能完成硬件操作、对象创建、字符串处理或状态更新。

行	源码	说明
437	<code>lv_obj_set_style_bg_opa(bar_fill, LV_OPA_COVER, LV_PART_MAIN);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
438	<code>lv_obj_set_style_bg_color(bar_fill, lv_color_hex(0xFF9900), LV_PART_MAIN);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
439		空行，用来分隔逻辑块，让代码更容易阅读。
440	<code>return bar_fill;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
441	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
442		空行，用来分隔逻辑块，让代码更容易阅读。
443	<code>static void ui_set_bar_percent(lv_obj_t *bar, rt_uint8_t percent)</code>	声明函数 <code>ui_set_bar_percent</code> ，告诉编译器后面会有这个接口。
444	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
445	<code>lv_coord_t bar_w = LCD_WIDTH - 52;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
446		空行，用来分隔逻辑块，让代码更容易阅读。
447	<code>if (bar == RT_NULL)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
448	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
449	<code>return;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
450	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
451	<code>if (percent > 100)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
452	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
453	<code>percent = 100;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
454	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
455		空行，用来分隔逻辑块，让代码更容易阅读。
456	<code>lv_obj_set_width(bar, (lv_coord_t)((bar_w * percent) / 100));</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
457	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
458		空行，用来分隔逻辑块，让代码更容易阅读。
459	<code>static void ui_format_rate(char *buf, rt_size_t size, rt_uint32_t bytes_per_sec)</code>	声明函数 <code>ui_format_rate</code> ，告诉编译器后面会有这个接口。
460	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
461	<code>const char *unit = "B/s";</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
462	<code>rt_uint32_t value = bytes_per_sec;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
463		空行，用来分隔逻辑块，让代码更容易阅读。
464	<code>if (buf == RT_NULL size == 0)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。

行	源码	说明
465	{	代码块开始，进入 当前结构 的内部逻辑。
466	return;	从当前函数返回结果；错误码或状态值会交给调用者判断。
467	}	代码块结束，离开 当前结构 的内部逻辑。
468		空行，用来分隔逻辑块，让代码更容易阅读。
469	if (bytes_per_sec >= 1024U * 1024U * 1024U)	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
470	{	代码块开始，进入 当前结构 的内部逻辑。
471	value = bytes_per_sec / (1024U * 1024U * 1024U);	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
472	unit = "GB/s";	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
473	}	代码块结束，离开 当前结构 的内部逻辑。
474	else if (bytes_per_sec >= 1024U * 1024U)	声明函数 if，告诉编译器后面会有这个接口。
475	{	代码块开始，进入 当前结构 的内部逻辑。
476	value = bytes_per_sec / (1024U * 1024U);	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
477	unit = "MB/s";	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
478	}	代码块结束，离开 当前结构 的内部逻辑。
479	else if (bytes_per_sec >= 1024U)	声明函数 if，告诉编译器后面会有这个接口。
480	{	代码块开始，进入 当前结构 的内部逻辑。
481	value = bytes_per_sec / 1024U;	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
482	unit = "kB/s";	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
483	}	代码块结束，离开 当前结构 的内部逻辑。
484		空行，用来分隔逻辑块，让代码更容易阅读。
485	rt_sprintf(buf, size, "%u%s", value, unit);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
486	}	代码块结束，离开 当前结构 的内部逻辑。
487		空行，用来分隔逻辑块，让代码更容易阅读。
488	static void ui_update_datetime(void)	声明函数 ui_update_datetime，告诉编译器后面会有这个接口。
489	{	代码块开始，进入 当前结构 的内部逻辑。
490	time_t now;	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
491	struct tm *tm_now;	定义结构体类型，把一组相关数据字段放在一起。
492	char text[24];	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
493		空行，用来分隔逻辑块，让代码更容易阅读。

行	源码	说明
494	<code>now = time(RT_NULL);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
495	<code>tm_now = localtime(&now);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
496	<code>if (tm_now == RT_NULL now < 24 * 60 * 60)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
497	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
498	<code>lv_label_set_text(g_panel_date_value, "日期: --");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
499	<code>lv_label_set_text(g_panel_time_value, "时间: --:--");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
500	<code>return;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
501	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
502		空行，用来分隔逻辑块，让代码更容易阅读。
503	<code>rt_snprintf(text, sizeof(text), "日期: %04d-%02d-%02d",</code>	初始化表中的一项，用于配置数组、结构体或常量表。
504	<code>tm_now->tm_year + 1900,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
505	<code>tm_now->tm_mon + 1,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
506	<code>tm_now->tm_mday);</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
507	<code>lv_label_set_text(g_panel_date_value, text);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
508		空行，用来分隔逻辑块，让代码更容易阅读。
509	<code>rt_snprintf(text, sizeof(text), "时间: %02d:%02d",</code>	初始化表中的一项，用于配置数组、结构体或常量表。
510	<code>tm_now->tm_hour,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
511	<code>tm_now->tm_min);</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
512	<code>lv_label_set_text(g_panel_time_value, text);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
513	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
514		空行，用来分隔逻辑块，让代码更容易阅读。
515	<code>static void ui_create_home_page(lv_obj_t *parent)</code>	声明函数 <code>ui_create_home_page</code> ，告诉编译器后面会有这个接口。
516	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
517	<code>lv_obj_t *page;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
518	<code>lv_obj_t *label;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
519		空行，用来分隔逻辑块，让代码更容易阅读。

行	源码	说明
520	<code>page = lv_obj_create(parent);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
521	<code>lv_obj_remove_style_all(page);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
522	<code>lv_obj_set_pos(page, UI_MARGIN + 2, 6);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
523	<code>lv_obj_set_size(page, LCD_WIDTH - 2 * UI_MARGIN - 4, LCD_HEIGHT - UI_NAV_H - 8);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
524	<code>lv_obj_clear_flag(page, LV_OBJ_FLAG_SCROLLABLE);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
525	<code>g_pages[APP_UI_PAGE_HOME] = page;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
526		空行，用来分隔逻辑块，让代码更容易阅读。
527	<code>g_home_title = ui_create_label(page, "首页", 6, 10);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
528	<code>lv_obj_set_style_text_color(g_home_title, lv_color_hex(0x101010), LV_PART_MAIN);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
529		空行，用来分隔逻辑块，让代码更容易阅读。
530	<code>ui_create_heartbeat(APP_UI_PAGE_HOME, page);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
531		空行，用来分隔逻辑块，让代码更容易阅读。
532	<code>label = lv_img_create(page);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
533	<code>lv_img_set_src(label, &home_avatar);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
534	<code>lv_obj_align(label, LV_ALIGN_TOP_RIGHT, -34, 8);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
535		空行，用来分隔逻辑块，让代码更容易阅读。
536	<code>g_panel_date_value = ui_create_label(page, "日期：--", 6, 32);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
537	<code>lv_obj_set_style_text_color(g_panel_date_value, lv_color_hex(0x22C55E),</code>	初始化表中的一行，用于配置数组、结构体或常量表。
538	<code>LV_PART_MAIN);</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
539	<code>g_panel_time_value = ui_create_label(page, "时间：--:--", 6, 50);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
540	<code>lv_obj_set_style_text_color(g_panel_time_value, lv_color_hex(0x22C55E),</code>	初始化表中的一行，用于配置数组、结构体或常量表。
541	<code>LV_PART_MAIN);</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
542		空行，用来分隔逻辑块，让代码更容易阅读。
543	<code>g_panel_cpu_bar = ui_create_meter_row(page, "处理器", "--%", 0, 76,</code>	初始化表中的一行，用于配置数组、结构体或常量表。
544	<code>&g_panel_cpu_value);</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。

行	源码	说明
545	<code>g_panel_mem_bar = ui_create_meter_row(page, "内存", "--%", 0, 122,</code>	初始化表中的一项, 用于配置数组、结构体或常量表。
546	<code>&g_panel_mem_value);</code>	普通 C 语句或声明, 服务于当前函数/模块的局部逻辑。
547		空行, 用来分隔逻辑块, 让代码更容易阅读。
548	<code>g_panel_net_name = ui_create_label(page, "网络", 6, 170);</code>	赋值语句: 更新变量、结构体字段或缓冲区内容, 为后续逻辑准备数据。
549	<code>lv_obj_set_style_text_color(g_panel_net_name, lv_color_hex(0x101010), LV_PART_MAIN);</code>	函数调用语句: 执行一个已有接口, 可能完成硬件操作、对象创建、字符串处理或状态更新。
550	<code>g_panel_net_value = ui_create_label(page, "下载:--B/s 上传:--B/s", 56, 170);</code>	赋值语句: 更新变量、结构体字段或缓冲区内容, 为后续逻辑准备数据。
551	<code>lv_obj_set_style_text_color(g_panel_net_value, lv_color_hex(0x101010),</code>	初始化表中的一项, 用于配置数组、结构体或常量表。
552	<code>LV_PART_MAIN);</code>	普通 C 语句或声明, 服务于当前函数/模块的局部逻辑。
553		空行, 用来分隔逻辑块, 让代码更容易阅读。
554	<code>label = ui_create_label(page, "地址", 6, 194);</code>	赋值语句: 更新变量、结构体字段或缓冲区内容, 为后续逻辑准备数据。
555	<code>lv_obj_set_style_text_color(label, lv_color_hex(0x101010), LV_PART_MAIN);</code>	函数调用语句: 执行一个已有接口, 可能完成硬件操作、对象创建、字符串处理或状态更新。
556	<code>g_panel_ip_value = ui_create_label(page, "--", 56, 194);</code>	赋值语句: 更新变量、结构体字段或缓冲区内容, 为后续逻辑准备数据。
557	<code>lv_obj_set_style_text_color(g_panel_ip_value, lv_color_hex(0x101010),</code>	初始化表中的一项, 用于配置数组、结构体或常量表。
558	<code>LV_PART_MAIN);</code>	普通 C 语句或声明, 服务于当前函数/模块的局部逻辑。
559		空行, 用来分隔逻辑块, 让代码更容易阅读。
560	<code>g_panel_battery_name = ui_create_label(page, "电量", 6, 216);</code>	赋值语句: 更新变量、结构体字段或缓冲区内容, 为后续逻辑准备数据。
561	<code>lv_obj_set_style_text_color(g_panel_battery_name, lv_color_hex(0x101010), LV_PART_MAIN);</code>	函数调用语句: 执行一个已有接口, 可能完成硬件操作、对象创建、字符串处理或状态更新。
562	<code>g_panel_battery_value = ui_create_label(page, "--%", 56, 216);</code>	赋值语句: 更新变量、结构体字段或缓冲区内容, 为后续逻辑准备数据。
563	<code>lv_obj_set_style_text_color(g_panel_battery_value, lv_color_hex(0x101010),</code>	初始化表中的一项, 用于配置数组、结构体或常量表。
564	<code>LV_PART_MAIN);</code>	普通 C 语句或声明, 服务于当前函数/模块的局部逻辑。
565		空行, 用来分隔逻辑块, 让代码更容易阅读。
566	<code>g_panel_pc_value = ui_create_label(page, "电脑: 离线", 6, 238);</code>	赋值语句: 更新变量、结构体字段或缓冲区内容, 为后续逻辑准备数据。
567	<code>lv_obj_set_style_text_color(g_panel_pc_value, lv_color_hex(0x808080),</code>	初始化表中的一项, 用于配置数组、结构体或常量表。
568	<code>LV_PART_MAIN);</code>	普通 C 语句或声明, 服务于当前函数/模块的局部逻辑。

行	源码	说明
569	<code>ui_apply_home_lang();</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
570	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
571		空行，用来分隔逻辑块，让代码更容易阅读。
572	<code>static void ui_create_nav(lv_obj_t *parent)</code>	声明函数 <code>ui_create_nav</code> ，告诉编译器后面会有这个接口。
573	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
574	<code>static const char *names[] = {"消息", "首页", "选项"};</code>	定义当前文件私有的静态变量或函数，并给出初始值；外部文件不能直接访问。
575	<code>lv_coord_t y = LCD_HEIGHT - UI_NAV_H;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
576	<code>lv_coord_t x = UI_MARGIN;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
577	<code>lv_coord_t w = LCD_WIDTH - 2 * UI_MARGIN;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
578	<code>lv_coord_t tab_w = w / 3;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
579	<code>int i;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
580		空行，用来分隔逻辑块，让代码更容易阅读。
581	<code>ui_line(parent, x, y, w, 2);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
582	<code>ui_line(parent, UI_MARGIN, y, 2, UI_NAV_H - 2);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
583	<code>ui_line(parent, LCD_WIDTH - UI_MARGIN - 2, y, 2, UI_NAV_H - 2);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
584		空行，用来分隔逻辑块，让代码更容易阅读。
585	<code>for (i = 0; i < 3; ++i)</code>	for 循环，按固定次数或范围重复执行，用于遍历数组、列表或填充缓冲区。
586	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
587	<code>lv_obj_t *box = lv_obj_create(parent);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
588	<code>lv_obj_t *label = lv_label_create(box);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
589		空行，用来分隔逻辑块，让代码更容易阅读。
590	<code>g_nav_boxes[i] = box;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
591	<code>lv_obj_remove_style_all(box);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
592	<code>lv_obj_set_pos(box, (lv_coord_t)(x + 2 + i * tab_w), y + 2);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
593	<code>lv_obj_set_size(box, (i == 2) ? (w - i * tab_w - 4) : (tab_w - 2),</code>	初始化表中的一项，用于配置数组、结构体或常量表。

行	源码	说明
594	<code>UI_NAV_H - 4);</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
595	<code>lv_obj_set_style_bg_opa(box, LV_OPA_COVER, LV_PART_MAIN);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
596	<code>lv_obj_set_style_bg_color(box, lv_color_hex(0x808080), LV_PART_MAIN);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
597		空行，用来分隔逻辑块，让代码更容易阅读。
598	<code>g_nav_labels[i] = label;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
599	<code>lv_label_set_text(label, names[i]);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
600	<code>lv_obj_set_width(label, tab_w - 10);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
601	<code>lv_obj_set_style_text_align(label, LV_TEXT_ALIGN_CENTER, LV_PART_MAIN);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
602	<code>lv_obj_center(label);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
603		空行，用来分隔逻辑块，让代码更容易阅读。
604	<code>if (i > 0)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
605	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
606	<code>ui_line(parent, (lv_coord_t)(x + i * tab_w), y, 2, UI_NAV_H - 2);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
607	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
608	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
609	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
610		空行，用来分隔逻辑块，让代码更容易阅读。
611	<code>static void ui_update_nav(void)</code>	声明函数 <code>ui_update_nav</code> ，告诉编译器后面会有这个接口。
612	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
613	<code>int i;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
614		空行，用来分隔逻辑块，让代码更容易阅读。
615	<code>for (i = 0; i < 3; ++i)</code>	for 循环，按固定次数或范围重复执行，用于遍历数组、列表或填充缓冲区。
616	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
617	<code>if (g_nav_labels[i] == RT_NULL)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
618	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
619	<code>continue;</code>	跳过本轮循环剩余语句，直接进入下一轮。
620	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
621		空行，用来分隔逻辑块，让代码更容易阅读。

行	源码	说明
622	<code>if (g_nav_boxes[i] != RT_NULL)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
623	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
624	<code>lv_obj_set_style_bg_color(g_nav_boxes[i],</code>	初始化表中的一项，用于配置数组、结构体或常量表。
625	<code>(i == (int)g_current_page) ?</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
626	<code>lv_color_hex(0xFF9900) :</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
627	<code>lv_color_hex(0x808080),</code>	初始化表中的一项，用于配置数组、结构体或常量表。
628	<code>LV_PART_MAIN);</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
629	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
630		空行，用来分隔逻辑块，让代码更容易阅读。
631	<code>lv_obj_set_style_text_color(g_nav_labels[i], lv_color_hex(0x101010),</code>	初始化表中的一项，用于配置数组、结构体或常量表。
632	<code>LV_PART_MAIN);</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
633	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
634	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
635		空行，用来分隔逻辑块，让代码更容易阅读。
636	<code>static lv_coord_t ui_msg_item_height(void)</code>	声明函数 <code>ui_msg_item_height</code> ，告诉编译器后面会有这个接口。
637	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
638	<code>return UI_TEXT_FONT->line_height;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
639	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
640		空行，用来分隔逻辑块，让代码更容易阅读。
641	<code>static void ui_copy_label_text(char *dst, rt_size_t dst_size, const char *src)</code>	声明函数 <code>ui_copy_label_text</code> ，告诉编译器后面会有这个接口。
642	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
643	<code>rt_size_t i;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
644	<code>rt_size_t copy_len;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
645		空行，用来分隔逻辑块，让代码更容易阅读。
646	<code>if (dst == RT_NULL dst_size == 0)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
647	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
648	<code>return;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
649	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。

行	源码	说明
650		空行，用来分隔逻辑块，让代码更容易阅读。
651	copy_len = rt_strlen(src);	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
652	if (copy_len >= dst_size)	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
653	{	代码块开始，进入 当前结构 的内部逻辑。
654	copy_len = dst_size - 1;	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
655	}	代码块结束，离开 当前结构 的内部逻辑。
656		空行，用来分隔逻辑块，让代码更容易阅读。
657	while (copy_len > 0 && (((const rt_uint8_t *)src)[copy_len] & 0xC0) == 0x80)	while 循环，只要条件成立就持续执行；线程主循环通常写成 while (1)。
658	{	代码块开始，进入 当前结构 的内部逻辑。
659	copy_len--;	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
660	}	代码块结束，离开 当前结构 的内部逻辑。
661		空行，用来分隔逻辑块，让代码更容易阅读。
662	for (i = 0; i < copy_len; ++i)	for 循环，按固定次数或范围重复执行，用于遍历数组、列表或填充缓冲区。
663	{	代码块开始，进入 当前结构 的内部逻辑。
664	if (src[i] == '#' src[i] == '\r' src[i] == '\n')	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
665	{	代码块开始，进入 当前结构 的内部逻辑。
666	dst[i] = ' ';	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
667	}	代码块结束，离开 当前结构 的内部逻辑。
668	else	条件判断的兜底分支，前面的条件都不满足时执行。
669	{	代码块开始，进入 当前结构 的内部逻辑。
670	dst[i] = src[i];	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
671	}	代码块结束，离开 当前结构 的内部逻辑。
672	}	代码块结束，离开 当前结构 的内部逻辑。
673		空行，用来分隔逻辑块，让代码更容易阅读。
674	dst[i] = '\0';	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
675	}	代码块结束，离开 当前结构 的内部逻辑。
676		空行，用来分隔逻辑块，让代码更容易阅读。
677	static void ui_update_heartbeat(void)	声明函数 ui_update_heartbeat，告诉编译器后面会有这个接口。
678	{	代码块开始，进入 当前结构 的内部逻辑。

行	源码	说明
679	<code>rt_uint32_t phase = rt_tick_get_millisecond() % UI_HEARTBEAT_PERIOD_MS;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
680		空行，用来分隔逻辑块，让代码更容易阅读。
681	<code>if (phase < UI_HEARTBEAT_ON_MS)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
682	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
683	<code>for (int i = 0; i < 3; ++i)</code>	for 循环，按固定次数或范围重复执行，用于遍历数组、列表或填充缓冲区。
684	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
685	<code>lv_obj_set_style_bg_color(g_heartbeat_boxes[i], lv_color_hex(0x22C55E),</code>	初始化表中的一项，用于配置数组、结构体或常量表。
686	<code>LV_PART_MAIN);</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
687	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
688	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
689	<code>else</code>	条件判断的兜底分支，前面的条件都不满足时执行。
690	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
691	<code>for (int i = 0; i < 3; ++i)</code>	for 循环，按固定次数或范围重复执行，用于遍历数组、列表或填充缓冲区。
692	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
693	<code>lv_obj_set_style_bg_color(g_heartbeat_boxes[i], lv_color_hex(0x101010),</code>	初始化表中的一项，用于配置数组、结构体或常量表。
694	<code>LV_PART_MAIN);</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
695	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
696	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
697	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
698		空行，用来分隔逻辑块，让代码更容易阅读。
699	<code>static void ui_update_message(void)</code>	声明函数 ui_update_message，告诉编译器后面会有这个接口。
700	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
701	<code>rt_uint32_t seq = my_mqtt_get_message_seq();</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
702	<code>rt_size_t count;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
703	<code>rt_size_t visible_count;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
704		空行，用来分隔逻辑块，让代码更容易阅读。
705	<code>if (seq == g_seen_message_seq)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
706	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。

行	源码	说明
707	<code>return;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
708	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
709		空行，用来分隔逻辑块，让代码更容易阅读。
710	<code>count = my_mqtt_get_history_count();</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
711	<code>visible_count = (count > UI_MSG_COUNT) ? UI_MSG_COUNT : count;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
712	<code>for (int i = 0; i < UI_MSG_COUNT; ++i)</code>	<code>for</code> 循环，按固定次数或范围重复执行，用于遍历数组、列表或填充缓冲区。
713	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
714	<code>rt_size_t display_index;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
715	<code>char prefix[UI_TOPIC_BUF_SIZE];</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
716	<code>char text[UI_PAYLOAD_BUF_SIZE];</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
717	<code>char safe_text[UI_PAYLOAD_BUF_SIZE];</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
718	<code>rt_bool_t is_new = RT_FALSE;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
719		空行，用来分隔逻辑块，让代码更容易阅读。
720	<code>if ((rt_size_t)i >= visible_count)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
721	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
722	<code>lv_label_set_text(g_topic_labels[i], "");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
723	<code>lv_label_set_text(g_payload_line1_labels[i], "");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
724	<code>lv_label_set_text(g_payload_line2_labels[i], "");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
725	<code>continue;</code>	跳过本轮循环剩余语句，直接进入下一轮。
726	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
727		空行，用来分隔逻辑块，让代码更容易阅读。
728	<code>display_index = visible_count - 1 - (rt_size_t)i;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
729	<code>if (!my_mqtt_get_history_item(display_index, prefix, sizeof(prefix),</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
730	<code>text, sizeof(text), &is_new))</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
731	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
732	<code>lv_label_set_text(g_topic_labels[i], "");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。

行	源码	说明
733	<code>lv_label_set_text(g_payload_line1_labels[i], "");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
734	<code>lv_label_set_text(g_payload_line2_labels[i], "");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
735	<code>continue;</code>	跳过本轮循环剩余语句，直接进入下一轮。
736	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
737		空行，用来分隔逻辑块，让代码更容易阅读。
738	<code>ui_copy_label_text(safe_text, sizeof(safe_text), text);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
739	<code>lv_obj_set_style_text_color(g_topic_labels[i],</code>	初始化表中的一项，用于配置数组、结构体或常量表。
740	<code>prefix[0] == '>' ? lv_color_hex(0xFF9900) :</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
741	<code>prefix[0] == '!' ? lv_color_hex(0xD00000) :</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
742	<code>lv_color_hex(0x22C55E),</code>	初始化表中的一项，用于配置数组、结构体或常量表。
743	<code>LV_PART_MAIN);</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
744	<code>lv_obj_set_style_text_color(g_payload_line1_labels[i],</code>	初始化表中的一项，用于配置数组、结构体或常量表。
745	<code>is_new ? lv_color_hex(0x101010) :</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
746	<code>lv_color_hex(0x606060),</code>	初始化表中的一项，用于配置数组、结构体或常量表。
747	<code>LV_PART_MAIN);</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
748		空行，用来分隔逻辑块，让代码更容易阅读。
749	<code>lv_label_set_text(g_topic_labels[i], prefix);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
750	<code>lv_label_set_text(g_payload_line1_labels[i], safe_text);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
751	<code>lv_label_set_text(g_payload_line2_labels[i], "");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
752	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
753		空行，用来分隔逻辑块，让代码更容易阅读。
754	<code>g_seen_message_seq = seq;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
755	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
756		空行，用来分隔逻辑块，让代码更容易阅读。
757	<code>static void ui_apply_home_lang(void)</code>	声明函数 <code>ui_apply_home_lang</code> ，告诉编译器后面会有这个接口。
758	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。

行	源码	说明
759	<code>if (g_home_title != RT_NULL)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
760	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
761	<code>lv_obj_set_style_text_font(g_home_title, UI_TEXT_FONT, LV_PART_MAIN);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
762	<code>lv_label_set_text(g_home_title, "首页");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
763	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
764	<code>if (g_panel_date_value != RT_NULL)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
765	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
766	<code>lv_obj_set_style_text_font(g_panel_date_value, UI_TEXT_FONT, LV_PART_MAIN);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
767	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
768	<code>if (g_panel_time_value != RT_NULL)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
769	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
770	<code>lv_obj_set_style_text_font(g_panel_time_value, UI_TEXT_FONT, LV_PART_MAIN);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
771	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
772	<code>if (g_panel_net_name != RT_NULL)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
773	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
774	<code>lv_obj_set_style_text_font(g_panel_net_name, UI_TEXT_FONT, LV_PART_MAIN);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
775	<code>lv_label_set_text(g_panel_net_name, "网络");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
776	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
777	<code>if (g_panel_battery_name != RT_NULL)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
778	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
779	<code>lv_obj_set_style_text_font(g_panel_battery_name, UI_TEXT_FONT, LV_PART_MAIN);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
780	<code>lv_label_set_text(g_panel_battery_name, "电量");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
781	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
782	<code>if (g_panel_pc_value != RT_NULL)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
783	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
784	<code>lv_obj_set_style_text_font(g_panel_pc_value, UI_TEXT_FONT, LV_PART_MAIN);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
785	<code>lv_label_set_text(g_panel_pc_value, "电脑：离线");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。

行	源码	说明
786	}	代码块结束, 离开 当前结构 的内部逻辑。
787	}	代码块结束, 离开 当前结构 的内部逻辑。
788	static void ui_update_home(void)	声明函数 ui_update_home, 告诉编译器后面会有这个接口。
789	{	代码块开始, 进入 当前结构 的内部逻辑。
790	my_mqtt_pc_status_t status;	普通 C 语句或声明, 服务于当前函数/模块的局部逻辑。
791	char text[64];	普通 C 语句或声明, 服务于当前函数/模块的局部逻辑。
792	char down_text[16];	普通 C 语句或声明, 服务于当前函数/模块的局部逻辑。
793	char up_text[16];	普通 C 语句或声明, 服务于当前函数/模块的局部逻辑。
794	rt_uint32_t age_ms;	普通 C 语句或声明, 服务于当前函数/模块的局部逻辑。
795	rt_bool_t online;	普通 C 语句或声明, 服务于当前函数/模块的局部逻辑。
796		空行, 用来分隔逻辑块, 让代码更容易阅读。
797	ui_update_datetime();	函数调用语句: 执行一个已有接口, 可能完成硬件操作、对象创建、字符串处理或状态更新。
798		空行, 用来分隔逻辑块, 让代码更容易阅读。
799	my_mqtt_get_pc_status(&status);	函数调用语句: 执行一个已有接口, 可能完成硬件操作、对象创建、字符串处理或状态更新。
800	age_ms = rt_tick_get_millisecond() - status.last_tick_ms;	赋值语句: 更新变量、结构体字段或缓冲区内容, 为后续逻辑准备数据。
801	online = (status.online && age_ms < 10000U) ? RT_TRUE : RT_FALSE;	赋值语句: 更新变量、结构体字段或缓冲区内容, 为后续逻辑准备数据。
802		空行, 用来分隔逻辑块, 让代码更容易阅读。
803	if (status.seq == g_seen_pc_status_seq && online == g_seen_pc_online)	条件判断: 只有条件成立时才执行后面的分支, 常用于参数检查、错误处理或状态切换。
804	{	代码块开始, 进入 当前结构 的内部逻辑。
805	return;	从当前函数返回结果; 错误码或状态值会交给调用者判断。
806	}	代码块结束, 离开 当前结构 的内部逻辑。
807		空行, 用来分隔逻辑块, 让代码更容易阅读。
808	rt_sprintf(text, sizeof(text), "%u%%", status.cpu);	函数调用语句: 执行一个已有接口, 可能完成硬件操作、对象创建、字符串处理或状态更新。
809	lv_label_set_text(g_panel_cpu_value, text);	函数调用语句: 执行一个已有接口, 可能完成硬件操作、对象创建、字符串处理或状态更新。
810	ui_set_bar_percent(g_panel_cpu_bar, status.cpu);	函数调用语句: 执行一个已有接口, 可能完成硬件操作、对象创建、字符串处理或状态更新。
811		空行, 用来分隔逻辑块, 让代码更容易阅读。

行	源码	说明
812	<code>rt_sprintf(text, sizeof(text), "%u%", status.mem);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
813	<code>lv_label_set_text(g_panel_mem_value, text);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
814	<code>ui_set_bar_percent(g_panel_mem_bar, status.mem);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
815		空行，用来分隔逻辑块，让代码更容易阅读。
816	<code>ui_format_rate(down_text, sizeof(down_text), status.down);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
817	<code>ui_format_rate(up_text, sizeof(up_text), status.up);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
818	<code>rt_sprintf(text, sizeof(text), "下载:%s 上传:%s", down_text, up_text);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
819	<code>lv_label_set_text(g_panel_net_value, text);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
820		空行，用来分隔逻辑块，让代码更容易阅读。
821	<code>rt_sprintf(text, sizeof(text), "%s", status.ip);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
822	<code>lv_label_set_text(g_panel_ip_value, text);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
823		空行，用来分隔逻辑块，让代码更容易阅读。
824	<code>if (status.battery < 0)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
825	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
826	<code>lv_label_set_text(g_panel_battery_value, "--%");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
827	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
828	<code>else</code>	条件判断的兜底分支，前面的条件都不满足时执行。
829	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
830	<code>rt_sprintf(text, sizeof(text), "%d%", status.battery);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
831	<code>lv_label_set_text(g_panel_battery_value, text);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
832	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
833		空行，用来分隔逻辑块，让代码更容易阅读。
834	<code>lv_label_set_text(g_panel_pc_value, online ? "电脑：在线" : "电脑：离线");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
835	<code>lv_obj_set_style_text_color(g_panel_pc_value,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
836	<code>online ? lv_color_hex(0x22C55E) :</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
837	<code>lv_color_hex(0x808080),</code>	初始化表中的一项，用于配置数组、结构体或常量表。

行	源码	说明
838	LV_PART_MAIN);	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
839		空行，用来分隔逻辑块，让代码更容易阅读。
840	g_seen_pc_status_seq = status.seq;	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
841	g_seen_pc_online = online;	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
842	}	代码块结束，离开 当前结构 的内部逻辑。
843		空行，用来分隔逻辑块，让代码更容易阅读。
844		空行，用来分隔逻辑块，让代码更容易阅读。
845		空行，用来分隔逻辑块，让代码更容易阅读。
846		空行，用来分隔逻辑块，让代码更容易阅读。
847		空行，用来分隔逻辑块，让代码更容易阅读。
848		空行，用来分隔逻辑块，让代码更容易阅读。
849		空行，用来分隔逻辑块，让代码更容易阅读。

my_mqtt.h

MQTT 模块对外接口，定义 PC 状态结构和启动/读取状态函数。

本文件共 22 行。下面的表格左侧是原始源码，中间是行号，右侧是逐行说明。资源数组文件的数组数据较长，说明会重点解释其作为图片/字体资源的角色。

行	源码	说明
1	<code>#ifndef __MY_MQTT_H__</code>	条件编译开始：只有配置宏满足时，下面这段代码才会参与编译。
2	<code>#define __MY_MQTT_H__</code>	宏定义，用名字替代固定值，方便统一修改和表达含义。
3		空行，用来分隔逻辑块，让代码更容易阅读。
4	<code>#include <rtthread.h></code>	引入头文件 <rtthread.h>，让当前文件可以使用其中声明的类型、宏和函数。
5		空行，用来分隔逻辑块，让代码更容易阅读。
6	<code>typedef struct</code>	定义结构体类型，把一组相关数据字段放在一起。
7	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
8	<code> rt_uint8_t cpu;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
9	<code> rt_uint8_t mem;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
10	<code> rt_uint32_t down;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
11	<code> rt_uint32_t up;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
12	<code> rt_int16_t battery;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
13	<code> char ip[16];</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
14	<code> rt_bool_t online;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
15	<code> rt_uint32_t seq;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
16	<code> rt_uint32_t last_tick_ms;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
17	<code>} my_mqtt_pc_status_t;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
18		空行，用来分隔逻辑块，让代码更容易阅读。
19	<code>int my_mqtt_start(void);</code>	声明函数 my_mqtt_start，告诉编译器后面会有这个接口。
20	<code>void my_mqtt_get_pc_status(my_mqtt_pc_status_t *status);</code>	声明函数 my_mqtt_get_pc_status，告诉编译器后面会有这个接口。
21		空行，用来分隔逻辑块，让代码更容易阅读。

行	源码	说明
22	#endif /* __MY_MQTT_H__ */	条件编译结束。

my_mqtt.c

MQTT、远程命令和文件系统小终端。连接巴法云 MQTT，订阅主题，解析 PC 状态和命令，调用 UI 切页，访问 littlefs。

本文件共 1531 行。下面的表格左侧是原始源码，中间是行号，右侧是逐行说明。资源数组文件的数组数据较长，说明会重点解释其作为图片/字体资源的角色。

行	源码	说明
1	#include <stdlib.h>	引入头文件 <stdlib.h>，让当前文件可以使用其中声明的类型、宏和函数。
2	#include <string.h>	引入头文件 <string.h>，让当前文件可以使用其中声明的类型、宏和函数。
3	#include <stdint.h>	引入头文件 <stdint.h>，让当前文件可以使用其中声明的类型、宏和函数。
4		空行，用来分隔逻辑块，让代码更容易阅读。
5	#include <rtthread.h>	引入头文件 <rtthread.h>，让当前文件可以使用其中声明的类型、宏和函数。
6	#include <rthw.h>	引入头文件 <rthw.h>，让当前文件可以使用其中声明的类型、宏和函数。
7	#include <dfs_posix.h>	引入头文件 <dfs_posix.h>，让当前文件可以使用其中声明的类型、宏和函数。
8	#include <libc/libc_fcntl.h>	引入头文件 <libc/libc_fcntl.h>，让当前文件可以使用其中声明的类型、宏和函数。
9	#include <arpa/inet.h>	引入头文件 <arpa/inet.h>，让当前文件可以使用其中声明的类型、宏和函数。
10	#include <netdb.h>	引入头文件 <netdb.h>，让当前文件可以使用其中声明的类型、宏和函数。
11	#include <sys/socket.h>	引入头文件 <sys/socket.h>，让当前文件可以使用其中声明的类型、宏和函数。
12		空行，用来分隔逻辑块，让代码更容易阅读。
13	#include "paho_mqtt.h"	引入头文件 "paho_mqtt.h"，让当前文件可以使用其中声明的类型、宏和函数。
14	#include "my_mqtt.h"	引入头文件 "my_mqtt.h"，让当前文件可以使用其中声明的类型、宏和函数。
15	#include "ui.h"	引入头文件 "ui.h"，让当前文件可以使用其中声明的类型、宏和函数。
16		空行，用来分隔逻辑块，让代码更容易阅读。
17	#define MQTT_NOTIFY_URI "tcp://bemfa.com:9501"	宏定义，用名字替代固定值，方便统一修改和表达含义。
18	#define MQTT_TOPIC_NOTIFY "rtstm32001"	宏定义，用名字替代固定值，方便统一修改和表达含义。
19	#define MQTT_TOPIC_PC_STATUS "rtpcstatus001"	宏定义，用名字替代固定值，方便统一修改和表达含义。
20	#define MQTT_TOPIC_TODO "rttodo001"	宏定义，用名字替代固定值，方便统一修改和表达含义。

行	源码	说明
21	<code>#define MQTT_NOTIFY_CLIENTID "e683645fd0944bd9a2a8b450de95f762"</code>	宏定义，用名字替代固定值，方便统一修改和表达含义。
22	<code>#define MQTT_HISTORY_COUNT 12</code>	宏定义，用名字替代固定值，方便统一修改和表达含义。
23	<code>#define MQTT_TOPIC_SIZE 24</code>	宏定义，用名字替代固定值，方便统一修改和表达含义。
24	<code>#define MQTT_PAYLOAD_SIZE 96</code>	宏定义，用名字替代固定值，方便统一修改和表达含义。
25	<code>#define MQTT_HEARTBEAT_MS 5000U</code>	宏定义，用名字替代固定值，方便统一修改和表达含义。
26	<code>#define MQTT_HEARTBEAT_STACK 1024</code>	线程栈大小配置，栈越大越不容易溢出，但会占更多 RAM。
27	<code>#define MQTT_HEARTBEAT_PRIO 25</code>	线程优先级配置；RT-Thread 数字越小优先级越高。
28		空行，用来分隔逻辑块，让代码更容易阅读。
29	<code>#define HTTP_GET_BUF_SIZE 768</code>	宏定义，用名字替代固定值，方便统一修改和表达含义。
30		空行，用来分隔逻辑块，让代码更容易阅读。
31	<code>static int http_get_connect(const char *host, int port)</code>	声明函数 <code>http_get_connect</code> ，告诉编译器后面会有这个接口。
32	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
33	<code>int sock;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
34	<code>struct sockaddr_in server_addr;</code>	定义结构体类型，把一组相关数据字段放在一起。
35	<code>struct hostent *host_entry;</code>	定义结构体类型，把一组相关数据字段放在一起。
36		空行，用来分隔逻辑块，让代码更容易阅读。
37	<code>rt_memset(&server_addr, 0, sizeof(server_addr));</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
38	<code>server_addr.sin_family = AF_INET;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
39	<code>server_addr.sin_port = htons((uint16_t)port);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
40		空行，用来分隔逻辑块，让代码更容易阅读。
41	<code>if (inet_aton(host, &server_addr.sin_addr) == 0)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
42	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
43	<code>host_entry = gethostbyname(host);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
44	<code>if (host_entry == RT_NULL host_entry->h_addr == RT_NULL)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
45	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。

行	源码	说明
46	<code>rt_kprintf("http_get: unknown host %s\n", host);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
47	<code>return -1;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
48	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
49	<code>rt_memcpy(&server_addr.sin_addr, host_entry->h_addr,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
50	<code>sizeof(server_addr.sin_addr));</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
51	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
52		空行，用来分隔逻辑块，让代码更容易阅读。
53	<code>sock = socket(AF_INET, SOCK_STREAM, 0);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
54	<code>if (sock < 0)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
55	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
56	<code>rt_kprintf("http_get: socket failed\n");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
57	<code>return -1;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
58	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
59		空行，用来分隔逻辑块，让代码更容易阅读。
60	<code>if (connect(sock, (struct sockaddr *)&server_addr,</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
61	<code>sizeof(server_addr)) < 0)</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
62	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
63	<code>rt_kprintf("http_get: connect failed\n");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
64	<code>close(sock);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
65	<code>return -1;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
66	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
67		空行，用来分隔逻辑块，让代码更容易阅读。
68	<code>return sock;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
69	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
70		空行，用来分隔逻辑块，让代码更容易阅读。
71	<code>static char *http_get_find_body(char *buf, int len)</code>	定义当前文件私有的静态对象或函数，限制作用域，降低模块耦合。
72	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。

行	源码	说明
73	<code>int i;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
74		空行，用来分隔逻辑块，让代码更容易阅读。
75	<code>for (i = 0; i <= len - 4; ++i)</code>	for 循环，按固定次数或范围重复执行，用于遍历数组、列表或填充缓冲区。
76	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
77	<code>if (buf[i] == '\r' && buf[i + 1] == '\n' &&</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
78	<code>buf[i + 2] == '\r' && buf[i + 3] == '\n')</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
79	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
80	<code>return &buf[i + 4];</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
81	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
82	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
83		空行，用来分隔逻辑块，让代码更容易阅读。
84	<code>return RT_NULL;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
85	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
86		空行，用来分隔逻辑块，让代码更容易阅读。
87	<code>static int http_get_save_file(const char *host, int port,</code>	定义当前文件私有的静态对象或函数，限制作用域，降低模块耦合。
88	<code>const char *remote_path,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
89	<code>const char *local_path)</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
90	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
91	<code>int sock;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
92	<code>int fd;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
93	<code>int received;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
94	<code>int header_len = 0;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
95	<code>int body_len;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
96	<code>int total = 0;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
97	<code>char *buf;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
98	<code>char *body;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。

行	源码	说明
99	<code>char request[160];</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
100		空行，用来分隔逻辑块，让代码更容易阅读。
101	<code>if (remote_path[0] != '/')</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
102	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
103	<code>rt_kprintf("http_get: remote path must start with /\n");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
104	<code>return -1;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
105	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
106		空行，用来分隔逻辑块，让代码更容易阅读。
107	<code>buf = rt_malloc(HTTP_GET_BUF_SIZE);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
108	<code>if (buf == RT_NULL)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
109	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
110	<code>rt_kprintf("http_get: no memory\n");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
111	<code>return -1;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
112	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
113		空行，用来分隔逻辑块，让代码更容易阅读。
114	<code>fd = open(local_path, O_WRONLY O_CREAT O_TRUNC, 0);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
115	<code>if (fd < 0)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
116	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
117	<code>rt_kprintf("http_get: open %s failed\n", local_path);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
118	<code>rt_free(buf);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
119	<code>return -1;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
120	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
121		空行，用来分隔逻辑块，让代码更容易阅读。
122	<code>sock = http_get_connect(host, port);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
123	<code>if (sock < 0)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
124	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
125	<code>close(fd);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。

行	源码	说明
126	rt_free(buf);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
127	return -1;	从当前函数返回结果；错误码或状态值会交给调用者判断。
128	}	代码块结束，离开 当前结构 的内部逻辑。
129		空行，用来分隔逻辑块，让代码更容易阅读。
130	rt_snprintf(request, sizeof(request),	初始化表中的一项，用于配置数组、结构体或常量表。
131	"GET %s HTTP/1.0\r\nHost: %s\r\nConnection: close\r\n\r\n",	初始化表中的一项，用于配置数组、结构体或常量表。
132	remote_path, host);	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
133	if (send(sock, request, rt_strlen(request), 0) < 0)	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
134	{	代码块开始，进入 当前结构 的内部逻辑。
135	rt_kprintf("http_get: send request failed\n");	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
136	close(sock);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
137	close(fd);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
138	rt_free(buf);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
139	return -1;	从当前函数返回结果；错误码或状态值会交给调用者判断。
140	}	代码块结束，离开 当前结构 的内部逻辑。
141		空行，用来分隔逻辑块，让代码更容易阅读。
142	while (1)	while 循环，只要条件成立就持续执行；线程主循环通常写成 while (1)。
143	{	代码块开始，进入 当前结构 的内部逻辑。
144	received = recv(sock, buf + header_len,	初始化表中的一项，用于配置数组、结构体或常量表。
145	HTTP_GET_BUF_SIZE - header_len, 0);	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
146	if (received <= 0)	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
147	{	代码块开始，进入 当前结构 的内部逻辑。
148	rt_kprintf("http_get: header recv failed\n");	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
149	close(sock);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
150	close(fd);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。

行	源码	说明
151	<code>rt_free(buf);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
152	<code>return -1;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
153	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
154		空行，用来分隔逻辑块，让代码更容易阅读。
155	<code>header_len += received;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
156	<code>body = http_get_find_body(buf, header_len);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
157	<code>if (body != RT_NULL)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
158	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
159	<code>body_len = header_len - (int)(body - buf);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
160	<code>if (rt_strncmp(buf, "HTTP/1.0 200", 12) != 0 &&</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
161	<code>rt_strncmp(buf, "HTTP/1.1 200", 12) != 0)</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
162	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
163	<code>rt_kprintf("http_get: server returned non-200\n");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
164	<code>close(sock);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
165	<code>close(fd);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
166	<code>rt_free(buf);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
167	<code>return -1;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
168	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
169	<code>if (body_len > 0 && write(fd, body, body_len) != body_len)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
170	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
171	<code>rt_kprintf("http_get: write failed\n");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
172	<code>close(sock);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
173	<code>close(fd);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
174	<code>rt_free(buf);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
175	<code>return -1;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。

行	源码	说明
176	}	代码块结束，离开 当前结构 的内部逻辑。
177	total += body_len;	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
178	break;	跳出当前循环或 switch 分支。
179	}	代码块结束，离开 当前结构 的内部逻辑。
180		空行，用来分隔逻辑块，让代码更容易阅读。
181	if (header_len == HTTP_GET_BUF_SIZE)	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
182	{	代码块开始，进入 当前结构 的内部逻辑。
183	rt_kprintf("http_get: HTTP header too large\n");	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
184	close(sock);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
185	close(fd);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
186	rt_free(buf);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
187	return -1;	从当前函数返回结果；错误码或状态值会交给调用者判断。
188	}	代码块结束，离开 当前结构 的内部逻辑。
189	}	代码块结束，离开 当前结构 的内部逻辑。
190		空行，用来分隔逻辑块，让代码更容易阅读。
191	while ((received = recv(sock, buf, HTTP_GET_BUF_SIZE, 0)) > 0)	while 循环，只要条件成立就持续执行；线程主循环通常写成 while (1)。
192	{	代码块开始，进入 当前结构 的内部逻辑。
193	if (write(fd, buf, received) != received)	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
194	{	代码块开始，进入 当前结构 的内部逻辑。
195	rt_kprintf("http_get: write failed\n");	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
196	close(sock);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
197	close(fd);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
198	rt_free(buf);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
199	return -1;	从当前函数返回结果；错误码或状态值会交给调用者判断。
200	}	代码块结束，离开 当前结构 的内部逻辑。
201	total += received;	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
202	if ((total % (64 * 1024)) < HTTP_GET_BUF_SIZE)	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。

行	源码	说明
203	{	代码块开始，进入 当前结构 的内部逻辑。
204	rt_kprintf("http_get: %d bytes\n", total);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
205	}	代码块结束，离开 当前结构 的内部逻辑。
206	}	代码块结束，离开 当前结构 的内部逻辑。
207		空行，用来分隔逻辑块，让代码更容易阅读。
208	close(sock);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
209	close(fd);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
210	rt_free(buf);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
211	rt_kprintf("http_get: saved %d bytes to %s\n", total, local_path);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
212		空行，用来分隔逻辑块，让代码更容易阅读。
213	return total;	从当前函数返回结果；错误码或状态值会交给调用者判断。
214	}	代码块结束，离开 当前结构 的内部逻辑。
215		空行，用来分隔逻辑块，让代码更容易阅读。
216	static int http_get(int argc, char **argv)	声明函数 http_get，告诉编译器后面会有这个接口。
217	{	代码块开始，进入 当前结构 的内部逻辑。
218	int port;	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
219		空行，用来分隔逻辑块，让代码更容易阅读。
220	if (argc != 5)	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
221	{	代码块开始，进入 当前结构 的内部逻辑。
222	rt_kprintf("Usage: http_get <host> <port> <remote_path> <local_path>\n");	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
223	rt_kprintf("eg: http_get 192.168.10.9 8000 /test.bin /test.bin\n");	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
224	return -1;	从当前函数返回结果；错误码或状态值会交给调用者判断。
225	}	代码块结束，离开 当前结构 的内部逻辑。
226		空行，用来分隔逻辑块，让代码更容易阅读。
227	port = atoi(argv[2]);	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
228	if (port <= 0 port > 65535)	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
229	{	代码块开始，进入 当前结构 的内部逻辑。
230	rt_kprintf("http_get: invalid port\n");	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。

行	源码	说明
231	<code>return -1;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
232	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
233		空行，用来分隔逻辑块，让代码更容易阅读。
234	<code>return http_get_save_file(argv[1], port, argv[3], argv[4]) >= 0 ? 0 : -1;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
235	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
236	<code>MSH_CMD_EXPORT(http_get, download file by HTTP GET);</code>	把 C 函数导出为 RT-Thread MSH 命令。
237	<code>static MQTTClient mqtt_notify_client;</code>	定义当前文件私有的静态对象或函数，限制作用域，降低模块耦合。
238	<code>static rt_bool_t mqtt_notify_started = RT_FALSE;</code>	定义当前文件私有的静态变量或函数，并给出初始值；外部文件不能直接访问。
239	<code>static rt_thread_t mqtt_heartbeat_thread = RT_NULL;</code>	定义当前文件私有的静态变量或函数，并给出初始值；外部文件不能直接访问。
240	<code>static rt_uint32_t mqtt_heartbeat_seq = 0;</code>	定义当前文件私有的静态变量或函数，并给出初始值；外部文件不能直接访问。
241	<code>static rt_uint32_t mqtt_message_seq = 0;</code>	定义当前文件私有的静态变量或函数，并给出初始值；外部文件不能直接访问。
242	<code>static my_mqtt_pc_status_t mqtt_pc_status = {</code>	定义当前文件私有的静态变量或函数，并给出初始值；外部文件不能直接访问。
243	<code>0, 0, 0, 0, -1, "--", RT_FALSE, 0, 0</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
244	<code>};</code>	结构体、枚举或初始化表结束。
245		空行，用来分隔逻辑块，让代码更容易阅读。
246	<code>struct mqtt_history_item</code>	定义结构体类型，把一组相关数据字段放在一起。
247	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
248	<code>char topic[MQTT_TOPIC_SIZE];</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
249	<code>char payload[MQTT_PAYLOAD_SIZE];</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
250	<code>};</code>	结构体、枚举或初始化表结束。
251		空行，用来分隔逻辑块，让代码更容易阅读。
252	<code>static struct mqtt_history_item mqtt_history[MQTT_HISTORY_COUNT] = {</code>	定义当前文件私有的静态变量或函数，并给出初始值；外部文件不能直接访问。
253	<code>{ "<", "terminal ready" }</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
254	<code>};</code>	结构体、枚举或初始化表结束。
255	<code>static rt_size_t mqtt_history_used = 1;</code>	定义当前文件私有的静态变量或函数，并给出初始值；外部文件不能直接访问。
256		空行，用来分隔逻辑块，让代码更容易阅读。
257	<code>static const char *mqtt_topics[] = {</code>	定义当前文件私有的静态变量或函数，并给出初始值；外部文件不能直接访问。

行	源码	说明
258	MQTT_TOPIC_NOTIFY,	初始化表中的一项，用于配置数组、结构体或常量表。
259	MQTT_TOPIC_PC_STATUS,	初始化表中的一项，用于配置数组、结构体或常量表。
260	MQTT_TOPIC_TODO,	初始化表中的一项，用于配置数组、结构体或常量表。
261	};	结构体、枚举或初始化表结束。
262		空行，用来分隔逻辑块，让代码更容易阅读。
263	static void mqtt_copy_text(char *dst, rt_size_t dst_size,	定义当前文件私有的静态对象或函数，限制作用域，降低模块耦合。
264	const char *src, rt_size_t src_len)	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
265	{	代码块开始，进入 当前结构 的内部逻辑。
266	rt_size_t copy_len = src_len;	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
267		空行，用来分隔逻辑块，让代码更容易阅读。
268	if (dst == RT_NULL dst_size == 0)	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
269	{	代码块开始，进入 当前结构 的内部逻辑。
270	return;	从当前函数返回结果；错误码或状态值会交给调用者判断。
271	}	代码块结束，离开 当前结构 的内部逻辑。
272		空行，用来分隔逻辑块，让代码更容易阅读。
273	if (copy_len >= dst_size)	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
274	{	代码块开始，进入 当前结构 的内部逻辑。
275	copy_len = dst_size - 1;	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
276		空行，用来分隔逻辑块，让代码更容易阅读。
277	while (copy_len > 0 &&	while 循环，只要条件成立就持续执行；线程主循环通常写成 while (1)。
278	((const rt_uint8_t *)src)[copy_len - 1] & 0xC0) == 0x80)	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
279	{	代码块开始，进入 当前结构 的内部逻辑。
280	copy_len--;	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
281	}	代码块结束，离开 当前结构 的内部逻辑。
282	}	代码块结束，离开 当前结构 的内部逻辑。
283		空行，用来分隔逻辑块，让代码更容易阅读。
284	rt_memcpy(dst, src, copy_len);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
285	dst[copy_len] = '\0';	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。

行	源码	说明
286	}	代码块结束，离开 当前结构 的内部逻辑。
287		空行，用来分隔逻辑块，让代码更容易阅读。
288	static void mqtt_push_history(const char *topic, rt_size_t topic_len,	定义当前文件私有的静态对象或函数，限制作用域，降低模块耦合。
289	const char *payload, rt_size_t payload_len)	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
290	{	代码块开始，进入 当前结构 的内部逻辑。
291	rt_size_t i;	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
292		空行，用来分隔逻辑块，让代码更容易阅读。
293	if (mqtt_history_used < MQTT_HISTORY_COUNT)	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
294	{	代码块开始，进入 当前结构 的内部逻辑。
295	mqtt_history_used++;	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
296	}	代码块结束，离开 当前结构 的内部逻辑。
297		空行，用来分隔逻辑块，让代码更容易阅读。
298	for (i = mqtt_history_used - 1; i > 0; --i)	for 循环，按固定次数或范围重复执行，用于遍历数组、列表或填充缓冲区。
299	{	代码块开始，进入 当前结构 的内部逻辑。
300	mqtt_history[i] = mqtt_history[i - 1];	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
301	}	代码块结束，离开 当前结构 的内部逻辑。
302		空行，用来分隔逻辑块，让代码更容易阅读。
303	mqtt_copy_text(mqtt_history[0].topic, sizeof(mqtt_history[0].topic),	初始化表中的一项，用于配置数组、结构体或常量表。
304	topic, topic_len);	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
305	mqtt_copy_text(mqtt_history[0].payload, sizeof(mqtt_history[0].payload),	初始化表中的一项，用于配置数组、结构体或常量表。
306	payload, payload_len);	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
307	mqtt_message_seq++;	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
308	}	代码块结束，离开 当前结构 的内部逻辑。
309		空行，用来分隔逻辑块，让代码更容易阅读。
310	static void mqtt_push_terminal_line(const char *prefix, const char *text)	声明函数 mqtt_push_terminal_line，告诉编译器后面会有这个接口。
311	{	代码块开始，进入 当前结构 的内部逻辑。
312	mqtt_push_history(prefix, rt_strlen(prefix), text, rt_strlen(text));	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
313	}	代码块结束，离开 当前结构 的内部逻辑。

行	源码	说明
314		空行，用来分隔逻辑块，让代码更容易阅读。
315	<code>static void mqtt_terminal_output(const char *prefix, const char *text)</code>	声明函数 <code>mqtt_terminal_output</code> ，告诉编译器后面会有这个接口。
316	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
317	<code> rt_kprintf("%s %s\n", prefix, text);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
318	<code> mqtt_push_terminal_line(prefix, text);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
319	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
320		空行，用来分隔逻辑块，让代码更容易阅读。
321	<code>static rt_bool_t mqtt_is_heartbeat_payload(const char *payload, rt_size_t len)</code>	声明函数 <code>mqtt_is_heartbeat_payload</code> ，告诉编译器后面会有这个接口。
322	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
323	<code> if (payload == RT_NULL)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
324	<code> {</code>	代码块开始，进入 当前结构 的内部逻辑。
325	<code> return RT_FALSE;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
326	<code> }</code>	代码块结束，离开 当前结构 的内部逻辑。
327		空行，用来分隔逻辑块，让代码更容易阅读。
328	<code> if (len >= 3 && payload[0] == 'h' && payload[1] == 'b' && payload[2] == '=')</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
329	<code> {</code>	代码块开始，进入 当前结构 的内部逻辑。
330	<code> return RT_TRUE;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
331	<code> }</code>	代码块结束，离开 当前结构 的内部逻辑。
332		空行，用来分隔逻辑块，让代码更容易阅读。
333	<code> if (len == 9 && rt_strcmp(payload, "heartbeat", 9) == 0)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
334	<code> {</code>	代码块开始，进入 当前结构 的内部逻辑。
335	<code> return RT_TRUE;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
336	<code> }</code>	代码块结束，离开 当前结构 的内部逻辑。
337		空行，用来分隔逻辑块，让代码更容易阅读。
338	<code> return RT_FALSE;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
339	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
340		空行，用来分隔逻辑块，让代码更容易阅读。
341	<code>static rt_bool_t mqtt_ascii_iequals(const char *a, const char *b)</code>	声明函数 <code>mqtt_ascii_iequals</code> ，告诉编译器后面会有这个接口。
342	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。

行	源码	说明
343	<code>char ca;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
344	<code>char cb;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
345		空行，用来分隔逻辑块，让代码更容易阅读。
346	<code>if (a == RT_NULL b == RT_NULL)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
347	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
348	<code>return RT_FALSE;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
349	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
350		空行，用来分隔逻辑块，让代码更容易阅读。
351	<code>while (*a != '\0' && *b != '\0')</code>	while 循环，只要条件成立就持续执行；线程主循环通常写成 while (1)。
352	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
353	<code>ca = *a;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
354	<code>cb = *b;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
355	<code>if (ca >= 'A' && ca <= 'Z')</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
356	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
357	<code>ca = (char)(ca - 'A' + 'a');</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
358	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
359	<code>if (cb >= 'A' && cb <= 'Z')</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
360	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
361	<code>cb = (char)(cb - 'A' + 'a');</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
362	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
363	<code>if (ca != cb)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
364	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
365	<code>return RT_FALSE;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
366	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
367	<code>a++;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
368	<code>b++;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
369	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
370		空行，用来分隔逻辑块，让代码更容易阅读。

行	源码	说明
371	<code>return (*a == '\0' && *b == '\0') ? RT_TRUE : RT_FALSE;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
372	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
373		空行，用来分隔逻辑块，让代码更容易阅读。
374	<code>static int mqtt_read_int_field(const char *payload, const char *key, int *value)</code>	声明函数 <code>mqtt_read_int_field</code> ，告诉编译器后面会有这个接口。
375	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
376	<code>const char *pos = rt_strstr(payload, key);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
377	<code>char *endptr;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
378		空行，用来分隔逻辑块，让代码更容易阅读。
379	<code>if (pos == RT_NULL value == RT_NULL)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
380	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
381	<code>return -RT_ERROR;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
382	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
383		空行，用来分隔逻辑块，让代码更容易阅读。
384	<code>pos += rt_strlen(key);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
385	<code>if (*pos != '=')</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
386	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
387	<code>return -RT_ERROR;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
388	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
389		空行，用来分隔逻辑块，让代码更容易阅读。
390	<code>pos++;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
391	<code>*value = strtol(pos, &endptr, 10);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
392	<code>if (endptr == pos)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
393	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
394	<code>return -RT_ERROR;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
395	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
396		空行，用来分隔逻辑块，让代码更容易阅读。
397	<code>return RT_EOK;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
398	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。

行	源码	说明
399		空行，用来分隔逻辑块，让代码更容易阅读。
400	static int mqtt_read_text_field(const char *payload, const char *key,	定义当前文件私有的静态对象或函数，限制作用域，降低模块耦合。
401	char *value, rt_size_t value_size)	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
402	{	代码块开始，进入 当前结构 的内部逻辑。
403	const char *pos = rt_strstr(payload, key);	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
404	rt_size_t len = 0;	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
405		空行，用来分隔逻辑块，让代码更容易阅读。
406	if (pos == RT_NULL value == RT_NULL value_size == 0)	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
407	{	代码块开始，进入 当前结构 的内部逻辑。
408	return -RT_ERROR;	从当前函数返回结果；错误码或状态值会交给调用者判断。
409	}	代码块结束，离开 当前结构 的内部逻辑。
410		空行，用来分隔逻辑块，让代码更容易阅读。
411	pos += rt_strlen(key);	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
412	if (*pos != '=')	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
413	{	代码块开始，进入 当前结构 的内部逻辑。
414	return -RT_ERROR;	从当前函数返回结果；错误码或状态值会交给调用者判断。
415	}	代码块结束，离开 当前结构 的内部逻辑。
416		空行，用来分隔逻辑块，让代码更容易阅读。
417	pos++;	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
418	while (pos[len] != '\0' && pos[len] != ',' && len < value_size - 1)	while 循环，只要条件成立就持续执行；线程主循环通常写成 while (1)。
419	{	代码块开始，进入 当前结构 的内部逻辑。
420	len++;	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
421	}	代码块结束，离开 当前结构 的内部逻辑。
422		空行，用来分隔逻辑块，让代码更容易阅读。
423	if (len == 0)	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
424	{	代码块开始，进入 当前结构 的内部逻辑。
425	return -RT_ERROR;	从当前函数返回结果；错误码或状态值会交给调用者判断。
426	}	代码块结束，离开 当前结构 的内部逻辑。

行	源码	说明
427		空行，用来分隔逻辑块，让代码更容易阅读。
428	<code>mqtt_copy_text(value, value_size, pos, len);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
429		空行，用来分隔逻辑块，让代码更容易阅读。
430	<code>return RT_EOK;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
431	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
432		空行，用来分隔逻辑块，让代码更容易阅读。
433	<code>static rt_uint8_t mqtt_percent_limit(int value)</code>	声明函数 <code>mqtt_percent_limit</code> ，告诉编译器后面会有这个接口。
434	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
435	<code>if (value < 0)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
436	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
437	<code>return 0;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
438	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
439	<code>if (value > 100)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
440	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
441	<code>return 100;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
442	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
443		空行，用来分隔逻辑块，让代码更容易阅读。
444	<code>return (rt_uint8_t)value;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
445	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
446		空行，用来分隔逻辑块，让代码更容易阅读。
447	<code>static void mqtt_handle_pc_status(const char *payload, rt_size_t payload_len)</code>	声明函数 <code>mqtt_handle_pc_status</code> ，告诉编译器后面会有这个接口。
448	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
449	<code>char buf[MQTT_PAYLOAD_SIZE];</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
450	<code>int value;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
451		空行，用来分隔逻辑块，让代码更容易阅读。
452	<code>mqtt_copy_text(buf, sizeof(buf), payload, payload_len);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
453		空行，用来分隔逻辑块，让代码更容易阅读。
454	<code>if (mqtt_read_int_field(buf, "cpu", &value) == RT_EOK)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
455	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。

行	源码	说明
456	<code>mqtt_pc_status.cpu = mqtt_percent_limit(value);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
457	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
458	<code>if (mqtt_read_int_field(buf, "mem", &value) == RT_EOK)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
459	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
460	<code>mqtt_pc_status.mem = mqtt_percent_limit(value);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
461	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
462	<code>if (mqtt_read_int_field(buf, "down", &value) == RT_EOK)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
463	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
464	<code>mqtt_pc_status.down = (value < 0) ? 0 : (rt_uint32_t)value;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
465	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
466	<code>if (mqtt_read_int_field(buf, "up", &value) == RT_EOK)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
467	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
468	<code>mqtt_pc_status.up = (value < 0) ? 0 : (rt_uint32_t)value;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
469	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
470	<code>if (mqtt_read_int_field(buf, "bat", &value) == RT_EOK)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
471	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
472	<code>if (value < 0)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
473	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
474	<code>mqtt_pc_status.battery = -1;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
475	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
476	<code>else if (value > 100)</code>	声明函数 if，告诉编译器后面会有这个接口。
477	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
478	<code>mqtt_pc_status.battery = 100;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
479	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
480	<code>else</code>	条件判断的兜底分支，前面的条件都不满足时执行。
481	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
482	<code>mqtt_pc_status.battery = (rt_int16_t)value;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
483	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
484	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。

行	源码	说明
485	<code>if (mqtt_read_text_field(buf, "ip", mqtt_pc_status.ip,</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
486	<code>sizeof(mqtt_pc_status.ip)) != RT_EOK &&</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
487	<code>mqtt_pc_status.ip[0] == '\0')</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
488	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
489	<code>mqtt_copy_text(mqtt_pc_status.ip, sizeof(mqtt_pc_status.ip),</code>	初始化表中的一项，用于配置数组、结构体或常量表。
490	<code>"--", 2);</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
491	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
492		空行，用来分隔逻辑块，让代码更容易阅读。
493	<code>mqtt_pc_status.online = RT_TRUE;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
494	<code>mqtt_pc_status.last_tick_ms = rt_tick_get_millisecond();</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
495	<code>mqtt_pc_status.seq++;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
496	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
497		空行，用来分隔逻辑块，让代码更容易阅读。
498	<code>static rt_bool_t mqtt_text_equals(const char *data, rt_size_t len, const char *cmd)</code>	声明函数 <code>mqtt_text_equals</code> ，告诉编译器后面会有这个接口。
499	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
500	<code>return rt_strlen(cmd) == len && rt_strncmp(data, cmd, len) == 0;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
501	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
502		空行，用来分隔逻辑块，让代码更容易阅读。
503	<code>static char *mqtt_trim_text(char *text)</code>	定义当前文件私有的静态对象或函数，限制作用域，降低模块耦合。
504	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
505	<code>char *end;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
506		空行，用来分隔逻辑块，让代码更容易阅读。
507	<code>if (text == RT_NULL)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
508	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
509	<code>return RT_NULL;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
510	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
511		空行，用来分隔逻辑块，让代码更容易阅读。
512	<code>while (*text == ' ' *text == '\t' *text == '\n' *text == '\n')</code>	<code>while</code> 循环，只要条件成立就持续执行；线程主循环通常写成 <code>while (1)</code> 。

行	源码	说明
513	{	代码块开始，进入 当前结构 的内部逻辑。
514	text++;	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
515	}	代码块结束，离开 当前结构 的内部逻辑。
516		空行，用来分隔逻辑块，让代码更容易阅读。
517	end = text + rt_strlen(text);	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
518	while (end > text &&	while 循环，只要条件成立就持续执行；线程主循环通常写成 while (1)。
519	(end[-1] == ' ' end[-1] == '\t'	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
520	end[-1] == '\r' end[-1] == '\n'))	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
521	{	代码块开始，进入 当前结构 的内部逻辑。
522	end--;	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
523	}	代码块结束，离开 当前结构 的内部逻辑。
524	*end = '\0';	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
525		空行，用来分隔逻辑块，让代码更容易阅读。
526	return text;	从当前函数返回结果；错误码或状态值会交给调用者判断。
527	}	代码块结束，离开 当前结构 的内部逻辑。
528		空行，用来分隔逻辑块，让代码更容易阅读。
529	static rt_bool_t mqtt_cmd_is(const char *cmd, const char *name)	声明函数 mqtt_cmd_is，告诉编译器后面会有这个接口。
530	{	代码块开始，进入 当前结构 的内部逻辑。
531	return mqtt_ascii_iequals(cmd, name);	声明函数 mqtt_ascii_iequals，告诉编译器后面会有这个接口。
532	}	代码块结束，离开 当前结构 的内部逻辑。
533		空行，用来分隔逻辑块，让代码更容易阅读。
534	static rt_bool_t mqtt_cmd_has_arg(const char *cmd, const char *name,	定义当前文件私有的静态对象或函数，限制作用域，降低模块耦合。
535	char **arg)	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
536	{	代码块开始，进入 当前结构 的内部逻辑。
537	rt_size_t len = rt_strlen(name);	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
538	rt_size_t i;	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
539		空行，用来分隔逻辑块，让代码更容易阅读。
540	for (i = 0; i < len; ++i)	for 循环，按固定次数或范围重复执行，用于遍历数组、列表或填充缓冲区。

行	源码	说明
541	{	代码块开始，进入 当前结构 的内部逻辑。
542	char ca = cmd[i];	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
543	char cb = name[i];	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
544		空行，用来分隔逻辑块，让代码更容易阅读。
545	if (ca >= 'A' && ca <= 'Z')	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
546	{	代码块开始，进入 当前结构 的内部逻辑。
547	ca = (char)(ca - 'A' + 'a');	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
548	}	代码块结束，离开 当前结构 的内部逻辑。
549	if (cb >= 'A' && cb <= 'Z')	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
550	{	代码块开始，进入 当前结构 的内部逻辑。
551	cb = (char)(cb - 'A' + 'a');	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
552	}	代码块结束，离开 当前结构 的内部逻辑。
553	if (ca != cb)	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
554	{	代码块开始，进入 当前结构 的内部逻辑。
555	return RT_FALSE;	从当前函数返回结果；错误码或状态值会交给调用者判断。
556	}	代码块结束，离开 当前结构 的内部逻辑。
557	}	代码块结束，离开 当前结构 的内部逻辑。
558		空行，用来分隔逻辑块，让代码更容易阅读。
559	if (cmd[len] != ' ' && cmd[len] != '\t')	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
560	{	代码块开始，进入 当前结构 的内部逻辑。
561	return RT_FALSE;	从当前函数返回结果；错误码或状态值会交给调用者判断。
562	}	代码块结束，离开 当前结构 的内部逻辑。
563		空行，用来分隔逻辑块，让代码更容易阅读。
564	if (arg != RT_NULL)	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
565	{	代码块开始，进入 当前结构 的内部逻辑。
566	*arg = mqtt_trim_text((char *)cmd + len);	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
567	}	代码块结束，离开 当前结构 的内部逻辑。
568		空行，用来分隔逻辑块，让代码更容易阅读。
569	return RT_TRUE;	从当前函数返回结果；错误码或状态值会交给调用者判断。

行	源码	说明
570	}	代码块结束, 离开 当前结构 的内部逻辑。
571		空行, 用来分隔逻辑块, 让代码更容易阅读。
572	static void mqtt_handle_switch_cmd(const char *page_name)	声明函数 mqtt_handle_switch_cmd, 告诉编译器后面会有这个接口。
573	{	代码块开始, 进入 当前结构 的内部逻辑。
574	if (mqtt_ascii_iequals(page_name, "message"))	条件判断: 只有条件成立时才执行后面的分支, 常用于参数检查、错误处理或状态切换。
575	{	代码块开始, 进入 当前结构 的内部逻辑。
576	app_ui_request_page(APP_UI_PAGE_MESSAGE);	函数调用语句: 执行一个已有接口, 可能完成硬件操作、对象创建、字符串处理或状态更新。
577	mqtt_terminal_output("<", "switch Message OK");	函数调用语句: 执行一个已有接口, 可能完成硬件操作、对象创建、字符串处理或状态更新。
578	}	代码块结束, 离开 当前结构 的内部逻辑。
579	else if (mqtt_ascii_iequals(page_name, "home"))	声明函数 if, 告诉编译器后面会有这个接口。
580	{	代码块开始, 进入 当前结构 的内部逻辑。
581	app_ui_request_page(APP_UI_PAGE_HOME);	函数调用语句: 执行一个已有接口, 可能完成硬件操作、对象创建、字符串处理或状态更新。
582	mqtt_terminal_output("<", "switch Home OK");	函数调用语句: 执行一个已有接口, 可能完成硬件操作、对象创建、字符串处理或状态更新。
583	}	代码块结束, 离开 当前结构 的内部逻辑。
584	else if (mqtt_ascii_iequals(page_name, "options"))	声明函数 if, 告诉编译器后面会有这个接口。
585	{	代码块开始, 进入 当前结构 的内部逻辑。
586	app_ui_request_active(APP_UI_ACTIVE_OPTIONS);	函数调用语句: 执行一个已有接口, 可能完成硬件操作、对象创建、字符串处理或状态更新。
587	app_ui_request_page(APP_UI_PAGE_OPTIONS);	函数调用语句: 执行一个已有接口, 可能完成硬件操作、对象创建、字符串处理或状态更新。
588	mqtt_terminal_output("<", "switch Options OK");	函数调用语句: 执行一个已有接口, 可能完成硬件操作、对象创建、字符串处理或状态更新。
589	}	代码块结束, 离开 当前结构 的内部逻辑。
590	else if (mqtt_ascii_iequals(page_name, "settings"))	声明函数 if, 告诉编译器后面会有这个接口。
591	{	代码块开始, 进入 当前结构 的内部逻辑。
592	app_ui_request_active(APP_UI_ACTIVE_SETTING);	函数调用语句: 执行一个已有接口, 可能完成硬件操作、对象创建、字符串处理或状态更新。
593	app_ui_request_page(APP_UI_PAGE_OPTIONS);	函数调用语句: 执行一个已有接口, 可能完成硬件操作、对象创建、字符串处理或状态更新。
594	mqtt_terminal_output("<", "switch Settings OK");	函数调用语句: 执行一个已有接口, 可能完成硬件操作、对象创建、字符串处理或状态更新。
595	}	代码块结束, 离开 当前结构 的内部逻辑。
596	else if (mqtt_ascii_iequals(page_name, "about"))	声明函数 if, 告诉编译器后面会有这个接口。
597	{	代码块开始, 进入 当前结构 的内部逻辑。
598	app_ui_request_active(APP_UI_ACTIVE_ABOUT);	函数调用语句: 执行一个已有接口, 可能完成硬件操作、对象创建、字符串处理或状态更新。

行	源码	说明
599	<code>app_ui_request_page(APP_UI_PAGE_OPTIONS);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
600	<code>mqtt_terminal_output("<", "switch About OK");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
601	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
602	<code>else</code>	条件判断的兜底分支，前面的条件都不满足时执行。
603	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
604	<code>char reply[MQTT_PAYLOAD_SIZE];</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
605		空行，用来分隔逻辑块，让代码更容易阅读。
606	<code>rt_sprintf(reply, sizeof(reply), "unknown page: %s", page_name);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
607	<code>mqtt_terminal_output("!", reply);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
608	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
609	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
610		空行，用来分隔逻辑块，让代码更容易阅读。
611	<code>static void mqtt_handle_clear_cmd(void)</code>	声明函数 <code>mqtt_handle_clear_cmd</code> ，告诉编译器后面会有这个接口。
612	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
613	<code>mqtt_history_used = 0;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
614	<code>mqtt_message_seq++;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
615	<code>mqtt_terminal_output("<", "clear OK");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
616	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
617		空行，用来分隔逻辑块，让代码更容易阅读。
618	<code>static void mqtt_handle_help_cmd(void)</code>	声明函数 <code>mqtt_handle_help_cmd</code> ，告诉编译器后面会有这个接口。
619	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
620	<code>mqtt_terminal_output("<", "Page:");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
621	<code>mqtt_terminal_output("<", "switch message/home");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
622	<code>mqtt_terminal_output("<", "switch options");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
623	<code>mqtt_terminal_output("<", "switch settings/about");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
624	<code>mqtt_terminal_output("<", "Setting:");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。

行	源码	说明
625	<code>mqtt_terminal_output("<", "setting list");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
626	<code>mqtt_terminal_output("<", "File:");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
627	<code>mqtt_terminal_output("<", "ls/pwd/cd/mkdir/rm");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
628	<code>mqtt_terminal_output("<", "cat/echo/df");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
629	<code>mqtt_terminal_output("<", "System: ps/free/reboot/clear");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
630	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
631		空行，用来分隔逻辑块，让代码更容易阅读。
632	<code>static void mqtt_handle_setting_cmd(const char *arg)</code>	声明函数 <code>mqtt_handle_setting_cmd</code> ，告诉编译器后面会有这个接口。
633	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
634	<code>if (arg == RT_NULL arg[0] == '\0' mqtt_ascii_iequals(arg, "list"))</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
635	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
636	<code>mqtt_terminal_output("<", "English UI only");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
637	<code>return;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
638	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
639		空行，用来分隔逻辑块，让代码更容易阅读。
640	<code>mqtt_terminal_output("!", "unknown setting, try: setting list");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
641	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
642		空行，用来分隔逻辑块，让代码更容易阅读。
643	<code>static void mqtt_handle_free_cmd(void)</code>	声明函数 <code>mqtt_handle_free_cmd</code> ，告诉编译器后面会有这个接口。
644	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
645	<code>rt_uint32_t total;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
646	<code>rt_uint32_t used;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
647	<code>rt_uint32_t max_used;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
648	<code>char line[MQTT_PAYLOAD_SIZE];</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
649		空行，用来分隔逻辑块，让代码更容易阅读。
650	<code>rt_memory_info(&total, &used, &max_used);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。

行	源码	说明
651	<code>rt_sprintf(line, sizeof(line), "total: %u bytes", total);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
652	<code>mqtt_terminal_output("<", line);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
653	<code>rt_sprintf(line, sizeof(line), "used : %u bytes", used);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
654	<code>mqtt_terminal_output("<", line);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
655	<code>rt_sprintf(line, sizeof(line), "max : %u bytes", max_used);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
656	<code>mqtt_terminal_output("<", line);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
657	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
658		空行，用来分隔逻辑块，让代码更容易阅读。
659	<code>static void mqtt_handle_pwd_cmd(void)</code>	声明函数 <code>mqtt_handle_pwd_cmd</code> ，告诉编译器后面会有这个接口。
660	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
661	<code>char cwd[DFS_PATH_MAX];</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
662		空行，用来分隔逻辑块，让代码更容易阅读。
663	<code>if (getcwd(cwd, sizeof(cwd)) == RT_NULL)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
664	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
665	<code>mqtt_terminal_output("!", "pwd failed");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
666	<code>return;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
667	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
668		空行，用来分隔逻辑块，让代码更容易阅读。
669	<code>mqtt_terminal_output("<", cwd);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
670	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
671		空行，用来分隔逻辑块，让代码更容易阅读。
672	<code>static void mqtt_handle_cd_cmd(const char *path)</code>	声明函数 <code>mqtt_handle_cd_cmd</code> ，告诉编译器后面会有这个接口。
673	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
674	<code>char cwd[DFS_PATH_MAX];</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
675	<code>char line[MQTT_PAYLOAD_SIZE];</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
676		空行，用来分隔逻辑块，让代码更容易阅读。
677	<code>if (path == RT_NULL path[0] == '\0')</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。

行	源码	说明
678	{	代码块开始, 进入 当前结构 的内部逻辑。
679	path = "/";	赋值语句: 更新变量、结构体字段或缓冲区内容, 为后续逻辑准备数据。
680	}	代码块结束, 离开 当前结构 的内部逻辑。
681		空行, 用来分隔逻辑块, 让代码更容易阅读。
682	if (chdir(path) != 0)	条件判断: 只有条件成立时才执行后面的分支, 常用于参数检查、错误处理或状态切换。
683	{	代码块开始, 进入 当前结构 的内部逻辑。
684	rt_sprintf(line, sizeof(line), "cd failed: %s", path);	函数调用语句: 执行一个已有接口, 可能完成硬件操作、对象创建、字符串处理或状态更新。
685	mqtt_terminal_output("!", line);	函数调用语句: 执行一个已有接口, 可能完成硬件操作、对象创建、字符串处理或状态更新。
686	return;	从当前函数返回结果; 错误码或状态值会交给调用者判断。
687	}	代码块结束, 离开 当前结构 的内部逻辑。
688		空行, 用来分隔逻辑块, 让代码更容易阅读。
689	if (getcwd(cwd, sizeof(cwd)) == RT_NULL)	条件判断: 只有条件成立时才执行后面的分支, 常用于参数检查、错误处理或状态切换。
690	{	代码块开始, 进入 当前结构 的内部逻辑。
691	mqtt_terminal_output("<", "cd OK");	函数调用语句: 执行一个已有接口, 可能完成硬件操作、对象创建、字符串处理或状态更新。
692	return;	从当前函数返回结果; 错误码或状态值会交给调用者判断。
693	}	代码块结束, 离开 当前结构 的内部逻辑。
694		空行, 用来分隔逻辑块, 让代码更容易阅读。
695	rt_sprintf(line, sizeof(line), "cwd: %s", cwd);	函数调用语句: 执行一个已有接口, 可能完成硬件操作、对象创建、字符串处理或状态更新。
696	mqtt_terminal_output("<", line);	函数调用语句: 执行一个已有接口, 可能完成硬件操作、对象创建、字符串处理或状态更新。
697	}	代码块结束, 离开 当前结构 的内部逻辑。
698		空行, 用来分隔逻辑块, 让代码更容易阅读。
699	static void mqtt_handle_mkdir_cmd(const char *path)	声明函数 mqtt_handle_mkdir_cmd, 告诉编译器后面会有这个接口。
700	{	代码块开始, 进入 当前结构 的内部逻辑。
701	char line[MQTT_PAYLOAD_SIZE];	普通 C 语句或声明, 服务于当前函数/模块的局部逻辑。
702		空行, 用来分隔逻辑块, 让代码更容易阅读。
703	if (path == RT_NULL path[0] == '\0')	条件判断: 只有条件成立时才执行后面的分支, 常用于参数检查、错误处理或状态切换。
704	{	代码块开始, 进入 当前结构 的内部逻辑。
705	mqtt_terminal_output("!", "mkdir: missing path");	函数调用语句: 执行一个已有接口, 可能完成硬件操作、对象创建、字符串处理或状态更新。

行	源码	说明
706	<code>return;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
707	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
708		空行，用来分隔逻辑块，让代码更容易阅读。
709	<code>if (mkdir(path, 0) != 0)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
710	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
711	<code>rt_sprintf(line, sizeof(line), "mkdir failed: %s", path);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
712	<code>mqtt_terminal_output("!", line);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
713	<code>return;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
714	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
715		空行，用来分隔逻辑块，让代码更容易阅读。
716	<code>rt_sprintf(line, sizeof(line), "mkdir OK: %s", path);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
717	<code>mqtt_terminal_output("<", line);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
718	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
719		空行，用来分隔逻辑块，让代码更容易阅读。
720	<code>static void mqtt_handle_rm_cmd(const char *path)</code>	声明函数 <code>mqtt_handle_rm_cmd</code> ，告诉编译器后面会有这个接口。
721	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
722	<code>char line[MQTT_PAYLOAD_SIZE];</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
723		空行，用来分隔逻辑块，让代码更容易阅读。
724	<code>if (path == RT_NULL path[0] == '\0')</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
725	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
726	<code>mqtt_terminal_output("!", "rm: missing path");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
727	<code>return;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
728	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
729		空行，用来分隔逻辑块，让代码更容易阅读。
730	<code>if (unlink(path) != 0 && rmdir(path) != 0)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
731	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
732	<code>rt_sprintf(line, sizeof(line), "rm failed: %s", path);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
733	<code>mqtt_terminal_output("!", line);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。

行	源码	说明
734	<code>return;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
735	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
736		空行，用来分隔逻辑块，让代码更容易阅读。
737	<code>rt_sprintf(line, sizeof(line), "rm OK: %s", path);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
738	<code>mqtt_terminal_output("<", line);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
739	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
740		空行，用来分隔逻辑块，让代码更容易阅读。
741	<code>static void mqtt_handle_cat_cmd(const char *path)</code>	声明函数 <code>mqtt_handle_cat_cmd</code> ，告诉编译器后面会有这个接口。
742	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
743	<code>int fd;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
744	<code>int len;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
745	<code>char buf[64];</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
746	<code>char line[MQTT_PAYLOAD_SIZE];</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
747		空行，用来分隔逻辑块，让代码更容易阅读。
748	<code>if (path == RT_NULL path[0] == '\0')</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
749	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
750	<code>mqtt_terminal_output("!", "cat: missing file");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
751	<code>return;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
752	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
753		空行，用来分隔逻辑块，让代码更容易阅读。
754	<code>fd = open(path, O_RDONLY, 0);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
755	<code>if (fd < 0)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
756	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
757	<code>rt_sprintf(line, sizeof(line), "cat: cannot open %s", path);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
758	<code>mqtt_terminal_output("!", line);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
759	<code>return;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
760	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。

行	源码	说明
761		空行，用来分隔逻辑块，让代码更容易阅读。
762	<code>while ((len = read(fd, buf, sizeof(buf) - 1)) > 0)</code>	<code>while</code> 循环，只要条件成立就持续执行；线程主循环通常写成 <code>while (1)</code> 。
763	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
764	<code>buf[len] = '\0';</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
765	<code>mqtt_terminal_output("<", buf);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
766	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
767		空行，用来分隔逻辑块，让代码更容易阅读。
768	<code>close(fd);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
769	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
770		空行，用来分隔逻辑块，让代码更容易阅读。
771	<code>static void mqtt_handle_echo_redirect(const char *text, const char *path,</code>	定义当前文件私有的静态对象或函数，限制作用域，降低模块耦合。
772	<code>rt_bool_t append)</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
773	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
774	<code>int fd;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
775	<code>int flags;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
776	<code>int len;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
777	<code>char line[MQTT_PAYLOAD_SIZE];</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
778		空行，用来分隔逻辑块，让代码更容易阅读。
779	<code>if (path == RT_NULL path[0] == '\0')</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
780	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
781	<code>mqtt_terminal_output("!", "echo: missing file");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
782	<code>return;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
783	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
784		空行，用来分隔逻辑块，让代码更容易阅读。
785	<code>if (text == RT_NULL)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
786	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
787	<code>text = "";</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
788	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。

行	源码	说明
789		空行，用来分隔逻辑块，让代码更容易阅读。
790	flags = O_WRONLY O_CREAT;	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
791	if (append)	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
792	{	代码块开始，进入 当前结构 的内部逻辑。
793	flags = O_APPEND;	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
794	}	代码块结束，离开 当前结构 的内部逻辑。
795	else	条件判断的兜底分支，前面的条件都不满足时执行。
796	{	代码块开始，进入 当前结构 的内部逻辑。
797	flags = O_TRUNC;	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
798	}	代码块结束，离开 当前结构 的内部逻辑。
799		空行，用来分隔逻辑块，让代码更容易阅读。
800	fd = open(path, flags, 0);	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
801	if (fd < 0)	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
802	{	代码块开始，进入 当前结构 的内部逻辑。
803	rt_sprintf(line, sizeof(line), "echo: cannot open %s", path);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
804	mqtt_terminal_output("!", line);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
805	return;	从当前函数返回结果；错误码或状态值会交给调用者判断。
806	}	代码块结束，离开 当前结构 的内部逻辑。
807		空行，用来分隔逻辑块，让代码更容易阅读。
808	len = write(fd, text, rt_strlen(text));	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
809	if (len >= 0)	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
810	{	代码块开始，进入 当前结构 的内部逻辑。
811	write(fd, "\n", 1);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
812	}	代码块结束，离开 当前结构 的内部逻辑。
813	close(fd);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
814		空行，用来分隔逻辑块，让代码更容易阅读。
815	if (len < 0)	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
816	{	代码块开始，进入 当前结构 的内部逻辑。

行	源码	说明
817	<code>rt_sprintf(line, sizeof(line), "echo: write failed: %s", path);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
818	<code>mqtt_terminal_output("!", line);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
819	<code>return;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
820	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
821		空行，用来分隔逻辑块，让代码更容易阅读。
822	<code>rt_sprintf(line, sizeof(line), "echo %s: %s",</code>	初始化表中的一项，用于配置数组、结构体或常量表。
823	<code>append ? "append OK" : "write OK", path);</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
824	<code>mqtt_terminal_output("<", line);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
825	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
826		空行，用来分隔逻辑块，让代码更容易阅读。
827	<code>static void mqtt_handle_echo_cmd(const char *text)</code>	声明函数 <code>mqtt_handle_echo_cmd</code> ，告诉编译器后面会有这个接口。
828	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
829	<code>char *redirect;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
830	<code>rt_bool_t append = RT_FALSE;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
831	<code>char *path;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
832		空行，用来分隔逻辑块，让代码更容易阅读。
833	<code>if (text == RT_NULL)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
834	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
835	<code>text = "";</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
836	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
837		空行，用来分隔逻辑块，让代码更容易阅读。
838	<code>redirect = rt_strstr(text, ">>");</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
839	<code>if (redirect != RT_NULL)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
840	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
841	<code>append = RT_TRUE;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
842	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
843	<code>else</code>	条件判断的兜底分支，前面的条件都不满足时执行。

行	源码	说明
844	{	代码块开始，进入 当前结构 的内部逻辑。
845	redirect = rt_strstr(text, ">");	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
846	}	代码块结束，离开 当前结构 的内部逻辑。
847		空行，用来分隔逻辑块，让代码更容易阅读。
848	if (redirect != RT_NULL)	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
849	{	代码块开始，进入 当前结构 的内部逻辑。
850	*redirect = '\0';	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
851	path = redirect + (append ? 2 : 1);	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
852	mqtt_handle_echo_redirect(mqtt_trim_text((char *)text),	初始化表中的一项，用于配置数组、结构体或常量表。
853	mqtt_trim_text(path),	初始化表中的一项，用于配置数组、结构体或常量表。
854	append);	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
855	return;	从当前函数返回结果；错误码或状态值会交给调用者判断。
856	}	代码块结束，离开 当前结构 的内部逻辑。
857		空行，用来分隔逻辑块，让代码更容易阅读。
858	mqtt_terminal_output("<", text);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
859	}	代码块结束，离开 当前结构 的内部逻辑。
860		空行，用来分隔逻辑块，让代码更容易阅读。
861	static void mqtt_handle_df_cmd(const char *path)	声明函数 mqtt_handle_df_cmd，告诉编译器后面会有这个接口。
862	{	代码块开始，进入 当前结构 的内部逻辑。
863	struct statfs fs;	定义结构体类型，把一组相关数据字段放在一起。
864	char line[MQTT_PAYLOAD_SIZE];	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
865	rt_uint32_t total_kb;	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
866	rt_uint32_t free_kb;	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
867		空行，用来分隔逻辑块，让代码更容易阅读。
868	if (path == RT_NULL path[0] == '\0')	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
869	{	代码块开始，进入 当前结构 的内部逻辑。
870	path = "/";	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。

行	源码	说明
871	}	代码块结束，离开 当前结构 的内部逻辑。
872		空行，用来分隔逻辑块，让代码更容易阅读。
873	if (statfs(path, &fs) != 0)	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
874	{	代码块开始，进入 当前结构 的内部逻辑。
875	rt_snprintf(line, sizeof(line), "df failed: %s", path);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
876	mqtt_terminal_output("!", line);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
877	return;	从当前函数返回结果；错误码或状态值会交给调用者判断。
878	}	代码块结束，离开 当前结构 的内部逻辑。
879		空行，用来分隔逻辑块，让代码更容易阅读。
880	total_kb = (rt_uint32_t)((fs.f_bsize * fs.f_blocks) / 1024);	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
881	free_kb = (rt_uint32_t)((fs.f_bsize * fs.f_bfree) / 1024);	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
882		空行，用来分隔逻辑块，让代码更容易阅读。
883	rt_snprintf(line, sizeof(line), "fs: %s", path);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
884	mqtt_terminal_output("<", line);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
885	rt_snprintf(line, sizeof(line), "total: %uKB", total_kb);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
886	mqtt_terminal_output("<", line);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
887	rt_snprintf(line, sizeof(line), "free : %uKB", free_kb);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
888	mqtt_terminal_output("<", line);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
889	}	代码块结束，离开 当前结构 的内部逻辑。
890		空行，用来分隔逻辑块，让代码更容易阅读。
891	static const char *mqtt_thread_status_text(rt_uint8_t stat)	定义当前文件私有的静态对象或函数，限制作用域，降低模块耦合。
892	{	代码块开始，进入 当前结构 的内部逻辑。
893	stat &= RT_THREAD_STAT_MASK;	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
894		空行，用来分隔逻辑块，让代码更容易阅读。
895	if (stat == RT_THREAD_READY)	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
896	{	代码块开始，进入 当前结构 的内部逻辑。
897	return "ready";	从当前函数返回结果；错误码或状态值会交给调用者判断。

行	源码	说明
898	}	代码块结束，离开 当前结构 的内部逻辑。
899	if (stat == RT_THREAD_SUSPEND)	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
900	{	代码块开始，进入 当前结构 的内部逻辑。
901	return "suspend";	从当前函数返回结果；错误码或状态值会交给调用者判断。
902	}	代码块结束，离开 当前结构 的内部逻辑。
903	if (stat == RT_THREAD_INIT)	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
904	{	代码块开始，进入 当前结构 的内部逻辑。
905	return "init";	从当前函数返回结果；错误码或状态值会交给调用者判断。
906	}	代码块结束，离开 当前结构 的内部逻辑。
907	if (stat == RT_THREAD_CLOSE)	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
908	{	代码块开始，进入 当前结构 的内部逻辑。
909	return "close";	从当前函数返回结果；错误码或状态值会交给调用者判断。
910	}	代码块结束，离开 当前结构 的内部逻辑。
911	if (stat == RT_THREAD_RUNNING)	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
912	{	代码块开始，进入 当前结构 的内部逻辑。
913	return "running";	从当前函数返回结果；错误码或状态值会交给调用者判断。
914	}	代码块结束，离开 当前结构 的内部逻辑。
915		空行，用来分隔逻辑块，让代码更容易阅读。
916	return "unknown";	从当前函数返回结果；错误码或状态值会交给调用者判断。
917	}	代码块结束，离开 当前结构 的内部逻辑。
918		空行，用来分隔逻辑块，让代码更容易阅读。
919	static void mqtt_handle_ps_cmd(void)	声明函数 mqtt_handle_ps_cmd，告诉编译器后面会有这个接口。
920	{	代码块开始，进入 当前结构 的内部逻辑。
921	struct rt_object_information *info;	定义结构体类型，把一组相关数据字段放在一起。
922	rt_list_t *node;	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
923	char line[MQTT_PAYLOAD_SIZE];	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
924	rt_size_t shown = 0;	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
925		空行，用来分隔逻辑块，让代码更容易阅读。

行	源码	说明
926	<code>info = rt_object_get_information(RT_Object_Class_Thread);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
927	<code>if (info == RT_NULL)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
928	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
929	<code>mqtt_terminal_output("!", "ps failed");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
930	<code>return;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
931	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
932		空行，用来分隔逻辑块，让代码更容易阅读。
933	<code>mqtt_terminal_output("<", "thread pri stat");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
934	<code>for (node = info->object_list.next;</code>	for 循环，按固定次数或范围重复执行，用于遍历数组、列表或填充缓冲区。
935	<code>node != &info->object_list;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
936	<code>node = node->next)</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
937	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
938	<code>struct rt_thread *thread;</code>	定义结构体类型，把一组相关数据字段放在一起。
939		空行，用来分隔逻辑块，让代码更容易阅读。
940	<code>thread = rt_list_entry(node, struct rt_thread, list);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
941	<code>rt_sprintf(line, sizeof(line), "%-10.*s %3d %s",</code>	初始化表中的一行，用于配置数组、结构体或常量表。
942	<code>RT_NAME_MAX, thread->name,</code>	初始化表中的一行，用于配置数组、结构体或常量表。
943	<code>thread->current_priority,</code>	初始化表中的一行，用于配置数组、结构体或常量表。
944	<code>mqtt_thread_status_text(thread->stat));</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
945	<code>mqtt_terminal_output("<", line);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
946	<code>shown++;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
947	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
948		空行，用来分隔逻辑块，让代码更容易阅读。
949	<code>if (shown == 0)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
950	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
951	<code>mqtt_terminal_output("<", "(no thread)");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。

行	源码	说明
952	}	代码块结束，离开 当前结构 的内部逻辑。
953	}	代码块结束，离开 当前结构 的内部逻辑。
954		空行，用来分隔逻辑块，让代码更容易阅读。
955	static void mqtt_handle_reboot_cmd(void)	声明函数 mqtt_handle_reboot_cmd，告诉编译器后面会有这个接口。
956	{	代码块开始，进入 当前结构 的内部逻辑。
957	mqtt_terminal_output("<", "rebooting...");	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
958	rt_thread_mdelay(100);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
959	rt_hw_cpu_reset();	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
960	}	代码块结束，离开 当前结构 的内部逻辑。
961		空行，用来分隔逻辑块，让代码更容易阅读。
962	static void mqtt_handle_ls_cmd(const char *path)	声明函数 mqtt_handle_ls_cmd，告诉编译器后面会有这个接口。
963	{	代码块开始，进入 当前结构 的内部逻辑。
964	DIR *dir;	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
965	struct dirent *entry;	定义结构体类型，把一组相关数据字段放在一起。
966	char line[MQTT_PAYLOAD_SIZE];	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
967	rt_size_t shown = 0;	赋值语句：更新变量、结构体字段或缓冲区内内容，为后续逻辑准备数据。
968		空行，用来分隔逻辑块，让代码更容易阅读。
969	if (path == RT_NULL path[0] == '\0')	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
970	{	代码块开始，进入 当前结构 的内部逻辑。
971	path = "/";	赋值语句：更新变量、结构体字段或缓冲区内内容，为后续逻辑准备数据。
972	}	代码块结束，离开 当前结构 的内部逻辑。
973		空行，用来分隔逻辑块，让代码更容易阅读。
974	dir = opendir(path);	赋值语句：更新变量、结构体字段或缓冲区内内容，为后续逻辑准备数据。
975	if (dir == RT_NULL)	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
976	{	代码块开始，进入 当前结构 的内部逻辑。
977	rt_sprintf(line, sizeof(line), "ls: cannot open %s", path);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
978	mqtt_terminal_output("!", line);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。

行	源码	说明
979	<code>return;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
980	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
981		空行，用来分隔逻辑块，让代码更容易阅读。
982	<code>rt_sprintf(line, sizeof(line), "ls %s", path);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
983	<code>mqtt_terminal_output("<", line);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
984		空行，用来分隔逻辑块，让代码更容易阅读。
985	<code>while ((entry = readdir(dir)) != RT_NULL)</code>	while 循环，只要条件成立就持续执行；线程主循环通常写成 while (1)。
986	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
987	<code>rt_sprintf(line, sizeof(line), "%c %s",</code>	初始化表中的一项，用于配置数组、结构体或常量表。
988	<code>entry->d_type == FT_DIRECTORY ? 'd' : '-',</code>	初始化表中的一项，用于配置数组、结构体或常量表。
989	<code>entry->d_name);</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
990	<code>mqtt_terminal_output("<", line);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
991	<code>shown++;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
992	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
993		空行，用来分隔逻辑块，让代码更容易阅读。
994	<code>closedir(dir);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
995		空行，用来分隔逻辑块，让代码更容易阅读。
996	<code>if (shown == 0)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
997	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
998	<code>mqtt_terminal_output("<", "(empty)");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
999	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
1000	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
1001		空行，用来分隔逻辑块，让代码更容易阅读。
1002	<code>static rt_bool_t mqtt_handle_command(const char *topic, rt_size_t topic_len,</code>	定义当前文件私有的静态对象或函数，限制作用域，降低模块耦合。
1003	<code>const char *payload, rt_size_t payload_len)</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
1004	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
1005	<code>char cmd[MQTT_PAYLOAD_SIZE];</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。

行	源码	说明
1006	<code>char *trimmed_cmd;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
1007	<code>char *arg = RT_NULL;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
1008		空行，用来分隔逻辑块，让代码更容易阅读。
1009	<code>if (!mqtt_text_equals(topic, topic_len, MQTT_TOPIC_NOTIFY))</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
1010	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
1011	<code>return RT_FALSE;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
1012	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
1013		空行，用来分隔逻辑块，让代码更容易阅读。
1014	<code>mqtt_copy_text(cmd, sizeof(cmd), payload, payload_len);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
1015	<code>trimmed_cmd = mqtt_trim_text(cmd);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
1016	<code>if (trimmed_cmd == RT_NULL trimmed_cmd[0] == '\0')</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
1017	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
1018	<code>return RT_TRUE;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
1019	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
1020		空行，用来分隔逻辑块，让代码更容易阅读。
1021	<code>mqtt_terminal_output(">", trimmed_cmd);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
1022		空行，用来分隔逻辑块，让代码更容易阅读。
1023	<code>if (mqtt_cmd_has_arg(trimmed_cmd, "switch", &arg))</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
1024	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
1025	<code>mqtt_handle_switch_cmd(arg);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
1026	<code>return RT_TRUE;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
1027	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
1028		空行，用来分隔逻辑块，让代码更容易阅读。
1029	<code>if (mqtt_cmd_is(trimmed_cmd, "setting"))</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
1030	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
1031	<code>mqtt_handle_setting_cmd("list");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
1032	<code>return RT_TRUE;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
1033	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。

行	源码	说明
1034		空行，用来分隔逻辑块，让代码更容易阅读。
1035	<code>if (mqtt_cmd_has_arg(trimmed_cmd, "setting", &arg))</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
1036	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
1037	<code>mqtt_handle_setting_cmd(arg);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
1038	<code>return RT_TRUE;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
1039	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
1040		空行，用来分隔逻辑块，让代码更容易阅读。
1041		空行，用来分隔逻辑块，让代码更容易阅读。
1042	<code>if (mqtt_cmd_is(trimmed_cmd, "ls"))</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
1043	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
1044	<code>mqtt_handle_ls_cmd(".");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
1045	<code>return RT_TRUE;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
1046	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
1047		空行，用来分隔逻辑块，让代码更容易阅读。
1048	<code>if (mqtt_cmd_has_arg(trimmed_cmd, "ls", &arg))</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
1049	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
1050	<code>mqtt_handle_ls_cmd(arg);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
1051	<code>return RT_TRUE;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
1052	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
1053		空行，用来分隔逻辑块，让代码更容易阅读。
1054	<code>if (mqtt_cmd_is(trimmed_cmd, "pwd"))</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
1055	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
1056	<code>mqtt_handle_pwd_cmd();</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
1057	<code>return RT_TRUE;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
1058	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
1059		空行，用来分隔逻辑块，让代码更容易阅读。
1060	<code>if (mqtt_cmd_is(trimmed_cmd, "cd"))</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
1061	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。

行	源码	说明
1062	<code>mqtt_handle_cd_cmd("/");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
1063	<code>return RT_TRUE;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
1064	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
1065		空行，用来分隔逻辑块，让代码更容易阅读。
1066	<code>if (mqtt_cmd_has_arg(trimmed_cmd, "cd", &arg))</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
1067	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
1068	<code>mqtt_handle_cd_cmd(arg);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
1069	<code>return RT_TRUE;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
1070	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
1071		空行，用来分隔逻辑块，让代码更容易阅读。
1072	<code>if (mqtt_cmd_has_arg(trimmed_cmd, "mkdir", &arg))</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
1073	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
1074	<code>mqtt_handle_mkdir_cmd(arg);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
1075	<code>return RT_TRUE;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
1076	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
1077		空行，用来分隔逻辑块，让代码更容易阅读。
1078	<code>if (mqtt_cmd_has_arg(trimmed_cmd, "rm", &arg))</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
1079	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
1080	<code>mqtt_handle_rm_cmd(arg);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
1081	<code>return RT_TRUE;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
1082	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
1083		空行，用来分隔逻辑块，让代码更容易阅读。
1084	<code>if (mqtt_cmd_has_arg(trimmed_cmd, "cat", &arg))</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
1085	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
1086	<code>mqtt_handle_cat_cmd(arg);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
1087	<code>return RT_TRUE;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
1088	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
1089		空行，用来分隔逻辑块，让代码更容易阅读。

行	源码	说明
1090	<code>if (mqtt_cmd_is(trimmed_cmd, "echo"))</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
1091	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
1092	<code>mqtt_handle_echo_cmd("");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
1093	<code>return RT_TRUE;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
1094	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
1095		空行，用来分隔逻辑块，让代码更容易阅读。
1096	<code>if (mqtt_cmd_has_arg(trimmed_cmd, "echo", &arg))</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
1097	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
1098	<code>mqtt_handle_echo_cmd(arg);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
1099	<code>return RT_TRUE;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
1100	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
1101		空行，用来分隔逻辑块，让代码更容易阅读。
1102	<code>if (mqtt_cmd_is(trimmed_cmd, "df"))</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
1103	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
1104	<code>mqtt_handle_df_cmd("/");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
1105	<code>return RT_TRUE;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
1106	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
1107		空行，用来分隔逻辑块，让代码更容易阅读。
1108	<code>if (mqtt_cmd_has_arg(trimmed_cmd, "df", &arg))</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
1109	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
1110	<code>mqtt_handle_df_cmd(arg);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
1111	<code>return RT_TRUE;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
1112	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
1113		空行，用来分隔逻辑块，让代码更容易阅读。
1114	<code>if (mqtt_cmd_is(trimmed_cmd, "ps"))</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
1115	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
1116	<code>mqtt_handle_ps_cmd();</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
1117	<code>return RT_TRUE;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。

行	源码	说明
1118	}	代码块结束，离开 当前结构 的内部逻辑。
1119		空行，用来分隔逻辑块，让代码更容易阅读。
1120	if (mqtt_cmd_is(trimmed_cmd, "free"))	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
1121	{	代码块开始，进入 当前结构 的内部逻辑。
1122	mqtt_handle_free_cmd();	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
1123	return RT_TRUE;	从当前函数返回结果；错误码或状态值会交给调用者判断。
1124	}	代码块结束，离开 当前结构 的内部逻辑。
1125		空行，用来分隔逻辑块，让代码更容易阅读。
1126	if (mqtt_cmd_is(trimmed_cmd, "help"))	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
1127	{	代码块开始，进入 当前结构 的内部逻辑。
1128	mqtt_handle_help_cmd();	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
1129	return RT_TRUE;	从当前函数返回结果；错误码或状态值会交给调用者判断。
1130	}	代码块结束，离开 当前结构 的内部逻辑。
1131		空行，用来分隔逻辑块，让代码更容易阅读。
1132	if (mqtt_cmd_is(trimmed_cmd, "clear"))	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
1133	{	代码块开始，进入 当前结构 的内部逻辑。
1134	mqtt_handle_clear_cmd();	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
1135	return RT_TRUE;	从当前函数返回结果；错误码或状态值会交给调用者判断。
1136	}	代码块结束，离开 当前结构 的内部逻辑。
1137		空行，用来分隔逻辑块，让代码更容易阅读。
1138	if (mqtt_cmd_is(trimmed_cmd, "reboot"))	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
1139	{	代码块开始，进入 当前结构 的内部逻辑。
1140	mqtt_handle_reboot_cmd();	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
1141	return RT_TRUE;	从当前函数返回结果；错误码或状态值会交给调用者判断。
1142	}	代码块结束，离开 当前结构 的内部逻辑。
1143		空行，用来分隔逻辑块，让代码更容易阅读。
1144	mqtt_terminal_output("!", "unknown cmd, try: help");	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
1145	return RT_TRUE;	从当前函数返回结果；错误码或状态值会交给调用者判断。

行	源码	说明
1146	}	代码块结束，离开 当前结构 的内部逻辑。
1147		空行，用来分隔逻辑块，让代码更容易阅读。
1148	void my_mqtt_get_history_text(char *buf, rt_size_t size)	声明函数 my_mqtt_get_history_text，告诉编译器后面会有这个接口。
1149	{	代码块开始，进入 当前结构 的内部逻辑。
1150	rt_size_t i;	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
1151	rt_size_t used = 0;	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
1152		空行，用来分隔逻辑块，让代码更容易阅读。
1153	if (buf == RT_NULL size == 0)	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
1154	{	代码块开始，进入 当前结构 的内部逻辑。
1155	return;	从当前函数返回结果；错误码或状态值会交给调用者判断。
1156	}	代码块结束，离开 当前结构 的内部逻辑。
1157		空行，用来分隔逻辑块，让代码更容易阅读。
1158	buf[0] = '\0';	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
1159		空行，用来分隔逻辑块，让代码更容易阅读。
1160	for (i = mqtt_history_used; i > 0; --i)	for 循环，按固定次数或范围重复执行，用于遍历数组、列表或填充缓冲区。
1161	{	代码块开始，进入 当前结构 的内部逻辑。
1162	rt_size_t index = i - 1;	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
1163	int written = rt_sprintf(buf + used, size - used,	初始化表中的一项，用于配置数组、结构体或常量表。
1164	"%s %s:%s#\n%s %s#s",	初始化表中的一项，用于配置数组、结构体或常量表。
1165	(index == 0) ? "ff9900" : "808080",	初始化表中的一项，用于配置数组、结构体或常量表。
1166	mqtt_history[index].topic,	初始化表中的一项，用于配置数组、结构体或常量表。
1167	(index == 0) ? "(new)" : "",	初始化表中的一项，用于配置数组、结构体或常量表。
1168	(index == 0) ? "ff9900" : "808080",	初始化表中的一项，用于配置数组、结构体或常量表。
1169	mqtt_history[index].payload,	初始化表中的一项，用于配置数组、结构体或常量表。
1170	(i > 1) ? "\n\n" : "");	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
1171		空行，用来分隔逻辑块，让代码更容易阅读。

行	源码	说明
1172	<code>if (written < 0)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
1173	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
1174	<code>break;</code>	跳出当前循环或 switch 分支。
1175	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
1176		空行，用来分隔逻辑块，让代码更容易阅读。
1177	<code>if ((rt_size_t)written >= size - used)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
1178	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
1179	<code>buf[size - 1] = '\0';</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
1180	<code>break;</code>	跳出当前循环或 switch 分支。
1181	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
1182		空行，用来分隔逻辑块，让代码更容易阅读。
1183	<code>used += (rt_size_t)written;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
1184	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
1185	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
1186		空行，用来分隔逻辑块，让代码更容易阅读。
1187	<code>rt_uint32_t my_mqtt_get_message_seq(void)</code>	声明函数 <code>my_mqtt_get_message_seq</code> ，告诉编译器后面会有这个接口。
1188	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
1189	<code>return mqtt_message_seq;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
1190	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
1191		空行，用来分隔逻辑块，让代码更容易阅读。
1192	<code>rt_size_t my_mqtt_get_history_count(void)</code>	声明函数 <code>my_mqtt_get_history_count</code> ，告诉编译器后面会有这个接口。
1193	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
1194	<code>return mqtt_history_used;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
1195	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
1196		空行，用来分隔逻辑块，让代码更容易阅读。
1197	<code>rt_bool_t my_mqtt_get_history_item(rt_size_t display_index,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
1198	<code>char *topic, rt_size_t topic_size,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
1199	<code>char *payload, rt_size_t payload_size,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
1200	<code>rt_bool_t *is_new)</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。

行	源码	说明
1201	{	代码块开始, 进入 当前结构 的内部逻辑。
1202	rt_size_t index;	普通 C 语句或声明, 服务于当前函数/模块的局部逻辑。
1203		空行, 用来分隔逻辑块, 让代码更容易阅读。
1204	if (display_index >= mqtt_history_used)	条件判断: 只有条件成立时才执行后面的分支, 常用于参数检查、错误处理或状态切换。
1205	{	代码块开始, 进入 当前结构 的内部逻辑。
1206	return RT_FALSE;	从当前函数返回结果; 错误码或状态值会交给调用者判断。
1207	}	代码块结束, 离开 当前结构 的内部逻辑。
1208		空行, 用来分隔逻辑块, 让代码更容易阅读。
1209	index = display_index;	赋值语句: 更新变量、结构体字段或缓冲区内容, 为后续逻辑准备数据。
1210	mqtt_copy_text(topic, topic_size, mqtt_history[index].topic,	初始化表中的一项, 用于配置数组、结构体或常量表。
1211	rt_strlen(mqtt_history[index].topic));	函数调用语句: 执行一个已有接口, 可能完成硬件操作、对象创建、字符串处理或状态更新。
1212	mqtt_copy_text(payload, payload_size, mqtt_history[index].payload,	初始化表中的一项, 用于配置数组、结构体或常量表。
1213	rt_strlen(mqtt_history[index].payload));	函数调用语句: 执行一个已有接口, 可能完成硬件操作、对象创建、字符串处理或状态更新。
1214		空行, 用来分隔逻辑块, 让代码更容易阅读。
1215	if (is_new != RT_NULL)	条件判断: 只有条件成立时才执行后面的分支, 常用于参数检查、错误处理或状态切换。
1216	{	代码块开始, 进入 当前结构 的内部逻辑。
1217	*is_new = (index == 0) ? RT_TRUE : RT_FALSE;	赋值语句: 更新变量、结构体字段或缓冲区内容, 为后续逻辑准备数据。
1218	}	代码块结束, 离开 当前结构 的内部逻辑。
1219		空行, 用来分隔逻辑块, 让代码更容易阅读。
1220	return RT_TRUE;	从当前函数返回结果; 错误码或状态值会交给调用者判断。
1221	}	代码块结束, 离开 当前结构 的内部逻辑。
1222		空行, 用来分隔逻辑块, 让代码更容易阅读。
1223	void my_mqtt_get_pc_status(my_mqtt_pc_status_t *status)	声明函数 my_mqtt_get_pc_status, 告诉编译器后面会有这个接口。
1224	{	代码块开始, 进入 当前结构 的内部逻辑。
1225	if (status == RT_NULL)	条件判断: 只有条件成立时才执行后面的分支, 常用于参数检查、错误处理或状态切换。
1226	{	代码块开始, 进入 当前结构 的内部逻辑。
1227	return;	从当前函数返回结果; 错误码或状态值会交给调用者判断。
1228	}	代码块结束, 离开 当前结构 的内部逻辑。

行	源码	说明
1229		空行，用来分隔逻辑块，让代码更容易阅读。
1230	<code>*status = mqtt_pc_status;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
1231	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
1232		空行，用来分隔逻辑块，让代码更容易阅读。
1233	<code>static void mqtt_heartbeat_entry(void *parameter)</code>	声明函数 <code>mqtt_heartbeat_entry</code> ，告诉编译器后面会有这个接口。
1234	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
1235	<code>char payload[48];</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
1236		空行，用来分隔逻辑块，让代码更容易阅读。
1237	<code>(void)parameter;</code>	显式标记参数未使用，避免编译器产生未使用参数警告。
1238		空行，用来分隔逻辑块，让代码更容易阅读。
1239	<code>while (mqtt_notify_started)</code>	<code>while</code> 循环，只要条件成立就持续执行；线程主循环通常写成 <code>while (1)</code> 。
1240	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
1241	<code>rt_thread_mdelay(MQTT_HEARTBEAT_MS);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
1242		空行，用来分隔逻辑块，让代码更容易阅读。
1243	<code>if (!mqtt_notify_started !mqtt_notify_client.isconnected)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
1244	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
1245	<code>continue;</code>	跳过本轮循环剩余语句，直接进入下一轮。
1246	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
1247		空行，用来分隔逻辑块，让代码更容易阅读。
1248	<code>mqtt_heartbeat_seq++;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
1249	<code>rt_sprintf(payload, sizeof(payload), "hb=%u,tick=%u",</code>	初始化表中的一行，用于配置数组、结构体或常量表。
1250	<code>mqtt_heartbeat_seq,</code>	初始化表中的一行，用于配置数组、结构体或常量表。
1251	<code>(rt_uint32_t)rt_tick_get_millisecond());</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
1252		空行，用来分隔逻辑块，让代码更容易阅读。
1253	<code>if (paho_mqtt_publish(&mqtt_notify_client, QOS1,</code>	向 MQTT 主题发布消息。
1254	<code>MQTT_TOPIC_NOTIFY, payload) != RT_EOK)</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
1255	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
1256	<code>rt_kprintf("[mqtt] heartbeat publish failed\n");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
1257	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。

行	源码	说明
1258	}	代码块结束，离开 当前结构 的内部逻辑。
1259		空行，用来分隔逻辑块，让代码更容易阅读。
1260	mqtt_heartbeat_thread = RT_NULL;	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
1261	}	代码块结束，离开 当前结构 的内部逻辑。
1262		空行，用来分隔逻辑块，让代码更容易阅读。
1263	static void mqtt_start_heartbeat(void)	声明函数 mqtt_start_heartbeat，告诉编译器后面会有这个接口。
1264	{	代码块开始，进入 当前结构 的内部逻辑。
1265	if (mqtt_heartbeat_thread != RT_NULL)	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
1266	{	代码块开始，进入 当前结构 的内部逻辑。
1267	return;	从当前函数返回结果；错误码或状态值会交给调用者判断。
1268	}	代码块结束，离开 当前结构 的内部逻辑。
1269		空行，用来分隔逻辑块，让代码更容易阅读。
1270	mqtt_heartbeat_thread = rt_thread_create("mqtt_hb",	初始化表中的一项，用于配置数组、结构体或常量表。
1271	mqtt_heartbeat_entry,	初始化表中的一项，用于配置数组、结构体或常量表。
1272	RT_NULL,	初始化表中的一项，用于配置数组、结构体或常量表。
1273	MQTT_HEARTBEAT_STACK,	初始化表中的一项，用于配置数组、结构体或常量表。
1274	MQTT_HEARTBEAT_PRIO,	初始化表中的一项，用于配置数组、结构体或常量表。
1275	10);	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
1276	if (mqtt_heartbeat_thread == RT_NULL)	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
1277	{	代码块开始，进入 当前结构 的内部逻辑。
1278	rt_kprintf("[mqtt] heartbeat thread create failed\n");	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
1279	return;	从当前函数返回结果；错误码或状态值会交给调用者判断。
1280	}	代码块结束，离开 当前结构 的内部逻辑。
1281		空行，用来分隔逻辑块，让代码更容易阅读。
1282	rt_thread_startup(mqtt_heartbeat_thread);	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
1283	}	代码块结束，离开 当前结构 的内部逻辑。
1284		空行，用来分隔逻辑块，让代码更容易阅读。
1285	static void mqtt_notify_message_cb(MQTTClient *client, MessageData *msg_data)	声明函数 mqtt_notify_message_cb，告诉编译器后面会有这个接口。

行	源码	说明
1286	{	代码块开始，进入 当前结构 的内部逻辑。
1287	rt_bool_t is_pc_status;	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
1288	rt_bool_t is_notify;	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
1289	rt_bool_t is_heartbeat;	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
1290		空行，用来分隔逻辑块，让代码更容易阅读。
1291	(void)client;	显式标记参数未使用，避免编译器产生未使用参数警告。
1292		空行，用来分隔逻辑块，让代码更容易阅读。
1293	is_pc_status = mqtt_text_equals(msg_data->topicName->lenstring.data,	初始化表中的一项，用于配置数组、结构体或常量表。
1294	msg_data->topicName->lenstring.len,	初始化表中的一项，用于配置数组、结构体或常量表。
1295	MQTT_TOPIC_PC_STATUS);	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
1296	is_notify = mqtt_text_equals(msg_data->topicName->lenstring.data,	初始化表中的一项，用于配置数组、结构体或常量表。
1297	msg_data->topicName->lenstring.len,	初始化表中的一项，用于配置数组、结构体或常量表。
1298	MQTT_TOPIC_NOTIFY);	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
1299	is_heartbeat = is_notify &&	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
1300	mqtt_is_heartbeat_payload((const char *)msg_data->message->payload,	初始化表中的一项，用于配置数组、结构体或常量表。
1301	msg_data->message->payloadlen);	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
1302	if (is_pc_status)	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
1303	{	代码块开始，进入 当前结构 的内部逻辑。
1304	mqtt_handle_pc_status((const char *)msg_data->message->payload,	初始化表中的一项，用于配置数组、结构体或常量表。
1305	msg_data->message->payloadlen);	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
1306	}	代码块结束，离开 当前结构 的内部逻辑。
1307		空行，用来分隔逻辑块，让代码更容易阅读。
1308	if (!is_pc_status && !is_heartbeat &&	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
1309	!mqtt_handle_command(msg_data->topicName->lenstring.data,	初始化表中的一项，用于配置数组、结构体或常量表。
1310	msg_data->topicName->lenstring.len,	初始化表中的一项，用于配置数组、结构体或常量表。

行	源码	说明
1311	<code>(const char *)msg_data->message->payload,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
1312	<code>msg_data->message->payloadlen))</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
1313	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
1314	<code>char text[MQTT_PAYLOAD_SIZE];</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
1315	<code>rt_size_t used;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
1316		空行，用来分隔逻辑块，让代码更容易阅读。
1317	<code>mqtt_copy_text(text, sizeof(text),</code>	初始化表中的一项，用于配置数组、结构体或常量表。
1318	<code>msg_data->topicName->lenstring.data,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
1319	<code>msg_data->topicName->lenstring.len);</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
1320	<code>used = rt_strlen(text);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
1321	<code>if (used < sizeof(text) - 2)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
1322	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
1323	<code>text[used++] = ':';</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
1324	<code>text[used++] = ' ';</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
1325	<code>text[used] = '\\0';</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
1326	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
1327	<code>mqtt_copy_text(text + used, sizeof(text) - used,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
1328	<code>(const char *)msg_data->message->payload,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
1329	<code>msg_data->message->payloadlen);</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
1330	<code>mqtt_push_terminal_line("<", text);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
1331	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
1332		空行，用来分隔逻辑块，让代码更容易阅读。
1333	<code>rt_kprintf("[mqtt] topic: %.*s\\n",</code>	初始化表中的一项，用于配置数组、结构体或常量表。
1334	<code>msg_data->topicName->lenstring.len,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
1335	<code>msg_data->topicName->lenstring.data);</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。

行	源码	说明
1336	<code>rt_kprintf("[mqtt] payload: %.*s\n",</code>	初始化表中的一项，用于配置数组、结构体或常量表。
1337	<code>msg_data->message->payloadlen,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
1338	<code>(char *)msg_data->message->payload);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
1339	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
1340		空行，用来分隔逻辑块，让代码更容易阅读。
1341	<code>static void mqtt_notify_online_cb(MQTTClient *client)</code>	声明函数 <code>mqtt_notify_online_cb</code> ，告诉编译器后面会有这个接口。
1342	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
1343	<code>(void)client;</code>	显式标记参数未使用，避免编译器产生未使用参数警告。
1344	<code>rt_kprintf("[mqtt] online, subscribed: %s, %s, %s\n",</code>	初始化表中的一项，用于配置数组、结构体或常量表。
1345	<code>MQTT_TOPIC_NOTIFY, MQTT_TOPIC_PC_STATUS, MQTT_TOPIC_TODO);</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
1346	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
1347		空行，用来分隔逻辑块，让代码更容易阅读。
1348	<code>static void mqtt_notify_offline_cb(MQTTClient *client)</code>	声明函数 <code>mqtt_notify_offline_cb</code> ，告诉编译器后面会有这个接口。
1349	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
1350	<code>(void)client;</code>	显式标记参数未使用，避免编译器产生未使用参数警告。
1351	<code>rt_kprintf("[mqtt] offline\n");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
1352	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
1353		空行，用来分隔逻辑块，让代码更容易阅读。
1354	<code>int my_mqtt_start(void)</code>	声明函数 <code>my_mqtt_start</code> ，告诉编译器后面会有这个接口。
1355	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
1356	<code>int result;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
1357	<code>rt_size_t i;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
1358	<code>MQTTPacket_connectData condata = MQTTPacket_connectData_initializer;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
1359		空行，用来分隔逻辑块，让代码更容易阅读。
1360	<code>if (mqtt_notify_started)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
1361	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
1362	<code>rt_kprintf("[mqtt] already started\n");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。

行	源码	说明
1363	<code>return 0;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
1364	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
1365		空行，用来分隔逻辑块，让代码更容易阅读。
1366	<code>rt_memset(&mqtt_notify_client, 0, sizeof(mqtt_notify_client));</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
1367		空行，用来分隔逻辑块，让代码更容易阅读。
1368	<code>mqtt_notify_client.uri = MQTT_NOTIFY_URI;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
1369	<code>rt_memcpy(&mqtt_notify_client.condata, &condata, sizeof(condata));</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
1370	<code>mqtt_notify_client.condata.clientID.cstring = MQTT_NOTIFY_CLIENTID;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
1371	<code>mqtt_notify_client.condata.keepAliveInterval = 30;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
1372	<code>mqtt_notify_client.condata.cleansession = 1;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
1373		空行，用来分隔逻辑块，让代码更容易阅读。
1374	<code>mqtt_notify_client.buf_size = 1024;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
1375	<code>mqtt_notify_client.readbuf_size = 1024;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
1376	<code>mqtt_notify_client.buf = rt_calloc(1, mqtt_notify_client.buf_size);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
1377	<code>mqtt_notify_client.readbuf = rt_calloc(1, mqtt_notify_client.readbuf_size);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
1378	<code>if (mqtt_notify_client.buf == RT_NULL mqtt_notify_client.readbuf == RT_NULL)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
1379	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
1380	<code>rt_kprintf("[mqtt] no memory for buffers\n");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
1381	<code>if (mqtt_notify_client.buf)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
1382	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
1383	<code>rt_free(mqtt_notify_client.buf);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
1384	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
1385	<code>if (mqtt_notify_client.readbuf)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
1386	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
1387	<code>rt_free(mqtt_notify_client.readbuf);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
1388	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。

行	源码	说明
1389	<code>return -RT_ENOMEM;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
1390	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
1391		空行，用来分隔逻辑块，让代码更容易阅读。
1392	<code>mqtt_notify_client.online_callback = mqtt_notify_online_cb;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
1393	<code>mqtt_notify_client.offline_callback = mqtt_notify_offline_cb;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
1394	<code>for (i = 0; i < sizeof(mqtt_topics) / sizeof(mqtt_topics[0]); ++i)</code>	for 循环，按固定次数或范围重复执行，用于遍历数组、列表或填充缓冲区。
1395	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
1396	<code>mqtt_notify_client.messageHandlers[i].topicFilter = rt_strdup(mqtt_topics[i]);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
1397	<code>mqtt_notify_client.messageHandlers[i].callback = mqtt_notify_message_cb;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
1398	<code>mqtt_notify_client.messageHandlers[i].qos = QOS1;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
1399	<code>if (mqtt_notify_client.messageHandlers[i].topicFilter == RT_NULL)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
1400	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
1401	<code>rt_kprintf("[mqtt] no memory for topic\n");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
1402	<code>while (i > 0)</code>	while 循环，只要条件成立就持续执行；线程主循环通常写成 while (1)。
1403	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
1404	<code>i--;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
1405	<code>rt_free(mqtt_notify_client.messageHandlers[i].topicFilter);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
1406	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
1407	<code>rt_free(mqtt_notify_client.buf);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
1408	<code>rt_free(mqtt_notify_client.readbuf);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
1409	<code>rt_memset(&mqtt_notify_client, 0, sizeof(mqtt_notify_client));</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
1410	<code>return -RT_ENOMEM;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
1411	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
1412	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
1413		空行，用来分隔逻辑块，让代码更容易阅读。
1414	<code>rt_kprintf("[mqtt] connecting %s\n", MQTT_NOTIFY_URI);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。

行	源码	说明
1415	<code>result = paho_mqtt_start(&mqtt_notify_client);</code>	启动 Paho MQTT 线程/连接流程。
1416	<code>if (result != RT_EOK)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
1417	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
1418	<code>rt_kprintf("[mqtt] start failed: %d\n", result);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
1419	<code>rt_free(mqtt_notify_client.buf);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
1420	<code>rt_free(mqtt_notify_client.readbuf);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
1421	<code>for (i = 0; i < sizeof(mqtt_topics) / sizeof(mqtt_topics[0]); ++i)</code>	for 循环，按固定次数或范围重复执行，用于遍历数组、列表或填充缓冲区。
1422	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
1423	<code>rt_free(mqtt_notify_client.messageHandlers[i].topicFilter);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
1424	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
1425	<code>rt_memset(&mqtt_notify_client, 0, sizeof(mqtt_notify_client));</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
1426	<code>return result;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
1427	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
1428		空行，用来分隔逻辑块，让代码更容易阅读。
1429	<code>mqtt_notify_started = RT_TRUE;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
1430	<code>mqtt_start_heartbeat();</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
1431		空行，用来分隔逻辑块，让代码更容易阅读。
1432	<code>return 0;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
1433	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
1434		空行，用来分隔逻辑块，让代码更容易阅读。
1435	<code>static void mqtt_join_args(char *buf, rt_size_t size, int argc, char **argv, int start)</code>	声明函数 mqtt_join_args，告诉编译器后面会有这个接口。
1436	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
1437	<code>int i;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
1438	<code>rt_size_t used = 0;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
1439		空行，用来分隔逻辑块，让代码更容易阅读。
1440	<code>if (buf == RT_NULL size == 0)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
1441	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。

行	源码	说明
1442	<code>return;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
1443	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
1444	<code>buf[0] = '\0';</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
1445		空行，用来分隔逻辑块，让代码更容易阅读。
1446	<code>for (i = start; i < argc; ++i)</code>	for 循环，按固定次数或范围重复执行，用于遍历数组、列表或填充缓冲区。
1447	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
1448	<code>int written;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
1449		空行，用来分隔逻辑块，让代码更容易阅读。
1450	<code>if (used >= size - 1)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
1451	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
1452	<code>break;</code>	跳出当前循环或 switch 分支。
1453	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
1454	<code>written = rt_snprintf(buf + used, size - used, "%s%s",</code>	初始化表中的一项，用于配置数组、结构体或常量表。
1455	<code>used == 0 ? "" : " ", argv[i]);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
1456	<code>if (written <= 0)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
1457	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
1458	<code>break;</code>	跳出当前循环或 switch 分支。
1459	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
1460	<code>used += (rt_size_t)written;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
1461	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
1462	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
1463		空行，用来分隔逻辑块，让代码更容易阅读。
1464	<code>static int setting(int argc, char **argv)</code>	声明函数 setting，告诉编译器后面会有这个接口。
1465	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
1466	<code>char arg[MQTT_PAYLOAD_SIZE];</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
1467		空行，用来分隔逻辑块，让代码更容易阅读。
1468	<code>if (argc <= 1)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
1469	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
1470	<code>mqtt_handle_setting_cmd("list");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。

行	源码	说明
1471	<code>return 0;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
1472	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
1473		空行，用来分隔逻辑块，让代码更容易阅读。
1474	<code>mqtt_join_args(arg, sizeof(arg), argc, argv, 1);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
1475	<code>mqtt_handle_setting_cmd(arg);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
1476	<code>return 0;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
1477	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
1478	<code>MSH_CMD_EXPORT(setting, UI setting command);</code>	把 C 函数导出为 RT-Thread MSH 命令。
1479		空行，用来分隔逻辑块，让代码更容易阅读。
1480	<code>static int switch_cmd(int argc, char **argv)</code>	声明函数 <code>switch_cmd</code> ，告诉编译器后面会有这个接口。
1481	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
1482	<code>if (argc != 2)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
1483	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
1484	<code>rt_kprintf("Usage: switch <message home options settings about>\n");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
1485	<code>return -1;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
1486	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
1487		空行，用来分隔逻辑块，让代码更容易阅读。
1488	<code>mqtt_handle_switch_cmd(argv[1]);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
1489	<code>return 0;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
1490	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
1491	<code>MSH_CMD_EXPORT_ALIAS(switch_cmd, switch, UI switch command);</code>	把 C 函数导出为 RT-Thread MSH 命令。
1492	<code>static int mqtt_start(int argc, char **argv)</code>	声明函数 <code>mqtt_start</code> ，告诉编译器后面会有这个接口。
1493	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
1494	<code>(void)argc;</code>	显式标记参数未使用，避免编译器产生未使用参数警告。
1495	<code>(void)argv;</code>	显式标记参数未使用，避免编译器产生未使用参数警告。
1496		空行，用来分隔逻辑块，让代码更容易阅读。
1497	<code>return my_mqtt_start();</code>	声明函数 <code>my_mqtt_start</code> ，告诉编译器后面会有这个接口。
1498	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。

行	源码	说明
1499	MSH_CMD_EXPORT(mqtt_start, start mqtt subscribe);	把 C 函数导出为 RT-Thread MSH 命令。
1500		空行，用来分隔逻辑块，让代码更容易阅读。
1501	static int mqtt_stop(int argc, char **argv)	声明函数 mqtt_stop，告诉编译器后面会有这个接口。
1502	{	代码块开始，进入 当前结构 的内部逻辑。
1503	int result;	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
1504		空行，用来分隔逻辑块，让代码更容易阅读。
1505	(void)argc;	显式标记参数未使用，避免编译器产生未使用参数警告。
1506	(void)argv;	显式标记参数未使用，避免编译器产生未使用参数警告。
1507		空行，用来分隔逻辑块，让代码更容易阅读。
1508	if (!mqtt_notify_started)	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
1509	{	代码块开始，进入 当前结构 的内部逻辑。
1510	rt_kprintf("[mqtt] not started\n");	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
1511	return 0;	从当前函数返回结果；错误码或状态值会交给调用者判断。
1512	}	代码块结束，离开 当前结构 的内部逻辑。
1513		空行，用来分隔逻辑块，让代码更容易阅读。
1514	result = paho_mqtt_stop(&mqtt_notify_client);	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
1515	mqtt_notify_started = RT_FALSE;	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
1516		空行，用来分隔逻辑块，让代码更容易阅读。
1517	rt_kprintf("[mqtt] stop %s\n", result == RT_EOK ? "OK" : "failed");	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
1518		空行，用来分隔逻辑块，让代码更容易阅读。
1519	return result;	从当前函数返回结果；错误码或状态值会交给调用者判断。
1520	}	代码块结束，离开 当前结构 的内部逻辑。
1521	MSH_CMD_EXPORT(mqtt_stop, stop mqtt subscribe);	把 C 函数导出为 RT-Thread MSH 命令。
1522		空行，用来分隔逻辑块，让代码更容易阅读。
1523		空行，用来分隔逻辑块，让代码更容易阅读。
1524		空行，用来分隔逻辑块，让代码更容易阅读。
1525		空行，用来分隔逻辑块，让代码更容易阅读。
1526		空行，用来分隔逻辑块，让代码更容易阅读。
1527		空行，用来分隔逻辑块，让代码更容易阅读。
1528		空行，用来分隔逻辑块，让代码更容易阅读。

行	源码	说明
1529		空行，用来分隔逻辑块，让代码更容易阅读。
1530		空行，用来分隔逻辑块，让代码更容易阅读。
1531		空行，用来分隔逻辑块，让代码更容易阅读。

net_test.c

MSH 网络测试命令，提供 `udp_test` 发送 UDP 数据包验证网络链路。

本文件共 58 行。下面的表格左侧是原始源码，中间是行号，右侧是逐行说明。资源数组文件的数组数据较长，说明会重点解释其作为图片/字体资源的角色。

行	源码	说明
1	<code>#include <rtthread.h></code>	引入头文件 <rtthread.h>，让当前文件可以使用其中声明的类型、宏和函数。
2	<code>#include <sys/socket.h></code>	引入头文件 <sys/socket.h>，让当前文件可以使用其中声明的类型、宏和函数。
3	<code>#include <arpa/inet.h></code>	引入头文件 <arpa/inet.h>，让当前文件可以使用其中声明的类型、宏和函数。
4	<code>#include <stdlib.h></code>	引入头文件 <stdlib.h>，让当前文件可以使用其中声明的类型、宏和函数。
5	<code>#include <string.h></code>	引入头文件 <string.h>，让当前文件可以使用其中声明的类型、宏和函数。
6		空行，用来分隔逻辑块，让代码更容易阅读。
7	<code>static void udp_test(int argc, char **argv)</code>	声明函数 <code>udp_test</code> ，告诉编译器后面会有这个接口。
8	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
9	<code>int sock;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
10	<code>int ret;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
11	<code>int port;</code>	普通 C 语句或声明，服务于当前函数/模块的局部逻辑。
12	<code>struct sockaddr_in dst;</code>	定义结构体类型，把一组相关数据字段放在一起。
13	<code>const char *msg = "hello from rt-thread";</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
14		空行，用来分隔逻辑块，让代码更容易阅读。
15	<code>if (argc < 3)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
16	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
17	<code>rt_kprintf("usage: udp_test <ip> <port> [msg]\n");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
18	<code>return;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
19	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
20		空行，用来分隔逻辑块，让代码更容易阅读。
21	<code>port = atoi(argv[2]);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。

行	源码	说明
22	<code>if (port <= 0 port > 65535)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
23	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
24	<code>rt_kprintf("invalid port: %s\n", argv[2]);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
25	<code>return;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
26	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
27		空行，用来分隔逻辑块，让代码更容易阅读。
28	<code>if (argc >= 4)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
29	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
30	<code>msg = argv[3];</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
31	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
32		空行，用来分隔逻辑块，让代码更容易阅读。
33	<code>sock = socket(AF_INET, SOCK_DGRAM, 0);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
34	<code>if (sock < 0)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
35	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
36	<code>rt_kprintf("create udp socket failed\n");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
37	<code>return;</code>	从当前函数返回结果；错误码或状态值会交给调用者判断。
38	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
39		空行，用来分隔逻辑块，让代码更容易阅读。
40	<code>rt_memset(&dst, 0, sizeof(dst));</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
41	<code>dst.sin_family = AF_INET;</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
42	<code>dst.sin_port = htons(port);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
43	<code>dst.sin_addr.s_addr = inet_addr(argv[1]);</code>	赋值语句：更新变量、结构体字段或缓冲区内容，为后续逻辑准备数据。
44		空行，用来分隔逻辑块，让代码更容易阅读。
45	<code>ret = sendto(sock, msg, strlen(msg), 0,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
46	<code>(struct sockaddr *)&dst, sizeof(dst));</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
47	<code>if (ret < 0)</code>	条件判断：只有条件成立时才执行后面的分支，常用于参数检查、错误处理或状态切换。
48	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。

行	源码	说明
49	<code>rt_kprintf("udp send failed\n");</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
50	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
51	<code>else</code>	条件判断的兜底分支，前面的条件都不满足时执行。
52	<code>{</code>	代码块开始，进入 当前结构 的内部逻辑。
53	<code>rt_kprintf("udp send %d bytes to %s:%d\n", ret, argv[1], port);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
54	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
55		空行，用来分隔逻辑块，让代码更容易阅读。
56	<code>closesocket(sock);</code>	函数调用语句：执行一个已有接口，可能完成硬件操作、对象创建、字符串处理或状态更新。
57	<code>}</code>	代码块结束，离开 当前结构 的内部逻辑。
58	<code>MSH_CMD_EXPORT(udp_test, send udp packet: udp_test <ip> <port> [msg]);</code>	把 C 函数导出为 RT-Thread MSH 命令。

ui_config.c

兼容保留文件。语言切换移除后仍保留，避免旧工程文件列表引用它时构建失败。

本文件共 4 行。下面的表格左侧是原始源码，中间是行号，右侧是逐行说明。资源数组文件的数组数据较长，说明会重点解释其作为图片/字体资源的角色。

行	源码	说明
1	/*	块注释开始/内容，解释接下来代码为什么这样写。
2	* UI language switching was removed. This file is kept so generated	块注释内容，属于说明文字，不会被编译进程序逻辑。
3	* RT-Thread Studio makefiles that still list ui_config.c continue to build.	块注释内容，属于说明文字，不会被编译进程序逻辑。
4	*/	块注释结束。

avatar_img.c

LVGL 图片资源，把头像位图转换成 C 数组和 lv_img_dsc_t 描述符。

本文件共 217 行。下面的表格左侧是原始源码，中间是行号，右侧是逐行说明。资源数组文件的数组数据较长，说明会重点解释其作为图片/字体资源的角色。

行	源码	说明
1	#include "lvgl.h"	引入头文件 "lvgl.h"，让当前文件可以使用其中声明的类型、宏和函数。
2		空行，用来分隔逻辑块，让代码更容易阅读。
3	#ifndef LV_ATTRIBUTE_IMG_HOME_AVATAR	条件编译开始：只有配置宏满足时，下面这段代码才会参与编译。
4	#define LV_ATTRIBUTE_IMG_HOME_AVATAR	宏定义，用名字替代固定值，方便统一修改和表达含义。
5	#endif	条件编译结束。
6		空行，用来分隔逻辑块，让代码更容易阅读。
7	const LV_ATTRIBUTE_MEM_ALIGN LV_ATTRIBUTE_LARGE_CONST LV_ATTRIBUTE_IMG_HOME_AVATAR uint8_t home_avatar_map[] = {	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
8	0x75, 0xFC, 0x6D, 0xBC, 0x75, 0xFC, 0x8E, 0x5D, 0xBF, 0x1E, 0xE7, 0x9F, 0xE7, 0x9F, 0xCF, 0x3E,	初始化表中的一项，用于配置数组、结构体或常量表。
9	0xB6, 0xDD, 0xAE, 0xDD, 0x86, 0x3C, 0x75, 0xDB, 0x75, 0xFC, 0x7E, 0x1C, 0x7E, 0x1C, 0x75, 0xFC,	初始化表中的一项，用于配置数组、结构体或常量表。
10	0x75, 0xFC, 0x75, 0xFC, 0x86, 0x3C, 0x9E, 0x9D, 0xD7, 0x5E, 0xB6, 0xDC, 0x8D, 0xF9, 0x5D, 0x15,	初始化表中的一项，用于配置数组、结构体或常量表。
11	0x75, 0x77, 0xA6, 0x7B, 0xAE, 0x9C, 0x85, 0xF9, 0x5D, 0x36, 0x5D, 0x36, 0x6D, 0x57, 0x65, 0x36,	初始化表中的一项，用于配置数组、结构体或常量表。
12	0x5D, 0x36, 0x54, 0xD5, 0x4C, 0x73, 0x4C, 0x32, 0x54, 0x93, 0x6D, 0x14, 0x64, 0xF3, 0x64, 0xB3,	初始化表中的一项，用于配置数组、结构体或常量表。
13	0x86, 0x1C, 0x65, 0xBB, 0x75, 0xFC, 0x9E, 0x9D, 0xC7, 0x3E, 0xE7, 0xBF, 0xDF, 0x9E, 0xB6, 0xFD,	初始化表中的一项，用于配置数组、结构体或常量表。
14	0xA6, 0xBD, 0xAE, 0xBD, 0x8E, 0x3C, 0x7E, 0x1C, 0x7E, 0x1C, 0x7D, 0xFC, 0x7D, 0xFC, 0x7D, 0xFB,	初始化表中的一项，用于配置数组、结构体或常量表。
15	0x7E, 0x1B, 0x7E, 0x1C, 0x8E, 0x3C, 0x9E, 0x7C, 0xD7, 0x5E, 0xBE, 0xFC, 0xA6, 0x9A, 0x7D, 0xB8,	初始化表中的一项，用于配置数组、结构体或常量表。
16	0x96, 0x3A, 0xB6, 0xBC, 0x96, 0x19, 0x65, 0x36, 0x65, 0x56, 0x75, 0xB8, 0x85, 0xF9, 0x65, 0x56,	初始化表中的一项，用于配置数组、结构体或常量表。
17	0x54, 0xF5, 0x5D, 0x15, 0x54, 0xB4, 0x4C, 0x93, 0x54, 0x93, 0x54, 0x92, 0x64, 0xB3, 0x6C, 0xF3,	初始化表中的一项，用于配置数组、结构体或常量表。
18	0xAE, 0x9B, 0xA6, 0x7C, 0x9E, 0x7C, 0xAE, 0xBC, 0xC7, 0x1D, 0xEF, 0xBF, 0xEF, 0xBF, 0xC7, 0x3D,	初始化表中的一项，用于配置数组、结构体或常量表。
19	0x9E, 0x9C, 0x96, 0x5C, 0x96, 0x5C, 0x8E, 0x5C, 0x86, 0x1B, 0x7D, 0xFB, 0x86, 0x1C, 0x86, 0x3C,	初始化表中的一项，用于配置数组、结构体或常量表。
20	0x8E, 0x3B, 0xAE, 0xDD, 0xAE, 0xBC, 0xC7, 0x1C, 0xE7, 0x9E, 0xC7, 0x1C, 0xA6, 0x7A, 0x96, 0x39,	初始化表中的一项，用于配置数组、结构体或常量表。

行	源码	说明
21	0x9E, 0x5A, 0x9E, 0x3A, 0x85, 0xF8, 0x85, 0xF9, 0x9E, 0x5A, 0x8D, 0xF8, 0x8D, 0xF8, 0x7D, 0xD7,	初始化表中的一项，用于配置数组、结构体或常量表。
22	0x65, 0x55, 0x65, 0x35, 0x65, 0x35, 0x5C, 0xF4, 0x5C, 0xF4, 0x6D, 0x14, 0x7D, 0x55, 0x95, 0xB6,	初始化表中的一项，用于配置数组、结构体或常量表。
23	0xB6, 0x7A, 0xCF, 0x1C, 0xE7, 0x9E, 0xC7, 0x1C, 0xD7, 0x3D, 0xF7, 0xDF, 0xF7, 0xDF, 0xEF, 0xBE,	初始化表中的一项，用于配置数组、结构体或常量表。
24	0xB6, 0xBC, 0x8E, 0x1B, 0xA6, 0x9C, 0x9E, 0x7B, 0x86, 0x1B, 0x96, 0x5B, 0x8E, 0x3B, 0x8E, 0x3B,	初始化表中的一项，用于配置数组、结构体或常量表。
25	0xA6, 0x7C, 0xC6, 0xFC, 0xD7, 0x3C, 0xE7, 0x9D, 0xE7, 0x9E, 0xC6, 0xFB, 0x8D, 0xD7, 0x75, 0x96,	初始化表中的一项，用于配置数组、结构体或常量表。
26	0x6D, 0x56, 0x75, 0x97, 0x7D, 0xD7, 0x9E, 0x39, 0x9E, 0x39, 0x7D, 0xB7, 0x85, 0xD7, 0x8D, 0xF7,	初始化表中的一项，用于配置数组、结构体或常量表。
27	0x7D, 0xB6, 0x7D, 0x96, 0x8D, 0xD6, 0x65, 0x34, 0x6D, 0x54, 0x95, 0xF7, 0xA6, 0x16, 0x7C, 0xB0,	初始化表中的一项，用于配置数组、结构体或常量表。
28	0xB6, 0x9A, 0xD7, 0x5D, 0xEF, 0xDE, 0xEF, 0x9E, 0xF7, 0xBE, 0xF7, 0xDE, 0xE7, 0x7D, 0xDF, 0x7E,	初始化表中的一项，用于配置数组、结构体或常量表。
29	0xBE, 0xDC, 0x96, 0x3B, 0xAE, 0xBC, 0xD7, 0x3D, 0xD7, 0x5D, 0xD7, 0x5D, 0xCF, 0x1D, 0xCF, 0x3D,	初始化表中的一项，用于配置数组、结构体或常量表。
30	0xAE, 0x9A, 0xD7, 0x3C, 0xEF, 0x9D, 0xE7, 0x9D, 0xDF, 0x7D, 0xCF, 0x1B, 0x9E, 0x58, 0x7D, 0xB6,	初始化表中的一项，用于配置数组、结构体或常量表。
31	0x85, 0xB6, 0x75, 0x76, 0x6D, 0x55, 0x7D, 0x96, 0x85, 0xB6, 0x8D, 0xF7, 0x8D, 0xD6, 0x7D, 0x96,	初始化表中的一项，用于配置数组、结构体或常量表。
32	0x7D, 0x95, 0x85, 0xB6, 0x8D, 0xD6, 0x7D, 0x75, 0x8D, 0xB5, 0xAE, 0x58, 0x85, 0x11, 0x4B, 0x29,	初始化表中的一项，用于配置数组、结构体或常量表。
33	0xC6, 0xDB, 0xD7, 0x3C, 0xDF, 0x5C, 0xEF, 0x9D, 0xFF, 0xDF, 0xFF, 0xDF, 0xD7, 0x3C, 0xC7, 0x1C,	初始化表中的一项，用于配置数组、结构体或常量表。
34	0xBE, 0xDC, 0xB6, 0xBB, 0xDF, 0x5D, 0xF7, 0xBE, 0xF7, 0xDF, 0xEF, 0xBF, 0xDF, 0x7D, 0xCF, 0x1C,	初始化表中的一项，用于配置数组、结构体或常量表。
35	0xBE, 0xBA, 0xEF, 0x9D, 0xE7, 0x7C, 0xDF, 0x5C, 0xD7, 0x1B, 0xD7, 0x3B, 0xC6, 0xDA, 0xB6, 0x99,	初始化表中的一项，用于配置数组、结构体或常量表。
36	0xC6, 0xFB, 0x9E, 0x38, 0x75, 0x75, 0x75, 0x75, 0x75, 0x75, 0x85, 0xB6, 0x8D, 0xD6, 0x85, 0x95,	初始化表中的一项，用于配置数组、结构体或常量表。
37	0x7D, 0x95, 0x75, 0x54, 0x85, 0x95, 0x8D, 0xD6, 0x95, 0xD6, 0x9D, 0xF5, 0x64, 0x0D, 0x53, 0x8A,	初始化表中的一项，用于配置数组、结构体或常量表。
38	0xCF, 0x1B, 0xDF, 0x7C, 0xE7, 0x7C, 0xEF, 0x7D, 0xF7, 0x9D, 0xFF, 0xDE, 0xF7, 0xBE, 0xEF, 0x9D,	初始化表中的一项，用于配置数组、结构体或常量表。
39	0xDF, 0x3C, 0xDF, 0x7D, 0xEF, 0xBE, 0xF7, 0xDF, 0xF7, 0xDF, 0xEF, 0xBE, 0xC6, 0xDB, 0xA6, 0x38,	初始化表中的一项，用于配置数组、结构体或常量表。
40	0xB6, 0x99, 0xC6, 0xDA, 0xE7, 0x5C, 0xE7, 0x5C, 0xCE, 0xFA, 0xCF, 0x1A, 0xD7, 0x3B, 0xD7, 0x3B,	初始化表中的一项，用于配置数组、结构体或常量表。
41	0xE7, 0x7D, 0xD7, 0x3B, 0x7D, 0x95, 0x6D, 0x13, 0x7D, 0x74, 0x8D, 0xB5, 0x8D, 0xB5, 0x95, 0xD6,	初始化表中的一项，用于配置数组、结构体或常量表。
42	0x9D, 0xF6, 0x7D, 0x54, 0x75, 0x13, 0x7D, 0x11, 0x64, 0x4E, 0x5C, 0x0C, 0x64, 0x2C, 0x6C, 0x8D,	初始化表中的一项，用于配置数组、结构体或常量表。
43	0xC6, 0x99, 0xDF, 0x3B, 0xEF, 0xBD, 0xF7, 0xDE, 0xF7, 0xBE, 0xFF, 0xDE, 0xE7, 0x7D, 0xDF, 0x3C,	初始化表中的一项，用于配置数组、结构体或常量表。

行	源码	说明
44	0xE7, 0x7D, 0xE7, 0x5D, 0xE7, 0x7D, 0xEF, 0xBE, 0xE7, 0x7D, 0xDF, 0x5C, 0xC6, 0xD9, 0xAE, 0x58,	初始化表中的一项，用于配置数组、结构体或常量表。
45	0xAE, 0x37, 0xB6, 0x78, 0xEF, 0x7C, 0xF7, 0xDE, 0xE7, 0x7C, 0xE7, 0x5B, 0xE7, 0x7C, 0xE7, 0x7C,	初始化表中的一项，用于配置数组、结构体或常量表。
46	0xE7, 0x7C, 0xDF, 0x5B, 0x9D, 0xF6, 0x95, 0xD6, 0xCE, 0xDA, 0xD7, 0x1A, 0xBE, 0xB8, 0xCE, 0xF9,	初始化表中的一项，用于配置数组、结构体或常量表。
47	0x95, 0x73, 0x53, 0x6B, 0x53, 0xEC, 0x64, 0x2C, 0x64, 0x4C, 0x6C, 0x8D, 0x74, 0xAD, 0x6C, 0x4C,	初始化表中的一项，用于配置数组、结构体或常量表。
48	0xAD, 0xF7, 0xBE, 0x78, 0xD6, 0xFA, 0xE7, 0x5C, 0xEF, 0x9D, 0xEF, 0x9C, 0xB6, 0x78, 0xA6, 0x38,	初始化表中的一项，用于配置数组、结构体或常量表。
49	0xCF, 0x1B, 0xD7, 0x1B, 0xDF, 0x3C, 0xE7, 0x9D, 0xE7, 0x5C, 0xDF, 0x5B, 0xD7, 0x3A, 0xC6, 0xD9,	初始化表中的一项，用于配置数组、结构体或常量表。
50	0xCE, 0xD9, 0xF7, 0xBC, 0xF7, 0xDE, 0xFF, 0xFF, 0xF7, 0xDE, 0xEF, 0x9C, 0xE7, 0x7B, 0xEF, 0x7C,	初始化表中的一项，用于配置数组、结构体或常量表。
51	0xEF, 0x9C, 0xEF, 0x9C, 0xE7, 0x5B, 0xEF, 0xBD, 0xE7, 0x7C, 0xBE, 0x37, 0x8D, 0x11, 0x63, 0xEC,	初始化表中的一项，用于配置数组、结构体或常量表。
52	0x4B, 0x4A, 0x4B, 0x29, 0x64, 0x0C, 0x6C, 0x6C, 0x6C, 0x6C, 0x6C, 0x5B, 0xCB, 0x53, 0x6A, 0x53, 0x6A,	初始化表中的一项，用于配置数组、结构体或常量表。
53	0x8D, 0x53, 0xAE, 0x37, 0xB6, 0x37, 0xB6, 0x37, 0xBE, 0x78, 0xBE, 0x78, 0xA5, 0xD5, 0x8D, 0x74,	初始化表中的一项，用于配置数组、结构体或常量表。
54	0xA5, 0xF6, 0xC6, 0x98, 0xE7, 0x5C, 0xEF, 0x9D, 0xF7, 0xBD, 0xEF, 0x9C, 0xE7, 0x5B, 0xEF, 0x7B,	初始化表中的一项，用于配置数组、结构体或常量表。
55	0xD6, 0xF9, 0xBE, 0x36, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xDE, 0xF7, 0xBC, 0xF7, 0xBC, 0xEF, 0x9C,	初始化表中的一项，用于配置数组、结构体或常量表。
56	0xEF, 0x7C, 0xEF, 0x9C, 0xEF, 0x9D, 0xBE, 0x57, 0x64, 0x0D, 0x4B, 0x8A, 0x43, 0x29, 0x4B, 0x69,	初始化表中的一项，用于配置数组、结构体或常量表。
57	0x5B, 0xEB, 0x53, 0x8A, 0x63, 0xEB, 0x64, 0x0C, 0x63, 0xEB, 0x5B, 0x8A, 0x6C, 0x2D, 0x6C, 0x2D,	初始化表中的一项，用于配置数组、结构体或常量表。
58	0x7C, 0xD1, 0x95, 0x74, 0xB6, 0x37, 0xBE, 0x57, 0xB6, 0x36, 0xB6, 0x16, 0xAD, 0xF5, 0x9D, 0xB5,	初始化表中的一项，用于配置数组、结构体或常量表。
59	0xA5, 0xB4, 0xC6, 0x97, 0xE7, 0x5B, 0xEF, 0x9C, 0xF7, 0xBC, 0xF7, 0x9C, 0xEF, 0x7B, 0xFF, 0xFE,	初始化表中的一项，用于配置数组、结构体或常量表。
60	0xBE, 0x16, 0x3A, 0x67, 0xCE, 0x98, 0xF7, 0xBE, 0xFF, 0xFF, 0xEF, 0x9C, 0xDE, 0xFA, 0xFF, 0xDD,	初始化表中的一项，用于配置数组、结构体或常量表。
61	0xE7, 0x5B, 0xEF, 0x9D, 0xCE, 0xB8, 0x4B, 0x49, 0x4B, 0x69, 0x53, 0x89, 0x53, 0x6A, 0x6C, 0x0C,	初始化表中的一项，用于配置数组、结构体或常量表。
62	0x5B, 0xAB, 0x5B, 0xAA, 0x6C, 0x2D, 0x6C, 0x0C, 0x63, 0xEB, 0x63, 0xCB, 0x7C, 0x6E, 0x6B, 0xCC,	初始化表中的一项，用于配置数组、结构体或常量表。
63	0x6C, 0x70, 0x74, 0xB0, 0x9D, 0x93, 0xBE, 0x56, 0xC6, 0x77, 0xBE, 0x56, 0xB6, 0x16, 0xAD, 0xF5,	初始化表中的一项，用于配置数组、结构体或常量表。
64	0xAD, 0xF5, 0xCE, 0xB8, 0xE7, 0x19, 0xE7, 0x5A, 0xF7, 0xBC, 0xEF, 0x9B, 0xF7, 0xBB, 0xF7, 0xBD,	初始化表中的一项，用于配置数组、结构体或常量表。
65	0x84, 0x8F, 0x3A, 0x26, 0x4B, 0x2B, 0x64, 0x50, 0x7C, 0xB1, 0x84, 0xD2, 0x9D, 0x74, 0xF7, 0xDD,	初始化表中的一项，用于配置数组、结构体或常量表。
66	0xFF, 0xDD, 0xAD, 0xD4, 0x4B, 0x49, 0x53, 0x89, 0x64, 0x0B, 0x5B, 0xCB, 0x6C, 0x0C, 0x74, 0x2D,	初始化表中的一项，用于配置数组、结构体或常量表。

行	源码	说明
67	0x63, 0xAB, 0x6C, 0x0C, 0x5B, 0xAB, 0x63, 0xEC, 0x5B, 0x6A, 0x53, 0x49, 0x5B, 0x8B, 0x5B, 0x6A,	初始化表中的一项，用于配置数组、结构体或常量表。
68	0x7C, 0xB0, 0x7C, 0xB0, 0x84, 0xF1, 0x9D, 0x53, 0xB6, 0x15, 0xC6, 0x97, 0xC6, 0x76, 0xAD, 0xD4,	初始化表中的一项，用于配置数组、结构体或常量表。
69	0xBE, 0x35, 0xCE, 0xB7, 0xE7, 0x39, 0xEF, 0x59, 0xF7, 0x9A, 0xF7, 0xBA, 0xFF, 0xFD, 0x9C, 0xF0,	初始化表中的一项，用于配置数组、结构体或常量表。
70	0x29, 0xA4, 0x63, 0xCD, 0x5B, 0xAD, 0x53, 0xAD, 0x53, 0x4C, 0x53, 0x8C, 0x53, 0xCE, 0xD6, 0xD9,	初始化表中的一项，用于配置数组、结构体或常量表。
71	0xB5, 0xF5, 0x42, 0xA7, 0x42, 0xC7, 0x53, 0x89, 0x5B, 0xAA, 0x5B, 0xAB, 0x74, 0x4D, 0x63, 0xAB,	初始化表中的一项，用于配置数组、结构体或常量表。
72	0x6C, 0x0C, 0x74, 0x2D, 0x7C, 0x6E, 0x6B, 0xEC, 0x4A, 0xC7, 0x4A, 0xC8, 0x53, 0x29, 0x5B, 0x69,	初始化表中的一项，用于配置数组、结构体或常量表。
73	0x84, 0xD0, 0x8D, 0x11, 0x8D, 0x11, 0x84, 0xD0, 0x95, 0x31, 0xAD, 0xD3, 0xAD, 0x92, 0x52, 0xE8,	初始化表中的一项，用于配置数组、结构体或常量表。
74	0xDF, 0x18, 0xE7, 0x39, 0xE7, 0x59, 0xEF, 0x79, 0xF7, 0x9A, 0xFF, 0xBB, 0xFF, 0xFD, 0xCE, 0x76,	初始化表中的一项，用于配置数组、结构体或常量表。
75	0x29, 0xC4, 0x3A, 0x26, 0x21, 0x85, 0x29, 0xA5, 0x32, 0x47, 0x2A, 0x06, 0x29, 0xE6, 0x5B, 0x4A,	初始化表中的一项，用于配置数组、结构体或常量表。
76	0x3A, 0xA7, 0x42, 0xE8, 0x53, 0x4A, 0x4B, 0x4A, 0x42, 0xE8, 0x3A, 0x86, 0x42, 0xE8, 0x53, 0x48,	初始化表中的一项，用于配置数组、结构体或常量表。
77	0x53, 0x28, 0x74, 0x4D, 0x84, 0xAE, 0x63, 0xCB, 0x63, 0xAA, 0x63, 0x8A, 0x63, 0x8A, 0x74, 0x4C,	初始化表中的一项，用于配置数组、结构体或常量表。
78	0x8C, 0xF0, 0xB6, 0x33, 0xAD, 0xD2, 0xA5, 0x92, 0xAD, 0xD3, 0xD6, 0xF5, 0x8C, 0x8D, 0x3A, 0x25,	初始化表中的一项，用于配置数组、结构体或常量表。
79	0xBE, 0x34, 0xD6, 0xD7, 0xD6, 0x96, 0xFF, 0xFC, 0xFF, 0xFC, 0xDE, 0xD7, 0xBD, 0xF3, 0x95, 0x0F,	初始化表中的一项，用于配置数组、结构体或常量表。
80	0x3A, 0x85, 0x3A, 0x85, 0x42, 0xA5, 0x42, 0xE5, 0x42, 0xC5, 0x53, 0x66, 0x63, 0xE7, 0x64, 0x07,	初始化表中的一项，用于配置数组、结构体或常量表。
81	0x74, 0x48, 0x7C, 0xC9, 0x74, 0x88, 0x6C, 0x08, 0x7C, 0x89, 0x63, 0xC8, 0x4A, 0xE6, 0x42, 0xE6,	初始化表中的一项，用于配置数组、结构体或常量表。
82	0x31, 0xE4, 0x4A, 0xE7, 0x63, 0xC9, 0x84, 0x8B, 0x9D, 0x8E, 0x95, 0x2D, 0xA5, 0x6E, 0xBE, 0x10,	初始化表中的一项，用于配置数组、结构体或常量表。
83	0xB6, 0x53, 0xD6, 0xF4, 0xE7, 0x97, 0xF7, 0xB8, 0xF7, 0xB8, 0xF7, 0xB8, 0x6B, 0x49, 0x21, 0x83,	初始化表中的一项，用于配置数组、结构体或常量表。
84	0x5B, 0x29, 0x94, 0xEF, 0x73, 0xEB, 0xAD, 0xB1, 0x84, 0x8C, 0x53, 0x26, 0x42, 0xA4, 0x42, 0xA3,	初始化表中的一项，用于配置数组、结构体或常量表。
85	0x4A, 0xE4, 0x4B, 0x04, 0x53, 0xA6, 0x64, 0x27, 0x64, 0x27, 0x5B, 0xA5, 0x5B, 0xA5, 0x84, 0xE9,	初始化表中的一项，用于配置数组、结构体或常量表。
86	0x9D, 0x8A, 0xAD, 0xEA, 0xAD, 0xE9, 0x95, 0x27, 0x74, 0x05, 0x84, 0x86, 0x8D, 0x09, 0x95, 0x29,	初始化表中的一项，用于配置数组、结构体或常量表。
87	0x95, 0x08, 0xA5, 0x89, 0x9D, 0x29, 0x53, 0x06, 0x6B, 0xC8, 0xA5, 0x8B, 0xB5, 0xCC, 0x94, 0xCB,	初始化表中的一项，用于配置数组、结构体或常量表。
88	0xE7, 0x74, 0xEF, 0x95, 0xEF, 0x97, 0xFF, 0xF8, 0xFF, 0xFA, 0xDF, 0x16, 0x63, 0x69, 0x32, 0x04,	初始化表中的一项，用于配置数组、结构体或常量表。
89	0x4A, 0xE6, 0x53, 0x67, 0x4B, 0x45, 0x4B, 0x45, 0x4B, 0x65, 0x4B, 0x24, 0x4B, 0x25, 0x53, 0x66,	初始化表中的一项，用于配置数组、结构体或常量表。

行	源码	说明
90	0x4B, 0x05, 0x53, 0x25, 0x53, 0x25, 0x5B, 0xA6, 0x7C, 0xA9, 0x84, 0xC8, 0x63, 0xA5, 0x53, 0x23,	初始化表中的一项，用于配置数组、结构体或常量表。
91	0x7C, 0x67, 0x9D, 0x28, 0xB5, 0x88, 0xAD, 0x6A, 0x8C, 0xAA, 0x7B, 0xC4, 0x73, 0xA3, 0x9D, 0x08,	初始化表中的一项，用于配置数组、结构体或常量表。
92	0xBD, 0xEA, 0xB5, 0x8A, 0xB5, 0xAA, 0x5B, 0x26, 0x5B, 0x46, 0xBD, 0xE9, 0xD6, 0x6C, 0x84, 0x4A,	初始化表中的一项，用于配置数组、结构体或常量表。
93	0xEF, 0x94, 0xE7, 0x55, 0xF7, 0xB7, 0xFF, 0xF9, 0xBE, 0x12, 0x42, 0x86, 0x29, 0xC3, 0x32, 0x24,	初始化表中的一项，用于配置数组、结构体或常量表。
94	0x32, 0x04, 0x3A, 0x44, 0x42, 0xA5, 0x4B, 0x26, 0x5B, 0xC7, 0x64, 0x28, 0x5B, 0xA6, 0x53, 0x65,	初始化表中的一项，用于配置数组、结构体或常量表。
95	0x63, 0xE7, 0x5B, 0x85, 0x63, 0xA6, 0x5B, 0x85, 0x6B, 0xE6, 0x95, 0x28, 0x95, 0x08, 0x73, 0xE5,	初始化表中的一项，用于配置数组、结构体或常量表。
96	0x52, 0xE3, 0x8C, 0x66, 0xCE, 0x09, 0xD6, 0x69, 0xD6, 0x69, 0xDE, 0x8A, 0x8C, 0x25, 0x62, 0xE2,	初始化表中的一项，用于配置数组、结构体或常量表。
97	0xA5, 0x29, 0xAD, 0x6B, 0xA5, 0x0A, 0xC6, 0x0D, 0xCE, 0x2B, 0xDE, 0x8D, 0xFF, 0x50, 0xFF, 0xB0,	初始化表中的一项，用于配置数组、结构体或常量表。
98	0xEF, 0xB4, 0xFF, 0xF7, 0xDE, 0xF4, 0xBE, 0x12, 0x8C, 0xAD, 0x21, 0xC4, 0x3A, 0x46, 0x29, 0xE4,	初始化表中的一项，用于配置数组、结构体或常量表。
99	0x29, 0xE4, 0x29, 0xC4, 0x29, 0xC4, 0x29, 0xC4, 0x29, 0xE3, 0x42, 0xC5, 0x5B, 0xC7, 0x5B, 0xA7,	初始化表中的一项，用于配置数组、结构体或常量表。
100	0x63, 0xA6, 0x74, 0x28, 0x5B, 0x04, 0x5B, 0x04, 0x52, 0xC3, 0x6B, 0x64, 0xA5, 0x07, 0xB5, 0x87,	初始化表中的一项，用于配置数组、结构体或常量表。
101	0x84, 0x05, 0x6B, 0x23, 0xB5, 0x26, 0xE6, 0xA9, 0xE6, 0xE9, 0xEE, 0xE9, 0xE6, 0xEC, 0x7B, 0xA4,	初始化表中的一项，用于配置数组、结构体或常量表。
102	0x7B, 0xA5, 0xCE, 0x29, 0xD6, 0x8A, 0xDE, 0x6C, 0xE6, 0xCD, 0xEE, 0xEF, 0xEF, 0x0F, 0xE6, 0xED,	初始化表中的一项，用于配置数组、结构体或常量表。
103	0xFF, 0xF4, 0xAD, 0xB0, 0x63, 0x89, 0x42, 0x86, 0x42, 0x86, 0x32, 0x45, 0x32, 0x25, 0x32, 0x05,	初始化表中的一项，用于配置数组、结构体或常量表。
104	0x32, 0x25, 0x32, 0x05, 0x32, 0x25, 0x32, 0x04, 0x32, 0x24, 0x32, 0x45, 0x3A, 0x44, 0x63, 0xC9,	初始化表中的一项，用于配置数组、结构体或常量表。
105	0x64, 0x0B, 0x5B, 0xA8, 0x63, 0xC7, 0x73, 0xE7, 0x7B, 0xE7, 0x7B, 0xA4, 0x7B, 0xE4, 0x83, 0xE3,	初始化表中的一项，用于配置数组、结构体或常量表。
106	0x83, 0xC3, 0x6B, 0x22, 0x73, 0x62, 0x8C, 0x04, 0x9C, 0x85, 0xA4, 0xC5, 0xA5, 0x08, 0xAD, 0x27,	初始化表中的一项，用于配置数组、结构体或常量表。
107	0x94, 0x87, 0x94, 0x6A, 0xB5, 0x8C, 0xAD, 0x2A, 0x9C, 0xA7, 0x9C, 0xA8, 0xAD, 0x29, 0x9C, 0xA6,	初始化表中的一项，用于配置数组、结构体或常量表。
108	0x63, 0x6A, 0x32, 0x25, 0x21, 0xA4, 0x29, 0xC4, 0x29, 0xE4, 0x31, 0xE4, 0x31, 0xE4, 0x32, 0x04,	初始化表中的一项，用于配置数组、结构体或常量表。
109	0x3A, 0x87, 0x3A, 0xA7, 0x32, 0x46, 0x32, 0x25, 0x3A, 0x64, 0x4B, 0x26, 0x42, 0x85, 0x32, 0x04,	初始化表中的一项，用于配置数组、结构体或常量表。
110	0x32, 0x05, 0x3A, 0xA6, 0x53, 0x67, 0x7C, 0x4A, 0x9D, 0x0C, 0xBE, 0x30, 0xC6, 0x4D, 0xC6, 0x8F,	初始化表中的一项，用于配置数组、结构体或常量表。
111	0xB5, 0xCA, 0xAD, 0x47, 0xA5, 0x07, 0x9C, 0xC7, 0x8C, 0x26, 0x84, 0x05, 0x7B, 0xC5, 0x6B, 0x24,	初始化表中的一项，用于配置数组、结构体或常量表。
112	0x4A, 0x22, 0x52, 0x84, 0x63, 0x04, 0x73, 0x65, 0x6B, 0x66, 0x8C, 0x68, 0xA5, 0x29, 0x8C, 0x67,	初始化表中的一项，用于配置数组、结构体或常量表。

行	源码	说明
113	0x21, 0x63, 0x29, 0xC4, 0x29, 0xA4, 0x29, 0xA4, 0x29, 0xA4, 0x29, 0xA4, 0x29, 0xA3, 0x29, 0xA4,	初始化表中的一项，用于配置数组、结构体或常量表。
114	0x29, 0xC4, 0x32, 0x44, 0x42, 0xC5, 0x42, 0xE4, 0x42, 0xA4, 0x3A, 0x45, 0x29, 0xC4, 0x29, 0xC3,	初始化表中的一项，用于配置数组、结构体或常量表。
115	0x29, 0xA3, 0x29, 0xC3, 0x29, 0xC3, 0x31, 0xE3, 0x4A, 0x65, 0x7C, 0x29, 0x95, 0x0A, 0xA5, 0x8F,	初始化表中的一项，用于配置数组、结构体或常量表。
116	0xBE, 0x2F, 0xC6, 0x6E, 0xCE, 0x8E, 0x84, 0x47, 0x52, 0xA4, 0x63, 0x45, 0x5B, 0x05, 0x6B, 0x86,	初始化表中的一项，用于配置数组、结构体或常量表。
117	0x29, 0x62, 0x39, 0xE3, 0x5B, 0x25, 0x31, 0xA2, 0x31, 0x82, 0x52, 0xC4, 0x5B, 0x06, 0x4A, 0x65,	初始化表中的一项，用于配置数组、结构体或常量表。
118	0x29, 0x81, 0x21, 0x81, 0x29, 0xA1, 0x29, 0xC1, 0x29, 0xC1, 0x32, 0x01, 0x3A, 0x42, 0x42, 0x83,	初始化表中的一项，用于配置数组、结构体或常量表。
119	0x4A, 0xE4, 0x4B, 0x23, 0x53, 0x64, 0x53, 0xA4, 0x5B, 0xE5, 0x5B, 0xA7, 0x53, 0x46, 0x4A, 0xE4,	初始化表中的一项，用于配置数组、结构体或常量表。
120	0x4A, 0xE3, 0x53, 0x24, 0x5B, 0x85, 0x63, 0xC6, 0x84, 0x69, 0xB5, 0x71, 0xB5, 0xD2, 0xC6, 0x74,	初始化表中的一项，用于配置数组、结构体或常量表。
121	0xBD, 0xAD, 0xAD, 0xAD, 0x94, 0xE8, 0x7C, 0x05, 0x84, 0x68, 0x95, 0x09, 0x84, 0xA8, 0x8C, 0xE9,	初始化表中的一项，用于配置数组、结构体或常量表。
122	0x63, 0xA7, 0x53, 0x06, 0x7C, 0x69, 0x74, 0x29, 0x7C, 0x48, 0x7C, 0x68, 0x74, 0x07, 0x6B, 0x87,	初始化表中的一项，用于配置数组、结构体或常量表。
123	0x3A, 0x8C, 0x32, 0x49, 0x3A, 0x89, 0x32, 0x47, 0x3A, 0x67, 0x42, 0xE8, 0x4B, 0x09, 0x53, 0x49,	初始化表中的一项，用于配置数组、结构体或常量表。
124	0x5B, 0xAA, 0x5B, 0xAB, 0x5B, 0xCC, 0x5B, 0xCB, 0x5B, 0xAB, 0x63, 0xCC, 0x5B, 0x69, 0x63, 0xCA,	初始化表中的一项，用于配置数组、结构体或常量表。
125	0x7C, 0x6B, 0x8C, 0xEC, 0x8C, 0xCB, 0x7C, 0x49, 0x74, 0x2A, 0x7C, 0x4C, 0x7C, 0x0D, 0x9C, 0xD2,	初始化表中的一项，用于配置数组、结构体或常量表。
126	0x8C, 0x8E, 0x84, 0xAB, 0x7B, 0xEA, 0x7C, 0x0A, 0x73, 0xE8, 0x73, 0xE8, 0x7C, 0x8B, 0x9D, 0x6E,	初始化表中的一项，用于配置数组、结构体或常量表。
127	0xAD, 0xD1, 0xA5, 0x90, 0x95, 0x4D, 0x9D, 0x4C, 0x95, 0x0B, 0x8D, 0x0C, 0x84, 0xAA, 0x95, 0x0A,	初始化表中的一项，用于配置数组、结构体或常量表。
128	0x43, 0x73, 0x43, 0x52, 0x43, 0x72, 0x43, 0x51, 0x4B, 0x71, 0x4B, 0x92, 0x4B, 0xB2, 0x53, 0xD2,	初始化表中的一项，用于配置数组、结构体或常量表。
129	0x5B, 0xD2, 0x53, 0xF1, 0x5B, 0xF1, 0x5B, 0xF1, 0x5B, 0xF1, 0x5B, 0xF1, 0x64, 0x30, 0x6C, 0x50,	初始化表中的一项，用于配置数组、结构体或常量表。
130	0x74, 0x91, 0x84, 0xF1, 0x84, 0xF0, 0x7C, 0x8E, 0x7C, 0x4E, 0x7C, 0x4E, 0x73, 0xCD, 0x8C, 0x70,	初始化表中的一项，用于配置数组、结构体或常量表。
131	0x84, 0xB0, 0x95, 0x30, 0x8D, 0x0F, 0x8C, 0xEF, 0x84, 0x8D, 0x8C, 0xAD, 0x95, 0x2F, 0xA5, 0xB2,	初始化表中的一项，用于配置数组、结构体或常量表。
132	0xAD, 0xF3, 0xB5, 0xF3, 0xAD, 0xB2, 0xAD, 0xB1, 0xA5, 0xD2, 0xAD, 0xF2, 0xA5, 0xB1, 0xAD, 0xD0,	初始化表中的一项，用于配置数组、结构体或常量表。
133	0x75, 0x35, 0x64, 0xB3, 0x54, 0x11, 0x43, 0x70, 0x4B, 0x6F, 0x4B, 0x90, 0x53, 0xD0, 0x5C, 0x10,	初始化表中的一项，用于配置数组、结构体或常量表。
134	0x5C, 0x10, 0x5C, 0x30, 0x64, 0x50, 0x64, 0x70, 0x64, 0x90, 0x64, 0x71, 0x5C, 0x30, 0x5C, 0x0F,	初始化表中的一项，用于配置数组、结构体或常量表。
135	0x5B, 0xEF, 0x5C, 0x0F, 0x64, 0x2E, 0x64, 0x2D, 0x74, 0x8E, 0x9D, 0x74, 0x9D, 0x95, 0xB6, 0x17,	初始化表中的一项，用于配置数组、结构体或常量表。

行	源码	说明
136	0xAD, 0xF3, 0x95, 0x70, 0x8D, 0x2E, 0x84, 0xED, 0x84, 0xEE, 0x8D, 0x2D, 0x8D, 0x2C, 0x95, 0x6D,	初始化表中的一项，用于配置数组、结构体或常量表。
137	0x84, 0xCC, 0x7C, 0x6B, 0x95, 0x6D, 0x8D, 0x0C, 0x8D, 0x2D, 0x95, 0x2C, 0x8C, 0xEB, 0x8C, 0xCC,	初始化表中的一项，用于配置数组、结构体或常量表。
138	0x8E, 0x38, 0x8E, 0x37, 0x85, 0xF7, 0x64, 0x93, 0x53, 0xB1, 0x4B, 0x91, 0x53, 0x91, 0x53, 0xB2,	初始化表中的一项，用于配置数组、结构体或常量表。
139	0x53, 0xD1, 0x5B, 0xF1, 0x5C, 0x32, 0x64, 0x52, 0x5C, 0x52, 0x64, 0x54, 0x5C, 0x53, 0x64, 0x53,	初始化表中的一项，用于配置数组、结构体或常量表。
140	0x5C, 0x53, 0x5C, 0x33, 0x5C, 0x12, 0x5B, 0xF2, 0x64, 0x32, 0x7C, 0xF4, 0x85, 0x32, 0x8D, 0x54,	初始化表中的一项，用于配置数组、结构体或常量表。
141	0x8D, 0x75, 0x8D, 0x75, 0x8D, 0x94, 0x74, 0xB2, 0x64, 0x10, 0x74, 0x50, 0x6C, 0x4E, 0x74, 0x8F,	初始化表中的一项，用于配置数组、结构体或常量表。
142	0x53, 0x4C, 0x5B, 0x8C, 0x7C, 0x8E, 0x5B, 0x8C, 0x5B, 0x6C, 0x6C, 0x2D, 0x6C, 0x0D, 0x63, 0x8C,	初始化表中的一项，用于配置数组、结构体或常量表。
143	0x8E, 0x58, 0x8E, 0x58, 0x8E, 0x37, 0x8E, 0x57, 0x85, 0xF7, 0x7D, 0x76, 0x75, 0x36, 0x75, 0x15,	初始化表中的一项，用于配置数组、结构体或常量表。
144	0x74, 0xD5, 0x6C, 0x94, 0x64, 0x53, 0x64, 0x53, 0x64, 0x33, 0x6C, 0x74, 0x64, 0x75, 0x64, 0x54,	初始化表中的一项，用于配置数组、结构体或常量表。
145	0x64, 0x75, 0x6C, 0xB5, 0x6C, 0xD6, 0x74, 0xF6, 0x75, 0x16, 0x8D, 0xB8, 0x9E, 0x17, 0x9E, 0x39,	初始化表中的一项，用于配置数组、结构体或常量表。
146	0x95, 0xF8, 0x95, 0xD7, 0x95, 0xB7, 0x95, 0xB7, 0x7D, 0x15, 0x74, 0xB3, 0x74, 0x92, 0x6C, 0x51,	初始化表中的一项，用于配置数组、结构体或常量表。
147	0x63, 0xF0, 0x64, 0x10, 0x64, 0x10, 0x64, 0x10, 0x6C, 0x10, 0x74, 0x70, 0x7C, 0xB1, 0x74, 0x90,	初始化表中的一项，用于配置数组、结构体或常量表。
148	0x7C, 0x6D, 0x7C, 0x6E, 0x7C, 0x6D, 0x7C, 0x6D, 0x7C, 0x6D, 0x8D, 0x0F, 0x84, 0xCE, 0x84, 0xCF,	初始化表中的一项，用于配置数组、结构体或常量表。
149	0x7C, 0x6F, 0x95, 0x95, 0x9E, 0x9A, 0x9E, 0x5A, 0x9E, 0x7B, 0x9E, 0x9A, 0x9E, 0x5A, 0x85, 0x57,	初始化表中的一项，用于配置数组、结构体或常量表。
150	0x85, 0x78, 0x7D, 0x37, 0x7D, 0x38, 0x85, 0x78, 0x85, 0x78, 0x95, 0xD9, 0xB6, 0xFB, 0xB6, 0xFB,	初始化表中的一项，用于配置数组、结构体或常量表。
151	0xAE, 0xBB, 0xAE, 0xBB, 0xAE, 0xBA, 0xA6, 0x39, 0x8D, 0x77, 0x7D, 0x15, 0x7D, 0x15, 0x7C, 0xF4,	初始化表中的一项，用于配置数组、结构体或常量表。
152	0x74, 0xB3, 0x74, 0xB4, 0x7C, 0xD4, 0x7C, 0xD4, 0x74, 0xD4, 0x7C, 0xD3, 0x7C, 0xD4, 0x74, 0xB3,	初始化表中的一项，用于配置数组、结构体或常量表。
153	0x5A, 0x21, 0x62, 0x42, 0x6A, 0x85, 0x6A, 0x85, 0x62, 0x66, 0x62, 0x86, 0x6A, 0x84, 0x6A, 0xA3,	初始化表中的一项，用于配置数组、结构体或常量表。
154	0x6A, 0xC4, 0x6A, 0xE5, 0x95, 0x2F, 0x95, 0x51, 0x84, 0x2D, 0xA6, 0x58, 0xAF, 0x1D, 0xAF, 0x1E,	初始化表中的一项，用于配置数组、结构体或常量表。
155	0xAF, 0x3F, 0xA6, 0xFF, 0x9E, 0xBE, 0x9E, 0x5C, 0x96, 0x3C, 0xA6, 0x9C, 0xBF, 0x7D, 0xBF, 0x7D,	初始化表中的一项，用于配置数组、结构体或常量表。
156	0xB7, 0x3D, 0xB6, 0xFD, 0xB6, 0xFC, 0xAE, 0xBB, 0xA6, 0x5A, 0x8D, 0xB8, 0x85, 0x37, 0x7D, 0x16,	初始化表中的一项，用于配置数组、结构体或常量表。
157	0x7C, 0xF5, 0x7D, 0x16, 0x7D, 0x16, 0x7C, 0xF5, 0x7C, 0xF5, 0x7C, 0xF5, 0x7C, 0xF5, 0x7C, 0xF4,	初始化表中的一项，用于配置数组、结构体或常量表。
158	0x62, 0xE3, 0x52, 0x21, 0x51, 0xC2, 0x51, 0xC3, 0x62, 0x66, 0x62, 0xE6, 0x6B, 0x26, 0x73, 0x87,	初始化表中的一项，用于配置数组、结构体或常量表。

行	源码	说明
159	0x84, 0x09, 0x84, 0x08, 0x84, 0x06, 0x8C, 0x84, 0x9C, 0xE5, 0xB5, 0xC9, 0xB5, 0xEB, 0xBE, 0x70,	初始化表中的一项，用于配置数组、结构体或常量表。
160	0xB6, 0x73, 0xB6, 0xB5, 0xBF, 0x39, 0xC7, 0xBD, 0xBF, 0xDF, 0xBF, 0xBF, 0xBF, 0xDF, 0xC7, 0xDF,	初始化表中的一项，用于配置数组、结构体或常量表。
161	0xC7, 0xDF, 0xC7, 0xBF, 0xC7, 0x7F, 0xB7, 0x3E, 0xAE, 0xBD, 0x9E, 0x5B, 0x95, 0xFA, 0x8D, 0x99,	初始化表中的一项，用于配置数组、结构体或常量表。
162	0x8D, 0x78, 0x85, 0x37, 0x85, 0x16, 0x85, 0x15, 0x84, 0xF5, 0x84, 0xF5, 0x7C, 0xF5, 0x7C, 0xF4,	初始化表中的一项，用于配置数组、结构体或常量表。
163	0x6B, 0x25, 0x73, 0x26, 0x62, 0x22, 0x62, 0x02, 0x62, 0x22, 0x5A, 0x02, 0x59, 0xE1, 0x51, 0xC1,	初始化表中的一项，用于配置数组、结构体或常量表。
164	0x6A, 0x84, 0x73, 0x06, 0x72, 0xE5, 0x73, 0x24, 0x73, 0x44, 0x73, 0x42, 0x7B, 0x81, 0x94, 0x84,	初始化表中的一项，用于配置数组、结构体或常量表。
165	0xAD, 0x25, 0xAD, 0x26, 0x8C, 0x04, 0x94, 0x87, 0x94, 0x88, 0x94, 0xEA, 0x9D, 0x2D, 0xA5, 0x2E,	初始化表中的一项，用于配置数组、结构体或常量表。
166	0xA5, 0x70, 0xAD, 0xF3, 0xAE, 0x57, 0xAE, 0x99, 0xAE, 0xBA, 0xA6, 0x7B, 0x9E, 0x3B, 0x9E, 0x1A,	初始化表中的一项，用于配置数组、结构体或常量表。
167	0x95, 0xD9, 0x95, 0xB9, 0x8D, 0x78, 0x8D, 0x58, 0x8D, 0x58, 0x8D, 0x57, 0x85, 0x16, 0x85, 0x16,	初始化表中的一项，用于配置数组、结构体或常量表。
168	0x62, 0x43, 0x62, 0x02, 0x59, 0xE1, 0x6A, 0xA3, 0x59, 0xE0, 0x59, 0xE1, 0x61, 0xE1, 0x62, 0x21,	初始化表中的一项，用于配置数组、结构体或常量表。
169	0x62, 0x01, 0x6A, 0x42, 0x72, 0x83, 0x72, 0xA3, 0x72, 0x83, 0x72, 0x83, 0x62, 0x41, 0x5A, 0x40,	初始化表中的一项，用于配置数组、结构体或常量表。
170	0x62, 0xA1, 0x8C, 0x26, 0xA5, 0x48, 0x94, 0xA5, 0x94, 0x45, 0x94, 0x48, 0xA4, 0xE9, 0x83, 0x43,	初始化表中的一项，用于配置数组、结构体或常量表。
171	0x72, 0xE2, 0x72, 0xE2, 0x62, 0xC2, 0x6B, 0x42, 0x7B, 0xA3, 0x9D, 0x09, 0xAD, 0xCB, 0xA5, 0x4A,	初始化表中的一项，用于配置数组、结构体或常量表。
172	0x9D, 0x0A, 0x94, 0xEC, 0xA5, 0xAF, 0xA5, 0x6D, 0xA5, 0x6D, 0xA5, 0xAD, 0xAD, 0xCE, 0x95, 0x0C,	初始化表中的一项，用于配置数组、结构体或常量表。
173	0x6A, 0x41, 0x6A, 0x62, 0x7B, 0x05, 0xB5, 0x2B, 0x9C, 0x29, 0x7B, 0x04, 0x72, 0xA2, 0x6A, 0x41,	初始化表中的一项，用于配置数组、结构体或常量表。
174	0x6A, 0x61, 0x72, 0xA2, 0x7A, 0xE4, 0x83, 0x45, 0x72, 0x83, 0x62, 0x01, 0x6A, 0x62, 0x6A, 0xA2,	初始化表中的一项，用于配置数组、结构体或常量表。
175	0x62, 0x81, 0x5A, 0x20, 0x94, 0x23, 0xA5, 0x05, 0x94, 0x65, 0xA5, 0x27, 0xAD, 0x87, 0xAD, 0x67,	初始化表中的一项，用于配置数组、结构体或常量表。
176	0xAD, 0x68, 0xB5, 0xAA, 0xA5, 0x8D, 0xA5, 0x6C, 0xB5, 0xC8, 0xB5, 0xC7, 0xAD, 0xA6, 0xBE, 0x08,	初始化表中的一项，用于配置数组、结构体或常量表。
177	0xBE, 0x08, 0xB5, 0xE9, 0xAD, 0xE9, 0xB5, 0xC7, 0xB5, 0xE7, 0xB6, 0x08, 0xB6, 0x4C, 0xB5, 0xA7,	初始化表中的一项，用于配置数组、结构体或常量表。
178	0xCD, 0xEE, 0xDE, 0x50, 0xDE, 0x91, 0xDE, 0x70, 0xF7, 0x54, 0xEE, 0xF3, 0xDE, 0x50, 0xBD, 0x6F,	初始化表中的一项，用于配置数组、结构体或常量表。
179	0xBD, 0x6E, 0xCD, 0xEE, 0xCD, 0xCE, 0xEE, 0xF3, 0xBD, 0x2C, 0x93, 0xE8, 0xA4, 0x69, 0x8B, 0xA5,	初始化表中的一项，用于配置数组、结构体或常量表。
180	0x72, 0xE2, 0x6A, 0xA1, 0x83, 0x83, 0x9C, 0x65, 0x73, 0x02, 0x83, 0x84, 0x94, 0x65, 0x94, 0x65,	初始化表中的一项，用于配置数组、结构体或常量表。
181	0x94, 0x85, 0x94, 0xA7, 0x94, 0x67, 0x94, 0x86, 0xA5, 0x06, 0xA5, 0x06, 0xA5, 0x06, 0xAD, 0x88,	初始化表中的一项，用于配置数组、结构体或常量表。

行	源码	说明
182	0xB5, 0xE9, 0xB6, 0x0A, 0xB5, 0xE9, 0xB5, 0xE8, 0xB5, 0xE8, 0xB6, 0x4C, 0xBE, 0x6B, 0xB5, 0xC8,	初始化表中的一项，用于配置数组、结构体或常量表。
183	0xFF, 0xB7, 0xFF, 0x96, 0xFF, 0x96, 0xF7, 0x75, 0xF7, 0x54, 0xF7, 0x74, 0xFF, 0x96, 0xFF, 0xD9,	初始化表中的一项，用于配置数组、结构体或常量表。
184	0xFF, 0xB8, 0xFF, 0x98, 0xFF, 0xB8, 0xF7, 0x56, 0xFF, 0x97, 0xFF, 0xB8, 0xFF, 0xB7, 0xF7, 0x54,	初始化表中的一项，用于配置数组、结构体或常量表。
185	0xEE, 0xF2, 0xDE, 0x6F, 0xC5, 0x8B, 0xBD, 0x69, 0x9C, 0x25, 0x8B, 0xA4, 0x8B, 0x83, 0x8B, 0xC3,	初始化表中的一项，用于配置数组、结构体或常量表。
186	0x8B, 0xE4, 0x8B, 0xE4, 0x83, 0xE3, 0x8C, 0x46, 0x8C, 0x66, 0x84, 0x04, 0x84, 0x05, 0x8C, 0x25,	初始化表中的一项，用于配置数组、结构体或常量表。
187	0x9C, 0xA4, 0x84, 0x04, 0x94, 0x86, 0xA5, 0x48, 0xAD, 0xCA, 0xAE, 0x0D, 0xAD, 0x88, 0xB5, 0x89,	初始化表中的一项，用于配置数组、结构体或常量表。
188	0xEF, 0x36, 0xF7, 0x37, 0xF7, 0x76, 0xFF, 0x97, 0xF7, 0x13, 0xEE, 0xF3, 0xF7, 0x57, 0xFF, 0x98,	初始化表中的一项，用于配置数组、结构体或常量表。
189	0xFF, 0x77, 0xFF, 0x77, 0xF7, 0x57, 0xF7, 0x57, 0xF7, 0x36, 0xF7, 0x15, 0xEF, 0x36, 0xDE, 0xD4,	初始化表中的一项，用于配置数组、结构体或常量表。
190	0xC5, 0xD1, 0xC6, 0x14, 0xC6, 0x34, 0xBE, 0x12, 0xC6, 0x31, 0xBD, 0xF1, 0xBE, 0x12, 0xBD, 0xD2,	初始化表中的一项，用于配置数组、结构体或常量表。
191	0xB5, 0xD3, 0xA5, 0x52, 0xA5, 0xB3, 0x9D, 0x93, 0x95, 0x53, 0x7C, 0xD1, 0x7C, 0x4E, 0x7C, 0x0C,	初始化表中的一项，用于配置数组、结构体或常量表。
192	0x7B, 0xE8, 0x6B, 0x03, 0x72, 0xE1, 0x7B, 0x41, 0x83, 0xE3, 0x84, 0x05, 0x9D, 0x29, 0xA5, 0x8A,	初始化表中的一项，用于配置数组、结构体或常量表。
193	0xD6, 0x31, 0xC5, 0xD0, 0xEF, 0x36, 0xE7, 0x17, 0xE7, 0x18, 0xE7, 0x18, 0xDF, 0x18, 0xDF, 0x19,	初始化表中的一项，用于配置数组、结构体或常量表。
194	0xCE, 0xB6, 0xC6, 0x96, 0xC6, 0x96, 0xBE, 0x97, 0xBE, 0x76, 0xBE, 0x76, 0xBE, 0xB9, 0xB6, 0xBA,	初始化表中的一项，用于配置数组、结构体或常量表。
195	0xB6, 0xDC, 0xBF, 0x1D, 0xBF, 0x5F, 0xBF, 0x7F, 0xC7, 0x9F, 0xD7, 0xFF, 0xD7, 0xFF, 0xD7, 0xFF,	初始化表中的一项，用于配置数组、结构体或常量表。
196	0xD7, 0xFF, 0xD7, 0xFF, 0xCF, 0xFF, 0xB7, 0xBF, 0xAF, 0x5F, 0xA7, 0x1F, 0x9E, 0xDF, 0x96, 0x5D,	初始化表中的一项，用于配置数组、结构体或常量表。
197	0x8D, 0xBA, 0x85, 0x36, 0x74, 0x92, 0x74, 0x0F, 0x6B, 0xAB, 0x63, 0x28, 0x7C, 0x2C, 0x84, 0x8E,	初始化表中的一项，用于配置数组、结构体或常量表。
198	0xC6, 0x95, 0xC6, 0xB5, 0xB6, 0x14, 0xB6, 0x35, 0xBE, 0x98, 0xBE, 0x97, 0xB6, 0x77, 0xB6, 0x57,	初始化表中的一项，用于配置数组、结构体或常量表。
199	0xB6, 0x77, 0xB6, 0x97, 0xB6, 0xB7, 0xB6, 0xB8, 0xBE, 0xB9, 0xB6, 0xB9, 0xBE, 0xFA, 0xBE, 0xFB,	初始化表中的一项，用于配置数组、结构体或常量表。
200	0xBF, 0x1C, 0xC7, 0x3D, 0xC7, 0x3E, 0xC7, 0x5F, 0xCF, 0x9F, 0xD7, 0xDF, 0xDF, 0xDF, 0xDF, 0xDF,	初始化表中的一项，用于配置数组、结构体或常量表。
201	0xD7, 0xDF, 0xD7, 0xDF, 0xCF, 0xBE, 0xC7, 0x9E, 0xBF, 0x7E, 0xAF, 0x3E, 0xA6, 0xDE, 0x9E, 0x7D,	初始化表中的一项，用于配置数组、结构体或常量表。
202	0x96, 0x3B, 0x8D, 0xFA, 0x8D, 0xD9, 0x85, 0xB9, 0x7D, 0x77, 0x7D, 0x36, 0x6C, 0xB4, 0x64, 0x52,	初始化表中的一项，用于配置数组、结构体或常量表。
203	0x52, 0xA4, 0x5A, 0xE7, 0x63, 0x0A, 0x6B, 0x6B, 0x84, 0x8E, 0x8C, 0xF0, 0x73, 0xEB, 0x84, 0x8E,	初始化表中的一项，用于配置数组、结构体或常量表。
204	0x8D, 0x0F, 0x84, 0xEE, 0xA6, 0x15, 0xB6, 0x98, 0xB6, 0x98, 0xB6, 0xB8, 0xBE, 0xDA, 0xBE, 0xFB,	初始化表中的一项，用于配置数组、结构体或常量表。

行	源码	说明
205	<code>0xBF, 0x1C, 0xC7, 0x3D, 0xC7, 0x5E, 0xC7, 0x5F, 0xCF, 0x7F, 0xCF, 0xBF, 0xD7, 0xBF, 0xD7, 0xDF,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
206	<code>0xDF, 0xBF, 0xDF, 0xDF, 0xD7, 0xBF, 0xCF, 0xBE, 0xD7, 0xBF, 0xC7, 0x7E, 0xB7, 0x1E, 0xA6, 0xBE,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
207	<code>0x9E, 0x7C, 0x96, 0x1A, 0x8D, 0xD8, 0x85, 0x97, 0x7D, 0x56, 0x75, 0x35, 0x6C, 0xF4, 0x6C, 0xD4,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
208	<code>};</code>	结构体、枚举或初始化表结束。
209		空行，用来分隔逻辑块，让代码更容易阅读。
210	<code>const lv_img_dsc_t home_avatar = {</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
211	<code>.header.always_zero = 0,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
212	<code>.header.w = 40,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
213	<code>.header.h = 40,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
214	<code>.header.cf = LV_IMG_CF_TRUE_COLOR,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
215	<code>.data_size = 40 * 40 * LV_COLOR_SIZE / 8,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
216	<code>.data = home_avatar_map,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
217	<code>};</code>	结构体、枚举或初始化表结束。

ui_font_zh_14.c

LVGL 中文字体资源，把字形、unicode 映射和字体描述符编译进固件。

本文件共 677 行。下面的表格左侧是原始源码，中间是行号，右侧是逐行说明。资源数组文件的数组数据较长，说明会重点解释其作为图片/字体资源的角色。

行	源码	说明
1	<code>/* Generated by tools/make_lvgl_rom_font.py. */</code>	块注释开始/内容，解释接下来代码为什么这样写。
2		空行，用来分隔逻辑块，让代码更容易阅读。
3	<code>#ifdef LV_LVGL_H_INCLUDE_SIMPLE</code>	条件编译开始：只有配置宏满足时，下面这段代码才会参与编译。
4	<code>#include "lvgl.h"</code>	引入头文件 "lvgl.h"，让当前文件可以使用其中声明的类型、宏和函数。
5	<code>#else</code>	条件编译分支：当前一组配置条件的另一个选择。
6	<code>#include "lvgl.h"</code>	引入头文件 "lvgl.h"，让当前文件可以使用其中声明的类型、宏和函数。
7	<code>#endif</code>	条件编译结束。
8		空行，用来分隔逻辑块，让代码更容易阅读。
9	<code>static LV_ATTRIBUTE_LARGE_CONST const uint8_t glyph_bitmap[] = {</code>	定义当前文件私有的静态变量或函数，并给出初始值；外部文件不能直接访问。
10	<code>0x00, 0x00, 0x04, 0xe0, 0x00, 0x00, 0x00, 0x00, 0x04, 0xe0, 0x00,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
11	<code>0x00, 0x00, 0x00, 0x00, 0x04, 0xe0, 0x00, 0x00, 0x00, 0x00, 0x04,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
12	<code>0xe0, 0x00, 0x00, 0x00, 0x00, 0x00, 0x04, 0xff, 0xff, 0xff, 0x80, 0x00,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
13	<code>0x00, 0x04, 0xe0, 0x00, 0x00, 0x00, 0x00, 0x00, 0x04, 0xe0, 0x00, 0x00,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
14	<code>0x00, 0x00, 0x00, 0x04, 0xe0, 0x00, 0x00, 0x00, 0x00, 0x00, 0x04, 0xe0,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
15	<code>0x00, 0x00, 0x00, 0x00, 0x00, 0x04, 0xe0, 0x00, 0x00, 0x00, 0x00, 0x00,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
16	<code>0x04, 0xe0, 0x00, 0x00, 0x00, 0x9f, 0xff, 0xff, 0xff, 0xff, 0xff, 0xf9,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
17	<code>0x9f, 0xff, 0xff, 0xff, 0xff, 0xff, 0xf9, 0x00, 0x00, 0x03, 0xf0, 0x00,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
18	<code>0x00, 0x00, 0x00, 0x00, 0x03, 0xf0, 0x00, 0x00, 0x00, 0x00, 0x00, 0x03,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
19	<code>0xf1, 0x00, 0x00, 0x00, 0x00, 0x00, 0x03, 0xfc, 0xa1, 0x00, 0x00, 0x00,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
20	<code>0x00, 0x03, 0xf1, 0x9e, 0x60, 0x00, 0x00, 0x00, 0x03, 0xf0, 0x04, 0xda,</code>	初始化表中的一项，用于配置数组、结构体或常量表。

行	源码	说明
21	0x10, 0x00, 0x00, 0x03, 0xf0, 0x00, 0x09, 0x70, 0x00, 0x00, 0x03, 0xf0,	初始化表中的一项，用于配置数组、结构体或常量表。
22	0x00, 0x00, 0x00, 0x00, 0x00, 0x03, 0xf0, 0x00, 0x00, 0x00, 0x00, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
23	0x03, 0xf0, 0x00, 0x00, 0x00, 0x00, 0x00, 0x03, 0xf0, 0x00, 0x00, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
24	0x00, 0x00, 0x00, 0x88, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x88, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
25	0x00, 0x00, 0x0c, 0xff, 0xff, 0xff, 0xff, 0xff, 0xd0, 0x0c, 0x40, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
26	0x88, 0x00, 0x03, 0xd0, 0x0c, 0x40, 0x00, 0x88, 0x00, 0x03, 0xd0, 0x0c,	初始化表中的一项，用于配置数组、结构体或常量表。
27	0x40, 0x00, 0x88, 0x00, 0x03, 0xd0, 0x0c, 0x40, 0x00, 0x88, 0x00, 0x03,	初始化表中的一项，用于配置数组、结构体或常量表。
28	0xd0, 0x0c, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xd0, 0x0c, 0x40, 0x00, 0x88,	初始化表中的一项，用于配置数组、结构体或常量表。
29	0x00, 0x03, 0xd0, 0x00, 0x00, 0x00, 0x88, 0x00, 0x00, 0x00, 0x00, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
30	0x00, 0x88, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x88, 0x00, 0x00, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
31	0x09, 0xff, 0xff, 0xff, 0xff, 0xff, 0xa0, 0x00, 0x00, 0x00, 0x5b, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
32	0x00, 0x00, 0x00, 0x00, 0x00, 0x5b, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
33	0x5b, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x5b, 0x00, 0x00, 0x00, 0xaf,	初始化表中的一项，用于配置数组、结构体或常量表。
34	0xff, 0xff, 0xff, 0xff, 0xff, 0xfb, 0x00, 0x00, 0x00, 0x5b, 0x00, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
35	0x00, 0x00, 0x00, 0x00, 0x5b, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x5b,	初始化表中的一项，用于配置数组、结构体或常量表。
36	0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x5b, 0x00, 0x00, 0x00, 0x00, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
37	0x00, 0x7b, 0x00, 0x00, 0x00, 0x00, 0x06, 0xff, 0xd5, 0x00, 0x00, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
38	0x00, 0x00, 0x00, 0xcd, 0x00, 0x00, 0x00, 0x00, 0x00, 0x09, 0xaa, 0x90,	初始化表中的一项，用于配置数组、结构体或常量表。
39	0x00, 0x00, 0x00, 0x00, 0xab, 0x00, 0xb8, 0x00, 0x00, 0x00, 0x2c, 0x90,	初始化表中的一项，用于配置数组、结构体或常量表。
40	0x00, 0x0a, 0xa1, 0x00, 0x07, 0xd5, 0x03, 0xa0, 0x00, 0x6d, 0x60, 0x79,	初始化表中的一项，用于配置数组、结构体或常量表。
41	0x10, 0x00, 0xb9, 0x00, 0x01, 0xa7, 0x00, 0x00, 0x00, 0x1b, 0x10, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
42	0x00, 0x03, 0xff, 0xff, 0xff, 0xff, 0xfa, 0x00, 0x00, 0x01, 0x40, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
43	0x08, 0xc2, 0x00, 0x00, 0x04, 0xdc, 0x55, 0xd7, 0x00, 0x00, 0x00, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。

行	源码	说明
44	0x05, 0xdf, 0x50, 0x00, 0x00, 0x00, 0x00, 0x00, 0x07, 0xc1, 0x00, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
45	0x00, 0x0c, 0x20, 0x07, 0x90, 0x00, 0x00, 0x00, 0x5c, 0x3f, 0xff, 0xff,	初始化表中的一项，用于配置数组、结构体或常量表。
46	0xff, 0xf3, 0x00, 0xc6, 0x00, 0x1e, 0x00, 0x00, 0x00, 0x03, 0xf3, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
47	0x5b, 0x00, 0x00, 0x00, 0x0c, 0xe3, 0xdf, 0xff, 0xff, 0xff, 0xfb, 0x7a,	初始化表中的一项，用于配置数组、结构体或常量表。
48	0xc3, 0x01, 0xe2, 0x00, 0x00, 0x00, 0x71, 0xc3, 0x04, 0xff, 0xff, 0xff,	初始化表中的一项，用于配置数组、结构体或常量表。
49	0xd0, 0x00, 0xc3, 0x00, 0x00, 0x00, 0x1c, 0x70, 0x00, 0xc3, 0x06, 0x91,	初始化表中的一项，用于配置数组、结构体或常量表。
50	0x01, 0xc7, 0x00, 0x00, 0xc3, 0x00, 0x6d, 0x8c, 0x70, 0x00, 0x00, 0xc3,	初始化表中的一项，用于配置数组、结构体或常量表。
51	0x00, 0x01, 0x9e, 0x50, 0x00, 0x00, 0xc3, 0x00, 0x00, 0x03, 0xb2, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
52	0x00, 0x2b, 0x0f, 0xff, 0xff, 0xff, 0x80, 0x00, 0x77, 0x0e, 0x00, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
53	0x06, 0x80, 0x00, 0xd1, 0x0e, 0x00, 0x00, 0x06, 0x80, 0x05, 0xc0, 0x0e,	初始化表中的一项，用于配置数组、结构体或常量表。
54	0x00, 0x00, 0x06, 0x80, 0x0c, 0xc0, 0x0f, 0xff, 0xff, 0xff, 0x80, 0x7b,	初始化表中的一项，用于配置数组、结构体或常量表。
55	0xc0, 0x0e, 0x00, 0xb3, 0x06, 0x80, 0x73, 0xc0, 0x00, 0x00, 0xb3, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
56	0x00, 0x02, 0xc4, 0xff, 0xff, 0xff, 0xff, 0xfb, 0x02, 0xc0, 0x00, 0x3c,	初始化表中的一项，用于配置数组、结构体或常量表。
57	0xcb, 0x50, 0x00, 0x02, 0xc0, 0x02, 0xd2, 0xb4, 0xa4, 0x00, 0x02, 0xc0,	初始化表中的一项，用于配置数组、结构体或常量表。
58	0x7d, 0x30, 0xb3, 0x0b, 0x70, 0x02, 0xc4, 0xb2, 0x00, 0xb3, 0x00, 0x88,	初始化表中的一项，用于配置数组、结构体或常量表。
59	0x00, 0x1c, 0x10, 0x00, 0xa3, 0x00, 0x00, 0x00, 0x7a, 0xdf, 0xff, 0xff,	初始化表中的一项，用于配置数组、结构体或常量表。
60	0xff, 0xfb, 0x01, 0xe2, 0x00, 0x00, 0x00, 0x00, 0x00, 0x0a, 0xe0, 0x1f,	初始化表中的一项，用于配置数组、结构体或常量表。
61	0xff, 0xff, 0xff, 0xe0, 0x6d, 0xe0, 0x00, 0x00, 0x00, 0x00, 0x00, 0x72,	初始化表中的一项，用于配置数组、结构体或常量表。
62	0xe0, 0x1f, 0xff, 0xff, 0xff, 0xe0, 0x00, 0xe0, 0x00, 0x00, 0x00, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
63	0x00, 0x00, 0xe0, 0x0e, 0xff, 0xff, 0xff, 0xc0, 0x00, 0xe0, 0x0e, 0x10,	初始化表中的一项，用于配置数组、结构体或常量表。
64	0x00, 0x03, 0xc0, 0x00, 0xe0, 0x0e, 0x10, 0x00, 0x03, 0xc0, 0x00, 0xe0,	初始化表中的一项，用于配置数组、结构体或常量表。
65	0x0e, 0xff, 0xff, 0xff, 0xc0, 0x00, 0xe0, 0x0e, 0x10, 0x00, 0x03, 0xc0,	初始化表中的一项，用于配置数组、结构体或常量表。
66	0x00, 0x3b, 0x00, 0x02, 0xb0, 0x00, 0x00, 0x00, 0x77, 0xff, 0xff, 0xff,	初始化表中的一项，用于配置数组、结构体或常量表。

行	源码	说明
67	0xff, 0xf6, 0x00, 0xd1, 0x00, 0x00, 0x00, 0x00, 0x00, 0x05, 0xd0, 0xd,	初始化表中的一项，用于配置数组、结构体或常量表。
68	0xff, 0xff, 0xff, 0x00, 0xd, 0xd0, 0xd, 0x10, 0x00, 0xe, 0x00, 0x79,	初始化表中的一项，用于配置数组、结构体或常量表。
69	0xd0, 0xd, 0xff, 0xff, 0xff, 0x00, 0x71, 0xd0, 0x00, 0x00, 0x00, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
70	0x00, 0x00, 0xd5, 0xff, 0xff, 0xff, 0xff, 0xf9, 0x00, 0xd5, 0x80, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
71	0x00, 0x00, 0x59, 0x00, 0xd0, 0xf, 0xff, 0xff, 0xff, 0x30, 0x00, 0xd0,	初始化表中的一项，用于配置数组、结构体或常量表。
72	0x00, 0x00, 0x78, 0x00, 0x00, 0x00, 0xd0, 0x00, 0xbf, 0xe5, 0x00, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
73	0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x85, 0xb1, 0x00, 0x59,	初始化表中的一项，用于配置数组、结构体或常量表。
74	0x00, 0x00, 0x00, 0xc2, 0x88, 0x4f, 0xff, 0xf8, 0xb2, 0x02, 0xc0, 0x1b,	初始化表中的一项，用于配置数组、结构体或常量表。
75	0x00, 0x59, 0x08, 0x90, 0x08, 0xb0, 0x00, 0x00, 0x59, 0x7a, 0x00, 0x1e,	初始化表中的一项，用于配置数组、结构体或常量表。
76	0xbd, 0xf5, 0xbf, 0xff, 0xff, 0xfb, 0x98, 0xb0, 0x85, 0x00, 0x7a, 0x10,	初始化表中的一项，用于配置数组、结构体或常量表。
77	0x00, 0x73, 0xb0, 0x85, 0x2b, 0xff, 0xff, 0xb0, 0x03, 0xb0, 0x86, 0xb8,	初始化表中的一项，用于配置数组、结构体或常量表。
78	0x90, 0x02, 0xb0, 0x03, 0xb0, 0x85, 0x04, 0xff, 0xff, 0xb0, 0x03, 0xb0,	初始化表中的一项，用于配置数组、结构体或常量表。
79	0x8b, 0xb4, 0x90, 0x02, 0xb0, 0x03, 0xb0, 0x9e, 0x54, 0xff, 0xff, 0xb0,	初始化表中的一项，用于配置数组、结构体或常量表。
80	0x03, 0xb0, 0xa3, 0x04, 0x90, 0x02, 0xb0, 0x00, 0x00, 0x97, 0x00, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
81	0x00, 0x00, 0x00, 0x00, 0x1c, 0x50, 0x00, 0x00, 0x00, 0x00, 0x00, 0x02,	初始化表中的一项，用于配置数组、结构体或常量表。
82	0xd1, 0x00, 0x00, 0x00, 0x00, 0x00, 0x01, 0xe9, 0x00, 0x00, 0x00, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
83	0x00, 0x06, 0xcd, 0x20, 0x00, 0x00, 0x00, 0x00, 0x0b, 0x56, 0xa0, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
84	0x00, 0x00, 0x00, 0x5d, 0x00, 0xc3, 0x00, 0x00, 0x00, 0x00, 0xd5, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
85	0x4c, 0x00, 0x00, 0x00, 0x0a, 0xb0, 0x00, 0x09, 0x80, 0x00, 0x00, 0x8c,	初始化表中的一项，用于配置数组、结构体或常量表。
86	0x10, 0x00, 0x01, 0xc5, 0x00, 0x1a, 0xc1, 0x00, 0x00, 0x00, 0x1d, 0x70,	初始化表中的一项，用于配置数组、结构体或常量表。
87	0x69, 0x00, 0x00, 0x00, 0x00, 0x01, 0xa6, 0x00, 0x1b, 0x10, 0x00, 0x06,	初始化表中的一项，用于配置数组、结构体或常量表。
88	0x80, 0x00, 0x00, 0x09, 0xc1, 0x00, 0x1e, 0x40, 0x00, 0x00, 0x00, 0xa3,	初始化表中的一项，用于配置数组、结构体或常量表。
89	0x00, 0xa9, 0x00, 0x00, 0x09, 0xff, 0xff, 0xff, 0xff, 0xa0, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。

行	源码	说明
90	0x00, 0x00, 0x97, 0x00, 0x00, 0x00, 0x00, 0x00, 0x97, 0x00, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
91	0x00, 0x6f, 0xff, 0xff, 0xff, 0xff, 0xff, 0xf7, 0x00, 0x00, 0x02, 0xed,	初始化表中的一项，用于配置数组、结构体或常量表。
92	0x10, 0x00, 0x00, 0x00, 0x00, 0x0b, 0x76, 0xb0, 0x00, 0x00, 0x00, 0x02,	初始化表中的一项，用于配置数组、结构体或常量表。
93	0xb9, 0x00, 0x8b, 0x20, 0x00, 0x03, 0x9d, 0x50, 0x00, 0x05, 0xd8, 0x20,	初始化表中的一项，用于配置数组、结构体或常量表。
94	0x5c, 0x60, 0x00, 0x00, 0x00, 0x17, 0xc4, 0x00, 0x00, 0x00, 0x88, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
95	0x00, 0x00, 0x00, 0x00, 0x00, 0x88, 0x00, 0x00, 0x00, 0x09, 0xff, 0xff,	初始化表中的一项，用于配置数组、结构体或常量表。
96	0xff, 0xff, 0xff, 0x80, 0x09, 0x70, 0x00, 0xa6, 0x00, 0x08, 0x80, 0x09,	初始化表中的一项，用于配置数组、结构体或常量表。
97	0x70, 0x00, 0xc4, 0x00, 0x08, 0x80, 0x09, 0x70, 0x03, 0xfb, 0x10, 0x08,	初始化表中的一项，用于配置数组、结构体或常量表。
98	0x80, 0x09, 0x70, 0x1c, 0x87, 0xd3, 0x08, 0x80, 0x09, 0x73, 0xd9, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
99	0x5e, 0x48, 0x80, 0x09, 0x9c, 0x50, 0x00, 0x04, 0xb8, 0x80, 0x09, 0x70,	初始化表中的一项，用于配置数组、结构体或常量表。
100	0x00, 0x00, 0x00, 0x08, 0x80, 0x09, 0x70, 0x00, 0x00, 0x00, 0x0a, 0x80,	初始化表中的一项，用于配置数组、结构体或常量表。
101	0x09, 0x70, 0x00, 0x00, 0x6f, 0xfd, 0x30, 0x00, 0x00, 0x00, 0x00, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
102	0x00, 0x00, 0x00, 0xe1, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0xe1, 0x0c,	初始化表中的一项，用于配置数组、结构体或常量表。
103	0xff, 0xff, 0xff, 0xf0, 0x00, 0xe1, 0x01, 0x06, 0x90, 0x01, 0xf0, 0x01,	初始化表中的一项，用于配置数组、结构体或常量表。
104	0xe7, 0xbc, 0x07, 0x90, 0x01, 0xe0, 0x9c, 0xe6, 0x20, 0x07, 0x80, 0x02,	初始化表中的一项，用于配置数组、结构体或常量表。
105	0xe0, 0x10, 0xe1, 0x00, 0x08, 0x70, 0x02, 0xd0, 0x00, 0xe1, 0x00, 0x0a,	初始化表中的一项，用于配置数组、结构体或常量表。
106	0x50, 0x03, 0xd0, 0x00, 0xe1, 0x09, 0xd0, 0x30, 0x04, 0xc0, 0x00, 0xe3,	初始化表中的一项，用于配置数组、结构体或常量表。
107	0xc5, 0x4d, 0x00, 0x05, 0xb0, 0x01, 0xfc, 0x21, 0xd6, 0x00, 0x06, 0xb0,	初始化表中的一项，用于配置数组、结构体或常量表。
108	0x02, 0xa0, 0x2c, 0x90, 0x00, 0x0a, 0x80, 0x00, 0x00, 0x97, 0x00, 0xcf,	初始化表中的一项，用于配置数组、结构体或常量表。
109	0xfc, 0x20, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x7f, 0xff, 0xff,	初始化表中的一项，用于配置数组、结构体或常量表。
110	0xff, 0x50, 0x00, 0xd3, 0x00, 0x3e, 0x10, 0x00, 0xd0, 0x30, 0xd3, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
111	0xc5, 0x05, 0x80, 0xd0, 0x30, 0xd3, 0x09, 0x90, 0x01, 0xe4, 0xd0, 0x30,	初始化表中的一项，用于配置数组、结构体或常量表。
112	0xd3, 0x1e, 0xff, 0xff, 0xfd, 0x1d, 0x30, 0xd3, 0x00, 0x00, 0x00, 0x04,	初始化表中的一项，用于配置数组、结构体或常量表。

行	源码	说明
113	0x0d, 0x30, 0xd3, 0x00, 0x02, 0xd0, 0x00, 0x0d, 0x30, 0xd3, 0x5f, 0xff,	初始化表中的一项，用于配置数组、结构体或常量表。
114	0xff, 0xff, 0x0d, 0x30, 0xd3, 0x00, 0x02, 0xd0, 0x00, 0x0d, 0x30, 0xd3,	初始化表中的一项，用于配置数组、结构体或常量表。
115	0x00, 0x02, 0xd0, 0x00, 0x00, 0x00, 0xd3, 0x02, 0x47, 0xe9, 0xbd, 0x30,	初始化表中的一项，用于配置数组、结构体或常量表。
116	0x01, 0xe2, 0x9d, 0xb9, 0x75, 0x31, 0x08, 0xfe, 0xb0, 0x00, 0x02, 0xb1,	初始化表中的一项，用于配置数组、结构体或常量表。
117	0x00, 0x0a, 0x40, 0x00, 0x00, 0x00, 0x98, 0x00, 0x5b, 0x00, 0x00, 0xaf,	初始化表中的一项，用于配置数组、结构体或常量表。
118	0xff, 0xff, 0xff, 0xff, 0xff, 0xfa, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
119	0x00, 0x0b, 0xff, 0xff, 0xf3, 0x2d, 0x05, 0xb0, 0x0b, 0x40, 0x00, 0xc3,	初始化表中的一项，用于配置数组、结构体或常量表。
120	0x2d, 0x05, 0xb0, 0x0b, 0xff, 0xff, 0xf3, 0x2d, 0x05, 0xb0, 0x0b, 0x40,	初始化表中的一项，用于配置数组、结构体或常量表。
121	0x00, 0xc3, 0x2d, 0x05, 0xb0, 0x0b, 0xff, 0xff, 0xf3, 0x2d, 0x05, 0xb0,	初始化表中的一项，用于配置数组、结构体或常量表。
122	0x0b, 0x40, 0x00, 0xc3, 0x2d, 0x05, 0xb0, 0x0b, 0x40, 0x00, 0xc3, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
123	0x07, 0xa0, 0x0b, 0x40, 0xcf, 0xd1, 0x0b, 0xfe, 0x50, 0x00, 0x00, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
124	0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x05, 0xb0, 0x00, 0x00, 0x9f,	初始化表中的一项，用于配置数组、结构体或常量表。
125	0xff, 0xfc, 0x05, 0xa0, 0x00, 0x00, 0x00, 0x88, 0x00, 0x05, 0xa0, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
126	0x00, 0x00, 0x88, 0x08, 0xff, 0xff, 0xff, 0xf0, 0x00, 0x88, 0x00, 0x06,	初始化表中的一项，用于配置数组、结构体或常量表。
127	0x90, 0x01, 0xf0, 0x00, 0x88, 0x00, 0x08, 0x80, 0x01, 0xe0, 0x00, 0x88,	初始化表中的一项，用于配置数组、结构体或常量表。
128	0x00, 0x0a, 0x60, 0x02, 0xe0, 0x00, 0x88, 0x00, 0x0d, 0x30, 0x02, 0xd0,	初始化表中的一项，用于配置数组、结构体或常量表。
129	0x03, 0xac, 0xbb, 0x6b, 0x00, 0x03, 0xc0, 0x9c, 0x96, 0x34, 0xe3, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
130	0x04, 0xc0, 0x00, 0x00, 0x5e, 0x50, 0x00, 0x08, 0xa0, 0x00, 0x03, 0xb2,	初始化表中的一项，用于配置数组、结构体或常量表。
131	0x00, 0xdf, 0xfd, 0x30, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
132	0x00, 0x03, 0xde, 0x30, 0x00, 0x00, 0x00, 0x01, 0x8d, 0x43, 0xc7, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
133	0x00, 0x03, 0x8d, 0x70, 0x00, 0x07, 0xc7, 0x20, 0x7b, 0x54, 0xff, 0xff,	初始化表中的一项，用于配置数组、结构体或常量表。
134	0xff, 0x56, 0xb6, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x06, 0xff,	初始化表中的一项，用于配置数组、结构体或常量表。
135	0xfe, 0x07, 0xff, 0xff, 0x70, 0x06, 0x80, 0x0e, 0x07, 0x80, 0x07, 0x70,	初始化表中的一项，用于配置数组、结构体或常量表。

行	源码	说明
136	0x06, 0x80, 0x0e, 0x07, 0x80, 0x07, 0x70, 0x06, 0x80, 0x0e, 0x07, 0x80,	初始化表中的一项，用于配置数组、结构体或常量表。
137	0x09, 0x70, 0x06, 0xff, 0xfe, 0x07, 0x8b, 0xfd, 0x20, 0x06, 0x80, 0x0e,	初始化表中的一项，用于配置数组、结构体或常量表。
138	0x07, 0x80, 0x00, 0x00, 0x00, 0x00, 0x00, 0x07, 0x80, 0x00, 0x00, 0x09,	初始化表中的一项，用于配置数组、结构体或常量表。
139	0xff, 0xff, 0x24, 0xff, 0xff, 0x70, 0x09, 0x50, 0x0b, 0x24, 0x90, 0x06,	初始化表中的一项，用于配置数组、结构体或常量表。
140	0x70, 0x09, 0xff, 0xff, 0x24, 0xff, 0xff, 0x70, 0x09, 0x50, 0x0b, 0x24,	初始化表中的一项，用于配置数组、结构体或常量表。
141	0x99, 0x26, 0x70, 0x00, 0x00, 0x00, 0xa4, 0x04, 0xd0, 0x00, 0x5f, 0xff,	初始化表中的一项，用于配置数组、结构体或常量表。
142	0xff, 0xff, 0xff, 0xff, 0xf6, 0x00, 0x03, 0x7c, 0x46, 0xc6, 0x31, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
143	0x7d, 0xda, 0x50, 0x00, 0x17, 0xbd, 0xe7, 0x09, 0xff, 0xff, 0x34, 0xff,	初始化表中的一项，用于配置数组、结构体或常量表。
144	0xff, 0x80, 0x08, 0x50, 0x0b, 0x34, 0x90, 0x06, 0x80, 0x08, 0xff, 0xff,	初始化表中的一项，用于配置数组、结构体或常量表。
145	0x34, 0xff, 0xff, 0x80, 0x08, 0x50, 0x0b, 0x34, 0x90, 0x06, 0x80, 0x0e,	初始化表中的一项，用于配置数组、结构体或常量表。
146	0xff, 0xff, 0xff, 0xff, 0xff, 0xe0, 0x0e, 0x20, 0x00, 0x00, 0x00, 0x02,	初始化表中的一项，用于配置数组、结构体或常量表。
147	0xe0, 0x0e, 0x20, 0x00, 0x00, 0x00, 0x02, 0xe0, 0x0e, 0x20, 0xdf, 0xff,	初始化表中的一项，用于配置数组、结构体或常量表。
148	0xfd, 0x02, 0xe0, 0x0e, 0x20, 0xd2, 0x00, 0x2d, 0x02, 0xe0, 0x0e, 0x20,	初始化表中的一项，用于配置数组、结构体或常量表。
149	0xd2, 0x00, 0x2d, 0x02, 0xe0, 0x0e, 0x20, 0xd2, 0x00, 0x2d, 0x02, 0xe0,	初始化表中的一项，用于配置数组、结构体或常量表。
150	0x0e, 0x20, 0xdf, 0xff, 0xfd, 0x02, 0xe0, 0x0e, 0x20, 0xd2, 0x00, 0x2d,	初始化表中的一项，用于配置数组、结构体或常量表。
151	0x02, 0xe0, 0x0e, 0x20, 0x00, 0x00, 0x00, 0x02, 0xe0, 0x0e, 0xff, 0xff,	初始化表中的一项，用于配置数组、结构体或常量表。
152	0xff, 0xff, 0xff, 0xe0, 0x0e, 0x20, 0x00, 0x00, 0x00, 0x02, 0xe0, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
153	0x00, 0x0d, 0x20, 0x00, 0x00, 0x00, 0x00, 0x00, 0x4d, 0x00, 0x00, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
154	0x00, 0x6f, 0xff, 0xff, 0xff, 0xff, 0xff, 0xfb, 0x00, 0x02, 0xe1, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
155	0x00, 0x00, 0x00, 0x00, 0x0b, 0x70, 0x00, 0xb5, 0x00, 0x00, 0x00, 0x6c,	初始化表中的一项，用于配置数组、结构体或常量表。
156	0x00, 0x00, 0xb5, 0x00, 0x00, 0x03, 0xe6, 0x5f, 0xff, 0xff, 0xff, 0xe0,	初始化表中的一项，用于配置数组、结构体或常量表。
157	0x2d, 0xc6, 0x00, 0x00, 0xb5, 0x00, 0x00, 0x85, 0x96, 0x00, 0x00, 0xb5,	初始化表中的一项，用于配置数组、结构体或常量表。
158	0x00, 0x00, 0x00, 0x96, 0x00, 0x00, 0xb5, 0x00, 0x00, 0x00, 0x96, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。

行	源码	说明
159	0x00, 0xb5, 0x00, 0x00, 0x00, 0x97, 0xff, 0xff, 0xff, 0xff, 0xfb, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
160	0xe1, 0x00, 0x00, 0x4b, 0x00, 0x00, 0x00, 0xe1, 0x06, 0x90, 0x4b, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
161	0x10, 0x00, 0xe1, 0x06, 0x90, 0x4d, 0x9c, 0xf0, 0xaf, 0xff, 0xa8, 0xdc,	初始化表中的一项，用于配置数组、结构体或常量表。
162	0xed, 0x74, 0xf0, 0x00, 0xe1, 0x7d, 0xb4, 0x5b, 0x00, 0xe0, 0x00, 0xe1,	初始化表中的一项，用于配置数组、结构体或常量表。
163	0x06, 0x90, 0x4b, 0x01, 0xe0, 0x00, 0xe1, 0x06, 0x90, 0x4b, 0x04, 0xd0,	初始化表中的一项，用于配置数组、结构体或常量表。
164	0x00, 0xe1, 0x06, 0x90, 0x4b, 0xde, 0x60, 0x00, 0xe7, 0xa6, 0x90, 0x4b,	初始化表中的一项，用于配置数组、结构体或常量表。
165	0x00, 0x3a, 0x28, 0xe9, 0x26, 0x90, 0x4b, 0x00, 0x5b, 0x98, 0x10, 0x05,	初始化表中的一项，用于配置数组、结构体或常量表。
166	0xc0, 0x00, 0x00, 0xa8, 0x00, 0x00, 0x00, 0xbe, 0xff, 0xfe, 0xb1, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
167	0xa5, 0x00, 0x00, 0xc0, 0x30, 0x00, 0x00, 0xa5, 0x00, 0x00, 0xc0, 0x30,	初始化表中的一项，用于配置数组、结构体或常量表。
168	0x00, 0x00, 0xa5, 0x00, 0xd1, 0xc0, 0x30, 0x00, 0xaf, 0xff, 0xf3, 0xd1,	初始化表中的一项，用于配置数组、结构体或常量表。
169	0xc0, 0x30, 0x00, 0x00, 0xa5, 0x00, 0xd1, 0xc0, 0xff, 0xf6, 0x00, 0xa5,	初始化表中的一项，用于配置数组、结构体或常量表。
170	0x00, 0xd1, 0xc0, 0x30, 0x00, 0x00, 0xa5, 0x00, 0xd1, 0xc0, 0x30, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
171	0x00, 0xa5, 0x00, 0xd1, 0xc0, 0x30, 0x00, 0x00, 0xa8, 0x96, 0xd1, 0xc0,	初始化表中的一项，用于配置数组、结构体或常量表。
172	0x30, 0x00, 0x17, 0xdc, 0x60, 0xd1, 0xc0, 0x30, 0x00, 0x89, 0x30, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
173	0xd1, 0xc0, 0x30, 0x00, 0x00, 0x00, 0xff, 0xff, 0xff, 0xff, 0xfc, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
174	0x87, 0x00, 0x00, 0x88, 0x00, 0x00, 0x00, 0xd4, 0x00, 0x00, 0x88, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
175	0x00, 0x02, 0xff, 0xff, 0xb0, 0x88, 0x00, 0x00, 0x07, 0x90, 0x06, 0x90,	初始化表中的一项，用于配置数组、结构体或常量表。
176	0x8b, 0x90, 0x00, 0x1e, 0x80, 0x08, 0x70, 0x89, 0xbb, 0x00, 0x7d, 0xc0,	初始化表中的一项，用于配置数组、结构体或常量表。
177	0xc0, 0x30, 0x88, 0x0a, 0xc1, 0x83, 0xc2, 0x1d, 0x00, 0x88, 0x00, 0x95,	初始化表中的一项，用于配置数组、结构体或常量表。
178	0x00, 0x78, 0x88, 0x00, 0x88, 0x00, 0x00, 0x00, 0xd, 0xd1, 0x00, 0x88,	初始化表中的一项，用于配置数组、结构体或常量表。
179	0x00, 0x00, 0x00, 0x3d, 0xc3, 0x00, 0x88, 0x00, 0x00, 0x07, 0xd3, 0x2c,	初始化表中的一项，用于配置数组、结构体或常量表。
180	0x83, 0x10, 0x00, 0x00, 0x38, 0x00, 0x00, 0x5a, 0xce, 0xee, 0xe9, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
181	0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x04, 0xa0, 0x00, 0x00, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。

行	源码	说明
182	0x00, 0x00, 0x4e, 0xff, 0xff, 0xff, 0xfd, 0x00, 0x07, 0xe9, 0xc2, 0x00,	初始化表中的一项, 用于配置数组、结构体或常量表。
183	0x06, 0xd4, 0x00, 0x4b, 0x20, 0x4c, 0xa8, 0xd8, 0x10, 0x00, 0x00, 0x14,	初始化表中的一项, 用于配置数组、结构体或常量表。
184	0x8c, 0xdc, 0xd8, 0x40, 0x00, 0x8c, 0xb8, 0x52, 0x00, 0x26, 0x9b, 0xc6,	初始化表中的一项, 用于配置数组、结构体或常量表。
185	0x00, 0xcf, 0xff, 0xff, 0xff, 0xfd, 0x00, 0x00, 0xc1, 0x00, 0x77, 0x00,	初始化表中的一项, 用于配置数组、结构体或常量表。
186	0x1d, 0x00, 0x00, 0xcf, 0xff, 0xff, 0xff, 0xfd, 0x00, 0x00, 0xc1, 0x00,	初始化表中的一项, 用于配置数组、结构体或常量表。
187	0x77, 0x00, 0x1d, 0x00, 0x00, 0xcf, 0xff, 0xff, 0xff, 0xfd, 0x00, 0x00,	初始化表中的一项, 用于配置数组、结构体或常量表。
188	0xc1, 0x00, 0x00, 0x00, 0x1d, 0x00, 0x00, 0xb3, 0x00, 0x00, 0x96, 0x00,	初始化表中的一项, 用于配置数组、结构体或常量表。
189	0x00, 0x00, 0xe2, 0x00, 0x01, 0xe1, 0x00, 0x00, 0x01, 0xe0, 0x00, 0x08,	初始化表中的一项, 用于配置数组、结构体或常量表。
190	0x70, 0x2b, 0x00, 0xbf, 0xff, 0xf6, 0x2c, 0x00, 0x0a, 0x80, 0x07, 0x80,	初始化表中的一项, 用于配置数组、结构体或常量表。
191	0xa5, 0xaa, 0x9b, 0xde, 0xe3, 0x0a, 0x50, 0xc6, 0xc8, 0x64, 0x21, 0x87,	初始化表中的一项, 用于配置数组、结构体或常量表。
192	0x0d, 0x11, 0xe0, 0x00, 0x00, 0x00, 0x00, 0x1e, 0x56, 0xa0, 0xcf, 0xff,	初始化表中的一项, 用于配置数组、结构体或常量表。
193	0xff, 0xc0, 0x02, 0xde, 0x40, 0xc3, 0x00, 0x03, 0xc0, 0x00, 0xae, 0xb1,	初始化表中的一项, 用于配置数组、结构体或常量表。
194	0xc3, 0x00, 0x03, 0xc0, 0x1a, 0xa1, 0x96, 0xcf, 0xff, 0xff, 0xc0, 0x77,	初始化表中的一项, 用于配置数组、结构体或常量表。
195	0x00, 0x00, 0xc3, 0x00, 0x03, 0xc0, 0x00, 0x00, 0x59, 0x00, 0x00, 0x00,	初始化表中的一项, 用于配置数组、结构体或常量表。
196	0x00, 0x00, 0x00, 0x98, 0x00, 0x00, 0x00, 0x00, 0x9f, 0xff, 0xff, 0xff,	初始化表中的一项, 用于配置数组、结构体或常量表。
197	0xff, 0xff, 0xfb, 0x00, 0x07, 0xb0, 0x00, 0x00, 0x00, 0x00, 0x00, 0x1e,	初始化表中的一项, 用于配置数组、结构体或常量表。
198	0x3f, 0xff, 0xff, 0xff, 0xc0, 0x00, 0x99, 0x00, 0x00, 0x04, 0xac, 0x40,	初始化表中的一项, 用于配置数组、结构体或常量表。
199	0x05, 0xf6, 0x00, 0x00, 0x4e, 0x50, 0x00, 0x3e, 0xc6, 0xef, 0xff, 0xff,	初始化表中的一项, 用于配置数组、结构体或常量表。
200	0xff, 0xfc, 0x84, 0x96, 0x00, 0x00, 0x4c, 0x00, 0x00, 0x00, 0x96, 0x00,	初始化表中的一项, 用于配置数组、结构体或常量表。
201	0x00, 0x4c, 0x00, 0x00, 0x00, 0x96, 0x00, 0x00, 0x5b, 0x00, 0x00, 0x00,	初始化表中的一项, 用于配置数组、结构体或常量表。
202	0x96, 0x00, 0xce, 0xe6, 0x00, 0x00, 0x0d, 0xff, 0xff, 0xff, 0xff, 0xff,	初始化表中的一项, 用于配置数组、结构体或常量表。
203	0x10, 0x00, 0x00, 0x00, 0x00, 0x00, 0x0f, 0x10, 0x00, 0x00, 0x00, 0x00,	初始化表中的一项, 用于配置数组、结构体或常量表。
204	0x00, 0x0f, 0x10, 0x05, 0xc0, 0x00, 0x00, 0x00, 0x0f, 0x10, 0x05, 0xc0,	初始化表中的一项, 用于配置数组、结构体或常量表。

行	源码	说明
205	0x00, 0x00, 0x00, 0x0f, 0x10, 0x05, 0xff, 0xff, 0xff, 0xff, 0xff, 0x10,	初始化表中的一项，用于配置数组、结构体或常量表。
206	0x05, 0xc0, 0x00, 0x00, 0x00, 0x0f, 0x10, 0x05, 0xc0, 0x00, 0x00, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
207	0x00, 0x00, 0x05, 0xc0, 0x00, 0x00, 0x00, 0x00, 0x76, 0x05, 0xc0, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
208	0x00, 0x00, 0x00, 0xa7, 0x04, 0xe1, 0x00, 0x00, 0x00, 0x03, 0xe3, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
209	0xae, 0xff, 0xff, 0xff, 0xfe, 0x80, 0x9f, 0xff, 0x60, 0x00, 0xe0, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
210	0x00, 0x00, 0x4d, 0x1c, 0xff, 0xff, 0xff, 0x30, 0x02, 0xd2, 0x00, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
211	0xe0, 0x0b, 0x30, 0x1d, 0x40, 0x7f, 0xff, 0xff, 0xff, 0xf9, 0x5f, 0xff,	初始化表中的一项，用于配置数组、结构体或常量表。
212	0x60, 0x00, 0xe0, 0x0b, 0x30, 0x00, 0x0a, 0x4c, 0xff, 0xff, 0xff, 0x30,	初始化表中的一项，用于配置数组、结构体或常量表。
213	0x56, 0x0d, 0x10, 0x00, 0xe0, 0x00, 0x00, 0x3c, 0x1d, 0x0f, 0xff, 0xff,	初始化表中的一项，用于配置数组、结构体或常量表。
214	0xff, 0x70, 0x0c, 0xa8, 0x00, 0x00, 0xe0, 0x00, 0x00, 0x04, 0xf5, 0x6f,	初始化表中的一项，用于配置数组、结构体或常量表。
215	0xff, 0xff, 0xff, 0xf1, 0x1b, 0xbe, 0x94, 0x21, 0xf1, 0x01, 0x11, 0x77,	初始化表中的一项，用于配置数组、结构体或常量表。
216	0x01, 0x8c, 0xee, 0xef, 0xff, 0xf8, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
217	0x00, 0x1f, 0xff, 0xff, 0xff, 0xff, 0xff, 0xf2, 0x00, 0x02, 0xe0, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
218	0x0b, 0x50, 0x00, 0x00, 0x02, 0xe0, 0x00, 0x0b, 0x50, 0x00, 0x00, 0x02,	初始化表中的一项，用于配置数组、结构体或常量表。
219	0xe0, 0x00, 0x0b, 0x50, 0x00, 0x00, 0x02, 0xe0, 0x00, 0x0b, 0x50, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
220	0xaf, 0xff, 0xff, 0xff, 0xff, 0xff, 0xfb, 0x00, 0x03, 0xd0, 0x00, 0x0b,	初始化表中的一项，用于配置数组、结构体或常量表。
221	0x50, 0x00, 0x00, 0x04, 0xc0, 0x00, 0x0b, 0x50, 0x00, 0x00, 0x08, 0x90,	初始化表中的一项，用于配置数组、结构体或常量表。
222	0x00, 0x0b, 0x50, 0x00, 0x00, 0x3e, 0x30, 0x00, 0x0b, 0x50, 0x00, 0x05,	初始化表中的一项，用于配置数组、结构体或常量表。
223	0xd5, 0x00, 0x00, 0x0b, 0x50, 0x00, 0x4b, 0x30, 0x00, 0x00, 0x0b, 0x50,	初始化表中的一项，用于配置数组、结构体或常量表。
224	0x00, 0x00, 0x00, 0x00, 0x88, 0x00, 0x00, 0x00, 0x02, 0xb2, 0x00, 0x88,	初始化表中的一项，用于配置数组、结构体或常量表。
225	0x00, 0x2b, 0x30, 0x00, 0x6d, 0x40, 0x88, 0x04, 0xd8, 0x00, 0x00, 0x05,	初始化表中的一项，用于配置数组、结构体或常量表。
226	0xa0, 0x88, 0x0b, 0x40, 0x00, 0x0c, 0xff, 0xff, 0xff, 0xff, 0xff, 0xa0,	初始化表中的一项，用于配置数组、结构体或常量表。
227	0x00, 0x00, 0x00, 0x00, 0x00, 0x06, 0xa0, 0x00, 0x00, 0x00, 0x00, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。

行	源码	说明
228	0x06, 0xa0, 0x06, 0xff, 0xff, 0xff, 0xff, 0xff, 0xa0, 0x00, 0x00, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
229	0x00, 0x00, 0x06, 0xa0, 0x00, 0x00, 0x00, 0x00, 0x00, 0x06, 0xa0, 0x3f,	初始化表中的一项，用于配置数组、结构体或常量表。
230	0xff, 0xff, 0xff, 0xff, 0xff, 0xa0, 0x00, 0x00, 0x00, 0x00, 0x00, 0x06,	初始化表中的一项，用于配置数组、结构体或常量表。
231	0xa0, 0x00, 0x00, 0x00, 0x77, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x96,	初始化表中的一项，用于配置数组、结构体或常量表。
232	0x00, 0x00, 0x00, 0x4f, 0xff, 0xff, 0xff, 0xff, 0xff, 0xf4, 0x00, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
233	0x05, 0xbb, 0x30, 0x00, 0x00, 0x00, 0x00, 0x7d, 0x11, 0xb6, 0x00, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
234	0x02, 0x7d, 0x8b, 0x90, 0x07, 0xc6, 0x10, 0x6b, 0x61, 0x00, 0x86, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
235	0x17, 0xc3, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x06, 0x83, 0xc1,	初始化表中的一项，用于配置数组、结构体或常量表。
236	0xc4, 0x00, 0x1b, 0x10, 0x0d, 0x33, 0xc0, 0x3c, 0x26, 0x67, 0xd1, 0x4a,	初始化表中的一项，用于配置数组、结构体或常量表。
237	0x03, 0xd0, 0x00, 0x0b, 0x60, 0x86, 0x00, 0x00, 0xbe, 0xff, 0xeb, 0x10,	初始化表中的一项，用于配置数组、结构体或常量表。
238	0x00, 0x00, 0x00, 0x06, 0x80, 0x00, 0x00, 0x00, 0x00, 0xff, 0xff, 0xff,	初始化表中的一项，用于配置数组、结构体或常量表。
239	0xff, 0xfd, 0x00, 0x00, 0xf0, 0x00, 0x00, 0x00, 0x2d, 0x00, 0x00, 0xff,	初始化表中的一项，用于配置数组、结构体或常量表。
240	0xff, 0xff, 0xff, 0xfd, 0x00, 0x00, 0xf0, 0x00, 0x00, 0x00, 0x2d, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
241	0x00, 0xff, 0xff, 0xff, 0xff, 0xfd, 0x00, 0x00, 0xf0, 0x00, 0x00, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
242	0x2d, 0x00, 0x00, 0xff, 0xff, 0xff, 0xff, 0xfd, 0x00, 0x00, 0xf0, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
243	0x00, 0x00, 0x2d, 0x00, 0x1c, 0x18, 0x70, 0x9b, 0x10, 0x06, 0x91, 0x77,	初始化表中的一项，用于配置数组、结构体或常量表。
244	0x08, 0x80, 0x00, 0x05, 0xb0, 0x88, 0x00, 0x03, 0xdf, 0xff, 0xfd, 0x60,	初始化表中的一项，用于配置数组、结构体或常量表。
245	0x00, 0x00, 0xd2, 0x0d, 0xff, 0xff, 0xff, 0xd0, 0x00, 0xd2, 0x00, 0xc4,	初始化表中的一项，用于配置数组、结构体或常量表。
246	0x00, 0x0a, 0x70, 0xaf, 0xff, 0xd0, 0x3d, 0x30, 0x9a, 0x00, 0x00, 0xd2,	初始化表中的一项，用于配置数组、结构体或常量表。
247	0x00, 0x03, 0xed, 0x90, 0x00, 0x00, 0xd2, 0x03, 0x8d, 0xb9, 0xd7, 0x20,	初始化表中的一项，用于配置数组、结构体或常量表。
248	0x14, 0xdc, 0xbc, 0x83, 0x00, 0x17, 0xb7, 0xab, 0xe5, 0x00, 0x00, 0x5a,	初始化表中的一项，用于配置数组、结构体或常量表。
249	0x00, 0x00, 0x00, 0xd2, 0x0b, 0xff, 0xff, 0xff, 0xf2, 0x00, 0xd2, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
250	0x00, 0x5a, 0x00, 0x00, 0x00, 0xd2, 0x9f, 0xff, 0xff, 0xff, 0xfc, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。

行	源码	说明
251	0xe1, 0x00, 0x00, 0x5a, 0x00, 0x00, 0x9e, 0xb0, 0x00, 0x00, 0x5a, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
252	0x00, 0x01, 0xe0, 0x00, 0xc, 0x10, 0x00, 0x00, 0x01, 0xe0, 0x00, 0x7f,	初始化表中的一项，用于配置数组、结构体或常量表。
253	0xff, 0xfd, 0x00, 0xaf, 0xff, 0x72, 0xa0, 0x00, 0xb6, 0x00, 0x01, 0xe0,	初始化表中的一项，用于配置数组、结构体或常量表。
254	0x0b, 0x10, 0x08, 0x90, 0x00, 0x01, 0xe0, 0x5e, 0xff, 0xff, 0xff, 0xa0,	初始化表中的一项，用于配置数组、结构体或常量表。
255	0x15, 0xee, 0x4b, 0x30, 0x86, 0x04, 0xa0, 0xaa, 0xe0, 0x0b, 0x30, 0x86,	初始化表中的一项，用于配置数组、结构体或常量表。
256	0x04, 0xa0, 0x01, 0xe0, 0x0b, 0x30, 0x85, 0x04, 0xa0, 0x01, 0xe0, 0xef,	初始化表中的一项，用于配置数组、结构体或常量表。
257	0xff, 0xff, 0xff, 0xfc, 0x01, 0xe0, 0x00, 0x04, 0xcb, 0x10, 0x00, 0x02,	初始化表中的一项，用于配置数组、结构体或常量表。
258	0xe0, 0x03, 0x9b, 0x13, 0xc6, 0x00, 0xbe, 0x91, 0xc9, 0x40, 0x00, 0x07,	初始化表中的一项，用于配置数组、结构体或常量表。
259	0xc4, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0xa4,	初始化表中的一项，用于配置数组、结构体或常量表。
260	0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x5b, 0x00, 0x00, 0x00, 0x8f, 0xff,	初始化表中的一项，用于配置数组、结构体或常量表。
261	0xff, 0xff, 0xff, 0xff, 0xf8, 0x00, 0x0c, 0x30, 0x00, 0x04, 0xd0, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
262	0x00, 0x08, 0x80, 0x00, 0x08, 0x90, 0x00, 0x00, 0x02, 0xd0, 0x00, 0x0e,	初始化表中的一项，用于配置数组、结构体或常量表。
263	0x20, 0x00, 0x00, 0x00, 0x99, 0x00, 0x89, 0x00, 0x00, 0x00, 0x00, 0x1d,	初始化表中的一项，用于配置数组、结构体或常量表。
264	0x65, 0xd1, 0x00, 0x00, 0x00, 0x00, 0x02, 0xee, 0x20, 0x00, 0x00, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
265	0x00, 0x4b, 0xbb, 0xc4, 0x00, 0x00, 0x01, 0x6c, 0xb4, 0x00, 0x5c, 0xc6,	初始化表中的一项，用于配置数组、结构体或常量表。
266	0x10, 0x4c, 0x72, 0x00, 0x00, 0x00, 0x38, 0xd3, 0x00, 0xcf, 0xff, 0xff,	初始化表中的一项，用于配置数组、结构体或常量表。
267	0xff, 0xfd, 0x00, 0x00, 0xc4, 0x00, 0x00, 0x00, 0x4d, 0x00, 0x00, 0xc4,	初始化表中的一项，用于配置数组、结构体或常量表。
268	0x00, 0x00, 0x00, 0x4d, 0x00, 0x00, 0xc4, 0x00, 0x00, 0x00, 0x4d, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
269	0x00, 0xc4, 0x00, 0x00, 0x00, 0x4d, 0x00, 0x00, 0xcf, 0xff, 0xff, 0xff,	初始化表中的一项，用于配置数组、结构体或常量表。
270	0xfd, 0x00, 0x00, 0xc4, 0x00, 0x00, 0x00, 0x4d, 0x00, 0x00, 0xc4, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
271	0x00, 0x00, 0x4d, 0x00, 0x00, 0xc4, 0x00, 0x00, 0x00, 0x4d, 0x00, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
272	0xc4, 0x00, 0x00, 0x00, 0x4d, 0x00, 0x00, 0xcf, 0xff, 0xff, 0xff, 0xfd,	初始化表中的一项，用于配置数组、结构体或常量表。
273	0x00, 0x00, 0xc4, 0x00, 0x00, 0x00, 0x4d, 0x00, 0x00, 0x00, 0x00, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。

行	源码	说明
274	0x00, 0x3d, 0x00, 0x3f, 0xff, 0xf7, 0x00, 0x00, 0x3d, 0x00, 0x3c, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
275	0x97, 0x00, 0x00, 0x3d, 0x00, 0x3c, 0x00, 0x98, 0xff, 0xff, 0xff, 0xfc,	初始化表中的一项，用于配置数组、结构体或常量表。
276	0x3c, 0x00, 0x97, 0x00, 0x00, 0x3d, 0x00, 0x3f, 0xff, 0xf7, 0x3b, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
277	0x3d, 0x00, 0x3c, 0x00, 0x97, 0x0c, 0x70, 0x3d, 0x00, 0x3c, 0x00, 0x97,	初始化表中的一项，用于配置数组、结构体或常量表。
278	0x04, 0xa0, 0x3d, 0x00, 0x3c, 0x00, 0x97, 0x00, 0x00, 0x3d, 0x00, 0x3f,	初始化表中的一项，用于配置数组、结构体或常量表。
279	0xff, 0xf7, 0x00, 0x00, 0x3d, 0x00, 0x3c, 0x00, 0x97, 0x00, 0x00, 0x6c,	初始化表中的一项，用于配置数组、结构体或常量表。
280	0x00, 0x00, 0x00, 0x00, 0x04, 0xff, 0xe6, 0x00, 0x01, 0xff, 0xff, 0xff,	初始化表中的一项，用于配置数组、结构体或常量表。
281	0xff, 0xff, 0x10, 0x01, 0xe0, 0x00, 0x00, 0x00, 0x0e, 0x10, 0x01, 0xff,	初始化表中的一项，用于配置数组、结构体或常量表。
282	0xff, 0xff, 0xff, 0xff, 0x10, 0x01, 0xe0, 0x00, 0x00, 0x00, 0x0e, 0x10,	初始化表中的一项，用于配置数组、结构体或常量表。
283	0x01, 0xff, 0xff, 0xff, 0xff, 0xff, 0x10, 0x01, 0xe0, 0x00, 0x00, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
284	0x0e, 0x10, 0x00, 0x00, 0x1e, 0x00, 0xf0, 0x00, 0x00, 0x08, 0x60, 0x1e,	初始化表中的一项，用于配置数组、结构体或常量表。
285	0x00, 0xf0, 0x06, 0x90, 0x02, 0xe5, 0x1e, 0x00, 0xf0, 0x4e, 0x30, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
286	0x4a, 0x1e, 0x00, 0xf0, 0xa4, 0x00, 0x00, 0x00, 0x1e, 0x00, 0xf0, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
287	0x00, 0x9f, 0xff, 0xff, 0xff, 0xff, 0xff, 0xf9, 0x00, 0x76, 0x00, 0x05,	初始化表中的一项，用于配置数组、结构体或常量表。
288	0xff, 0xff, 0xf3, 0xaf, 0xff, 0xff, 0xd5, 0x80, 0x00, 0x00, 0x0a, 0x55,	初始化表中的一项，用于配置数组、结构体或常量表。
289	0x80, 0x05, 0xff, 0xff, 0xf9, 0x2f, 0xff, 0xff, 0xb8, 0x50, 0x4a, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
290	0x01, 0x27, 0xa5, 0x5d, 0x10, 0x4a, 0x00, 0xad, 0xcc, 0xc8, 0x97, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
291	0x4a, 0x00, 0x00, 0xaf, 0xff, 0xff, 0xff, 0xf9, 0x00, 0x00, 0xa4, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
292	0x00, 0x00, 0x49, 0x00, 0x00, 0xaf, 0xff, 0xff, 0xff, 0xf9, 0x00, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
293	0xa4, 0x00, 0x00, 0x00, 0x49, 0x00, 0x00, 0xaf, 0xff, 0xff, 0xff, 0xf9,	初始化表中的一项，用于配置数组、结构体或常量表。
294	0x00, 0x00, 0xa4, 0x00, 0x00, 0x00, 0x49, 0x00, 0x06, 0x80, 0x08, 0x60,	初始化表中的一项，用于配置数组、结构体或常量表。
295	0xbf, 0xff, 0xf1, 0x8f, 0xff, 0xff, 0xf8, 0xb3, 0x00, 0xd1, 0x06, 0x80,	初始化表中的一项，用于配置数组、结构体或常量表。
296	0x08, 0x60, 0xb3, 0x00, 0xd1, 0x06, 0xff, 0xff, 0x60, 0xbf, 0xff, 0xf1,	初始化表中的一项，用于配置数组、结构体或常量表。

行	源码	说明
297	0x06, 0x80, 0x08, 0x60, 0xb3, 0x00, 0xd1, 0x06, 0xff, 0xff, 0x60, 0xb3,	初始化表中的一项，用于配置数组、结构体或常量表。
298	0x00, 0xd1, 0x06, 0x80, 0x08, 0x60, 0xc3, 0x00, 0xd1, 0x06, 0x80, 0x08,	初始化表中的一项，用于配置数组、结构体或常量表。
299	0x60, 0xdf, 0xff, 0xf1, 0xaf, 0xff, 0xff, 0xf8, 0xe0, 0x00, 0xd1, 0x01,	初始化表中的一项，用于配置数组、结构体或常量表。
300	0xb2, 0x59, 0x05, 0xc0, 0x00, 0xd1, 0x2d, 0x80, 0x07, 0x8c, 0x60, 0x01,	初始化表中的一项，用于配置数组、结构体或常量表。
301	0xe0, 0x76, 0x00, 0x00, 0x3a, 0x00, 0xee, 0xa0, 0x00, 0xc2, 0x00, 0x1c,	初始化表中的一项，用于配置数组、结构体或常量表。
302	0x00, 0x00, 0x00, 0x00, 0xc2, 0x00, 0x89, 0x00, 0x00, 0x00, 0x00, 0xc2,	初始化表中的一项，用于配置数组、结构体或常量表。
303	0x01, 0xef, 0xff, 0xff, 0xf4, 0xaf, 0xff, 0xda, 0x90, 0x00, 0x00, 0xc3,	初始化表中的一项，用于配置数组、结构体或常量表。
304	0x00, 0xe3, 0x1b, 0x05, 0x90, 0x00, 0xc3, 0x05, 0xed, 0x70, 0x0d, 0x40,	初始化表中的一项，用于配置数组、结构体或常量表。
305	0x00, 0xd2, 0x0b, 0xc3, 0xa2, 0x99, 0x2b, 0x00, 0xd2, 0x77, 0xc2, 0x06,	初始化表中的一项，用于配置数组、结构体或常量表。
306	0xc1, 0x09, 0x90, 0xe1, 0x70, 0xc2, 0x0b, 0xfe, 0xdc, 0xc3, 0xe1, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
307	0xc2, 0x00, 0x00, 0x00, 0x00, 0xf0, 0x00, 0xc2, 0x00, 0x00, 0x00, 0x05,	初始化表中的一项，用于配置数组、结构体或常量表。
308	0xd0, 0x00, 0xc2, 0x00, 0x00, 0xef, 0xec, 0x40, 0x1b, 0x10, 0x77, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
309	0xc4, 0x02, 0xb1, 0x08, 0xd2, 0x1d, 0x60, 0xc4, 0x1c, 0x90, 0x00, 0x85,	初始化表中的一项，用于配置数组、结构体或常量表。
310	0x03, 0xa0, 0xc4, 0x49, 0x00, 0x77, 0x00, 0x8f, 0xff, 0xff, 0xff, 0xe0,	初始化表中的一项，用于配置数组、结构体或常量表。
311	0x2d, 0x90, 0x87, 0x00, 0x00, 0x01, 0xe0, 0x02, 0xa0, 0x87, 0x00, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
312	0x01, 0xe0, 0x00, 0x00, 0x8f, 0xff, 0xff, 0xff, 0xe0, 0x04, 0xa0, 0x87,	初始化表中的一项，用于配置数组、结构体或常量表。
313	0x00, 0x00, 0x01, 0xe0, 0x08, 0xa0, 0x8f, 0xff, 0xff, 0xff, 0xe0, 0x0c,	初始化表中的一项，用于配置数组、结构体或常量表。
314	0x50, 0x87, 0x00, 0x00, 0x01, 0xe0, 0x2f, 0x10, 0x87, 0x00, 0x00, 0x04,	初始化表中的一项，用于配置数组、结构体或常量表。
315	0xe0, 0x4b, 0x00, 0x87, 0x00, 0x0e, 0xfe, 0x80, 0x00, 0x0b, 0x50, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
316	0x96, 0x69, 0x10, 0x00, 0x0b, 0x50, 0x00, 0xa6, 0x07, 0x90, 0x39, 0x0b,	初始化表中的一项，用于配置数组、结构体或常量表。
317	0x50, 0x00, 0xa6, 0x00, 0x00, 0x0d, 0x4b, 0x7f, 0xff, 0xff, 0xff, 0xf7,	初始化表中的一项，用于配置数组、结构体或常量表。
318	0x06, 0x8b, 0x50, 0x00, 0xc8, 0x00, 0x00, 0x00, 0x0b, 0x50, 0x00, 0xed,	初始化表中的一项，用于配置数组、结构体或常量表。
319	0x00, 0x00, 0x00, 0x9e, 0x50, 0x03, 0xcb, 0x30, 0x00, 0x1b, 0x7b, 0x50,	初始化表中的一项，用于配置数组、结构体或常量表。

行	源码	说明
320	0x09, 0x76, 0xa0, 0x00, 0x76, 0x0b, 0x50, 0x2e, 0x10, 0xd2, 0x00, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
321	0x0b, 0x50, 0xc6, 0x00, 0x5c, 0x00, 0x00, 0x0b, 0x6b, 0xa0, 0x00, 0x09,	初始化表中的一项，用于配置数组、结构体或常量表。
322	0xb1, 0x00, 0x0b, 0x99, 0x00, 0x00, 0x00, 0x96, 0xaf, 0xff, 0xe7, 0xff,	初始化表中的一项，用于配置数组、结构体或常量表。
323	0xff, 0xff, 0xe0, 0x00, 0xc2, 0x07, 0x80, 0x4b, 0x01, 0xe0, 0x00, 0xc2,	初始化表中的一项，用于配置数组、结构体或常量表。
324	0x07, 0xff, 0xff, 0xff, 0xe0, 0x00, 0xc2, 0x07, 0x80, 0x4b, 0x01, 0xe0,	初始化表中的一项，用于配置数组、结构体或常量表。
325	0x00, 0xc2, 0x07, 0x80, 0x4b, 0x01, 0xe0, 0x8f, 0xff, 0xd7, 0xff, 0xff,	初始化表中的一项，用于配置数组、结构体或常量表。
326	0xff, 0xe0, 0x00, 0xc2, 0x07, 0x80, 0x4b, 0x01, 0xe0, 0x00, 0xc2, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
327	0x00, 0x4b, 0x00, 0x00, 0x00, 0xc2, 0x0b, 0xff, 0xff, 0xff, 0xf3, 0x14,	初始化表中的一项，用于配置数组、结构体或常量表。
328	0xdb, 0xd0, 0x00, 0x4b, 0x00, 0x00, 0xab, 0x85, 0x10, 0x00, 0x4b, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
329	0x00, 0x00, 0x00, 0x8f, 0xff, 0xff, 0xff, 0xfc, 0x00, 0x00, 0x00, 0xf0,	初始化表中的一项，用于配置数组、结构体或常量表。
330	0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0xf0, 0x00, 0x00, 0x00, 0x1f, 0xff,	初始化表中的一项，用于配置数组、结构体或常量表。
331	0xff, 0xff, 0xff, 0xff, 0x10, 0x1f, 0x00, 0x00, 0xf0, 0x00, 0x0f, 0x10,	初始化表中的一项，用于配置数组、结构体或常量表。
332	0x1f, 0x00, 0x00, 0xf0, 0x00, 0x0f, 0x10, 0x1f, 0xff, 0xff, 0xff, 0xff,	初始化表中的一项，用于配置数组、结构体或常量表。
333	0xff, 0x10, 0x1f, 0x00, 0x00, 0xf0, 0x00, 0x0f, 0x10, 0x1f, 0x00, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
334	0xf0, 0x00, 0x0f, 0x10, 0x1f, 0xff, 0xff, 0xff, 0xff, 0xff, 0x10, 0x1f,	初始化表中的一项，用于配置数组、结构体或常量表。
335	0x00, 0x00, 0xf0, 0x00, 0x00, 0x67, 0x00, 0x00, 0x00, 0xe3, 0x00, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
336	0xb7, 0x00, 0x00, 0x00, 0x8e, 0xff, 0xfe, 0xc1, 0x05, 0xff, 0xff, 0xff,	初始化表中的一项，用于配置数组、结构体或常量表。
337	0xff, 0xff, 0x50, 0x05, 0xa0, 0x00, 0x77, 0x00, 0x0a, 0x50, 0x05, 0xff,	初始化表中的一项，用于配置数组、结构体或常量表。
338	0xff, 0xff, 0xff, 0xff, 0x50, 0x05, 0xa0, 0x00, 0x77, 0x00, 0x0a, 0x50,	初始化表中的一项，用于配置数组、结构体或常量表。
339	0x05, 0xff, 0xff, 0xff, 0xff, 0xff, 0x50, 0x05, 0xa0, 0x3d, 0x43, 0xc2,	初始化表中的一项，用于配置数组、结构体或常量表。
340	0x0a, 0x50, 0x00, 0x07, 0xc2, 0x00, 0x2b, 0x40, 0x00, 0x05, 0xc8, 0xe1,	初始化表中的一项，用于配置数组、结构体或常量表。
341	0x00, 0x0f, 0x8a, 0x30, 0x6a, 0x21, 0xe0, 0x00, 0x0f, 0x03, 0xa6, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
342	0x07, 0xb0, 0x00, 0x0f, 0x00, 0x00, 0x00, 0x7d, 0x20, 0x00, 0x0f, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。

行	源码	说明
343	0x00, 0x07, 0xa1, 0x00, 0x00, 0x0f, 0x00, 0x00, 0x00, 0x00, 0x00, 0x01,	初始化表中的一项, 用于配置数组、结构体或常量表。
344	0x12, 0x33, 0x00, 0x04, 0xee, 0xee, 0xee, 0xdc, 0xbb, 0x30, 0x00, 0x2a,	初始化表中的一项, 用于配置数组、结构体或常量表。
345	0x00, 0x77, 0x00, 0xb2, 0x00, 0x00, 0x0a, 0x30, 0x77, 0x04, 0xb0, 0x00,	初始化表中的一项, 用于配置数组、结构体或常量表。
346	0x5f, 0xff, 0xff, 0xff, 0xff, 0xff, 0xf5, 0x00, 0x00, 0x7c, 0xa9, 0xc7,	初始化表中的一项, 用于配置数组、结构体或常量表。
347	0x00, 0x00, 0x02, 0x8c, 0x70, 0x77, 0x07, 0xc8, 0x20, 0x7b, 0x60, 0x00,	初始化表中的一项, 用于配置数组、结构体或常量表。
348	0x77, 0x00, 0x06, 0xb7, 0x00, 0xcf, 0xff, 0xff, 0xff, 0xfc, 0x00, 0x00,	初始化表中的一项, 用于配置数组、结构体或常量表。
349	0xc1, 0x00, 0x77, 0x00, 0x1c, 0x00, 0x00, 0xcf, 0xff, 0xff, 0xff, 0xfc,	初始化表中的一项, 用于配置数组、结构体或常量表。
350	0x00, 0x00, 0xc1, 0x00, 0x77, 0x00, 0x1c, 0x00, 0x00, 0xcf, 0xff, 0xff,	初始化表中的一项, 用于配置数组、结构体或常量表。
351	0xff, 0xfc, 0x00, 0x00, 0xc1, 0x00, 0x00, 0x00, 0x1c, 0x00, 0x07, 0xff,	初始化表中的一项, 用于配置数组、结构体或常量表。
352	0xff, 0xff, 0xff, 0xff, 0x80, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,	初始化表中的一项, 用于配置数组、结构体或常量表。
353	0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,	初始化表中的一项, 用于配置数组、结构体或常量表。
354	0x00, 0x00, 0xbf, 0xff, 0xff, 0xff, 0xff, 0xff, 0xfb, 0x00, 0x00, 0x00,	初始化表中的一项, 用于配置数组、结构体或常量表。
355	0x3d, 0x00, 0x00, 0x00, 0x00, 0x1b, 0x00, 0x3d, 0x01, 0xb0, 0x00, 0x00,	初始化表中的一项, 用于配置数组、结构体或常量表。
356	0xa8, 0x00, 0x3d, 0x00, 0xaa, 0x00, 0x05, 0xc0, 0x00, 0x3d, 0x00, 0x0d,	初始化表中的一项, 用于配置数组、结构体或常量表。
357	0x60, 0x2d, 0x30, 0x00, 0x3d, 0x00, 0x03, 0xe3, 0x57, 0x00, 0x00, 0x6d,	初始化表中的一项, 用于配置数组、结构体或常量表。
358	0x00, 0x00, 0x77, 0x00, 0x01, 0xef, 0xe6, 0x00, 0x00, 0x00, 0x00, 0x00,	初始化表中的一项, 用于配置数组、结构体或常量表。
359	0x00, 0xb5, 0x00, 0x00, 0x00, 0x8f, 0xff, 0xff, 0xff, 0xff, 0xff, 0xf8,	初始化表中的一项, 用于配置数组、结构体或常量表。
360	0x00, 0x03, 0xb6, 0x10, 0x6b, 0x00, 0x00, 0x02, 0xc0, 0x19, 0xfe, 0xc2,	初始化表中的一项, 用于配置数组、结构体或常量表。
361	0x0c, 0x20, 0x02, 0xc5, 0xb7, 0x31, 0x6b, 0x3c, 0x20, 0x02, 0xff, 0xff,	初始化表中的一项, 用于配置数组、结构体或常量表。
362	0xff, 0xff, 0xff, 0x20, 0x00, 0x00, 0x00, 0xd4, 0x00, 0x00, 0x00, 0x0a,	初始化表中的一项, 用于配置数组、结构体或常量表。
363	0xff, 0xff, 0xff, 0xff, 0xff, 0xa0, 0x0a, 0x40, 0x6a, 0x11, 0xb4, 0x05,	初始化表中的一项, 用于配置数组、结构体或常量表。
364	0xa0, 0x0a, 0x41, 0xee, 0xed, 0xcd, 0x25, 0xa0, 0x0a, 0x40, 0x00, 0x00,	初始化表中的一项, 用于配置数组、结构体或常量表。
365	0x00, 0x06, 0xa0, 0x0a, 0x40, 0x00, 0x00, 0x03, 0xee, 0x50, 0x00, 0x01,	初始化表中的一项, 用于配置数组、结构体或常量表。

行	源码	说明
366	0x23, 0x45, 0x67, 0x89, 0x70, 0x0c, 0xed, 0xcc, 0xea, 0x87, 0x65, 0x30,	初始化表中的一项, 用于配置数组、结构体或常量表。
367	0x00, 0x00, 0x5d, 0x80, 0x00, 0x00, 0x00, 0x01, 0x5c, 0xa2, 0x00, 0x06,	初始化表中的一项, 用于配置数组、结构体或常量表。
368	0xc6, 0x00, 0x06, 0xfd, 0xbb, 0xcd, 0xec, 0x50, 0x00, 0x00, 0x00, 0x06,	初始化表中的一项, 用于配置数组、结构体或常量表。
369	0xca, 0x30, 0x00, 0x00, 0x00, 0x49, 0xb7, 0x22, 0x29, 0x81, 0x00, 0x07,	初始化表中的一项, 用于配置数组、结构体或常量表。
370	0xfe, 0xed, 0xdf, 0xed, 0xdc, 0x50, 0x00, 0x00, 0x00, 0x5a, 0x00, 0x01,	初始化表中的一项, 用于配置数组、结构体或常量表。
371	0xb1, 0x00, 0x3c, 0x30, 0x5a, 0x09, 0x80, 0x00, 0x08, 0xd5, 0x00, 0x7a,	初始化表中的一项, 用于配置数组、结构体或常量表。
372	0x01, 0x9d, 0x50, 0x79, 0x10, 0xbe, 0xe5, 0x00, 0x03, 0xc3, 0x00, 0x68,	初始化表中的一项, 用于配置数组、结构体或常量表。
373	0x00, 0x07, 0x90, 0xb4, 0x00, 0x00, 0xd3, 0x00, 0x06, 0x90, 0x2b, 0x20,	初始化表中的一项, 用于配置数组、结构体或常量表。
374	0x08, 0x82, 0xb0, 0x28, 0xc8, 0x9b, 0xd5, 0x3c, 0x0a, 0x6c, 0xcc, 0xd7,	初始化表中的一项, 用于配置数组、结构体或常量表。
375	0x53, 0x10, 0x9d, 0xdc, 0x00, 0x03, 0xc0, 0x00, 0x00, 0x00, 0xb2, 0x01,	初始化表中的一项, 用于配置数组、结构体或常量表。
376	0x26, 0xe8, 0x9b, 0xd9, 0x06, 0x60, 0x2e, 0xcb, 0xe7, 0x53, 0x20, 0x2e,	初始化表中的一项, 用于配置数组、结构体或常量表。
377	0xde, 0x90, 0x00, 0xc3, 0x07, 0x90, 0x5b, 0x61, 0x00, 0x00, 0x89, 0xac,	初始化表中的一项, 用于配置数组、结构体或常量表。
378	0x20, 0x00, 0x17, 0xd0, 0x00, 0x8f, 0x80, 0x00, 0x28, 0xd7, 0x12, 0x7d,	初始化表中的一项, 用于配置数组、结构体或常量表。
379	0xbb, 0xc4, 0x78, 0x87, 0x10, 0x5c, 0x72, 0x00, 0x8d, 0xd4, 0x00, 0x95,	初始化表中的一项, 用于配置数组、结构体或常量表。
380	0x00, 0x0c, 0x20, 0x00, 0x00, 0x01, 0xe1, 0x00, 0x8f, 0xff, 0xff, 0xb0,	初始化表中的一项, 用于配置数组、结构体或常量表。
381	0x09, 0x60, 0xc7, 0xdc, 0x00, 0x1c, 0x60, 0x3c, 0x07, 0x7a, 0x27, 0xb3,	初始化表中的一项, 用于配置数组、结构体或常量表。
382	0xc7, 0x00, 0x7d, 0xda, 0x00, 0x00, 0xbf, 0x90, 0x00, 0x00, 0x91, 0x11,	初始化表中的一项, 用于配置数组、结构体或常量表。
383	0x6c, 0xa6, 0xcc, 0x61, 0x1b, 0xad, 0x8c, 0x82, 0x00, 0x05, 0xa7, 0x5b,	初始化表中的一项, 用于配置数组、结构体或常量表。
384	0x61, 0x00, 0xff, 0xff, 0xff, 0xa0, 0x00, 0x00, 0x10, 0xf0, 0x00, 0x05,	初始化表中的一项, 用于配置数组、结构体或常量表。
385	0xa0, 0x04, 0x8c, 0xd1, 0xf0, 0x00, 0x05, 0xa0, 0x7b, 0x72, 0x00, 0xff,	初始化表中的一项, 用于配置数组、结构体或常量表。
386	0xff, 0xff, 0xa0, 0x00, 0x00, 0x00, 0xf0, 0x00, 0x05, 0xa0, 0x00, 0x68,	初始化表中的一项, 用于配置数组、结构体或常量表。
387	0x00, 0x00, 0xa4, 0x00, 0x00, 0x00, 0xc4, 0x00, 0x00, 0x5b, 0x00, 0x00,	初始化表中的一项, 用于配置数组、结构体或常量表。
388	0x04, 0xc0, 0x6f, 0xff, 0xff, 0xff, 0xf8, 0x0c, 0x40, 0x00, 0x0b, 0x60,	初始化表中的一项, 用于配置数组、结构体或常量表。

行	源码	说明
389	0x00, 0x00, 0x6a, 0x08, 0x80, 0x9b, 0x00, 0x67, 0x00, 0x9d, 0xeb, 0x18,	初始化表中的一项, 用于配置数组、结构体或常量表。
390	0xd1, 0x00, 0x1e, 0x30, 0x01, 0x81, 0x3e, 0xff, 0xee, 0xee, 0xd0, 0x1c,	初始化表中的一项, 用于配置数组、结构体或常量表。
391	0xde, 0x80, 0x4c, 0x08, 0x80, 0xa3, 0x5b, 0x61, 0x10, 0x5a, 0x08, 0x80,	初始化表中的一项, 用于配置数组、结构体或常量表。
392	0x00, 0x00, 0x29, 0xb0, 0xb6, 0x08, 0x80, 0x48, 0x3a, 0xc5, 0x2a, 0xb0,	初始化表中的一项, 用于配置数组、结构体或常量表。
393	0x07, 0x90, 0x79, 0x75, 0x02, 0xc6, 0x00, 0x03, 0xdf, 0xe4, 0x0c, 0xff,	初始化表中的一项, 用于配置数组、结构体或常量表。
394	0xff, 0xff, 0xff, 0xff, 0xc0, 0x0c, 0x20, 0x00, 0x00, 0x00, 0x03, 0xc0,	初始化表中的一项, 用于配置数组、结构体或常量表。
395	0x0c, 0x86, 0x04, 0x80, 0x00, 0xa6, 0xc0, 0x0c, 0x4d, 0x1a, 0x5b, 0x12,	初始化表中的一项, 用于配置数组、结构体或常量表。
396	0xd3, 0xc0, 0x0c, 0x26, 0xbd, 0x07, 0x88, 0x73, 0xc0, 0x0c, 0x20, 0xd7,	初始化表中的一项, 用于配置数组、结构体或常量表。
397	0x00, 0xdd, 0x13, 0xc0, 0x0c, 0x22, 0xeb, 0x00, 0x8c, 0x03, 0xc0, 0x0c,	初始化表中的一项, 用于配置数组、结构体或常量表。
398	0x2a, 0x6b, 0x52, 0xdc, 0x53, 0xc0, 0x0c, 0x7d, 0x04, 0x9b, 0x53, 0xd3,	初始化表中的一项, 用于配置数组、结构体或常量表。
399	0xc0, 0x0c, 0xb5, 0x00, 0x39, 0x00, 0x96, 0xc0, 0x0c, 0x30, 0x00, 0x00,	初始化表中的一项, 用于配置数组、结构体或常量表。
400	0x00, 0x05, 0xc0, 0x0c, 0x20, 0x00, 0x00, 0x0e, 0xfe, 0x60, 0x09, 0xff,	初始化表中的一项, 用于配置数组、结构体或常量表。
401	0xff, 0xff, 0xff, 0xff, 0x80, 0x09, 0x50, 0x2b, 0x00, 0xb2, 0x05, 0x80,	初始化表中的一项, 用于配置数组、结构体或常量表。
402	0x09, 0xff, 0xff, 0xff, 0xff, 0xff, 0x80, 0x00, 0x00, 0x00, 0x86, 0x00,	初始化表中的一项, 用于配置数组、结构体或常量表。
403	0x00, 0x00, 0x7f, 0xff, 0xff, 0xff, 0xff, 0xff, 0xf6, 0x00, 0x00, 0x00,	初始化表中的一项, 用于配置数组、结构体或常量表。
404	0x86, 0x00, 0x00, 0x00, 0x00, 0xbf, 0xff, 0xff, 0xff, 0xfd, 0x00, 0x00,	初始化表中的一项, 用于配置数组、结构体或常量表。
405	0xb3, 0x00, 0x00, 0x00, 0x1d, 0x00, 0x00, 0xbf, 0xff, 0xff, 0xff, 0xfd,	初始化表中的一项, 用于配置数组、结构体或常量表。
406	0x00, 0x00, 0xb3, 0x00, 0x00, 0x00, 0x1d, 0x00, 0x00, 0xb3, 0x00, 0x00,	初始化表中的一项, 用于配置数组、结构体或常量表。
407	0x00, 0x1d, 0x00, 0xcf, 0xff, 0xff, 0xff, 0xff, 0xff, 0xfb, 0x00, 0x79,	初始化表中的一项, 用于配置数组、结构体或常量表。
408	0x00, 0x01, 0xe0, 0x03, 0xb1, 0x07, 0x82, 0x4a, 0x01, 0xe1, 0x8b, 0x30,	初始化表中的一项, 用于配置数组、结构体或常量表。
409	0x7d, 0xcb, 0xa8, 0xa1, 0xfc, 0x50, 0x66, 0x00, 0x00, 0x00, 0x84, 0xe1,	初始化表中的一项, 用于配置数组、结构体或常量表。
410	0x00, 0xa7, 0x0e, 0xff, 0xff, 0x80, 0xae, 0xfe, 0xc2, 0x0e, 0x10, 0x07,	初始化表中的一项, 用于配置数组、结构体或常量表。
411	0x80, 0x00, 0x00, 0x00, 0x0e, 0xff, 0xff, 0x81, 0xe0, 0x01, 0xa5, 0x0e,	初始化表中的一项, 用于配置数组、结构体或常量表。

行	源码	说明
412	0x10, 0x07, 0x81, 0xe1, 0x7d, 0x60, 0x0e, 0xff, 0xff, 0x81, 0xed, 0x70,	初始化表中的一项，用于配置数组、结构体或常量表。
413	0x00, 0x0e, 0x10, 0x07, 0x81, 0xe0, 0x00, 0x67, 0x0e, 0x10, 0x09, 0x81,	初始化表中的一项，用于配置数组、结构体或常量表。
414	0xe1, 0x00, 0x97, 0x0e, 0x16, 0xfe, 0x30, 0xae, 0xff, 0xd2, 0x00, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
415	0x00, 0x00, 0x00, 0x00, 0x00, 0x0d, 0xff, 0xb0, 0x00, 0x3a, 0x00, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
416	0x0d, 0x12, 0xb0, 0x00, 0x0b, 0x20, 0x00, 0x0d, 0x12, 0xb8, 0xff, 0xff,	初始化表中的一项，用于配置数组、结构体或常量表。
417	0xff, 0xfc, 0x0d, 0xff, 0xb0, 0x00, 0x00, 0x00, 0x00, 0x0d, 0x12, 0xb0,	初始化表中的一项，用于配置数组、结构体或常量表。
418	0x07, 0x70, 0x86, 0x00, 0x0d, 0x12, 0xb0, 0xe1, 0xcb, 0xc0, 0xc2, 0x0d,	初始化表中的一项，用于配置数组、结构体或常量表。
419	0x12, 0xb0, 0xe0, 0x2e, 0x40, 0xc2, 0x0d, 0xff, 0xb0, 0xe0, 0xbc, 0xd1,	初始化表中的一项，用于配置数组、结构体或常量表。
420	0xc2, 0x0e, 0x02, 0xb0, 0xe8, 0x80, 0x78, 0xc2, 0x1e, 0x02, 0xb0, 0xe0,	初始化表中的一项，用于配置数组、结构体或常量表。
421	0x00, 0x00, 0xc2, 0x7a, 0x03, 0xb0, 0xff, 0xff, 0xff, 0xf2, 0x74, 0xce,	初始化表中的一项，用于配置数组、结构体或常量表。
422	0x70, 0x00, 0x00, 0x00, 0xc2, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
423	0x00, 0x00, 0xf0, 0x00, 0x0f, 0x00, 0x00, 0x9f, 0xff, 0xff, 0xff, 0xff,	初始化表中的一项，用于配置数组、结构体或常量表。
424	0xff, 0xfa, 0x00, 0x00, 0xf0, 0x00, 0x0f, 0x00, 0x00, 0x00, 0x00, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
425	0x78, 0x00, 0x00, 0x00, 0x00, 0xef, 0xff, 0xff, 0xff, 0xfe, 0x00, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
426	0xe1, 0x00, 0x78, 0x00, 0x1e, 0x00, 0x00, 0xe1, 0x00, 0x87, 0x00, 0x1e,	初始化表中的一项，用于配置数组、结构体或常量表。
427	0x00, 0xaf, 0xff, 0xff, 0xff, 0xff, 0xff, 0xfb, 0x00, 0x00, 0x04, 0xba,	初始化表中的一项，用于配置数组、结构体或常量表。
428	0x30, 0x00, 0x00, 0x00, 0x00, 0x6b, 0x11, 0xb3, 0x00, 0x00, 0x02, 0x7c,	初始化表中的一项，用于配置数组、结构体或常量表。
429	0x70, 0x00, 0x1a, 0x93, 0x00, 0x6c, 0x61, 0x00, 0x00, 0x00, 0x38, 0xc5,	初始化表中的一项，用于配置数组、结构体或常量表。
430	0x00, 0x01, 0xe0, 0x00, 0x0d, 0x10, 0x00, 0xaf, 0xff, 0xff, 0xff, 0xff,	初始化表中的一项，用于配置数组、结构体或常量表。
431	0xff, 0xfa, 0x00, 0x01, 0xe0, 0x00, 0x0d, 0x10, 0x00, 0x00, 0x86, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
432	0x00, 0x00, 0x00, 0x00, 0x7f, 0xff, 0xff, 0x90, 0xef, 0xff, 0xf3, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
433	0x96, 0x06, 0x90, 0xe0, 0x00, 0xc3, 0x00, 0xa5, 0x06, 0x80, 0xe0, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
434	0xc3, 0x00, 0xd2, 0x07, 0x80, 0xe0, 0x00, 0xc3, 0x02, 0xe0, 0x07, 0x80,	初始化表中的一项，用于配置数组、结构体或常量表。

行	源码	说明
435	0xe0, 0x00, 0xc3, 0x08, 0x80, 0x08, 0x70, 0xe0, 0x00, 0xc3, 0x3e, 0x10,	初始化表中的一项，用于配置数组、结构体或常量表。
436	0x0b, 0x50, 0xef, 0xff, 0xf3, 0x75, 0x0e, 0xeb, 0x00, 0xe0, 0x00, 0xc3,	初始化表中的一项，用于配置数组、结构体或常量表。
437	0x00, 0x00, 0x00, 0x97, 0x00, 0x00, 0x00, 0x9f, 0xff, 0xff, 0xff, 0xff,	初始化表中的一项，用于配置数组、结构体或常量表。
438	0xff, 0xfa, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x04, 0xff, 0xff,	初始化表中的一项，用于配置数组、结构体或常量表。
439	0xff, 0xff, 0xff, 0x50, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x04,	初始化表中的一项，用于配置数组、结构体或常量表。
440	0xff, 0xff, 0xff, 0xff, 0xff, 0x50, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
441	0x00, 0x00, 0xff, 0xff, 0xff, 0xff, 0xff, 0x00, 0x00, 0xf1, 0x00, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
442	0x00, 0x1f, 0x00, 0x00, 0xf1, 0x00, 0x00, 0x00, 0x1f, 0x00, 0x00, 0xff,	初始化表中的一项，用于配置数组、结构体或常量表。
443	0xff, 0xff, 0xff, 0xff, 0x00, 0x00, 0xf1, 0x00, 0x00, 0x00, 0x1f, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
444	0x04, 0xb3, 0x00, 0xcf, 0xff, 0xfa, 0x00, 0x00, 0x7a, 0x00, 0xd2, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
445	0x5a, 0x00, 0x00, 0x00, 0x02, 0xe0, 0x00, 0x5a, 0x00, 0x00, 0x00, 0x2c,	初始化表中的一项，用于配置数组、结构体或常量表。
446	0x80, 0x00, 0x5b, 0x00, 0xaf, 0xf1, 0x79, 0x00, 0x00, 0x1d, 0xfb, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
447	0xe1, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0xe1, 0x1f, 0xff, 0xff, 0xff,	初始化表中的一项，用于配置数组、结构体或常量表。
448	0x40, 0x00, 0xe1, 0x01, 0xc1, 0x00, 0x4d, 0x10, 0x00, 0xe3, 0xa0, 0x5b,	初始化表中的一项，用于配置数组、结构体或常量表。
449	0x14, 0xd3, 0x00, 0x01, 0xee, 0x80, 0x07, 0xee, 0x20, 0x00, 0x02, 0xb3,	初始化表中的一项，用于配置数组、结构体或常量表。
450	0x47, 0xcd, 0x7a, 0xc7, 0x41, 0x00, 0x04, 0xd8, 0x40, 0x00, 0x28, 0xc6,	初始化表中的一项，用于配置数组、结构体或常量表。
451	0x0b, 0x20, 0xaf, 0xff, 0xff, 0xff, 0xf2, 0x07, 0xd2, 0x00, 0x2c, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
452	0x00, 0x00, 0x00, 0x85, 0x5f, 0xff, 0xff, 0xff, 0x00, 0x00, 0x00, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
453	0x78, 0x00, 0x0e, 0x00, 0xbf, 0xc0, 0x00, 0x96, 0x00, 0x0e, 0x00, 0x03,	初始化表中的一项，用于配置数组、结构体或常量表。
454	0xc0, 0xff, 0xff, 0xff, 0xff, 0xfb, 0x03, 0xc0, 0x00, 0x00, 0x00, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
455	0x00, 0x03, 0xc0, 0x0c, 0xff, 0xff, 0xff, 0x70, 0x03, 0xc0, 0x0c, 0x30,	初始化表中的一项，用于配置数组、结构体或常量表。
456	0x00, 0x07, 0x70, 0x03, 0xc5, 0x7c, 0x30, 0x00, 0x07, 0x70, 0x04, 0xfc,	初始化表中的一项，用于配置数组、结构体或常量表。
457	0x2c, 0xff, 0xff, 0xff, 0x70, 0x06, 0x80, 0x0c, 0x30, 0x00, 0x07, 0x70,	初始化表中的一项，用于配置数组、结构体或常量表。

行	源码	说明
458	0x00, 0x04, 0xa0, 0x00, 0xc2, 0x89, 0x10, 0x2f, 0xff, 0xff, 0xf5, 0xc3,	初始化表中的一项，用于配置数组、结构体或常量表。
459	0x07, 0x90, 0x00, 0x04, 0xa0, 0x00, 0xb3, 0x00, 0x00, 0xaf, 0xff, 0xff,	初始化表中的一项，用于配置数组、结构体或常量表。
460	0xff, 0xff, 0xff, 0xfc, 0x00, 0x1c, 0x10, 0x00, 0x95, 0x00, 0x00, 0x5f,	初始化表中的一项，用于配置数组、结构体或常量表。
461	0xff, 0xff, 0xfb, 0x77, 0x07, 0x60, 0x06, 0xa0, 0xd0, 0x00, 0x59, 0x1d,	初始化表中的一项，用于配置数组、结构体或常量表。
462	0x20, 0x0e, 0xff, 0xff, 0xf9, 0x2d, 0xa7, 0x00, 0x00, 0x00, 0xd0, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
463	0x0d, 0xb0, 0x00, 0x01, 0x35, 0xea, 0xc9, 0x6e, 0x90, 0x39, 0x6d, 0xa8,	初始化表中的一项，用于配置数组、结构体或常量表。
464	0xe4, 0x29, 0xc2, 0xd4, 0x78, 0x00, 0x00, 0xd0, 0x5a, 0x10, 0x2c, 0xd2,	初始化表中的一项，用于配置数组、结构体或常量表。
465	0x00, 0xc0, 0x00, 0x00, 0xae, 0x10, 0x00, 0x02, 0xc0, 0x00, 0x07, 0x96,	初始化表中的一项，用于配置数组、结构体或常量表。
466	0xa0, 0x00, 0xaf, 0xff, 0xf2, 0x68, 0x00, 0x57, 0x00, 0x0a, 0x40, 0x09,	初始化表中的一项，用于配置数组、结构体或常量表。
467	0xbf, 0xff, 0xfa, 0x90, 0xd0, 0x67, 0x57, 0x00, 0x00, 0x00, 0x48, 0x68,	初始化表中的一项，用于配置数组、结构体或常量表。
468	0x67, 0x0a, 0xff, 0xd0, 0x00, 0xc1, 0x8f, 0xff, 0xca, 0x20, 0xd0, 0xd0,	初始化表中的一项，用于配置数组、结构体或常量表。
469	0xc1, 0x00, 0x67, 0x0a, 0xff, 0xd0, 0xd0, 0xc1, 0x03, 0x9c, 0xca, 0x20,	初始化表中的一项，用于配置数组、结构体或常量表。
470	0xd0, 0xd0, 0xc1, 0x9b, 0xb9, 0x1a, 0xff, 0xd0, 0xd0, 0xc1, 0x00, 0x67,	初始化表中的一项，用于配置数组、结构体或常量表。
471	0x0a, 0x20, 0xd0, 0x00, 0xd1, 0x00, 0x67, 0x0a, 0x7f, 0xa0, 0xde, 0xb0,	初始化表中的一项，用于配置数组、结构体或常量表。
472	0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x0a, 0x40, 0x00, 0x12, 0x35,	初始化表中的一项，用于配置数组、结构体或常量表。
473	0x7a, 0xd2, 0x04, 0xe2, 0xd0, 0xed, 0xb9, 0x75, 0x20, 0x00, 0x86, 0xd0,	初始化表中的一项，用于配置数组、结构体或常量表。
474	0x20, 0x00, 0x00, 0x00, 0x00, 0x00, 0xd0, 0x20, 0x00, 0x00, 0x00, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
475	0x00, 0xd0, 0xff, 0xff, 0xff, 0xb0, 0xaf, 0xe0, 0x0e, 0x6a, 0x20, 0xb0,	初始化表中的一项，用于配置数组、结构体或常量表。
476	0x60, 0x01, 0xe0, 0x2e, 0x06, 0xd7, 0xb9, 0x00, 0x01, 0xe0, 0x6b, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
477	0x5f, 0xe2, 0x00, 0x01, 0xe1, 0xd5, 0x4a, 0xc4, 0x7e, 0x70, 0x03, 0xe3,	初始化表中的一项，用于配置数组、结构体或常量表。
478	0x95, 0xb5, 0x00, 0x03, 0xb1, 0x5c, 0x7a, 0x30, 0x00, 0x00, 0x00, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
479	0x72, 0x05, 0xbe, 0xee, 0xef, 0xff, 0xf8, 0x00, 0x00, 0x00, 0x00, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
480	0x00, 0x00, 0x0a, 0x40, 0x05, 0xa0, 0x96, 0x00, 0x00, 0x04, 0xe2, 0x0a,	初始化表中的一项，用于配置数组、结构体或常量表。

行	源码	说明
481	0x80, 0x96, 0x00, 0x00, 0x00, 0x86, 0x2e, 0xff, 0xff, 0xff, 0xf0, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
482	0x00, 0x69, 0x00, 0x96, 0x00, 0x00, 0xaf, 0xe0, 0x00, 0x00, 0x96, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
483	0x00, 0x01, 0xe2, 0xff, 0xff, 0xff, 0xff, 0xf9, 0x01, 0xe0, 0x00, 0x87,	初始化表中的一项，用于配置数组、结构体或常量表。
484	0x09, 0x60, 0x00, 0x01, 0xe0, 0x00, 0xb5, 0x09, 0x60, 0x66, 0x01, 0xe0,	初始化表中的一项，用于配置数组、结构体或常量表。
485	0x2a, 0xc0, 0x09, 0x70, 0xa6, 0x05, 0xe1, 0xa9, 0x00, 0x04, 0xef, 0xc1,	初始化表中的一项，用于配置数组、结构体或常量表。
486	0x6d, 0x7c, 0x41, 0x00, 0x00, 0x00, 0x10, 0x72, 0x04, 0xbd, 0xee, 0xfe,	初始化表中的一项，用于配置数组、结构体或常量表。
487	0xee, 0xe8, 0x8f, 0xff, 0xff, 0xfa, 0xcf, 0xff, 0xe0, 0x00, 0x48, 0x76,	初始化表中的一项，用于配置数组、结构体或常量表。
488	0x00, 0x00, 0x00, 0xe0, 0x00, 0x48, 0x76, 0x00, 0x00, 0x00, 0xe0, 0x2f,	初始化表中的一项，用于配置数组、结构体或常量表。
489	0xff, 0xff, 0xf5, 0x00, 0x00, 0xe0, 0x2b, 0x58, 0x76, 0x85, 0x9f, 0xff,	初始化表中的一项，用于配置数组、结构体或常量表。
490	0xe0, 0x2b, 0x96, 0x76, 0x85, 0x95, 0x00, 0xe0, 0x2c, 0xd1, 0x4e, 0xf5,	初始化表中的一项，用于配置数组、结构体或常量表。
491	0x95, 0x00, 0x00, 0x2b, 0x00, 0x00, 0x85, 0x95, 0x00, 0x00, 0x2f, 0xff,	初始化表中的一项，用于配置数组、结构体或常量表。
492	0xff, 0xf5, 0x95, 0x00, 0x48, 0x2b, 0x00, 0x00, 0x85, 0x95, 0x00, 0x59,	初始化表中的一项，用于配置数组、结构体或常量表。
493	0x2f, 0xff, 0xff, 0xf5, 0x88, 0x00, 0xa7, 0x2b, 0x00, 0x00, 0x85, 0x2d,	初始化表中的一项，用于配置数组、结构体或常量表。
494	0xff, 0xc1, 0x00, 0x00, 0x01, 0x12, 0x23, 0x34, 0x20, 0x0b, 0xee, 0xdd,	初始化表中的一项，用于配置数组、结构体或常量表。
495	0xed, 0xcb, 0xaa, 0x40, 0x00, 0x00, 0x00, 0x77, 0x00, 0x00, 0x00, 0x9f,	初始化表中的一项，用于配置数组、结构体或常量表。
496	0xff, 0xff, 0xff, 0xff, 0xff, 0xfa, 0x00, 0x00, 0x00, 0x77, 0x00, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
497	0x00, 0x02, 0xff, 0xff, 0xff, 0xff, 0xff, 0x20, 0x02, 0xc0, 0x00, 0x77,	初始化表中的一项，用于配置数组、结构体或常量表。
498	0x00, 0x0b, 0x20, 0x02, 0xff, 0xff, 0xff, 0xff, 0xff, 0x20, 0x02, 0xc0,	初始化表中的一项，用于配置数组、结构体或常量表。
499	0x00, 0x77, 0x00, 0x0b, 0x20, 0x02, 0xff, 0xff, 0xff, 0xff, 0xff, 0x20,	初始化表中的一项，用于配置数组、结构体或常量表。
500	0x00, 0x00, 0x00, 0x77, 0x00, 0x00, 0x00, 0x0a, 0xff, 0xff, 0xff, 0xff,	初始化表中的一项，用于配置数组、结构体或常量表。
501	0xff, 0xa0, 0x00, 0x00, 0x00, 0x77, 0x00, 0x00, 0x00, 0xbf, 0xff, 0xff,	初始化表中的一项，用于配置数组、结构体或常量表。
502	0xff, 0xff, 0xff, 0xfa, 0x00, 0x8f, 0xff, 0xff, 0xff, 0xf9, 0x00, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
503	0x85, 0x00, 0x00, 0x00, 0x49, 0x00, 0x00, 0x8f, 0xff, 0xff, 0xff, 0xf9,	初始化表中的一项，用于配置数组、结构体或常量表。

行	源码	说明
504	0x00, 0x00, 0x85, 0x00, 0x00, 0x00, 0x49, 0x00, 0x9f, 0xff, 0xff, 0xff,	初始化表中的一项，用于配置数组、结构体或常量表。
505	0xff, 0xff, 0xfa, 0x00, 0x94, 0x00, 0x77, 0x00, 0x3a, 0x00, 0x00, 0x9f,	初始化表中的一项，用于配置数组、结构体或常量表。
506	0xff, 0xff, 0xff, 0xfa, 0x00, 0x00, 0x94, 0x00, 0x77, 0x00, 0x3a, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
507	0x00, 0x9f, 0xff, 0xff, 0xff, 0xfa, 0x00, 0x00, 0x00, 0x00, 0x77, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
508	0x00, 0x00, 0x0e, 0xff, 0xff, 0xff, 0xff, 0xff, 0xe0, 0x00, 0x00, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
509	0x77, 0x00, 0x00, 0x00, 0xbf, 0xff, 0xff, 0xff, 0xff, 0xff, 0xfb, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
510	0xc2, 0x00, 0x00, 0x0f, 0x00, 0x00, 0x05, 0xb0, 0x00, 0x00, 0x0f, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
511	0x00, 0x0c, 0xff, 0xf3, 0x00, 0x0f, 0x00, 0x00, 0x69, 0x00, 0x02, 0xff,	初始化表中的一项，用于配置数组、结构体或常量表。
512	0xff, 0xff, 0xf1, 0x8c, 0xff, 0xd2, 0xc0, 0x0f, 0x00, 0xd1, 0x00, 0xc2,	初始化表中的一项，用于配置数组、结构体或常量表。
513	0x02, 0xc0, 0x0f, 0x00, 0xd1, 0x00, 0xc2, 0x02, 0xc0, 0x0f, 0x00, 0xd1,	初始化表中的一项，用于配置数组、结构体或常量表。
514	0xaf, 0xff, 0xf5, 0xff, 0xff, 0xff, 0xf1, 0x00, 0xc2, 0x02, 0xc0, 0x0f,	初始化表中的一项，用于配置数组、结构体或常量表。
515	0x00, 0xd1, 0x00, 0xc2, 0x00, 0x00, 0x0f, 0x00, 0x00, 0x00, 0xc6, 0xb0,	初始化表中的一项，用于配置数组、结构体或常量表。
516	0x00, 0x0f, 0x00, 0x00, 0x01, 0xc8, 0x20, 0x00, 0x0f, 0x00, 0x00, 0x01,	初始化表中的一项，用于配置数组、结构体或常量表。
517	0xc4, 0x0b, 0xff, 0xff, 0xff, 0xf2, 0x00, 0x4b, 0x10, 0x00, 0x00, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
518	0xd2, 0x1e, 0x00, 0x00, 0x00, 0x00, 0x00, 0xd2, 0x1e, 0x00, 0xdf, 0xff,	初始化表中的一项，用于配置数组、结构体或常量表。
519	0xfd, 0x00, 0xd2, 0x1e, 0x00, 0xd0, 0x00, 0x1d, 0x00, 0xd2, 0x1e, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
520	0xdf, 0xff, 0xfd, 0x00, 0xd2, 0x1e, 0x00, 0xd0, 0x00, 0x1d, 0x00, 0xd2,	初始化表中的一项，用于配置数组、结构体或常量表。
521	0x1e, 0x00, 0xd0, 0x00, 0x1d, 0x00, 0xd2, 0x1e, 0x00, 0xdf, 0xff, 0xfd,	初始化表中的一项，用于配置数组、结构体或常量表。
522	0x00, 0xd2, 0x1e, 0x00, 0xd0, 0x00, 0x00, 0x00, 0xd2, 0x1e, 0x00, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
523	0x00, 0x00, 0x01, 0xe1, 0x1e, 0x00, 0x00, 0x00, 0x0d, 0xee, 0x90, 0x8f,	初始化表中的一项，用于配置数组、结构体或常量表。
524	0xff, 0xff, 0xff, 0xff, 0xff, 0xf8, 0x00, 0x00, 0x00, 0xe2, 0x00, 0x00,	初始化表中的一项，用于配置数组、结构体或常量表。
525	0x00, 0x00, 0x00, 0x05, 0xb0, 0x00, 0x00, 0x00, 0xc0, 0xff, 0xff, 0xff,	初始化表中的一项，用于配置数组、结构体或常量表。
526	0xff, 0xff, 0xc0, 0x0c, 0x20, 0x95, 0x00, 0x68, 0x03, 0xc0, 0x0c, 0x20,	初始化表中的一项，用于配置数组、结构体或常量表。

行	源码	说明
527	0x9f, 0xff, 0xf8, 0x03, 0xc0, 0x0c, 0x20, 0x95, 0x00, 0x68, 0x03, 0xc0,	初始化表中的一项, 用于配置数组、结构体或常量表。
528	0x0c, 0x20, 0x9f, 0xff, 0xf8, 0x03, 0xc0, 0x0c, 0x20, 0x95, 0x00, 0x68,	初始化表中的一项, 用于配置数组、结构体或常量表。
529	0x03, 0xc0, 0x0c, 0x20, 0x95, 0x00, 0x68, 0x03, 0xc0, 0x0c, 0xff, 0xff,	初始化表中的一项, 用于配置数组、结构体或常量表。
530	0xff, 0xff, 0xff, 0xc0, 0x0c, 0x20, 0x00, 0x00, 0x00, 0x03, 0xc0, 0x7f,	初始化表中的一项, 用于配置数组、结构体或常量表。
531	0xff, 0xff, 0xff, 0xff, 0xff, 0xf8, 0x00, 0x00, 0x00, 0xb6, 0x00, 0x00,	初始化表中的一项, 用于配置数组、结构体或常量表。
532	0x00, 0x00, 0x00, 0x00, 0xe1, 0x00, 0x00, 0x00, 0x00, 0xef, 0xff, 0xff,	初始化表中的一项, 用于配置数组、结构体或常量表。
533	0xff, 0xff, 0x20, 0x00, 0xe1, 0x00, 0x00, 0x00, 0x0e, 0x20, 0x00, 0xe1,	初始化表中的一项, 用于配置数组、结构体或常量表。
534	0x00, 0x79, 0x00, 0x0e, 0x20, 0x00, 0xe1, 0x00, 0x79, 0x00, 0x0e, 0x20,	初始化表中的一项, 用于配置数组、结构体或常量表。
535	0x00, 0xe1, 0x00, 0x97, 0x00, 0x0e, 0x20, 0x00, 0xe1, 0x01, 0xe9, 0x10,	初始化表中的一项, 用于配置数组、结构体或常量表。
536	0x0e, 0x20, 0x00, 0x00, 0x1b, 0xa9, 0xe9, 0x30, 0x00, 0x00, 0x49, 0xd7,	初始化表中的一项, 用于配置数组、结构体或常量表。
537	0x00, 0x06, 0xcb, 0x30, 0x5d, 0x95, 0x00, 0x00, 0x00, 0x05, 0xc3, 0x00,	初始化表中的一项, 用于配置数组、结构体或常量表。
538	0x00, 0x09, 0xff, 0xff, 0xff, 0xf9, 0xaf, 0xff, 0xe0, 0x00, 0x7a, 0x00,	初始化表中的一项, 用于配置数组、结构体或常量表。
539	0x00, 0x00, 0xc3, 0x00, 0x00, 0xa6, 0x00, 0x00, 0x00, 0xc3, 0x04, 0xff,	初始化表中的一项, 用于配置数组、结构体或常量表。
540	0xff, 0xff, 0xd0, 0x00, 0xc3, 0x04, 0xb0, 0x00, 0x02, 0xd0, 0x00, 0xc3,	初始化表中的一项, 用于配置数组、结构体或常量表。
541	0x04, 0xb0, 0x3c, 0x02, 0xd0, 0x00, 0xc3, 0x04, 0xb0, 0x3c, 0x02, 0xd0,	初始化表中的一项, 用于配置数组、结构体或常量表。
542	0x00, 0xc3, 0x04, 0xb0, 0x5b, 0x02, 0xd0, 0x13, 0xdb, 0xc8, 0xb0, 0x88,	初始化表中的一项, 用于配置数组、结构体或常量表。
543	0x02, 0xd0, 0xab, 0x85, 0x25, 0xb4, 0xed, 0x83, 0xd0, 0x00, 0x00, 0x02,	初始化表中的一项, 用于配置数组、结构体或常量表。
544	0x9d, 0x41, 0x9e, 0x70, 0x00, 0x00, 0x5c, 0x71, 0x00, 0x02, 0xa6, 0x00,	初始化表中的一项, 用于配置数组、结构体或常量表。
545	0x04, 0xa0, 0x00, 0x0a, 0x40, 0x00, 0x00, 0x00, 0xc5, 0x00, 0x6c, 0x00,	初始化表中的一项, 用于配置数组、结构体或常量表。
546	0x00, 0x9f, 0xff, 0xff, 0xff, 0xff, 0xff, 0xfa, 0x00, 0x00, 0x00, 0xe2,	初始化表中的一项, 用于配置数组、结构体或常量表。
547	0x00, 0x00, 0x00, 0x00, 0xff, 0xff, 0xff, 0xff, 0xff, 0x10, 0x00, 0xf0,	初始化表中的一项, 用于配置数组、结构体或常量表。
548	0x00, 0x00, 0x00, 0x0d, 0x10, 0x00, 0xff, 0xff, 0xff, 0xff, 0xff, 0x10,	初始化表中的一项, 用于配置数组、结构体或常量表。
549	0x00, 0xf0, 0x00, 0x00, 0x00, 0x0d, 0x10, 0x00, 0xff, 0xff, 0xff, 0xff,	初始化表中的一项, 用于配置数组、结构体或常量表。

行	源码	说明
550	<code>0xff, 0x10, 0x00, 0xf0, 0x00, 0x00, 0x00, 0x0d, 0x10, 0x00, 0xff, 0xff,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
551	<code>0xff, 0xff, 0xff, 0x10, 0x00, 0xf0, 0x00, 0x00, 0x00, 0x0d, 0x10,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
552	<code>};</code>	结构体、枚举或初始化表结束。
553		空行，用来分隔逻辑块，让代码更容易阅读。
554	<code>static const uint16_t unicode_list_0[] = {</code>	定义当前文件私有的静态变量或函数，并给出初始值；外部文件不能直接访问。
555	<code>0x0000, 0x0001, 0x0023, 0x0084, 0x00da, 0x0116, 0x01d3, 0x01d7, 0x0252, 0x029e, 0x035b, 0x0369, 0x037b, 0x03fd, 0x0426, 0x0443, 0x0495, 0x0673, 0x085e, 0x08d4, 0x091e, 0x0926, 0x0936, 0x0afa, 0x0afd, 0x0bc1, 0x0d4e, 0x0fe8, 0x10f0, 0x10f6, 0x1149, 0x11f7, 0x1265, 0x14df, 0x1558, 0x177d, 0x17db, 0x17ec, 0x1834, 0x1878, 0x1915, 0x197a, 0x1f7e, 0x24ac, 0x25fc, 0x272b, 0x2742, 0x2760, 0x2b30, 0x2bb1, 0x2ef1, 0x30b5, 0x30d2, 0x30d5, 0x3147, 0x3164, 0x32f3, 0x3307, 0x34e7, 0x34fa, 0x3bf6, 0x3db4, 0x3de3, 0x4173, 0x4189, 0x41ca, 0x41ff, 0x4343, 0x43c3, 0x43c5, 0x4695, 0x47ea, 0x4958, 0x4a6b, 0x4a6f, 0x4b8c</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
556	<code>};</code>	结构体、枚举或初始化表结束。
557		空行，用来分隔逻辑块，让代码更容易阅读。
558	<code>static const lv_font_fmt_txt_glyph_dsc_t glyph_dsc[] = {</code>	定义当前文件私有的静态变量或函数，并给出初始值；外部文件不能直接访问。
559	<code>{.bitmap_index = 0, .adv_w = 0, .box_w = 0, .box_h = 0, .ofs_x = 0, .ofs_y = 0},</code>	初始化表中的一项，用于配置数组、结构体或常量表。
560	<code>{.bitmap_index = 0, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+4E0A */</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
561	<code>{.bitmap_index = 84, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+4E0B */</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
562	<code>{.bitmap_index = 168, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+4E2D */</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
563	<code>{.bitmap_index = 252, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+4E8E */</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
564	<code>{.bitmap_index = 336, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+4EE4 */</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
565	<code>{.bitmap_index = 420, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+4F20 */</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
566	<code>{.bitmap_index = 504, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+4FDD */</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
567	<code>{.bitmap_index = 588, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+4FE1 */</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
568	<code>{.bitmap_index = 672, .adv_w = 224, .box_w = 14, .box_h = 13, .ofs_x = 0, .ofs_y = -2}, /* U+505C */</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
569	<code>{.bitmap_index = 763, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+50A8 */</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
570	<code>{.bitmap_index = 847, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+5165 */</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
571	<code>{.bitmap_index = 931, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+5173 */</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
572	<code>{.bitmap_index = 1015, .adv_w = 224, .box_w = 14, .box_h = 13, .ofs_x = 0, .ofs_y = -2}, /* U+5185 */</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。

行	源码	说明
573	<code>{.bitmap_index = 1106, .adv_w = 224, .box_w = 14, .box_h = 13, .ofs_x = 0, .ofs_y = -2}, /* U+5207 */</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
574	<code>{.bitmap_index = 1197, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+5230 */</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
575	<code>{.bitmap_index = 1281, .adv_w = 224, .box_w = 14, .box_h = 13, .ofs_x = 0, .ofs_y = -2}, /* U+524D */</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
576	<code>{.bitmap_index = 1372, .adv_w = 224, .box_w = 14, .box_h = 13, .ofs_x = 0, .ofs_y = -2}, /* U+529F */</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
577	<code>{.bitmap_index = 1463, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+547D */</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
578	<code>{.bitmap_index = 1547, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+5668 */</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
579	<code>{.bitmap_index = 1631, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+56DE */</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
580	<code>{.bitmap_index = 1715, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+5728 */</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
581	<code>{.bitmap_index = 1799, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+5730 */</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
582	<code>{.bitmap_index = 1883, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+5740 */</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
583	<code>{.bitmap_index = 1967, .adv_w = 224, .box_w = 14, .box_h = 13, .ofs_x = 0, .ofs_y = -2}, /* U+5904 */</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
584	<code>{.bitmap_index = 2058, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+5907 */</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
585	<code>{.bitmap_index = 2142, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+59CB */</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
586	<code>{.bitmap_index = 2226, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+5B58 */</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
587	<code>{.bitmap_index = 2310, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+5DF2 */</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
588	<code>{.bitmap_index = 2394, .adv_w = 224, .box_w = 14, .box_h = 13, .ofs_x = 0, .ofs_y = -2}, /* U+5EFA */</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
589	<code>{.bitmap_index = 2485, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+5F00 */</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
590	<code>{.bitmap_index = 2569, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+5F53 */</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
591	<code>{.bitmap_index = 2653, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+6001 */</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
592	<code>{.bitmap_index = 2737, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+606F */</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
593	<code>{.bitmap_index = 2821, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+62E9 */</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
594	<code>{.bitmap_index = 2905, .adv_w = 224, .box_w = 14, .box_h = 13, .ofs_x = 0, .ofs_y = -2}, /* U+6362 */</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
595	<code>{.bitmap_index = 2996, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+6587 */</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。

行	源码	说明
596	<code>{.bitmap_index = 3080, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+65E5 */</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
597	<code>{.bitmap_index = 3164, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+65F6 */</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
598	<code>{.bitmap_index = 3248, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+663E */</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
599	<code>{.bitmap_index = 3332, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+6682 */</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
600	<code>{.bitmap_index = 3416, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+671F */</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
601	<code>{.bitmap_index = 3500, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+6784 */</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
602	<code>{.bitmap_index = 3584, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+6D88 */</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
603	<code>{.bitmap_index = 3668, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+72B6 */</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
604	<code>{.bitmap_index = 3752, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+7406 */</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
605	<code>{.bitmap_index = 3836, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+7535 */</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
606	<code>{.bitmap_index = 3920, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+754C */</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
607	<code>{.bitmap_index = 4004, .adv_w = 224, .box_w = 14, .box_h = 14, .ofs_x = 0, .ofs_y = -2}, /* U+756A */</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
608	<code>{.bitmap_index = 4102, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+793A */</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
609	<code>{.bitmap_index = 4186, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+79BB */</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
610	<code>{.bitmap_index = 4270, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+7CFB */</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
611	<code>{.bitmap_index = 4354, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+7EBF */</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
612	<code>{.bitmap_index = 4438, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+7EDC */</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
613	<code>{.bitmap_index = 4522, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+7EDF */</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
614	<code>{.bitmap_index = 4606, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+7F51 */</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
615	<code>{.bitmap_index = 4690, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+7F6E */</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
616	<code>{.bitmap_index = 4774, .adv_w = 224, .box_w = 14, .box_h = 13, .ofs_x = 0, .ofs_y = -2}, /* U+80FD */</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
617	<code>{.bitmap_index = 4865, .adv_w = 224, .box_w = 14, .box_h = 13, .ofs_x = 0, .ofs_y = -2}, /* U+8111 */</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
618	<code>{.bitmap_index = 4956, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+82F1 */</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。

行	源码	说明
619	<code>{.bitmap_index = 5040, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+8304 */</code>	资源数据行: 把图片或字体的二进制内容以 C 数组形式编译进固件。
620	<code>{.bitmap_index = 5124, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+8A00 */</code>	资源数据行: 把图片或字体的二进制内容以 C 数组形式编译进固件。
621	<code>{.bitmap_index = 5208, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+8BBE */</code>	资源数据行: 把图片或字体的二进制内容以 C 数组形式编译进固件。
622	<code>{.bitmap_index = 5292, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+8BED */</code>	资源数据行: 把图片或字体的二进制内容以 C 数组形式编译进固件。
623	<code>{.bitmap_index = 5376, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+8F7D */</code>	资源数据行: 把图片或字体的二进制内容以 C 数组形式编译进固件。
624	<code>{.bitmap_index = 5460, .adv_w = 224, .box_w = 14, .box_h = 13, .ofs_x = 0, .ofs_y = -2}, /* U+8F93 */</code>	资源数据行: 把图片或字体的二进制内容以 C 数组形式编译进固件。
625	<code>{.bitmap_index = 5551, .adv_w = 224, .box_w = 14, .box_h = 13, .ofs_x = 0, .ofs_y = -2}, /* U+8FD4 */</code>	资源数据行: 把图片或字体的二进制内容以 C 数组形式编译进固件。
626	<code>{.bitmap_index = 5642, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+9009 */</code>	资源数据行: 把图片或字体的二进制内容以 C 数组形式编译进固件。
627	<code>{.bitmap_index = 5726, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+914D */</code>	资源数据行: 把图片或字体的二进制内容以 C 数组形式编译进固件。
628	<code>{.bitmap_index = 5810, .adv_w = 224, .box_w = 14, .box_h = 14, .ofs_x = 0, .ofs_y = -1}, /* U+91CD */</code>	资源数据行: 把图片或字体的二进制内容以 C 数组形式编译进固件。
629	<code>{.bitmap_index = 5908, .adv_w = 224, .box_w = 14, .box_h = 13, .ofs_x = 0, .ofs_y = -1}, /* U+91CF */</code>	资源数据行: 把图片或字体的二进制内容以 C 数组形式编译进固件。
630	<code>{.bitmap_index = 5999, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+949F */</code>	资源数据行: 把图片或字体的二进制内容以 C 数组形式编译进固件。
631	<code>{.bitmap_index = 6083, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+95F4 */</code>	资源数据行: 把图片或字体的二进制内容以 C 数组形式编译进固件。
632	<code>{.bitmap_index = 6167, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+9762 */</code>	资源数据行: 把图片或字体的二进制内容以 C 数组形式编译进固件。
633	<code>{.bitmap_index = 6251, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+9875 */</code>	资源数据行: 把图片或字体的二进制内容以 C 数组形式编译进固件。
634	<code>{.bitmap_index = 6335, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+9879 */</code>	资源数据行: 把图片或字体的二进制内容以 C 数组形式编译进固件。
635	<code>{.bitmap_index = 6419, .adv_w = 224, .box_w = 14, .box_h = 12, .ofs_x = 0, .ofs_y = -1}, /* U+9996 */</code>	资源数据行: 把图片或字体的二进制内容以 C 数组形式编译进固件。
636	<code>};</code>	结构体、枚举或初始化表结束。
637		空行, 用来分隔逻辑块, 让代码更容易阅读。
638	<code>static const lv_font_fmt_txt_cmap_t cmaps[] = {</code>	定义当前文件私有的静态变量或函数, 并给出初始值; 外部文件不能直接访问。
639	<code>{</code>	代码块开始, 进入 当前结构 的内部逻辑。
640	<code>.range_start = 0x4e0a, .range_length = 19341, .glyph_id_start = 1,</code>	初始化表中的一项, 用于配置数组、结构体或常量表。
641	<code>.unicode_list = unicode_list_0, .glyph_id_ofs_list = NULL, .list_length = 76, .type = LV_FONT_FMT_TXT_CMAP_SPARSE_TINY</code>	资源数据行: 把图片或字体的二进制内容以 C 数组形式编译进固件。
642	<code>}</code>	代码块结束, 离开 当前结构 的内部逻辑。
643	<code>};</code>	结构体、枚举或初始化表结束。

行	源码	说明
644		空行，用来分隔逻辑块，让代码更容易阅读。
645	<code>#if LV_VERSION_CHECK(8, 0, 0)</code>	条件编译开始：只有配置宏满足时，下面这段代码才会参与编译。
646	<code>static lv_font_fmt_txt_glyph_cache_t cache;</code>	定义当前文件私有的静态对象或函数，限制作用域，降低模块耦合。
647	<code>static const lv_font_fmt_txt_dsc_t font_dsc = {</code>	定义当前文件私有的静态变量或函数，并给出初始值；外部文件不能直接访问。
648	<code>#else</code>	条件编译分支：当前一组配置条件的另一个选择。
649	<code>static lv_font_fmt_txt_dsc_t font_dsc = {</code>	定义当前文件私有的静态变量或函数，并给出初始值；外部文件不能直接访问。
650	<code>#endif</code>	条件编译结束。
651	<code>.glyph_bitmap = glyph_bitmap,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
652	<code>.glyph_dsc = glyph_dsc,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
653	<code>.cmaps = cmaps,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
654	<code>.kern_dsc = NULL,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
655	<code>.kern_scale = 0,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
656	<code>.cmap_num = 1,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
657	<code>.bpp = 4,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
658	<code>.kern_classes = 0,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
659	<code>.bitmap_format = 0,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
660	<code>#if LV_VERSION_CHECK(8, 0, 0)</code>	条件编译开始：只有配置宏满足时，下面这段代码才会参与编译。
661	<code>.cache = &cache</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
662	<code>#endif</code>	条件编译结束。
663	<code>};</code>	结构体、枚举或初始化表结束。
664		空行，用来分隔逻辑块，让代码更容易阅读。
665	<code>extern const lv_font_t lv_font_montserrat_14;</code>	声明外部符号：它的定义在别的文件里，当前文件只是使用它。
666		空行，用来分隔逻辑块，让代码更容易阅读。
667	<code>const lv_font_t ui_font_zh_14 = {</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
668	<code>.get_glyph_dsc = lv_font_get_glyph_dsc_fmt_txt,</code>	初始化表中的一项，用于配置数组、结构体或常量表。

行	源码	说明
669	<code>.get_glyph_bitmap = lv_font_get_bitmap_fmt_txt,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
670	<code>.line_height = 16,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
671	<code>.base_line = 3,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
672	<code>.subpx = LV_FONT_SUBPX_NONE,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
673	<code>.underline_position = -1,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
674	<code>.underline_thickness = 1,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
675	<code>.dsc = &font_dsc,</code>	初始化表中的一项，用于配置数组、结构体或常量表。
676	<code>.fallback = &lv_font_montserrat_14</code>	资源数据行：把图片或字体的二进制内容以 C 数组形式编译进固件。
677	<code>};</code>	结构体、枚举或初始化表结束。